

中药材烘干热湿耦合建模、守恒求解与模型检验 ——基于质量坐标有限体积的四问统一求解

摘要

中药材干燥是产地加工的关键环节：水分偏高易霉变，干燥不足或过度都会影响品质。本文以长 25 cm、半径 2 cm 的圆柱药柱为对象，初始干基含水率 2.55 kg/kg、初温 28 °C，研究预热段与全过程的温度场与水分浓度场演变，并确定固定几何与计入失水收缩两种情形下的达标时间，为烘干时长的控制提供依据。

针对问题 1（0–1800 s 预热段），含水率变化很小、热湿近似解耦，材料参数按附录 2 取常数，热方程退化为线性圆柱 Robin 问题；由此写出圆柱 Robin 特征展开与时变环境 Duhamel 卷积的解析谱解，再与守恒型有限体积数值解逐点对照，最大偏差 $5.9 \times 10^{-7} \text{ °C}$ ，1800 s 末全场温差收敛到约 1 °C。解析解不含离散误差，两者一致说明所报温度场是模型的解，空间离散、边界处理与时间步进没有引入可观误差。

针对问题 2，材料参数按附录 3 取用，保留温度与水分浓度的双向耦合，用质量坐标下的守恒型有限体积格式求解：表面取半单元串联阻力，时间推进用后向欧拉加局部外推，非线性项用 Picard–Newton 交替迭代，步长自适应。判据 $Bi_m \in [1.1805, 47.7480]$ 表明内部扩散主导，把含水率作集总或平均处理会把达标时间低估 46.03 h（–80%），与题目要求的空间分布不符；因此保留全场分布，给出 72 h 内各时刻的温度与水分浓度场。

针对问题 3，在问题 2 的模型上加入达标判据（各处水分浓度首次低于 0.15 kg/kg 干基），由数值扫描确认中心为最慢点，按穿越区间及其右侧安全端点报告达标时刻： $t_f = 57.1799 \text{ h}$ （Richardson 外推 57.1636 h）。该值远大于常系数参考时标 15.70 h，量级与参数一致；其中对边界汇取值最敏感， $C_e = 0.1366$ 时增大 22.6 h，已检验因素的累计偏移量级约 1.71 h。

针对问题 4，几何按仿射收缩 $r = R(t)\sqrt{q}$ 演化，方程与固定域同形；为确定 $R(t)$ ，把失水收缩、毛细-膨压塌陷、结皮-成孔停滞三项机制写成半经验关系式，由湿度场逐步预测半径，预测不使用附件 2 的数据，附件 2 实测半径仅作交叉验证（RMSE 0.047 mm、偏差 –0.56%）。得 $t_f = 50.5369 \text{ h}$ ，比问题 3 快 11.6%：附录 4 的材料参数使固定半径下的烘干时间放大 2.26 倍（+78.57 h），半径收缩（ R^2 降 64%）则加速 60.9%，两者相抵后反而更快；耦合主解在满 72 h 真实轨迹上的全程 RMSE 0.088 mm。

关键词：热湿耦合 质量坐标 守恒有限体积 无量纲判据 模型检验 中药材干燥

一、 问题重述

1.1 问题背景

中药材的干燥是药材加工的关键环节：水分过高会导致霉变，干燥不足或过度都会影响品质。本题药材为圆柱形（长 $L_0 = 25$ cm、半径 $R_0 = 2$ cm），初始温度 28°C 、初始干基含水率 2.55 kg/kg，置于烘干房中干燥；烘房空气温度与空气水分浓度由附件 1 给出（0–14400 s、60 s 一步），药材失水时外半径随之收缩（附件 2 给出 0–72 h、1800 s 一步的半径，由 2.0 cm 缩至 1.198 cm 后进入近平台段）。烘干达标指药材各处水分浓度（干基含水率）均首次低于 0.15 kg/kg。

1.2 问题提出

四个问题依次为：

问题 1： 预热平衡阶段（0–1800 s）药材内温度与水分浓度的时空分布，材料参数取附录 2，按题目所给的表格格式列出；

问题 2： 整个烘干过程的温度与水分浓度分布情况，材料参数取附录 3；

问题 3： 固定几何下首次全部达标的烘干时间，结果按表 5 的格式给出；

问题 4： 计入失水收缩后的达标时间，外半径取附件 2、材料参数取附录 4，结果按表 6 的格式给出。

四个结果分别写入 result1.xlsx 至 result4.xlsx。

总体思路是建立统一模型，四问共用同一套热湿耦合控制方程，差别只在取法：材料参数来源、热湿耦合方式、几何是否演化以及结果形式四项不同。其中问题 1 是问题 2 的解耦特例，问题 3 在问题 2 的基础上增加阈值判据，问题 4 是移动域的推广；对收缩几何，先按有物理含义的半经验关系式由内而外地预测半径，附件 2 只作验证数据，再用力学驱动扩展模型讨论收缩的机制边界。几何模型与边界条件见图 1。

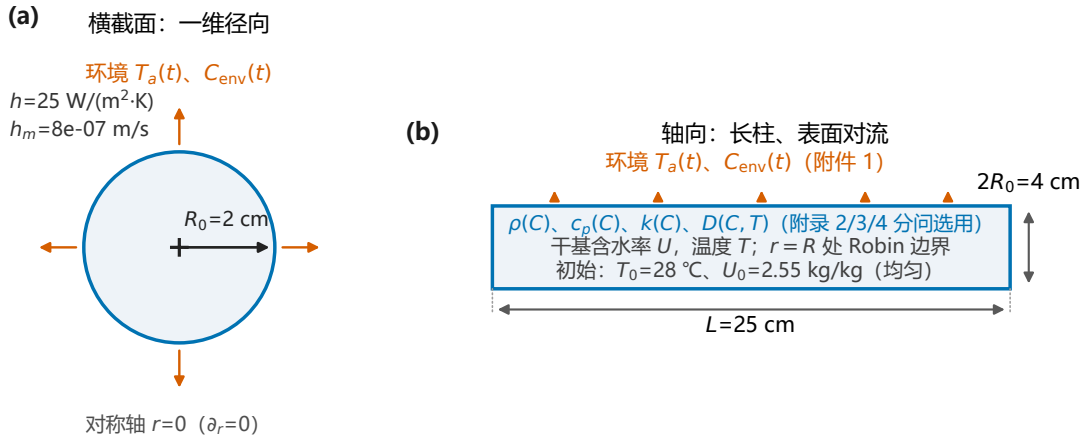


图 1 几何模型与边界条件。长 25 cm、半径 2 cm 的圆柱药柱，初始 $U_0 = 2.55$ kg/kg、 $T_0 = 301.15$ K；柱面与环境 $T_a(t)$ 、 $C_{env}(t)$ 经对流换热 $h = 25$ W/(m²·K) 与表面水分交换 $h_m = 8 \times 10^{-7}$ m/s 耦合；轴心对称、端面忽略（一维径向假设，见第三章）

二、 问题分析

2.1 四个问题的统一结构与难点

四个问题（以下简称四问）的物理过程相同，都包含内部导热与导湿、表面对流边界以及材料参数的强非线性。(i) 三套附录材料参数必须按问题分别使用，附录 2/3/4 的 ρ, c_p, k, D 互不相同，不得把三套参数调和、平均或取为常数。例如附录 4 与附录 3 的扩散系数比 $0.175 e^{0.15/U}$ 在高含水端约为 1/5.4（图 2），(ii) $D \propto e^{-\alpha/C} e^{-3850/T}$ 对局部含水率与温度强非线性，尾段 D 最多缩小到原先的 16.8 倍，形成低扩散率的“干壳”，决定达标时间的尾段行为；(iii) 环境数据只有 4 h，半径数据有 72 h，而烘干需 50 h 以上，边界外推方式与收缩关系式的取法都会改变结论的量级。

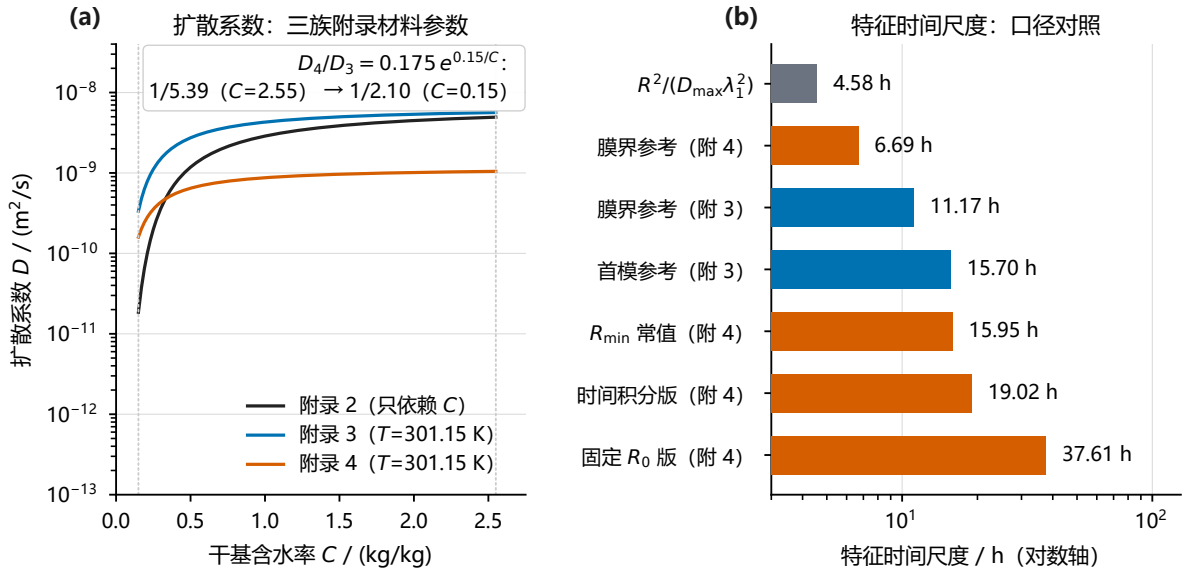


图 2 扩散系数非线性与特征时间尺度：(a) 三套附录材料参数的 D - C 半对数曲线——附录 2 只与 C 有关、不含 T ，附录 3/4 为 $D(C, T)$ ；附录 4 全程低于附录 3，尾段 $C \rightarrow 0.15$ 时 D 缩小 16.8 倍，该倍率按附录 3 的材料参数公式算出；(b) 各参考时标：常系数首模参考 15.7021 h、膜界 11.1720 h、问题 4 瞬时首模时间积分 19.016 h、固定 R_0 版 37.61 h、 $R_{\min}$ 常值版 15.95 h——达标时间的两个主值 57.18 h 与 50.54 h 均大于上述各时标

2.2 各问分析与建模路线

问题 1（预热段场）：0–1800 s 含水率变化很小，热湿近似解耦， D 不含 T 、按附录 2 取常数材料参数，热方程退化为线性圆柱 Robin 问题，解析谱解可直接写出，其形式为圆柱 Robin 特征展开与时变环境的 Duhamel 卷积。把解析解与数值解逐点对照，两者吻合到 $5.9 \times 10^{-7} ^\circ\text{C}$ ，说明数值解求的正是这组方程，问题 1 的温度场不是求解过程带进来的偏差；两边用的是同一套材料参数，所以这一比较不能说明材料参数的取值本身是否正确（图 13）。

问题 2/3（全程场与达标时间）：必须保留双向耦合（ D 含 C, T ）与空间分布。判据 $Bi_m \in [1.1805, 47.7480]$ 表明内部扩散主导，任何“集总/平均含水率”的处理都会把达标时间低估 46.03 h（-80%），无法得到本题的达标时间。求解路线是质量坐标守恒有限体积加自适应时间积分，并用解析退化、守恒恒等式与网格收敛一并检验；达标判据用未舍入值在重建场上判定，中心为最慢点，已由逐步数值扫描确认。

问题 4 (收缩下达标时间): 几何演化使方程成为移动域问题。在材料坐标下取仿射收缩 $r = R(t)\sqrt{q}$, 方程保持同形且无伪对流项; 半径 $R(t)$ 的给定方式直接决定问题 4 的结果。本文把“失水收缩、毛细-膨压塌陷、结皮-成孔停滞”三项有物理含义的机制写成半经验关系式, 由湿度场逐步内生预测 $R(t)$, 半径不使用附件 2 的数据; 附件 2 实测半径降为验证数据。另用力学驱动扩展模型讨论收缩的机制结构与精度边界, 该模型由有限应变径向平衡与可标定保水本构构成, 见第 5.5 节。

2.3 输入数据与预处理

附件 1 的环境时程共 241 点, 用分段线性插值取值, 4 h 后采用末值保持外推, 外推值取 $T_a = 50.165^\circ\text{C}$ 、 $C_{\text{env}} = 0.04986$; 其他取值方式作为对照, 已纳入敏感性分析。附件 2 半径用 PCHIP 保形插值, 插值结果半径为正且单调不增, 72 h 后末值延拓, 数据与插值行为见图 3。

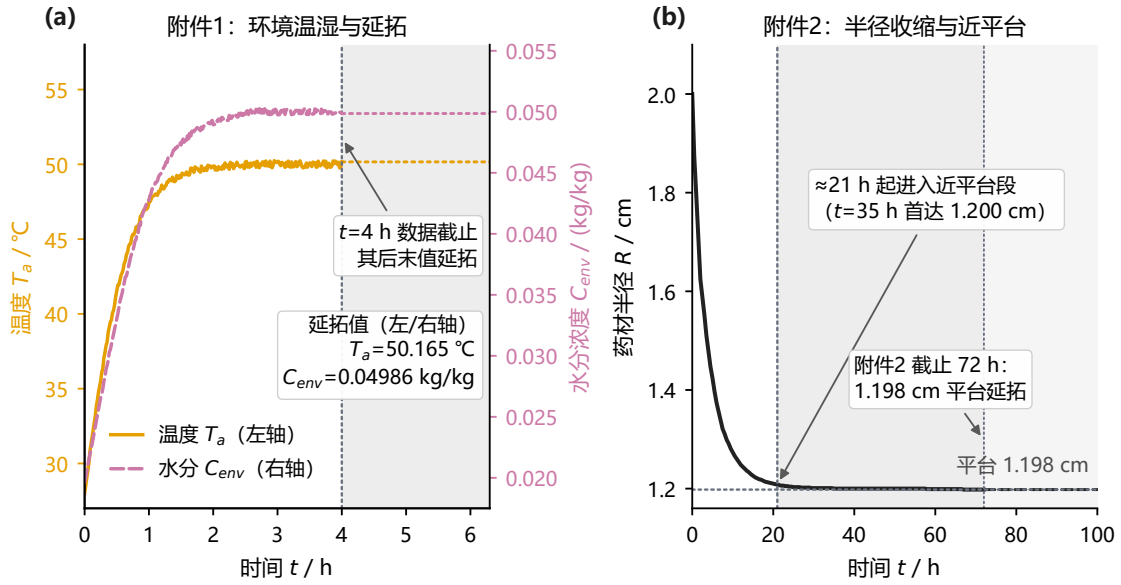


图 3 输入数据: 附件 1 给出 0–4 h、每 60 s 一步的烘房温度与空气水分浓度, 4 h 后按末值保持外推; 附件 2 给出 0–72 h 的外半径, 呈急缩、缓缩、平台三段结构, 约 21 h 起进入近平台段

三、模型假设

1. **一维径向、轴对称, 忽略端面与轴向。**依据: 端面与侧面的面积比 $R/L = 8\%$ 、轴向扩散路径 $L/2 = 12.5 \text{ cm}$ 是径向的 6.25 倍, 扩散时间尺度为径向的 $6.25^2 \approx 39$ 倍。
2. **湿边界取本文设定的等效边界条件:** 把附件 1 的 $C_{\text{env}}(t)$ 直接当作与 h_m 配套的 Robin 驱动势 $-D\partial_r U = h_m(U_s - C_{\text{env}})$ 。依据: 空气与固体含水率基准不同, 严格的边界条件需借助吸附等温线或水活度关系; 本文结论对这一取值最敏感, 其影响由 $C_e = 0.1366$ 的敏感性 +22.6261 h 给出, 见第七章。
3. **热方程取无潜热的有效显热近似。**依据: 潜热通量与显热通量同量级, 通量比为 2.06–2.49, 用第一问 1800 s 点的表面量复算, 通量比为 5.54; 作为对照的潜热情景下 t_f 由 57.1799 h 变为 62.5694 h (+5.3889 h), 见 5.3 节。

4. **材料参数逐点、按局部瞬时值取值：** ρ, c_p, k, D 按附录公式在局部 C 、局部 T （开尔文）取值； $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 自附录 2 延用到问题 2-4。
5. **环境外推：** $t > 14400 \text{ s}$ 取末值保持。对照取法有线性外推与末 1 h 均值，差异在 +1.62 h 以内，已纳入敏感性分析。
6. **几何：** 长度不变，内部取等比仿射收缩 $r = R(t)\sqrt{q}$ ； $R(t)$ 由半经验关系式逐步预测，72 h 后保持 1.198 cm。
7. **$\rho(C)$ 不用于质量—体积换算：** ρ 只用于有效体积热容 $a_{\text{eff}} = \rho c_p$ 与通量算子 $\rho_{sv} = \rho/(1+C)$ 。依据：附录 3/4 的 $\rho(C)$ 与附件 2 的 $R(t)$ 在干固体质量守恒下不相容，要求 $R \geq 1.301 \text{ cm}$ 而数据终值为 1.198 cm。

四、 符号说明

表 1 主要符号

符号	含义	单位
q	材料坐标（干物质标签）， $q = (X/R_0)^2$	—
$U(q, t)$	干基含水率（水分浓度）	kg/kg
$T(q, t)$	温度（开尔文）	K
$D(C, T)$	扩散系数（附录 2/3/4 公式）	m^2/s
ρ, c_p, k	密度、比热容、导热系数（逐点取值）	kg/m^3 、 $\text{J}/(\text{kg} \cdot \text{K})$ 、 $\text{W}/(\text{m} \cdot \text{K})$
a_{eff}	有效体积热容 ρc_p	$\text{J}/(\text{m}^3 \cdot \text{K})$
h, h_m	表面对流换热系数、表面传质系数	$\text{W}/(\text{m}^2 \cdot \text{K})$ 、 m/s
$T_a(t), C_{\text{env}}(t)$	环境温度、环境水分浓度（附件 1）	K、kg/kg
$R(t)$	药柱外半径（问题 4 中为待求量）	m
t_f	达标时间（ $\max_q U < 0.15$ 首次成立）	h
Le, Bi_m, Bi_h	无量纲判据： α/D 、 $h_m R/D$ 、 hR/k	—
J	体积变形梯度（力学扩展）	—
S	液相饱和度（力学扩展）	—
$p_c, \bar{p}_c$	毛细压及其储能平均（力学扩展）	（模型单位）

五、 模型建立与求解

5.1 统一控制方程与质量坐标

守恒律与材料坐标： 从干固体守恒与水分守恒出发，以干物质标签 $q \in [0, 1]$ 作为材料坐标，其中 $q = (X/R_0)^2$ 、 X 为初始物质点，描述径向一维热湿耦合过程；毛细多孔介质热湿传递的经典框架见 [1]。当干物质密度沿空间均匀、且内部取仿射收缩 $r = R(t)\sqrt{q}$ 时，方程与固定域同形、无伪对流项，推导见附录 A：

$$U_t = \partial_q \left(\frac{4qD}{R(t)^2} U_q \right), \quad a_{\text{eff}} T_t = \partial_q \left(\frac{4qk}{R(t)^2} T_q \right). \quad (5.1)$$

边界为移动表面 $q = 1$ 上的 Robin 条件，形式不变，其中跳条件里的几何运动项恒等相消：

$$-D \partial_r U = h_m (U_s - C_{\text{env}}(t)), \quad -k \partial_r T = h (T_s - T_a(t)), \quad (5.2)$$

$q = 0$ 处取正则对称条件；初值 $U \equiv 2.55$ 、 $T \equiv 301.15$ K。式 (5.1) 与边界式 (5.2) 的全部符号见表 1；四问的差别仅在取法（表 2）：

表 2 问题 1–4 的取法对照

取法	问题 1	问题 2	问题 3	问题 4
材料参数	附录 2（常数， $D = f(C)$ ）	附录 3	附录 3	附录 4
热湿耦合	解耦（ D 不含 T ）	双向耦合	双向耦合	双向耦合
几何	固定 $R = 0.02$ m	固定	固定	$R(t)$ 内生（半经验关系式，附件 2 交叉验证）
环境	附件 1（0–1800 s）	附件 1 + 外推	同左	同左
结果形式	场（表 1/2, result1）	场（表 3/4, result2）	决策量 t_f （表 5, result3）	决策量 t_f （表 6, result4）

数值方法：固定 q 网格单元中心有限体积 ($N=160$ 主网格)，内面通量共享；表面用半单元内阻与外对流阻串联，表面值 U_s, T_s 由 $8D_{\text{eff}}(U_s - U_N)/(R^2 \Delta q) = 2h_m(C_{\text{env}} - U_s)/R$ 联立重建，其中 D_{eff} 取半单元中点值，不把末单元平均值当作表面值。这样取值可避免强变的 $D \propto e^{-0.45/C}$ 在端点取值时产生内通量被强制为零的伪根。

5.2 问题 1：预热平衡段的场（0–1800 s）

模型：材料参数取附录 2 的常数值，热湿解耦，热方程为线性圆柱 Robin 问题；湿分在 1800 s 内变化小但非零，仍按同一框架求解。**求解与局部检验：**数值解与解析谱解逐点比较；后者取圆柱 Robin 特征展开加时变环境的 Duhamel 卷积，并用稳态减法消除级数尾巴，其自身误差 $< 10^{-8}^\circ\text{C}$ 。单元中心最大偏差为 $5.9 \times 10^{-7}^\circ\text{C}$ ($N=320$)，收敛阶为 2.00/1.99/1.99，见图 13。这一比较说明数值解确实求出了这组方程的准确解；由于两边用的是同一套材料参数，它不能说明材料参数的取值是否正确。**结果：**图 6 给出温度与水分浓度的径向分布（色阶按时间递增），图 4 给出时空云图，图 5 给出特征位置的时程与偏差、表面通量与表面扩散系数。结果表见表 6、表 7（result1.xlsx）。

5.3 问题 2/3：全程场与固定几何达标时间

模型：材料参数取附录 3，热与湿双向耦合，几何固定；达标判据

$$t_f = \inf\{t : \max_{q \in [0,1]} U(q, t) < 0.15 \text{ kg/kg}\}, \quad (5.3)$$

用未舍入值在全域重建场上判定，判据即式 (5.3)。本文不把中心最慢当作先验假设，而由逐步数值扫描确认：全部时刻的最大值均位于中心列，判定容差取 10^{-12} 。**结果（场）：**温湿剖面随时间的演化（色阶按时间递增，viridis）见图 6，水分场时空云图与干壳/湿核结构见图 7；干燥锋面自表面向内推进、表面先干、中心最慢。随表面变干， $D \propto e^{-0.45/C}$ 最多缩小 16.8 倍，形成低扩散率“干壳”，中心残余水分只能穿过它排出。

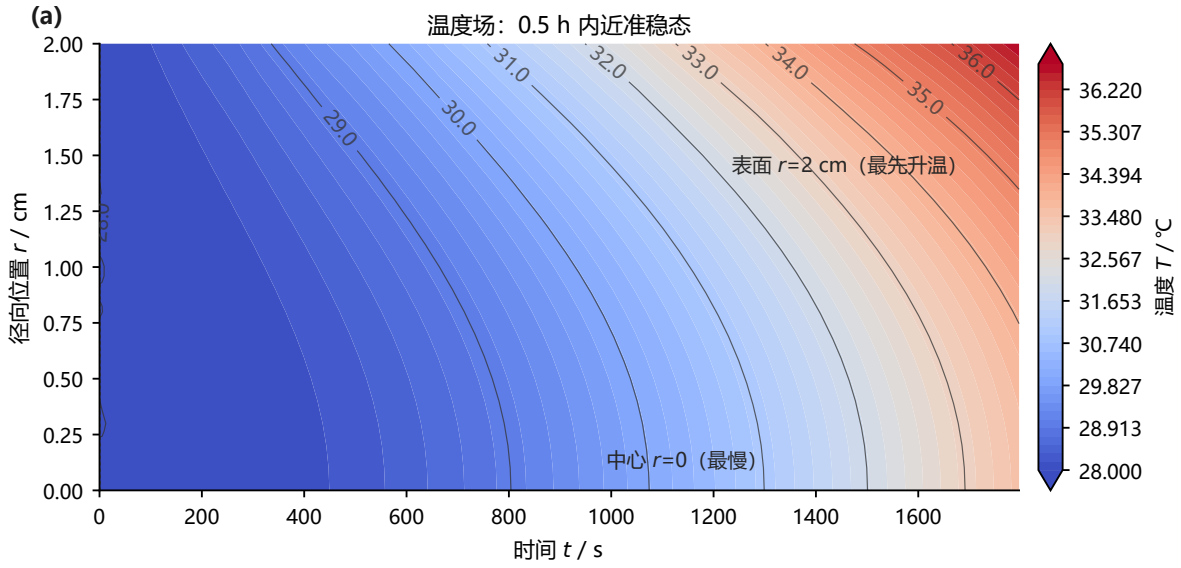


图 4 温度场时空演化：云图以时间为横轴、径向位置为纵轴，表面在约 600 s 内贴近环境、中心滞后明显，1800 s 末全场温差收敛到约 1°C 量级

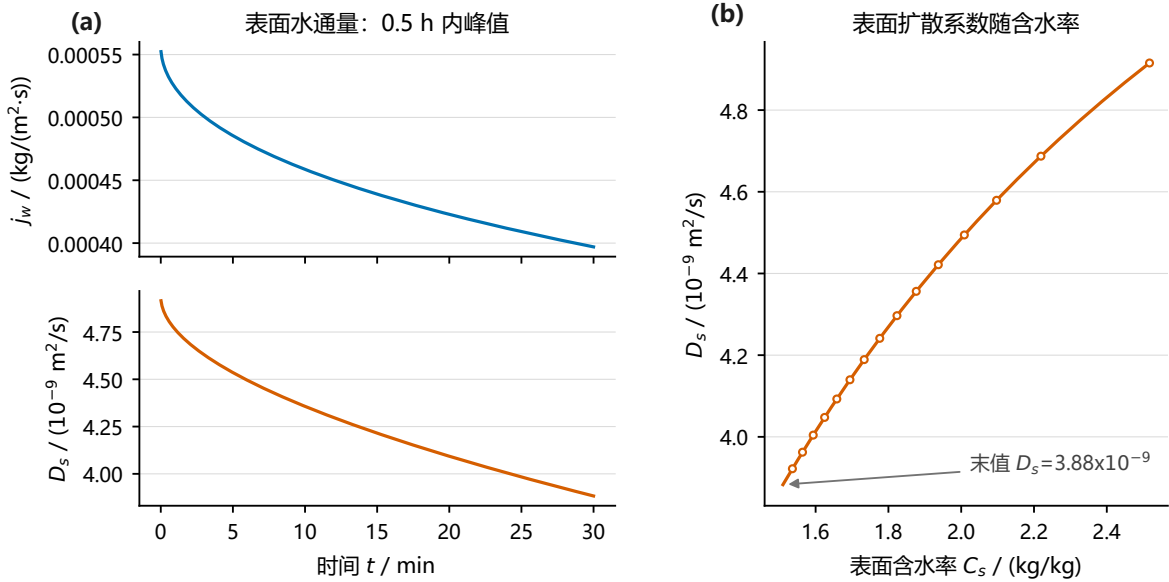


图 5 表面通量与表面扩散系数：(a) 表面水分通量 j_s 与表面扩散系数 $D_s = D(C_s)$ 的时程：通量初期最大而后衰减， D_s 随表面变干持续下降；(b) D_s 与 C_s 的对应关系，本过程的 C_s 只落在高含水端，故曲线平缓；图 2(a) 给出 $D \propto e^{-0.89/C}$ 在全量程上的强非线性

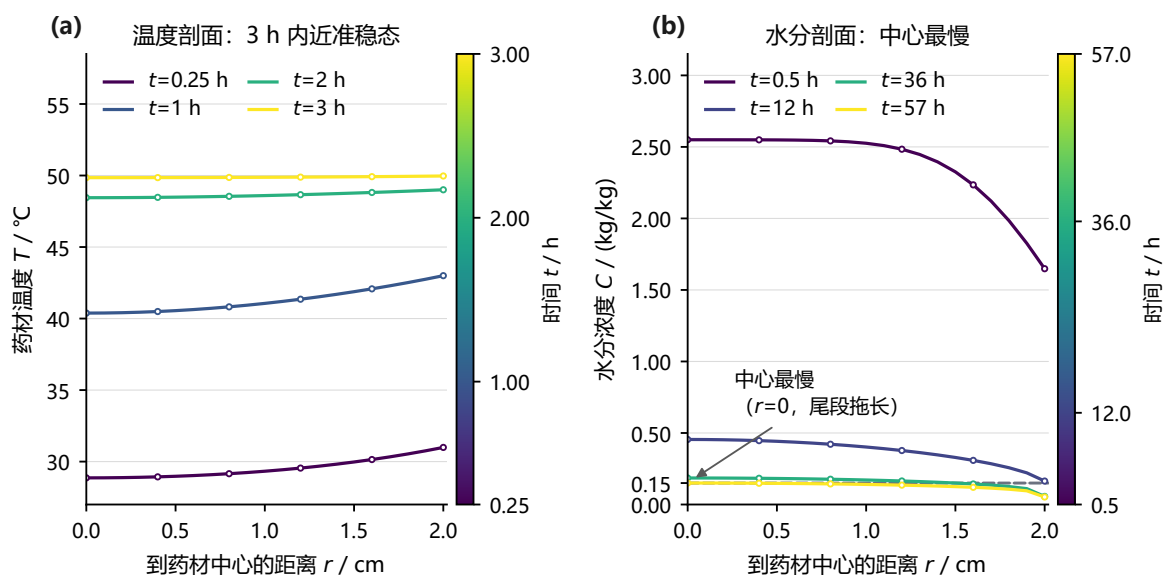


图 6 温湿剖面演化

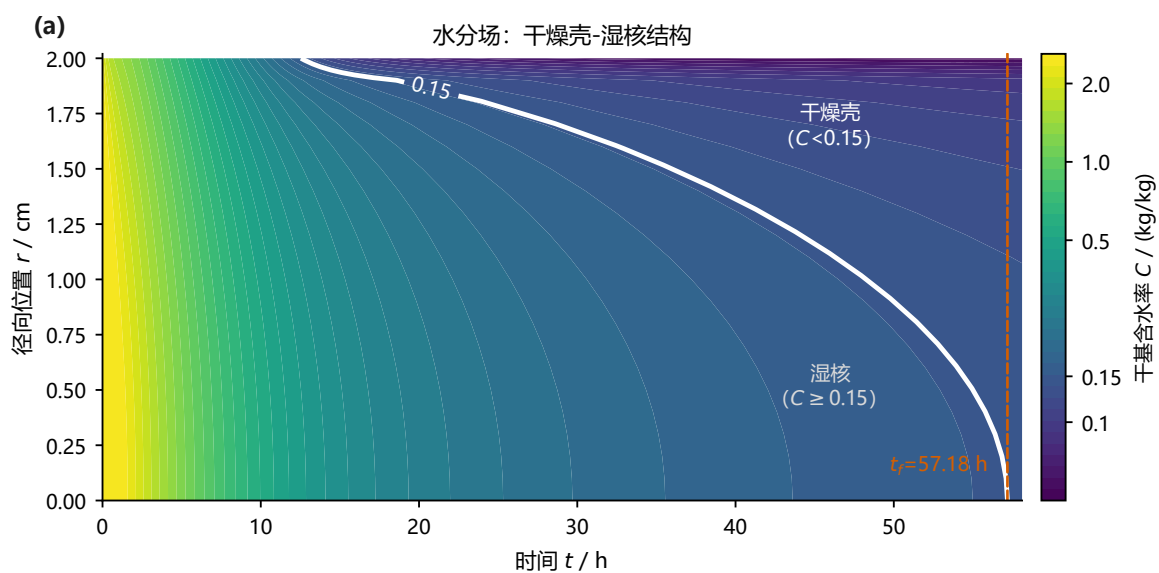


图 7 水分场时空演化

尾段 $\ln(U_{\max} - C_e)$ 呈指数衰减 ($R^2 = 0.9972$), 衰减率对应剖面加权等效 $D \approx 2.5 \times 10^{-10} \text{ m}^2/\text{s}$ 。

结果 (达标时间): 数值上报告阈值穿越点所在的区间, 右端加安全端点: 问题 3 为 $[205837.7, 205847.7] \text{ s}$, $t_f = 57.1799 \text{ h}$, 插值值 57.1795 h , 大于常系数参考时标 15.70 h , 满足该判据。收敛序列 ($N=40/80/160$) 为 $57.249/57.201/57.180 \text{ h}$, Richardson 外推 57.1636 h , 主值偏差 0.016 h 属空间误差量级。四位小数舍入后的 60 s 网格判定为 57.2333 h , 与未舍入值差 0.053 h , 属另一种数据使用方式的差异, 见第七章。阈值 $0.15 \text{ 干基} = 13.04\% \text{ 湿基}$, 与药材水分限度的通行标准 (约 13%) 相符 [3]。结果表见表 8、表 9 (result2.xlsx) 与表 10 (result3.xlsx); 达标时刻见图 15。

潜热对照情景: 第三章假设 3 的无潜热近似用情景检验界定: 表面相变汇 $-k\partial_r T|_R = h(T_s - T_a) + L_v j_w$, $j_w = \rho_{d,s} h_m (U_s - U_e)$ ($\rho_{d,s} = (650 + 128U_s)/(1 + U_s)$ 为表面局部干物质密度, $L_v = 2.4 \times 10^6 \text{ J/kg}$ 近似值)。对照情景沿用同一套附录 3 材料参数、同一边界数据与同一数值框架, t_f 由 57.1799 h 变为 **62.5694 h** (**$+5.3889 \text{ h}$** , $+9.4\%$), 全程最大逐时表面/中心温差 $29.6/29.2^\circ\text{C}$ 、最大平均含水率差 0.322 kg/kg 。故无潜热基准的 t_f 偏差量级约 5.4 h 。该情景中表面温度一度降至 13.4°C 、低于湿球温度, 说明 h_m 与能量边界条件尚需协调。

5.4 问题 4: 收缩移动域与内生收缩模型

几何与移动域: 仿射收缩 $r = R(t)\sqrt{q}$ 下方程同形, 见式 (5.1), $R(t)$ 为待确定的几何函数。附件 2 的三段结构是 A 急缩 $0-3.5 \text{ h}$ 占 61% 、B 缓缩 $3.5-21 \text{ h}$ 、C 近平台段约 21 h 起, 它不是单一失水收缩曲线: 理想等容收缩这一特定基准 $R_0\sqrt{(1+\bar{U})/(1+C_0)}$ 不能解释全部数据, 其反演平均含水率为 0.278 kg/kg , 低于模型同刻的 0.149 kg/kg , 在 $0.129 \rightarrow 0.155 \text{ kg/kg}$ 区段还存在成孔造成的偏差, 并与数据在约 16 h 交叉滞后, 附件 2 的时间范围是 $0-72 \text{ h}$, t_f 落在该区间内, 见图 8。

半经验收缩关系式: 问题 4 主模型由三项物理机制叠加而成, 每项对应一种物理过程, 半径由模型自身逐步预测, 共 5 个参数:

$$R(t) = \underbrace{R_0 \sqrt{\frac{1+\bar{U}(t)}{1+C_0}}}_{\text{失水收缩}} \cdot \underbrace{[1 - \varepsilon_c g_c(\bar{U})]}_{\text{毛细-膨压塌陷}} \cdot \underbrace{[1 + \varepsilon_p g_p(C_s)]}_{\text{结皮-成孔停滞}} \quad (5.4)$$

(i) 失水收缩: 这一项不含待定参数, 由水分守恒与固体、水等密度的假设给出; (ii) 毛细-膨压塌陷: $g_c(x) = (1-x)^a x^b$ ($x = (C_0 - \bar{U})/C_0$, 峰值归一), 其量级由毛细压实的估计支撑, 估计区间 $[0.028, 0.580] \text{ cm}$ 包含了 2 h 观测到的 0.182 cm 超额与 9.1% 的半径应变; (iii) 结皮-成孔停滞: 表面 C_s 达到 $C_{\text{glass}} \approx 0.21$ 后收缩停滞, 继续失水转化为孔隙, 这一阈值取 Gordon-Taylor 结构的典型参数情景值 [4], 已列入参数不确定清单; $g_p = \text{clip}((C_{\text{glass}} - C_s)/(C_{\text{glass}} - C_{s,\min}), 0, 1)^{k_p}$, $C_{s,\min} = 0.0488 \text{ kg/kg}$ 为完整轨迹读得的归一化常数。 g_c, g_p 是有物理含义的经验形状函数, 该式整体为半经验关系式; 5 个参数由最小二乘对附件 2 标定, C_{glass} 与 $\rho_{sk} = 1468$ 这 2 个量取情景值或典型范围值, 不参与拟合, 参数表见附录 B, 标定细节见附录 C。

标定与验证: 拟合优度 $\text{RMSE} = 0.047 \text{ mm}$, 约为附件 2 量化分辨率 0.01 mm 的 5 倍, 最大偏差 0.162 mm ; 滞后交叉约 17 h (计算值 17.0 h), 数据为约 16 h ; 平台高度 1.1969 cm , 数据为 1.198 cm ; 三段形态全部复现, 见图 9。孔隙份额自 9.5 h 起转为正值并单调增加, 成孔由此开始, 末期孔隙率为 30.9% 。过拟合检验表明: 参数标定与验证用的是同一批数据, $R_{\text{obs}} \rightarrow (\bar{U}, C_s)_{\text{主解}} \rightarrow \text{标定} \rightarrow R_{\text{pred}}$ 构成循环; 奇偶折半留出法只检验了同一条光滑曲线上的插值稳健; 若要独立验证, 需对每组候选参数从相同初值重解, 即完整耦合标定, 本文当前的做法是借助实测几何生成湿度场再作标定, 尚属近似。

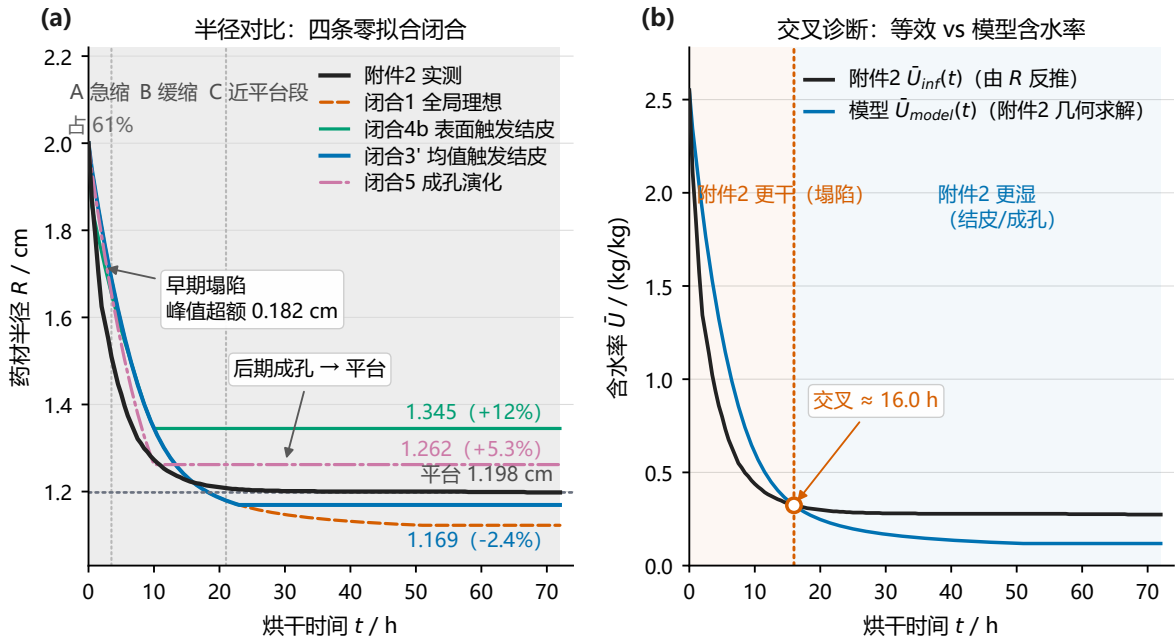


图8 附件2的收缩机制分析。附件2呈A急缩61%、B缓缩、C平台的三段复合结构，与约16 h滞后交叉；不作参数拟合的“理想收缩+玻璃化冻结”半经验关系式已由结构给出减速与平台，其中按平均含水率判断结皮的那一条给出平台1.169 cm、偏差-2.4%、冻结时刻偏差+12%；早期塌陷与后期成孔对应两组机制。数据源：问题4主解湿度场与附件2实测半径

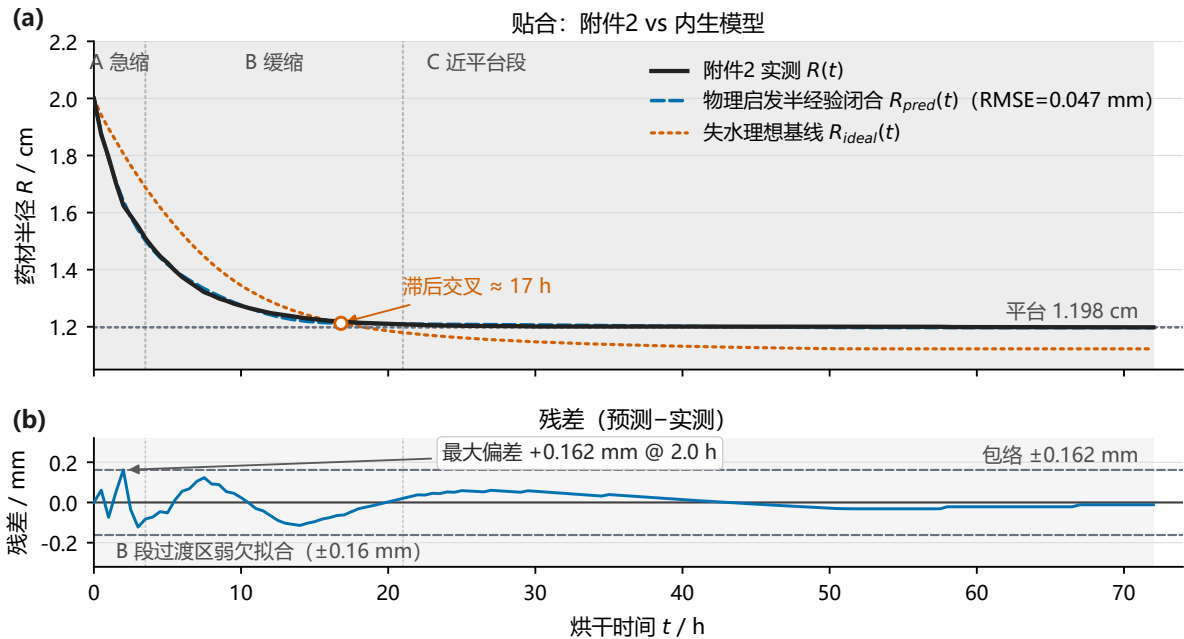


图9 内生收缩模型对附件2的吻合与验证：式(5.4)由三项物理机制叠加而成，含5个标定参数与2个不参与拟合的量，它复现了附件2的全程形态：RMSE 0.047 mm，约为量化分辨率的5倍；滞后交叉17.0 h，数据约16 h；平台1.1969 cm，数据1.198 cm；三段形态全部复现。残差子图显示偏差始终在 ± 0.16 mm以内，这是以实测几何单向驱动得到的拟合指标

耦合主解：附件 2 在此仅作验证数据。主结果采用完全内生的收缩， R 由湿度场逐步预测，每步以当前 $\bar{U}, C_s$ 代入式 (5.4)，不使用附件 2 的几何数据，得 $t_f = 50.5369$ h，该值取自 result4 内生版的第 3033 行；未对右端点舍入，达标时刻所在区间为 $[181917.7, 181932.7]$ s，瞬时首模近似给出的参考值为 19.02 h，该结果满足这一参考。把附件 2 实测几何作为输入的对照版得 $t_f = 50.8230$ h，与主解的偏差为 -0.286 h、 -0.56% ，这一比较属模型之间的对照，不是独立的实验验证。两个精度指标须分清：**0.047 mm** 是以实测几何单向驱动得到的拟合指标，耦合主解自身的指标是沿满 72 h 真实轨迹的全程 RMSE **0.088 mm**，其中 0-达标 **0.065**、达标-72 h **0.127 mm**；主解积分满 72 h，首次达标单独记录，未用常值填充。末期 $R_{\text{pred}}(72\text{ h}) = 1.1836$ ，附件 2 为 1.198 cm，60-72 h 再降 0.031 mm，末端已近似平台，但仍有残余缓降，与结构上的限定一致。半径预测所用代数关系的残差为：先在 R_{ideal} 下求出 C_s ，再代入修正式后未收敛的残差，其最大值 0.003 mm；末单元温度与重建表面温度之差最大 0.018 °C，对 t_f 的影响远在 0.01 h 以下。收敛序列 50.5450/50.5370/50.5369 h 的相邻差为 28.8 s 与 0.41 s，后者已小于尾段约 15 s 的步宽，故不能用表观收敛阶判断收敛；改用连续事件定位的 t^* 序列为 50.5422/50.5355/50.5362 h，N 由 80 增至 160 时变化 0.0007 h，可视为空间误差的量级，见第七章。中心/表面演化与半径收缩见图 10，截面演化见图 11；结果表见表 11 (result4.xlsx)。

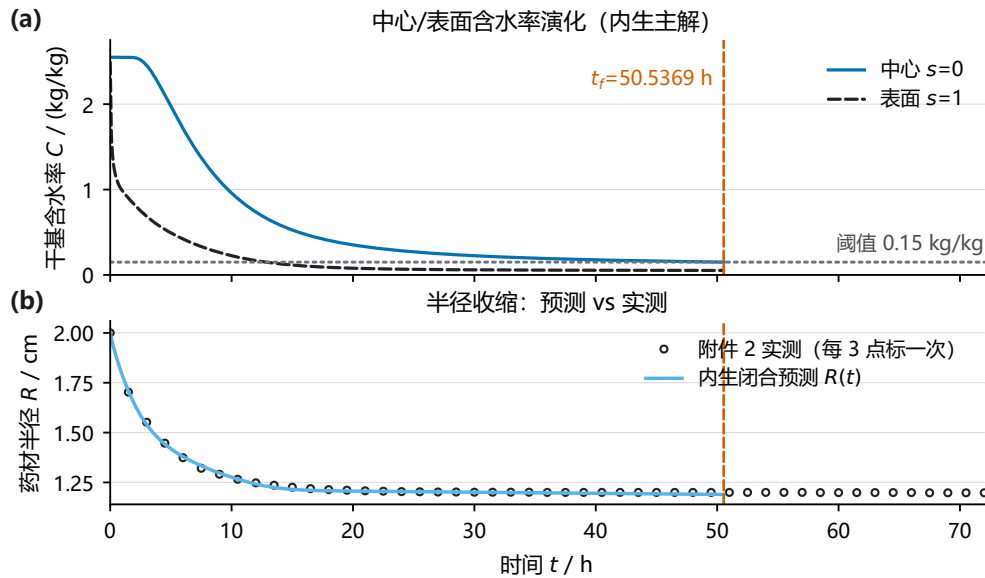


图 10 中心/表面含水率演化与半径收缩：(a) 中心与表面含水率曲线，配 0.15 kg/kg 阈值虚线与 $t_f = 50.5369$ h 达标竖线：表面先到干态、中心最后达标；(b) 内生半径 $R(t)$ 在 50.5 h 进入近平台，60-72 h 仅再降 0.031 mm，模型已积分至 72 h，曲线止于 $t_f = 50.54$ h

与问题 3 的大小关系：

- **附录材料参数组合效应：**附录 4 与附录 3 的 D 之比 $0.175e^{0.15/U}$ 由 $1/5.39$ ($U = 2.55$) 变到 $1/2.10$ ($U = 0.15$)；固定半径下 t_f 从 57.18 h 放大到 129.10 h，即 $\times 2.26$ 、 $+78.57$ h。该对照同时改变了全部热物理参数，因此不能严格归因于单个扩散系数的倍率。
- **收缩效应：** R^2 降 64%，扩散时间尺度缩至 0.36； $2/R$ 换热与传质面积比增 67%，把 129.10 h 拉回 50.54 h，即 -60.9% 。三组对照中，收缩带来的加速幅度最大。

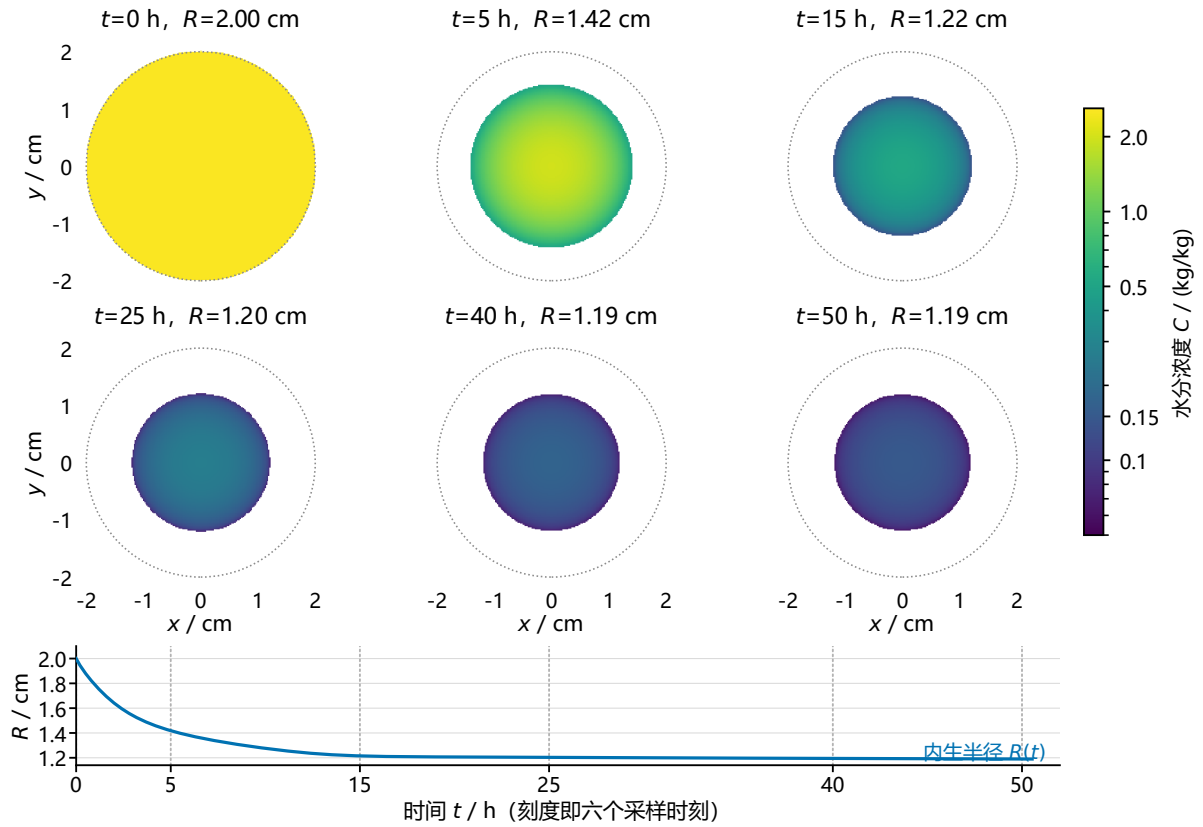


图 11 问题 4 的截面演化。图中给出 6 个代表时刻的圆盘截面热图，色标为水分浓度，取对数色阶以便分辨低含水端的壳核差异，色标上 0.15 处即题目的达标阈值。热图显示干燥自表面向中心推进，外半径同步收缩并进入平台：附件 2 实测的平台半径为 1.198 cm，模型逐步预测的末端半径为 1.19 cm，各面板标题给出该时刻的模型半径。图底的时间轴带给出内生半径 $R(t)$ 与六个采样时刻在曲线上的位置，六个盘的时间间隔是 5/10/10/15/10 h，而半径降幅集中在前两盘： R 在前 5 h 由 2.00 降到 1.42 cm，占 0.81 cm 总降幅的 72%，20 h 后即进入平台，因此在等间隔取样、等宽排版下视觉变化前快后慢；中心为最慢点， $t_f = 50.5369$ h 时中心首次低于 0.15 kg/kg

- **净效应**：材料参数组合效应带来的减速 $\times 2.26$ 被收缩加速反超，问题 4 反而比问题 3 快 11.6%，比固定半径的对照快 +6.64 h。

5.5 力学驱动扩展：收缩的力学机制

本节把半经验关系式推广为“由守恒律与有限应变力学约束、材料本构可标定的内生收缩模型”，作为**扩展研究**；主线为 5.4 节的半经验关系式，它对附件 2 的精度更高；

框架：材料点 X 的现时位置 $r(X, t)$ 为未知量，有限应变非仿射几何 $\lambda_r = r_X$ 、 $\lambda_\theta = r/X$ 、 $\lambda_z = 1$ 、 $J = \lambda_r \lambda_\theta$ ；孔隙非负直接进入方程： $g = J - s_0 - w \geq 0$ 由乘子 ζ 以互补条件活动集实施；毛细储能 $\psi_{\text{cap}} = (J - s_0) \int_S^1 p_c ds$ ，保水本构取幂律 $p_c = p^*(1 - S)/S^\beta$ ， $\bar{p}_c$ 有解析式，与数值积分的偏差不超过 5×10^{-16} ；骨架取对数应变的单松弛黏弹性模型，含内变量 Z ，用后向欧拉作局部消元，每个时间步只需解一次；初始平衡 $e_* = \bar{p}_c(S_0)/K_\infty$ 使初始总应力为零；总应力 $\sigma_i = H_i/J + \bar{p}_c - \zeta$ ，准静态径向平衡以弱式求解，表面取 $\sigma_{rr} = 0$ ，外半径 $R(t)$ 是待求量而非已知输入；与传质以 $U_t = \partial_q(D/r_q^2 U_q)$ 衔接，每步按力学、几何系数、传质的顺序迭代耦合；热算子按 $a_{\text{eff}} T_t = (1/(rr_q)) \partial_q(rk/r_q T_q)$ 的有限体积形式离散。判别实验表明应力传递是全局的，而不是逐点体积平衡：只干燥最外层时，湿内部 J 保持 1.000，而径向应力以 -0.047 全域均匀传递，表皮环向应力为 $+4.188$ ，表面节点内移 0.042 cm。

退化检查（表 3）：孔隙非负全程成立，体积衡等式达到机器精度，热算子与参考解的比较结果一致。在常数材料参数、无失水、阶跃环境下取同一后向欧拉步， $N=16/80/160$ 的最大场差为 0.0084/0.0023/0.0013 $^\circ\text{C}$ ，并随加密单调下降；牛顿步全部收敛，最大平衡残差为 9.5×10^{-11} 。

表 3 力学模型的退化检验（末列为实测判定）

检查	应出现的结果	实测	判定
不干燥 + 初始总应力平衡	尺寸无人为跳变	$\max \Delta r = 0.000$	✓
不计黏弹分支	恢复纯弹性问题	计算结果正常 ($R_{\text{end}} = 1.717$ cm)	✓
$\tau \ll$ 过程尺度	黏弹分支充分松弛	与纯弹性差 5.3×10^{-15} cm	✓
$\tau \gg$ 过程尺度	内变量近冻结	$\max Z / \max E = 4.3 \times 10^{-11}$	✓
指定仿射映射 $r = R\sqrt{q}$	传质系数恢复 $4qD/R^2$	相对误差 4.9×10^{-15}	✓
固定外形持续失水	g 按体积恒等式增长	相对误差 2.2×10^{-16}	✓
全程孔隙非负	任一时刻任一点 $g \geq 0$	$g_{\min} = +0.1446$ （无任何负孔隙）	✓
全局体积核验式	$R^2/R_0^2 = s_0 + \frac{\rho_{d0}}{\rho_w} \bar{U} + \int g dq$	残差 $\leq 2.7 \times 10^{-16}$	✓
热算子退化比对	与参考解同 BE 步一致，随加密下降	0.0084/0.0023/0.0013 $^\circ\text{C}$	✓

标定与复现程度：幂律保水—骨架结构对附件 2 标定， β 的下界为 0.02，4 个起点都收敛到同一解： $p^* = 1.0 \times 10^4$ 触到上界，此时轨迹由无量纲形状 $\bar{p}_c(S)/\bar{p}_c(S_0)$ 决定， p^* 处于渐近不变区，不可辨识属于预期； $\beta = 0.1059$ ； $\Pi_\tau = 1.0 \times 10^{-3}$ 触到下界。完整积分满 72 h 的 **RMSE** = 0.522 mm，其中急缩段 1.193、缓缩段 0.875、平台段 0.163 mm；分段为 0–达标 0.604、达标–72 h 0.212 mm。末期平台 1.1747，附件 2 为 1.198

cm, 60–72 h 仅再降 0.04 mm, 末端已真正进入平台; $t_f = 51.1667$ h, 较半经验主线 50.5369 h 偏差 +1.25%。奇偶折半留出为 0.587/0.591 mm, 说明没有过拟合; 分块自助法给出 β 的 95% 置信区间 [0.095, 0.91]; 孔隙非负全程成立, $g_{\min} = 0.1446$ 。平台来自吸力饱和平衡: 标定参数下表面 Deborah 数 $De \approx 1.0 \times 10^{-3} \ll 1$, 说明平台并非黏弹冻结所致, 见图 12。

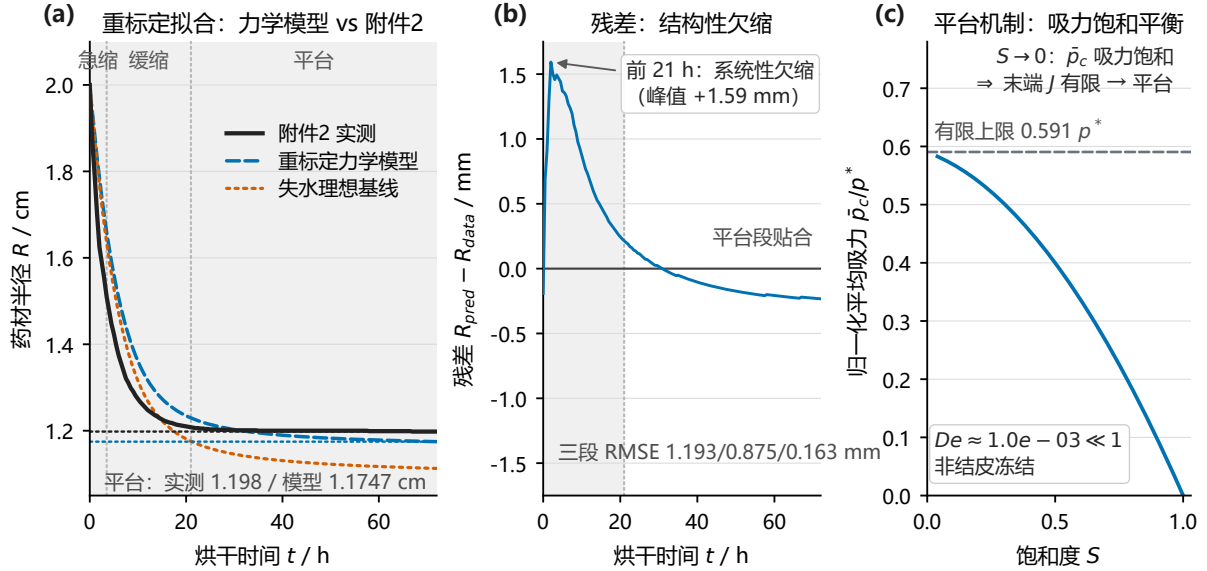


图 12 力学驱动内生收缩模型: 修正后的数值格式与幂律保水重标定 ($\beta = 0.106$) 对附件 2 的拟合结果为 RMSE 0.522 mm, 残差在三段上分别为急缩 1.193、缓缩 0.875、平台 0.163 mm, $t_f = 51.1667$ h; 末端 60–72 h 仅再降 0.04 mm, 已真正进入平台, 平台机制为吸力饱和平衡, 此时 $De \approx 10^{-3} \ll 1$ 。(c) 面板给出该平台机制: 归一化平均吸力 $\bar{p}_c/p^*$ 随饱和度 S 下降而单调上升, 并在 $S \rightarrow 0$ 时趋于有限上限 $p^*/[(1-\beta)(2-\beta)]$, 末端收缩进入平台即来自这一吸力饱和平衡; 气相体积份额全程非负, $g_{\min} = 0.1446$, 见表 3

由模型结构决定的上限与参数缺失: 在所选幂律保水—简化储能结构内, $\bar{p}_c$ 的有限上限为 $p^*/[(1-\beta)(2-\beta)]$, $\beta = 0.15$ 时仅为 $0.636 p^*$; 且吸力的作用范围是局部的, 只作用于干燥表皮, 湿内部则以其体积刚度抵抗由外层传递而来的压缩。由此带来两个后果: 第一, 附件 2 前 21 h 的早期全局塌陷无法仅用吸力驱动的局部收缩解释; 第二, 末端平衡 $J_{\text{end}}(\beta)$ 由 $J \cdot g(S(J); \beta) = g(S_0; \beta)$ 决定, $\beta = 0.02/0.05/0.10/0.15/0.85$ 时 $J_{\text{end}} = 0.368/0.356/0.336/0.319/0.218$ 。若要达到附件 2 的平台 $J = 0.36$, 须取 $\beta \lesssim 0.05$, 此时缓缩段过慢; 若要使中段足够快, 须取 $\beta \gtrsim 0.10$, 此时末端必然过缩。因此在这一结构内, 单一 β 无法同时给出快速的急缩与停滞的平台; 标定折中取 $\beta = 0.106$, 末端 $J = 0.3343$, 附件 2 的对应值为 0.36。这一上限由所选本构与储能结构决定, 其他保水曲线或体积驱动机制有可能达到更高的精度, 这一问题留作后续工作。**参数缺失**方面, 真实保水曲线 $p_c(S, T)$ 是其中影响最大的一项, 此外还有膨压类体积驱动、干态模量增长与结皮硬化、骨架模量与松弛谱、初始湿质量与真实密度; 体积核算只给出 $\rho_{d0} \leq 256$ kg/m³ 的上限。

结论: 当前的精度仍不如半经验主线, 其指标为 0.047/0.088 mm。在真实保水曲线与力学实测数据到位之前, 本文的力学模型还不能复现附件 2 的实测过程; 力学驱动的推广是后续可继续沿用的方向, 届时半经验收缩关系式可作为其低维验证基准。

六、 模型检验

6.1 无量纲判据 (Le 、 Bi_m 、 Bi_h)

沿矩形扫描域 $C \in [0.15, 2.55]$ 、 $T \in \{301.15, 323.315\}$ K、 $R = R_0$ ，逐点按附录公式计算 $\alpha = k/(\rho c_p)$ ，扫描区间即表 4 所列区间。干燥末段表面 $U < 0.15$ 的局部状态超出该扫描域，扫描未包含。表中回代偏差指采用该简化后 t_f 与全耦合解 57.2009 h 的实测差。

表 4 无量纲判据表：定义、扫描区间、阈值语义、简化决策与回代偏差

判据	定义	附录 3 区间	附录 4 区间	简化决策	回代偏差
Le	α/D	10.6860– 640.6171	31.2068– 556.1413	$Le \gg 1$: 热场准稳态 $T \approx T_a$	0.303 h
Bi_m	$h_m R/D$	1.1805–47.7480	6.3606– 100.3742	$Bi_m \rightarrow 0$: 集总 (内扩散无阻力)	–46.03 h
Bi_h	hR/k	1.0353–1.9263	1.8964–3.4226	$Bi_h \rightarrow 0$: 忽略表面热滞后	0.075 h

回代检验结果如下，每条给出采用该简化后 t_f 的偏移：

- $Le \gg 1$ 说明热场先趋于准稳态：温度场取 $T_a(t)$ 只求湿分， t_f 偏差 0.303 h (0.5%)；本文主模型仍保留双向耦合，此简化只作回代检验，且只在无潜热基准内评价，不适用于有蒸发冷却的情形 (5.3 节潜热情景表面温差达 29.6 °C)。
- Bi_m 区间 $\gtrsim O(1)$ 表明内部扩散主导：集总模型给出 $t_f = 11.17$ h，比全耦合解快 46.03 h (–80%)；可见把含水率作“集总或平均”处理都无法复现本题的 t_f ，模型必须保留空间分布。
- $Bi_h \sim O(1)$ 下达标时间对该简化不敏感：令 $h \rightarrow \infty$ 实测偏差 0.075 h (0.13%)，本文未采用该简化； $Bi_h \sim 1$ 本身并不表示空间温差小，同样不适用于有蒸发冷却的情形。

6.2 参考时标 (常系数与瞬时首模近似)

取 D 的路径点态最大 $D_{\max}$ (D 对 C, T 均单调增)，构造常系数 Robin 参考问题，其首模形式为

$$t_{\text{ref}} = \frac{R^2}{D_{\max} \lambda_1^2} \ln \frac{A_1 (C_0 - C_e)}{C_{\text{th}} - C_e}, \quad (6.1)$$

其中 λ_1 为 $\lambda J_1(\lambda) = Bi_m J_0(\lambda)$ 第一根 (圆柱 Robin 特征值问题，标准谱理论见 [2])， A_1 为中心响应系数。问题 3 (式 (6.1))： $D_{\max} = 1.359 \times 10^{-8}$ m²/s、 $Bi_m = 1.177$ ，得 $t_{\text{ref}} = 15.70$ h (300 模精确级数的达标时刻同为 15.7021 h，参考问题自身的首模近似误差可忽略)；膜界 11.17 h (忽略内部扩散阻力，系统性偏弱，仅作次级参照)。问题 4：把瞬时首模时标沿真实 $R(s)$ 轨迹积分，得参考值 19.02 h ($R_{\min}$ 常值版 15.95 h、固定 R_0 版 37.61 h)。两个主值 (57.18 h、50.54 h) 均远大于各自参考时标。

参考解与真实问题之间的差异： (i) 参考解把环境平衡势固定为末值 $C_e = 0.04986$ ，但真实边界初期低至 0.01963——同一 $D_{\max}$ 首模参考在固定初值边界下为 **14.5491** h，用于排除简化的时标随边界取法而变，比较的先后顺序并不固定； (ii) 对散度形式的变系数扩散方程， $D \leq D_{\max}$ 本身不构成逐点解比较； (iii) 收缩版积分瞬时第一特征值，而

$Bi(s)$ 随时间变化引起模态耦合，瞬时特征值积分不是精确的变边界谱解。若达标时间远低于参考时标，就应怀疑该结果是否可信。参考时标只能用于判断量级，据此可以排除偏差较大的简化。

6.3 检验结果

解析谱解与守恒有限体积数值解来自两种不同的数学结构，两者的逐点最大偏差为 $5.9 \times 10^{-7} ^\circ\text{C}$ (V1)。各项检验汇总于表 5，收敛与守恒的可视化见图 14。

表 5 验证汇总表 (V1–V6)：手段、数学结构、实测数字、能得到的结论与尚未检验的误差源

检验 (手段)	实测数字	能得到的结论 / 尚未检验的误差源
V1 问题 1 热场解析谱解 (特征展开 + Duhamel 卷积) 与有限体积数值解	单元中心逐点最大偏差 $5.9 \times 10^{-7} ^\circ\text{C}$ (N=320) ; 收敛阶 2.00/1.99/1.99	数值解求的正是该线性模型, 网格加密后按二阶收敛, 场值不是网格假象; 材料参数的取值不在本文检验范围内
V2 后验尾段诊断 (等温线性化 Bessel 首模结构对照)	尾段 $\ln(U_{\max} - C_e)$ 线性 $R^2 = 0.9972$; 衰减率 $\gamma = 3.55 \times 10^{-6}/\text{s}$ 对应等效 $D = 2.54 \times 10^{-10}$ ($C \approx 0.108$, 在尾段剖面区间内)	尾段的衰减确实由扩散首模控制 (等效 D_{eff} 由本解反演, 属构造一致、不是独立预测); 尚未检验的边界与几何离散误差
V3 参考时标判据检验 (常系数/瞬时首模近似, 见 6.2 节限定)	$t_f = 57.18 \text{ h} > 15.70 \text{ h}$; $t_f = 50.54 \text{ h} > 19.02 \text{ h}$	达标时间与参数给出的参考量级一致 (远大于参考时标), 据此可以排除偏差较大的简化; 尚未检验的变系数扩散与变动边界谱差异
V4 守恒等式 (总水量、冻结热容的显热离散平衡, 逐项核对)	水量窗口相对残差 $\leq 5.5 \times 10^{-11}$; 显热离散平衡全局吞吐残差 $\leq 1.1 \times 10^{-11}$	逐项核对说明边界输入的水量与内部贮存的变化一致, 水分与显热不会凭空增减, 达标时间不是数值泄漏造成的; 状态焓未独立定义, 真实能量守恒与潜热分析不在本文检验范围内
V5 退化测试 (移动域 $R(t) \equiv R_0$; D 冻结)	移动域退化为固定域: $\Delta t_f = 0.00 \text{ s}$ 、场差 0; D 冻结退回 Bessel 首模: 偏差 3.1×10^{-9} 、首模商 0.99996	移动域在 $R(t) \equiv R_0$ 时还原为固定域, 说明两个模型自洽; 常系数湿场退回 Bessel 首模, 是独立对照。模型的基本假设本身不在本文检验范围内
V6 网格/时间 Richardson 收敛	场级空间阶 1.90–2.45; t_f 序列 57.249/57.201/57.180 h, 外推偏差 0.016 h; 时间容差收紧十倍实测 $\Delta t_f = 0.0014 \text{ h}$	报告的 t_f 是连续方程的解, 不是网格假象; 单独使用不足以支撑可靠性

七、结果分析与敏感性

结果表格按 result1–4.xlsx 汇总, 见表 6–表 11; 达标时刻的三条曲线分别取问题 3 主解、问题 4 主解与附录 4 固定半径对照, 见图 15。

7.1 敏感性分析

每组都实测 Δt_f , 见表 12 与图 16。边界汇取值 (C_e) 是本题唯一能改变结论量级的因素: 改用解吸等温线 $C_e = 0.1366$ 后 t_f 增大 22.6 h (39.6%)。主结果一律采用本文

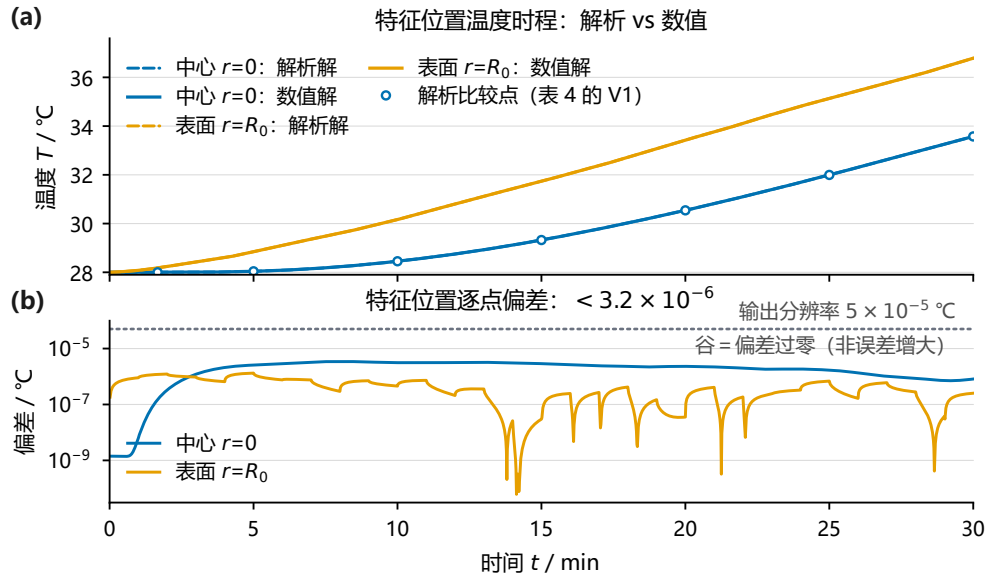


图 13 问题 1 解析比较 (V1): (a) 中心与表面两个特征位置的温度数值解与解析谱解时程; (b) 两个特征位置的逐点偏差时程, 量级 $10^{-6} ^\circ\text{C}$ 、全程低于 $3.2 \times 10^{-6} ^\circ\text{C}$, 虚线为四位小数输出的分辨率上限; 径向网格加密的二阶收敛见表 5 与图 14(a); 全域单元中心在 $N=320$ 的最大偏差为 $5.9 \times 10^{-7} ^\circ\text{C}$

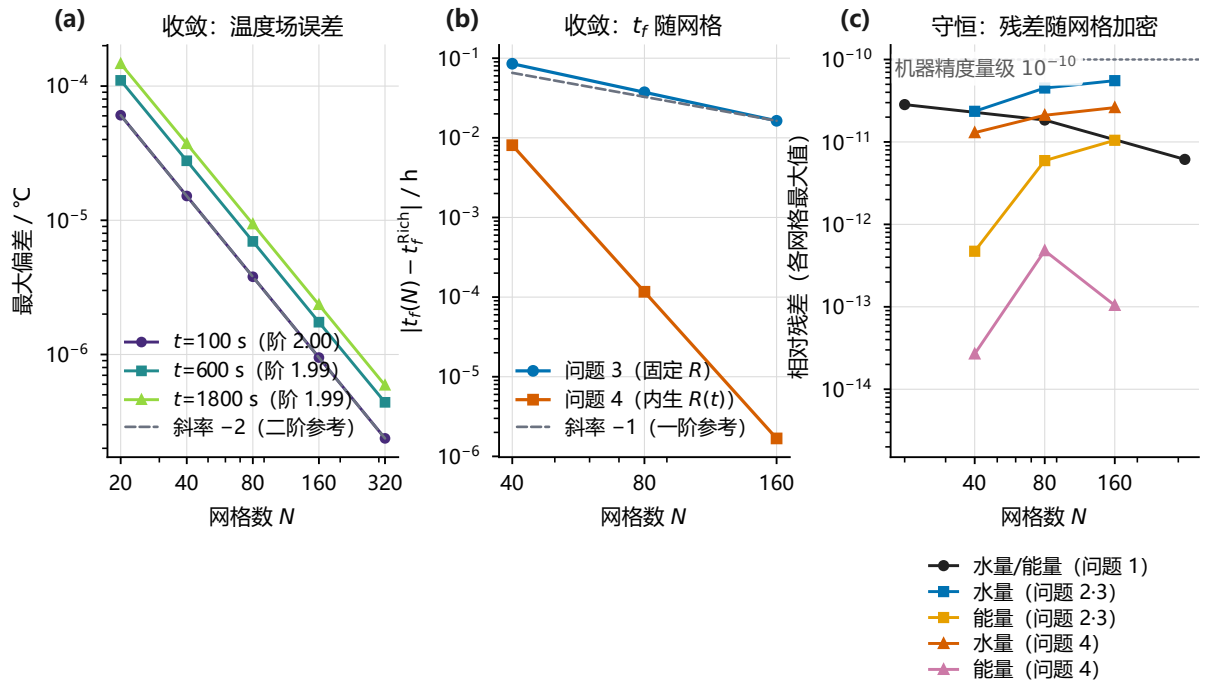


图 14 收敛与守恒检验: (a) 温度场误差随网格加密按二阶下降 (V1、V6); (b) t_f 随网格加密的收敛序列; (c) 残差随网格加密: 水量与显热离散平衡残差维持在 10^{-11} 量级附近 (V4)

表 6 问题 1（预热平衡阶段）药材温度（℃）。数字由 result 工作簿同一数据源抽取，保留四位小数

时间/s	0	0.5	1	1.5	2
100s	28.0001	28.0003	28.0040	28.0327	28.1801
300s	28.0408	28.0635	28.1514	28.3681	28.8489
600s	28.4533	28.5360	28.8039	29.3159	30.1652
900s	29.3243	29.4583	29.8755	30.6161	31.7304
1200s	30.5427	30.7098	31.2223	32.1126	33.4276
1500s	31.9957	32.1867	32.7660	33.7463	35.1203
1800s	33.5753	33.7720	34.3642	35.3621	36.7856

表 7 问题 1（预热平衡阶段）药材水分浓度（kg/kg）。数字由 result 工作簿同一数据源抽取，保留四位小数

时间/s	0	0.5	1	1.5	2
100s	2.5500	2.5500	2.5500	2.5500	2.2470
300s	2.5500	2.5500	2.5500	2.5492	2.0508
600s	2.5500	2.5500	2.5500	2.5353	1.8770
900s	2.5500	2.5500	2.5497	2.5045	1.7547
1200s	2.5500	2.5500	2.5482	2.4646	1.6586
1500s	2.5500	2.5499	2.5445	2.4206	1.5787
1800s	2.5500	2.5497	2.5383	2.3755	1.5102

表 8 问题 2（全烘干过程）药材温度（℃）。数字由 result 工作簿同一数据源抽取，保留四位小数

时间/h	0	0.5	1	1.5	2
0.5h	32.1892	32.3821	32.9659	33.9608	35.4130
1.0h	40.3816	40.5536	41.0605	41.8782	42.9976
1.5h	45.8468	45.9348	46.1930	46.6051	47.1401
2.0h	48.4502	48.4881	48.5984	48.7735	49.0033
2.5h	49.4670	49.4792	49.5136	49.5653	49.6609
3.0h	49.8495	49.8553	49.8746	49.9101	49.9664

表 9 问题 2（全烘干过程）药材水分浓度（kg/kg）。数字由 result 工作簿同一数据源抽取，保留四位小数

时间/h	0	0.5	1	1.5	2
0.5h	2.5499	2.5489	2.5256	2.3257	1.6486
1.0h	2.5258	2.4948	2.3578	2.0230	1.4711
1.5h	2.3861	2.3256	2.1344	1.8020	1.3476
2.0h	2.1709	2.1084	1.9236	1.6259	1.2311
2.5h	1.9566	1.9006	1.7360	1.4720	1.1166
3.0h	1.7662	1.7165	1.5702	1.3333	1.0081

表 10 问题 3 药材烘干过程的水分浓度（kg/kg）。结束行时刻为未舍入判定值 57.1799 h；表中 0.1500 为舍入显示，判定位用未舍入值

时间/h	0	0.5	1	1.5	2
6h	1.0156	0.9868	0.9006	0.7546	0.5335
12h	0.4548	0.4418	0.4019	0.3289	0.1639
18h	0.2979	0.2906	0.2679	0.2247	0.0842
24h	0.2371	0.2321	0.2161	0.1851	0.0663
30h	0.2053	0.2013	0.1887	0.1637	0.0597
36h	0.1854	0.1821	0.1714	0.1501	0.0565
42h	0.1717	0.1687	0.1594	0.1404	0.0547
48h	0.1615	0.1588	0.1504	0.1332	0.0536
54h	0.1535	0.1511	0.1434	0.1275	0.0528
烘干结束时间	0.1500	0.1477	0.1402	0.1249	0.0525

表 11 问题 4 药材烘干过程的水分浓度（kg/kg）。“—”表示该时刻该固定位置已在药材表面之外（半径收缩，留空不外推）。结束行时刻为未舍入判定值 50.5369 h（内生几何主解）

时间/h	0	0.5	1	1.5	药材表面
6h	1.7132	1.5336	1.0247	—	0.4202
12h	0.7362	0.6525	0.4044	—	0.1676
18h	0.4044	0.3645	0.2359	—	0.0886
24h	0.2825	0.2588	0.1777	—	0.0669
30h	0.2247	0.2079	0.1483	—	0.0592
36h	0.1915	0.1784	0.1305	—	0.0558
42h	0.1701	0.1593	0.1186	—	0.0539
48h	0.1551	0.1457	0.1100	—	0.0528
烘干结束时间	0.1500	0.1411	0.1071	—	0.0525

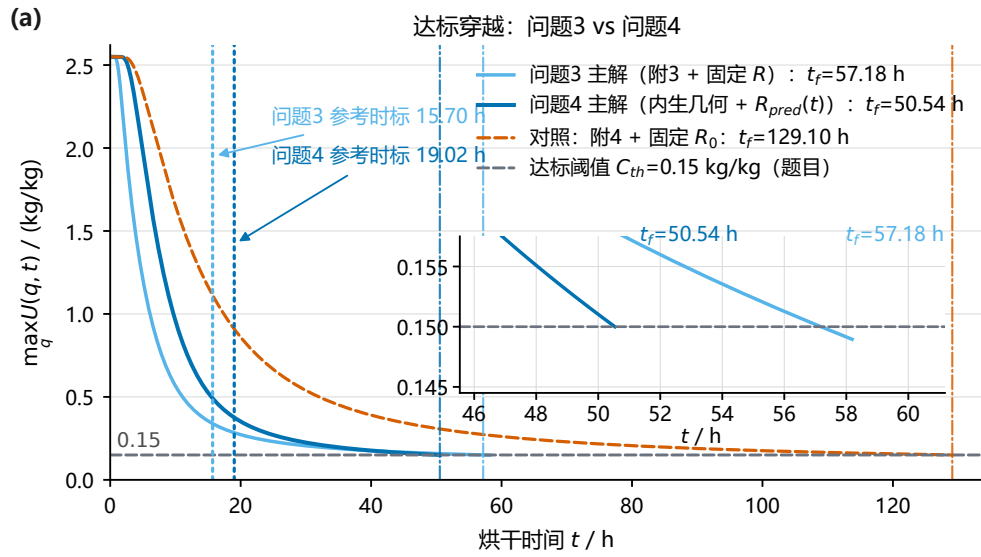


图 15 达标时刻： $\max_q U(q, t)$ 全程曲线与 0.15 阈值线；问题 3 主解 $t_f = 57.1799$ h、问题 4 主解 $t_f = 50.5369$ h（内生几何）与附录 4 固定半径对照 129.1047 h 三条曲线对比，横轴标出参考时标 15.70 h 与 19.02 h（常系数与瞬时首模近似）

设定的取法，超出题设的修正只进入敏感性分析并明确标注，其偏移不计入误差分量汇总（7.2 节）。半径插值两档实测仅差 0.0022 h（附件 2 平滑，两插值几乎重合），说明这一取法对 t_f 的影响很小。

表 12 敏感性汇总表（问题 3 取法、N=80 对照取法；问题 4 拆分沿用主取法）

组别	t_f (h)	Δt_f (h)
$C_e = 0.04986$ （本文设定，主）	57.2009	—
$C_e = 0.1366$ （山药解吸支，超出题设的修正）	79.8270	+22.6261（+39.6%）
环境外推：末值保持（主）	57.2009	—
环境外推：线性外推（末 1 h 斜率）	58.8203	+1.6194
环境外推：末 1 h 均值	57.5034	+0.3025
半径插值：PCHIP（主，问题 4，附件 2 实测几何对照）	50.8295	—
半径插值：线性（问题 4）	50.8317	+0.0022
问题 4 拆分：附录 4 固定 R	129.1047	+78.57（对内生主解）
问题 4 拆分：附录 4 + 内生 $R(t)$ （主解）	50.5369	—
问题 4 拆分：问题 3	57.1799	+6.64（对内生主解）

7.2 误差与不确定度

模型结构差异：潜热情景的 +5.3889 h 与边界汇取值 $C_e = 0.1366$ 的 +22.6261 h 单列，不计入上表的合成。前者是 5.3 节在情景假设集合下无潜热基准的 t_f 偏差量级，后者是超出题设的修正，只作敏感性分析。

尚未纳入误差分量汇总的因素：边界条件的全部可能形式（吸附与水活度关系缺少数据）、内生收缩标定参数（ k_p 的 95% CI 相对半宽 24%）、内部非仿射变形（5.5 节的

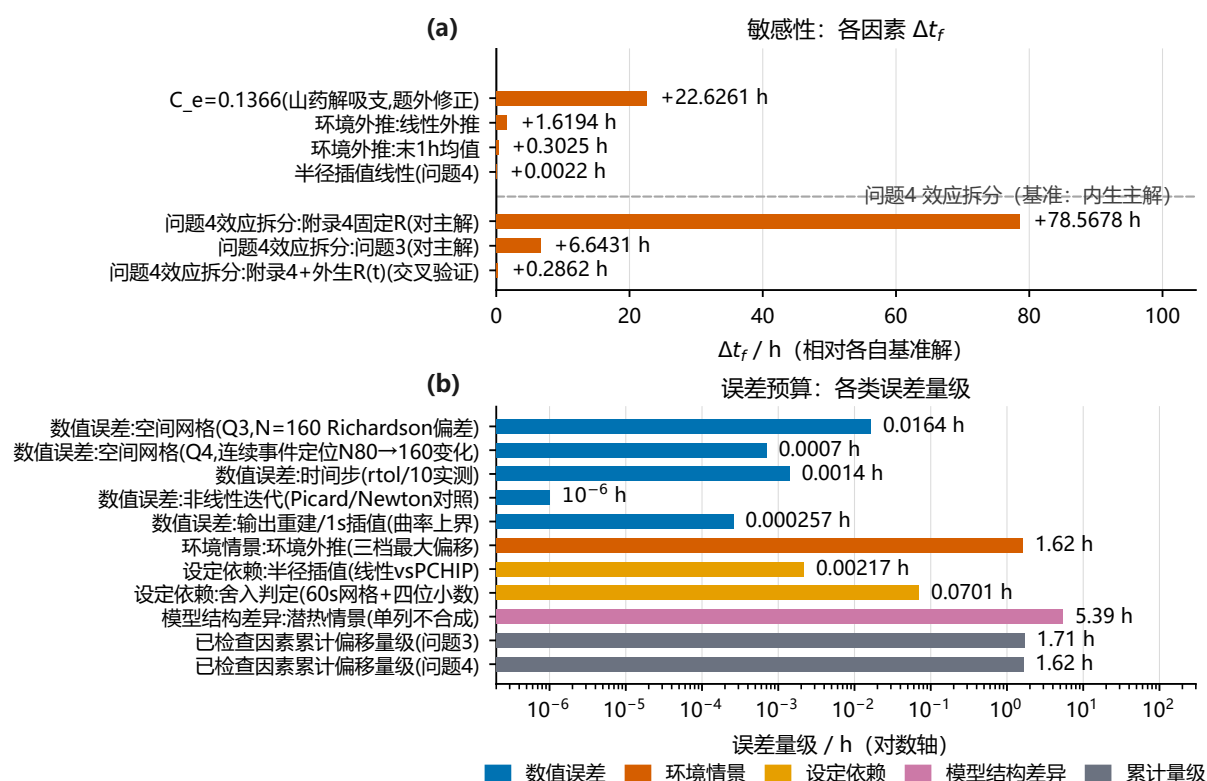


图 16 敏感性条形图: 各因素的 Δt_f 画成横向条形, 其中边界汇取值 $C_e = 0.1366$ (超出题设的修正) 的偏移 +22.63 h 最大, 环境外推 +1.62 h 次之, 半径插值仅 +0.002 h

表 13 误差分量汇总表: 按数值误差、环境情景与取法敏感性三类分别报告

类别	误差源	问题 3 (h)	问题 4 (h)
数值误差	空间网格 (Q3: N=160 Richardson 偏差; Q4: 连续事件定位 N=80→160 变化)	0.0164	0.0007
数值误差	时间步 (时间容差收紧十倍)	0.0014	0.0014
数值误差	非线性迭代 (Picard/Newton 对照)	1×10^{-6}	1×10^{-6}
数值误差	输出重建 / 1 s 插值 (曲率上界实测)	2.6×10^{-4}	2.6×10^{-4}
环境情景	环境外推 (三档最大偏移)	1.6194	1.6194
对取法的敏感性	半径插值 (线性与 PCHIP, 仅问题 4)	—	0.0022
对取法的敏感性	舍入判定 (60 s 网格 + 四位小数, 另一种数据使用方式)	0.0701	—
已检验因素累计偏移量级 (三角和)		≈ 1.71	≈ 1.62

模型结构分析)。故问题 3 的达标时间报告为 $t_f = 57.18$ h, 在所列模型假设与情景集合内, 已检验因素的累计偏移量级约 1.71 h。

八、模型评价与改进

8.1 优点

1. **统一性与自洽:** 四问同为这一套热湿耦合方程、仅取法不同 (表 2), 问题 1 是问题 2 的解耦特例、问题 4 是移动域推广, 四问自然衔接; 质量坐标下收缩几何方程同形、无伪对流项 (附录 A 双守恒推导)。
2. **这些检验相互独立:** 解析谱解与守恒有限体积是两种不同的数学结构, V1 给出逐点最大偏差 $5.9 \times 10^{-7} ^\circ\text{C}$ 与二阶收敛; 此外还有退化测试、守恒恒等式 (10^{-11} 量级) 与参考时标判据检验, 分别给出可以得到的结论与尚未检验的误差源。
3. **收缩由模型自身预测:** 问题 4 不读附件 2 而由半经验关系式逐步预测 $R(t)$ ($t_f = 50.5369$ h), 附件 2 降为验证数据后仍全程吻合 (0.088 mm); 参数标定与验证用的是同一批数据, 标定指标由实测几何单向驱动得到, 与耦合主解的验证指标是两个不同的量。
4. **不确定度的来源逐项列出:** 无量纲判据的回代检验给出每条简化的 t_f 偏移, 敏感性分析给出全部 Δt_f , 误差按三类分别报告, 潜热与边界条件两个情景各自单列。

8.2 缺点

1. **湿边界取本文设定的等效边界条件:** C_{env} 直接作 Robin 驱动势, 缺少吸附等温线的支撑, 是本题影响最大的一处假设 (超出题设的修正 +22.6261 h)。
2. **无潜热基准:** 潜热与显热同量级, 基准的 t_f 偏差量级约 5.4 h (表面相变汇情景), 更真实的边界条件需要题目未给的吸附与解吸数据。
3. **收缩关系式是半经验式:** g_c, g_p 为经验形状函数, 力学驱动扩展虽更严谨, 但在所选幂律保水—简化储能结构内存在精度上限 (末端 $J = 0.3343$, 附件 2 的对应值为 0.36), 缺少保水曲线与力学实测数据。
4. **几何与外推的理想化:** 模型取一维径向, 端面效应未给出定量界; 环境条件在 4 h 之后保持末值; 内部取等比收缩, 内部收缩不可识别。

8.3 改进

1. 针对缺点 1/2: 补测材料吸附/解吸等温线 (或水活度关系) 与相变焓数据, 把湿边界与能量边界提升为介质配套的边界条件;
2. 针对缺点 3: 补测真实保水曲线 $p_c(S, T)$ 与骨架模量/松弛谱, 沿力学驱动路线 (5.5 节框架已经建立) 重新标定, 半经验关系式作为低维验证基准;
3. 针对缺点 4: 补有限长圆柱对照量化端面效应、以更长时程环境数据替换外推、在力学框架内放松仿射假设 (非仿射已在扩展模型中实现);
4. 固定全部比较条件后构造可证明的包络或积分不等式, 把参考时标提升为已证明的严格下界。

参考文献

- [1] Luikov A V. *Heat and Mass Transfer in Capillary-Porous Bodies*. Pergamon Press, Oxford, 1966.
- [2] Crank J. *The Mathematics of Diffusion*. 2nd ed., Oxford University Press, 1975.
- [3] 国家药典委员会. 中华人民共和国药典（一部）. 北京: 中国医药科技出版社, 2020. (药材水分限度的通行标准约 13%)
- [4] Saravacos G D. Mass transfer properties of foods. In: *Engineering Properties of Foods* (M. A. Rao, S. S. Rizvi, eds.), Marcel Dekker, New York, 1986.

附录 A： 移动域方程的推导要点

本文的出发点同时包含干物质守恒与水分守恒。设 ρ_d 为单位总体积中的干物质质量、 v 为骨架径向速度、 j 为相对骨架的水通量 ($j = -\rho_d D \partial_r U$):

$$\partial_t \rho_d + \frac{1}{r} \partial_r (r \rho_d v) = 0, \quad \partial_t (\rho_d U) + \frac{1}{r} \partial_r [r (\rho_d U v + j)] = 0. \quad (\text{A.1})$$

第二式展开后用第一式相减，得物质点形式

$$\rho_d (U_t + v U_r) = \frac{1}{r} \partial_r (r \rho_d D U_r). \quad (\text{A.2})$$

引入材料坐标 q (干物质标签)，物理坐标 $r = \psi(q, t)$ ，骨架速度 $v = \psi_t|_q$ 。在干物质密度空间均匀与仿射收缩假设下，取 $\psi = R(t)\sqrt{q}$ ，即长度不变、内部等比。附件 2 只识别外半径，内部收缩不可识别，等比是最简取法。此时 $\rho_d \psi \psi_q$ 与 t 无关，式 (A.2) 化为材料坐标形式 $U_t|_q = \partial_q (4qD/R(t)^2 U_q)$ ——与固定域同形、无伪对流项。在移动表面 $q = 1$ 处，对表面薄层做水分衡算，几何运动项与通量跳条件中的对应项恒等相消，Robin 边界 $-D \partial_r U = h_m (U_s - C_{\text{env}})$ 形式不变。温度方程同理。两点说明：(i) 若由 $U_t|_r = (1/r) \partial_r (r D U_r)$ 出发、假定迁移项相消，所得写法不成立：该式右侧没有可相消项；上面的材料坐标形式由双守恒方程推出。(ii) 固定半径退化测试 (V5a) **无法检验** 上述推导：此时骨架速度 $v \equiv 0$ ，移动域与固定域方程逐字相同，因此该测试只说明两者在数值上一致，不能证明移动域方程本身正确。

附录 B: 参数取值表与收缩参数表

表 14 参数取值表

参数	取值/公式	取值方式	适用
$D(C)$	$7 \times 10^{-9} e^{-0.89/C}$	局部 C	问题 1
$D(C, T)$	$2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}$	局部 C 、局部 T (开尔文)	问题 2/3
$D(C, T)$	$4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T}$	局部 C 、局部 T (开尔文)	问题 4
h	$25 \text{ W}/(\text{m}^2 \cdot \text{K})$	常数	问题 1–4
h_m	$8 \times 10^{-7} \text{ m/s}$	常数	问题 1–4
$T_a(t), C_{\text{env}}(t)$	附件 1 (0–14400 s, 60 s 步)	分段线性插值	问题 1–4
$R(t)$	附件 2 (0–259200 s, 1800 s 步)	PCHIP 保形插值	问题 4
几何	$L_0 = 0.25 \text{ m}$, $R_0 = 0.02 \text{ m}$	一维径向 ($L_0/R_0 = 12.5$)	问题 1–4
初值	$U_0 = 2.55 \text{ kg/kg}$, $\Theta_0 = 301.15 \text{ K}$	全域均匀	问题 1–4
本文设定的假设			
湿边界处理	C_{env} 直接作 Robin 驱动势	等效边界条件 (第三章假设 2)	问题 1–4
环境外推	$t > 14400 \text{ s}$ 末值保持	本文设定 (敏感性见 7.1 节)	问题 2–4
h, h_m 延用	附录 2 值延用到问题 2–4	本文设定的参数延用假设	问题 2–4
半径延拓	$t > 259200 \text{ s}$ 保持 1.198 cm	本文设定	问题 4

表 15 内生收缩模型参数表与物理合理性

参数	数值	来源	物理合理性核对
ε_c (塌陷峰值应变)	0.1108	拟合附件 2	11.1% 在毛细压实区间 [1.4%, 29%] 内
a, b (塌陷形状指数)	1.668, 1.220	拟合附件 2	峰值相位 $\bar{U} \approx 1.47$ (约 4 h) 与急缩段一致
ε_p (停滞幅度)	0.0709	拟合附件 2	对应末期孔隙率 30.9% $\approx$ 附件 2 隐含 31.0%
k_p (停滞开启陡峭度)	3.787	拟合附件 2	开启相位由 C_s 穿越 C_{glass} 物理锚定
C_{glass} (玻璃化含水率)	0.21	Gordon–Taylor 结构典型参数情景值, 未拟合	估计区间 0.19–0.23, 列入参数不确定清单
ρ_{sk} (骨架密度, kg/m^3)	1468	生物物质骨架密度典型范围值, 未拟合	结果对其不敏感

表 16 量纲检查表

符号	单位	核对要点
U, C	kg/kg (干基)	边界驱动势 C_{env} 同基准 (本文设定)
D	m^2/s	$4qD/R^2$ 为 $1/\text{s}$, 与 U_t 一致
T	K (不是 $^{\circ}\text{C}$)	$e^{-3850/T}$ 的 3850 单位为 K; 读入与全程统一取开尔文并作单位校验
h, h_m	$\text{W}/(\text{m}^2 \cdot \text{K})$ 、 m/s	$-k\partial_r T = h(T_s - T_a)$ 两侧 W/m^2 ; $h_m \Delta U$ 为 m/s
a_{eff}	$\text{J}/(\text{m}^3 \cdot \text{K})$	ρc_p 逐点取值, ρ 不进湿分方程
q	—	$q = (X/R_0)^2 \in [0, 1]$, 干物质标签
$\varepsilon_c, \varepsilon_p$	— (应变)	式 (5.4) 的乘子修正

附录 C: 内生收缩模型的标定细节

内生收缩模型 (式 (5.4)) 的 5 个参数 $\varepsilon_c, a, b, \varepsilon_p, k_p$ 由 scipy 的 `least_squares` 在附件 2 的 145 个采样点上对 $R_{\text{pred}} - R_{\text{data}}$ 做带物理边界约束的最小二乘估计。 $C_{\text{glass}} = 0.21$ 取 Gordon-Taylor 结构典型参数情景值, $\rho_{sk} = 1468 \text{ kg}/\text{m}^3$ 取生物质骨架密度典型范围值, 两者不参与拟合, 均列入参数不确定清单。过拟合检验用奇偶折半留出法: 两折上训练与留出 RMSE 均在 $0.045\text{--}0.050 \text{ mm}$, 且两折参数互差 $< 10\%$ 。该检验只检验了同一光滑曲线上的插值, 因为湿度场由附件 2 实测几何生成, 参数标定与验证用的是同一批数据, 所以只能说明插值稳健, 不构成参数被独立识别的证明; 块 bootstrap (残差自相关块长 24、 $B = 200$) 95% 置信区间的相对半宽 ε_c 4.7%、 a 11.9%、 b 11.0%、 ε_p 3.8%、 k_p 23.9%, 分布有偏斜或多峰迹象; 变初值 $\times$ 变损失函数的最大参数偏差 $\leq 0.98\%$; 各机制在其主导区段单独拟合与全曲线拟合的幅度参数差 $< 2\%$, 独立识别需完整耦合标定。

参数的第二路线检验: $\varepsilon_c = 11.1\%$ 落在毛细压 $2\gamma/r_{\text{pore}}$ 对基体模量的理论应变区间 $[1.4\%, 28.8\%]$; ε_p 对应的末期孔隙率 30.9% 与由“平台半径和质量守恒”独立推出的 31.0% 一致; $C_{\text{glass}} = 0.21$ 与“附件 2 冻结起点约 21 h $\leftrightarrow$ 模型 $\bar{U}(21 \text{ h}) = 0.234$ ”交叉吻合; $\rho_{sk} = 1468$ 在生物质骨架密度典型范围内且结果对其不敏感。第二路线检验的结果是: ε_c 、 a 、 b 、 ε_p 、 C_{glass} 、 ρ_{sk} 均获得支持; k_p **存疑**: 它只有曲线拟合一条路, 没有独立的物理观测量, 置信区间也最宽, 达 24%; 但 k_p 只影响过渡带形状, 不触及达标时间与平台半径的结论。力学扩展模型的标定 (4 起点同一解、留出、bootstrap、末端平衡分析) 见 5.5 节正文与数据文件 数据/mechRefit.csv。

附录 D: 复现说明与代码结构

一条命令 `python run_all.py` 依次执行下列脚本; Windows 下需先设置环境变量 `PYTHONIOENCODING=utf-8`。全部脚本只用到 `numpy/scipy/pandas/openpyxl`。

- `run_q1.py`: 问题 1 求解 + V1 解析比较 + 守恒与退化检验 + `result1.xlsx`;
- `run_q23.py`: 问题 2/3 耦合求解 + t_f + V3/V4/V6 判据检验 + `result2/result3.xlsx` + 环境外推三档;
- `run_q4.py`: 问题 4 移动域 + 效应拆分 + V5 退化 + `result4.xlsx`; `run_q4_endo.py`: 内生几何主解 (满 72 h 积分、三段误差评价);

- `run_m4.py`: 无量纲判据回代检验 + 敏感性汇总 + 误差分量汇总 + V2 尾段 + 结果汇总;
- `latent_scenario.py`: 潜热对照情景 (5.3 节);
- `run_mech_refit.py` (数据): 力学扩展模型重标定 (5.5 节)。

全部结果文件确定性写出 (固定归档时间戳), 连续两次运行 `result1-4` 的 md5 完全一致 (`result1 22eb47f7...`、`result2 12cde4e8...`、`result3 fbe982ff...`、`result4 80c73431...`)。

支撑材料文件列表

交付文件夹为 论文/ (顶层目录, 63 个文件、7 个目录, 未压缩约 5.7 MB); 四个结果工作簿按题目要求命名并置于包根目录, 其余材料在 论文/支撑材料/ 下按源码仓库结构分组。下列清单与该交付文件夹内容逐项一致。

如何复现: 打开 (或解包) 后进入 论文/支撑材料/, 该目录完整镜像源码仓库的相对结构 (代码/、数据/、结果/、图表/、源码/), 脚本按仓库相对路径解析的输入输出路径、以及本文的插图与源码引用路径均**原样成立**, 无需调整目录。编译论文: `cd 支撑材料/源码 && xelatex 中药材烘干热湿耦合建模、守恒求解与模型检验.tex` (连跑两遍以解析交叉引用); 复现结果: `cd 支撑材料/代码 && python run_all.py`, 按既定顺序执行全部脚本并由脚本写出四个结果工作簿与 数据/ 的中间数据 (数据/ 下的 `.npz` 张量缓存合计约 23.5 MB, 超 20 MB 上限, 由该命令重新生成, 故未随包提供)。结果/ 是脚本的结果写入处, 交付结果工作簿同时置于包根目录, 故该目录在包内留空, 保留它是为了复跑时不必手工建目录。赛题原件按规范第十一条豁免未随包。

- 论文/ (6): 中药材烘干热湿耦合建模、守恒求解与模型检验.pdf、`result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4.xlsx`、AI 工具使用声明.docx;
- 论文/支撑材料/代码/ (13): `analytic_heat.py`、`endogenous_calibrated.py`、`latent_scenario.py`、`mech_shrinkage.py`、`plot_all.py`、`run_all.py`、`run_m4.py`、`run_q1.py`、`run_q23.py`、`run_q4.py`、`solver_q1.py`、`solver_q23.py`、`solver_q4.py`;
- 论文/支撑材料/数据/ (20): `run_mech_A.py`、`run_mech_mainline.py`、`run_mech_refit.py`、`run_q4_endo.py`、`run_q4_endo_fields.py`、交付口径-核对.py、结果总账.csv、`paper_tables.csv`、`q4_endo_eval.csv`、`q4_split.csv`、`q4_endo_convergence.csv`、`mechRefit.csv`、`sensitivity.csv`、`error_budget.csv`、`latent_scenario.csv`、`D_of_C.csv`、`surface_flux.csv`、交付口径.md、力学升级 _ 修复重标定.md、内生收缩模型.md;
- 论文/支撑材料/结果/ (0): 空目录 (脚本的结果写入处, 交付结果同时在包根目录);
- 论文/支撑材料/图表/ (17): `fig1_ 输入数据.pdf`、`fig2_ 问题 1 解析验证.pdf`、`fig3_ 温湿剖面.pdf`、`fig4_ 达标穿越.pdf`、`fig5_ 敏感性.pdf`、`fig6_ 收缩机制分析.pdf`、`fig7_ 内生收缩贴合.pdf`、`fig9_ 截面演化.pdf`、`fig10_ 力学升级.pdf`、`fig11_ 扩散系数与时间尺度.pdf`、`fig12_ 温度场时空云图.pdf`、`fig13_ 水分场时空云图.pdf`、`fig14_ 表面通量与表面扩散系数.pdf`、`fig15_ 收敛与守恒检验.pdf`、`fig16_ 中心表面演化与半径收缩.pdf`、`fig17_ 几何模型与边界条件示意图.pdf`、`_fig4_ 对照 _ 附 4 固定 R_maxU.csv`;

- 论文/支撑材料/源码/ (7): 中药材烘干热湿耦合建模、守恒求解与模型检验.tex、tables.tex、gen_tables.py、check_abstract.py、check_numbers.py、check_refs.py、check_table6.py。

其中 result1.xlsx、result3.xlsx、result4.xlsx 为原表原样入包；包根目录的 result2.xlsx 为降采样版（仓库内文件名为 result2_降采样.xlsx）：原表为 1 s 时间网格（温度与水分浓度 2 张表 $\times$ 209447 数据行 $\times$ 22 列，27.9 MB），超过支撑材料 20 MB 上限，故按每 1800 s（30 min）一帧抽取、每帧保留完整径向剖面（21 个节点），并加留首帧 1 s 与末帧 209447 s（ $t_f + 3600$ s 余量），共 118 帧；表名、列头、列序与原表一致。其余中间工作簿与张量（.npz）、位图、日志仍不入包。

核心求解步骤的程序摘录（守恒步进与表面重建，注释从略）如下。

代码 1 守恒耦合步进核心（solver 摘录）

```

1 # 表面值：半单元内阻与外对流阻串联（D_eff 取半单元中点值）
2 def surface_m(CN, Ts, Cenv, dq):
3     g = lambda Cs: 8.0*D_of(0.5*(Cs+CN), Ts)*(Cs-CN)/(R0**2*dq) \
4         - 2.0*HM*(Cenv-Cs)/R0
5     Cs = brentq(g, lo, hi, xtol=1e-15, rtol=1e-14)
6     return Cs, 2.0*HM*(Cenv-Cs)/R0
7
8 # 一个耦合后向欧拉步：热场（冻结系数，线性）→ 湿场（冻结温度，Newton）
9 def coupled_step(U, T, dt, Ta, Cenv, dq, newton):
10     ch = 4.0*qf*k_of(Uf)*dt/(R0**2*dq**2) # 面通量系数
11     Rh = R0**2*dq/(8.0*ks) + R0/(2.0*H) # 表面串联热阻
12     T_new = solve_banded((1,1), ab_T, rhs_T) # 热场线性求解
13     for _ in range(30): # 湿场 Newton/Picard
14         dU = solve_banded((1,1), J, -G)
15         U_new = U_new + dU
16     return U_new, T_new

```

附录 E： 核心程序代码

本附录列出建模用到的全部 19 个脚本的完整源码：代码/ 13 个共 5473 行、数据/ 6 个共 1502 行，合计 6975 行。全部源码均由 lstinputlisting 自源文件直接引入，未手工转录，与支撑材料中的同一份文件逐字节一致；数据、图表与结果文件的清单见前文「支撑材料文件列表」。

E.1 解析谱解

圆柱 Robin 特征展开与时变环境 Duhamel 卷积的解析解，用作问题 1 的独立参照（V1）。

代码 2 解析谱解（analytic_heat.py, 122 行）

```

1 """问题 1 热场（以及常 D 退化湿分场）的解析解：圆柱 Robin 特征展开 + Duhamel
   卷积。
2
3 方程：T_t = diff * (1/r)∂_r(r ∂_r T), r∈[0,R]；中心正则；
4 表面 -coef * T_r = h_ext * (T_s - g_env(t)) (Robin, Bi = h_ext * R/coef)。
5
6 特征值：λ J1(λ) = Bi * J0(λ), 特征函数 J0(λn r/R), μn = diff * λn2/R2。

```

```

7 令  $g(t) = g_{\text{env}}(t) - T(0)$ , 则
8
9  $T(r, t) = T(0) + \sum_n c_n(t) \cdot J_0(\lambda_n r/R)$ ,
10  $c_n(t) = K_n \cdot J_n(t)$ ,  $K_n = 2 \mu_n Bi / (J_0(\lambda_n)(\lambda_n^2 + Bi^2))$ ,
11  $J_n(t) = \int_0^t e^{-\mu_n(t-s)} g(s) ds$ 。
12
13 阶跃响应系数:  $A_n = 2Bi^2 / (\lambda_n J_1(\lambda_n)(Bi^2 + \lambda_n^2))$  (中心列),
14 表面列  $A_n J_0(\lambda_n) = 2Bi / (Bi^2 + \lambda_n^2)$ 。自检:  $\sum A_n = 1$ 、 $\sum A_n J_0(\lambda_n) = 1$ 。
15
16  $g$  分段线性时  $J_n$  有精确递推 (每段一步, 无时间截断误差),
17 与求解器的精确时间积分对齐  $\Rightarrow V1$  对拍隔离纯空间误差。
18 """
19 from __future__ import annotations
20
21 import numpy as np
22 from scipy.optimize import brentq
23 from scipy.special import jv
24
25
26 def robin_eigs(Bi: float, n_modes: int) -> np.ndarray:
27     """ $\lambda J_1(\lambda) - Bi \cdot J_0(\lambda) = 0$  的前  $n_{\text{modes}}$  个正根 (稠密扫描 + brentq) 。"""
28     def f(x):
29         return x * jv(1, x) - Bi * jv(0, x)
30
31     xmax = (n_modes + 1) * np.pi + 1.0
32     xs = np.linspace(1e-9, xmax, int(xmax / 0.002) + 1)
33     fs = f(xs)
34     roots = []
35     for i in range(len(xs) - 1):
36         if fs[i] == 0.0:
37             roots.append(xs[i])
38         elif fs[i] * fs[i + 1] < 0:
39             roots.append(brentq(f, xs[i], xs[i + 1], xtol=1e-15, rtol=1e-15))
40     if len(roots) >= n_modes:
41         break
42     roots = np.array(roots[:n_modes])
43     assert len(roots) == n_modes, f"只找到 {len(roots)} 个根"
44     d = np.diff(roots)
45     assert np.all((d > 2.5) & (d < 3.7)), "特征根间距异常, 可能漏根"
46     return roots
47
48
49 class RobinCylinder:
50     """圆柱 Robin 问题的特征展开解 (Duhamel 卷积, 分段线性驱动精确积分) 。"""
51
52     def __init__(self, Bi: float, diff: float, radius: float, n_modes: int = 300):
53         self.Bi, self.diff, self.R = Bi, diff, radius
54         self.lam = robin_eigs(Bi, n_modes)
55         self.mu = diff * self.lam**2 / radius**2
56         self.J0l = jv(0, self.lam)

```

```

57     J1l = jv(1, self.lam)
58     self.A = 2 * Bi**2 / (self.lam * J1l * (self.lam**2 + Bi**2))
59     self.Kn = 2 * self.mu * Bi / (self.J0l * (self.lam**2 + Bi**2))
60     # 自检: 阶跃响应趋于 1 (中心列与表面列)
61     self.sum_center = float(self.A.sum())
62     self.sum_surface = float((self.A * self.J0l).sum())
63     self.mean_mode = 2.0 * jv(1, self.lam) / self.lam #  $J_0(\lambda\sqrt{q})$  的体积平均
64
65     def _advance(self, J, dt, g0, g1):
66         e = np.exp(-self.mu * dt)
67         E1 = -e / self.mu #  $(1-e^{-\mu dt})/\mu$ 
68         E2 = (e + self.mu * dt) / self.mu**2 #  $(e^{-\mu dt}-1+\mu dt)/\mu^2$ 
69         m = (g1 - g0) / dt
70         return J * np.exp(-self.mu * dt) + g0 * E1 + m * E2
71
72     def _state(self, env_t, env_g, times):
73         """把模态卷积状态 J_n 精确推进到各时刻; 返回 (g(times), J(times))。"""
74         times = np.asarray(times, dtype=float)
75         J = np.zeros(len(self.lam))
76         t_cur, seg = float(env_t[0]), 0
77         g_out = np.zeros(len(times))
78         J_out = np.zeros((len(times), len(self.lam)))
79
80         def g_at(t): # 段内线性 (段界已对齐, 无跨段)
81             return env_g[seg] + (env_g[seg + 1] - env_g[seg]) * \
82                 (t - env_t[seg]) / (env_t[seg + 1] - env_t[seg])
83
84         for j, tau in enumerate(times):
85             while t_cur < tau - 1e-12:
86                 t_nxt = min(tau, float(env_t[seg + 1]))
87                 J = self._advance(J, t_nxt - t_cur, g_at(t_cur), g_at(t_nxt))
88                 t_cur = t_nxt
89                 if t_nxt >= float(env_t[seg + 1]) - 1e-12 and seg + 1 < len(
90                     env_t) - 1:
91                     seg += 1
92                 g_out[j] = g_at(tau)
93                 J_out[j] = J
94             return g_out, J_out
95
96     def eval(self, r, times, env_t, env_g, offset):
97         """r (m, 数组)、times (s, 升序数组) 处的解。
98
99         env_t/env_g: 驱动  $g(t)$  = 环境 - 初值 的分段线性采样点 (须含  $t=0$ )。
100         采用稳态减法加速级数收敛:
101              $T = \text{offset} + g(t) - \sum_n (K_n/\mu_n) \cdot J_n'(t) \cdot J_0(\lambda_n r/R)$ ,  $J_n' = g - \mu_n J_n$ ,
102             其中  $\sum (K_n/\mu_n) J_0 \equiv 1$  (阶跃稳态为均匀  $g$ ) 为精确恒等式,
103             尾巴项衰减从  $\lambda^{-2}$  (非交错) 提到  $\lambda^{-7/2}$  (交错)。
104         """
105         r = np.atleast_1d(np.asarray(r, dtype=float))
106         basis = jv(0, np.outer(self.lam, r / self.R)) # (M, len(r))

```

```

106     g_t, J_t = self._state(np.asarray(env_t, float), np.asarray(env_g,
107                               float), times)
107     coef = self.Kn / self.mu
108     out = np.zeros((len(times), len(r)))
109     for j in range(len(times)):
110         Jp = g_t[j] - self.mu * J_t[j]
111         out[j] = offset + g_t[j] - (coef * Jp) @ basis
112     return out
113
114     def mean(self, times, env_t, env_g, offset):
115         """体积平均 ( $\int_0^1 T(q)dq$ ) 随时间 (同一稳态减法形式) 。”””
116         g_t, J_t = self._state(np.asarray(env_t, float), np.asarray(env_g,
117                               float), times)
118         coef = self.Kn / self.mu * self.mean_mode
119         out = np.zeros(len(times))
120         for j in range(len(times)):
121             Jp = g_t[j] - self.mu * J_t[j]
122             out[j] = offset + g_t[j] - float(coef @ Jp)
123         return out

```

E.2 热湿耦合主求解器

质量坐标下的守恒型有限体积格式：半单元内阻与外对流阻串联重建表面值、后向欧拉加局部外推、Picard–Newton 交替迭代、自适应步长。

代码 3 热湿耦合主求解器 (solver_q23.py, 271 行)

```

1  """问题 2/3: 双向耦合质量坐标有限体积求解器 (附录 3 物性, 固定 R0) 。
2
3  与问题 1 的差别 (00-最终方案.md §1 开关表) : 物性全部随状态变、D=D(U,θ) 双向耦合。
4
5  方程 ( $q \in [0,1]$ ,  $r = R0 \cdot \sqrt{q}$ , 见 §2.1) :
6       $U_t = \partial_q(4qD(U,\theta)/R0^2 \cdot U_q)$ 
7       $a_{eff}(U) \cdot \theta_t = \partial_q(4qk(U)/R0^2 \cdot \theta_q)$ ,  $a_{eff} = \rho(U) \cdot c_p(U)$  (有效体积热容,  $\rho$  不进湿分方程)
8  附录 3 (逐点取值, 不得常数化) :
9       $\rho = 650+128C$ ;  $c_p = 1450+2736 \cdot C/(C+1)$ ;  $k = 0.21+0.38 \cdot C/(C+1)$ 
10      $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$ ,  $C$  取局部干基、 $T$  取局部开尔文。
11      $h=25$ 、 $h_m=8e-7$  从附录 2 延用 (声明假设, §2.2) 。
12
13  离散: 单元中心 FV;  $q=0$  面通量系数为 0 (正则对称自动满足) ;
14  表面半单元内阻 + 外对流阻串联,  $U_s/\theta_s$  由 Newton 联立重建。
15  时间: 后向欧拉 + 一步 vs 两半步自适应; 非线性: 外场 Gauss–Seidel 耦合,
16  热场线性求解, 湿场 Picard 起步、 $\max_q U < 0.6$  后切换含  $\partial D/\partial C$  的 Newton (尾段 D 变化 16.8 倍) 。
17  """
18  from __future__ import annotations
19
20  import math
21
22  import numpy as np

```



```

23 from scipy.linalg import solve_banded
24 from scipy.optimize import brentq
25
26 from solver_q1 import R0, H, HM, T0_K, C0, build_interp_weights,
    apply_interp # noqa: F401
27
28 TH = 0.15      # 达标阈值 kg/kg (问题 3)
29 PROBE = {"outer": 0, "calls": 0, "rej_outer": 0} # 诊断计数器
30
31
32 # ---- 附录 3 物性 (逐点取值) ----
33 def rho_cp(C):
34     """有效体积热容 a_eff =  $\rho \cdot c_p$ , J/(m3 · K)。"""
35     C = np.asarray(C, dtype=float)
36     return (650.0 + 128.0 * C) * (1450.0 + 2736.0 * C / (C + 1.0))
37
38
39 def k_of(C):
40     C = np.asarray(C, dtype=float)
41     return 0.21 + 0.38 * C / (C + 1.0)
42
43
44 def D_of(C, T):
45     """附录 3:  $D = 2.4e-3 \cdot \exp(-0.45/C) \cdot \exp(-3850/T)$  m2/s。T 必须开尔文。
46
47     迭代中间态可能短暂越出物理区间，钳制自变量防止 exp 溢出；
48     收敛解必在物理区间内（外围 assert 保证），钳制不影响收敛值。
49     """
50     C = np.clip(np.asarray(C, dtype=float), 1e-3, 20.0)
51     T = np.clip(np.asarray(T, dtype=float), 250.0, 700.0)
52     return 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / T)
53
54
55 def dD_dC(C, T):
56     C = np.asarray(C, dtype=float)
57     return D_of(C, T) * 0.45 / C**2
58
59
60 def surface_m(CN, Ts, Cenv, dq):
61     """湿分表面:  $F = 8 \cdot D_{\text{eff}} \cdot (C_s - CN) / (R0^2 \cdot dq) = 2h_m(Cenv - C_s) / R0$ , Brent
        保护求根。
62
63      $D_{\text{eff}} = D((C_s + CN)/2, T_s)$ : 半单元中点取值，与内面通量  $D_f = D((U_i + U_{i+1})/2)$  同阶。
64     附录 3 的  $D \propto e^{-0.45/C}$  强变，若用端点  $D(C_s)$ ,  $C_s \rightarrow Cenv$  时  $D \rightarrow 0$  会产生
65     "内通量被人为归零"的伪根（表面误判为瞬间干透）；中点取值消除伪根。
66     返回 (C_s, F_s, D_eff)。
67     """
68     def g(Cs):
69         De = float(D_of(0.5 * (Cs + CN), Ts))
70         return 8.0 * De * (Cs - CN) / (R0**2 * dq) - 2.0 * HM * (Cenv - Cs) /

```

```

R0
71
72 hi = CN
73 lo = max(1e-6, min(CN, Cenv) * 1e-3)
74 if g(hi) <= 0.0:          # Cenv ≥ CN (本情景不出现)：表面贴近环境
75     Cs = CN
76 else:
77     Cs = brentq(g, lo, hi, xtol=1e-15, rtol=1e-14, maxiter=100)
78 De = float(D_of(0.5 * (Cs + CN), Ts))
79 Fs = 2.0 * HM * (Cenv - Cs) / R0
80 return Cs, Fs, De
81
82
83 def coupled_step(U, T, dt, Ta, Cenv, dq, newton):
84     """一个耦合后向欧拉步。返回 (Unew, Tnew, Us, Fs_m, Ts, Fs_h, a_fr)。
85
86     外场 Gauss-Seidel: 热场 (系数冻结在 U_m, 线性) → 湿场 (θ 冻结, Newton/
87     Picard)。
88     a_fr 为热场求解实际使用的冻结体积热容 (供总焓恒等式逐项对账)。
89     """
90     N = U.size
91     qf = np.arange(1, N) * dq
92     U_m, T_m = U.copy(), T.copy()
93     Us_m, _, _ = surface_m(U_m[-1], T_m[-1], Cenv, dq)
94     a = rho_cp(U_m)
95     for _ in range(40):
96         PROBE["outer"] += 1
97         # ----- 热场 (线性, 系数取 U_m) -----
98         a = rho_cp(U_m)
99         kf = k_of(0.5 * (U_m[:-1] + U_m[1:]))
100         ch = 4.0 * qf * kf * dt / (R0**2 * dq**2) # 面通量系数 (不含 α_i)
101         ks = float(k_of(0.5 * (Us_m + U_m[-1]))) # 半单元中点取值 (同湿分口径)
102         Rh = R0**2 * dq / (8.0 * ks) + R0 / (2.0 * H)
103         diag = 1.0 + (np.concatenate(([0.0], ch)) + np.concatenate((ch,
104             [0.0]))) / a
105         diag[-1] += dt / (Rh * a[-1] * dq)
106         rhs = T.copy()
107         rhs[-1] += dt * Ta / (Rh * a[-1] * dq)
108         ab = np.zeros((3, N))
109         ab[0, 1:] = -ch / a[:-1]
110         ab[1, :] = diag
111         ab[2, :-1] = -ch / a[1:]
112         T_new = solve_banded((1, 1), ab, rhs)
113         Fs_h = (Ta - T_new[-1]) / Rh
114         Ts = T_new[-1] + Fs_h * R0**2 * dq / (8.0 * ks)
115         # ----- 湿场 (θ = T_new 冻结, Newton 或 Picard) -----
116         U_new = U_m.copy()
117         Tf = 0.5 * (T_new[:-1] + T_new[1:])
118         for _ in range(30):
119             Uf = 0.5 * (U_new[:-1] + U_new[1:])
120             Df = D_of(Uf, Tf)

```

```

119     b = 4.0 * qf * Df * dt / (R0**2 * dq**2)
120     if newton:
121         e = 0.5 * dD_dC(Uf, Tf) / Df * (U_new[1:] - U_new[:-1])
122     else:
123         e = np.zeros(N - 1)
124     Cs, Fs_m, Ds = surface_m(U_new[-1], Ts, Cenv, dq)
125     Rs = R0**2 * dq / (8.0 * Ds) + R0 / (2.0 * HM)
126     gs = dt / (Rs * dq)
127     fl = b * (U_new[1:] - U_new[:-1]) # 内面通量 (dt/dq 已乘)
128     G = U_new - U - np.concatenate((fl, [gs * (Cenv - U_new[-1])])) \
129         + np.concatenate(([0.0], fl))
130     bn = np.concatenate((b, [0.0]))
131     en = np.concatenate((e, [0.0]))
132     bp = np.concatenate(([0.0], b))
133     ep = np.concatenate(([0.0], e))
134     J = np.zeros((3, N))
135     J[0, 1:] = -bn[:-1] * (1.0 + en[:-1])
136     J[1, :] = 1.0 + bn * (1.0 - en) + bp * (1.0 + ep)
137     J[1, -1] += gs
138     J[2, :-1] = -bn[:-1] * (1.0 - en[:-1])
139     dU = solve_banded((1, 1), J, -G)
140     U_new = U_new + dU
141     if np.max(np.abs(dU)) < 1e-11:
142         break
143     dUmax = float(np.max(np.abs(U_new - U_m)))
144     dTmax = float(np.max(np.abs(T_new - T_m)))
145     U_m, T_m = U_new, T_new
146     if not (np.all(np.isfinite(U_m)) and np.all(np.isfinite(T_m))
147             and U_m.min() > -1.0 and U_m.max() < 10.0
148             and T_m.min() > 200.0 and T_m.max() < 700.0):
149         raise RuntimeError("迭代越界, 应缩小步长")
150     Us_m, _, _ = surface_m(U_m[-1], Ts, Cenv, dq)
151     if dUmax < 1e-10 and dTmax < 1e-8:
152         break
153     else:
154         raise RuntimeError("耦合外场未收敛, 应缩小步长")
155     Us, Fs_m, _ = surface_m(U_m[-1], Ts, Cenv, dq)
156     return U_m, T_m, Us, Fs_m, Ts, Fs_h, a
157
158
159 def solve_coupled(N, env_fun, t_end, out_times=(), dt0=1e-3, dt_max=1.0,
160                  atolU=1e-11, rtolU=1e-7, atolT=1e-8, rtolT=1e-7,
161                  newton_below=0.6, margin=0.0, record=None, win=600.0,
162                  progress=None, record_each=False):
163     """双向耦合自适应求解。
164
165     env_fun(t) → (Ta_K, Cenv); out_times 需精确落点记录 (record 回调收 21 列之
166         外的原场)。
167     达标检测: 每个接受步扫描 max_q U (含中心正则外推), 首次 <0.15 后记录穿越区
168         间,
169     继续推进 margin 秒后停止。record(t, U, T, Us, Ts) 在落点时刻调用。

```

```

168 返回 dict: crossing=(t-, u-, t+, u+), stats (守恒双恒等式、步数),
169 umax_locus (抽样时刻的 argmax 检验)。
170 """
171 dq = 1.0 / N
172 U = np.full(N, C0)
173 T = np.full(N, T0_K)
174 assert 200.0 < T0_K < 400.0, "初温必须开尔文"
175 t, dt = 0.0, dt0
176 out_list = sorted(out_times)
177 i_out = 0
178 n_steps = n_rej = 0
179 max_res_m = max_res_h = 0.0
180 g_dh_err = 0.0          # 全程  $\Sigma(dh - fh)$  (焓恒等式全局结算分子)
181 g_fh_abs = 0.0          # 全程  $\Sigma|fh|$  (总吞吐, 分母)
182 w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
183 win_end = win
184 crossing = None
185 t_prev = 0.0
186 umax_prev = C0
187 locus = []
188 worst_h = []
189 while t < t_end - 1e-12:
190     if crossing is not None and t >= crossing[2] + margin - 1e-12:
191         break
192     target = t + dt
193     if i_out < len(out_list):
194         target = min(target, out_list[i_out])
195     target = min(target, t_end)
196     dtc = target - t
197     if dtc <= 1e-15:
198         i_out += 1
199         continue
200     nw = float(np.max(U)) < newton_below
201     Ta_m, Ce_m = env_fun(t + 0.5 * dtc)
202     Ta_e, Ce_e = env_fun(t + dtc)
203     assert 200.0 < Ta_m < 700.0 and 200.0 < Ta_e < 700.0, "环境温度必须开
204         尔文"
205     # 一步 vs 两半步; 接受外推解  $2 \cdot$  半对 - 整步 (局部外推,  $BE \rightarrow$  二阶)。
206     # 外推态的守恒通量 =  $(F_{h1} + F_{h2} - F_f) \cdot dt$  (由各  $BE$  恒等式线性组合,
207         仍严格守恒)。
208     try:
209         Uf_, Tf_, _, Fmf, _, Fhf, af_ = coupled_step(U, T, dtc, Ta_e, Ce_e,
210             , dq, nw)
211         Uh, Th, Us1, Fm1, Ts1, Fh1, a1 = coupled_step(U, T, 0.5 * dtc,
212             Ta_m, Ce_m, dq, nw)
213         Uh2, Th2, Us2, Fm2, Ts2, Fh2, a2 = coupled_step(Uh, Th, 0.5 * dtc,
214             Ta_e, Ce_e, dq, nw)
215     except (RuntimeError, ValueError, FloatingPointError):
216         n_rej += 1
217         PROBE["rej_outer"] += 1
218         dt = dtc * 0.5

```

```

214         continue
215     n_steps += 3
216     errU = float(np.max(np.abs(Uh2 - Uf_)))
217     errT = float(np.max(np.abs(Th2 - Tf_)))
218     tolU = atolU + rtolU * float(np.max(np.abs(U)))
219     tolT = atolT + rtolT * float(np.max(np.abs(T)))
220     r = max(errU / tolU, errT / tolT)
221     if r <= 1.0:
222         U_ex = 2.0 * Uh2 - Uf_
223         T_ex = 2.0 * Th2 - Tf_
224         assert float(np.min(U_ex)) > 0.0, "外推产生非正水分"
225         umax_old = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
226         w_dm.append(dq * math.fsum(float(x) for x in (U_ex - U)))
227         w_fm.append(dtc * (Fm1 + Fm2 - Fmf))
228         w_dh.append(dq * (2.0 * math.fsum(float(a1[i]) * float(Th[i] - T[i]
229             ] for i in range(N))
230             + 2.0 * math.fsum(float(a2[i]) * float(Th2[i] - Th[i]
231             ] for i in range(N))
232             - math.fsum(float(af_[i]) * float(Tf_[i] - T[i]) for
233             i in range(N))))
234         w_fh.append(dtc * (Fh1 + Fh2 - Fhf))
235         w_fh_abs.append(dtc * (abs(Fh1) + abs(Fh2) + abs(Fhf)) / 3.0)
236         if t + dtc >= win_end - 1e-12:
237             dm, fm = math.fsum(w_dm), math.fsum(w_fm)
238             dh, fh = math.fsum(w_dh), math.fsum(w_fh)
239             if abs(fm) > 0:
240                 max_res_m = max(max_res_m, abs(dm - fm) / abs(fm))
241                 fh_abs = math.fsum(w_fh_abs)
242                 if fh_abs > 0: # 环境回落段净通量可近零, 分母用总吞吐防病态
243                     res_h = abs(dh - fh) / fh_abs
244                     if res_h > max_res_h:
245                         max_res_h = res_h
246                         worst_h[:] = [(res_h, win_end, dh, fh, fh_abs, len(w_dh))
247                             ]
248                     g_dh_err += dh - fh
249                     g_fh_abs += fh_abs
250                     w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
251                     win_end += win
252         U, T = U_ex, T_ex
253         t += dtc
254         umax = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
255         if crossing is None and umax < TH:
256             crossing = (t - dtc, umax_old, t, umax)
257         if record_each and record is not None:
258             record(t, U, T, Us2, Ts2)
259         if i_out < len(out_list) and abs(t - out_list[i_out]) < 1e-12:
260             if record is not None:
261                 record(t, U, T, Us2, Ts2)
262             if len(locus) < 40:
263                 locus.append((out_list[i_out], int(np.argmax(U))))
264             i_out += 1

```

```

261         fac = 0.9 / max(r, 1e-30) ** (1.0 / 3.0)
262         dt = min(dtc * min(2.0, max(0.3, fac)), dt_max)
263         if progress is not None and n_steps % 3000 == 0:
264             progress(t, dtc, r, float(np.max(U)), float(np.min(U)))
265     else:
266         n_rej += 1
267         dt = dtc * max(0.2, 0.9 / r ** (1.0 / 3.0))
268     stats = dict(n_steps=n_steps, n_rej=n_rej, max_res_m=max_res_m,
269                 max_res_h=max_res_h,
270                 t_end=t, worst_h=worst_h,
271                 glob_res_h=abs(g_dh_err) / g_fh_abs if g_fh_abs > 0 else 0.0)
271     return dict(U=U, T=T, crossing=crossing, stats=stats, locus=locus)

```

E.3 移动域求解器

仿射收缩 $r = R(t)\sqrt{q}$ 下的移动域求解与达标判定。

代码 4 移动域求解器 (solver_q4.py, 232 行)

```

1  """问题 4: 仿射收缩移动域 (材料坐标) + 附录 4 物性; 兼作 T6 对照与 V5 退化的通
   用求解器。
2
3  移动域方程 (00-最终方案.md §2.1, 材料坐标  $q$ ,  $r(q,t)=R(t) \cdot \sqrt{q}$ , 长度不变、内部等
   比为声明闭合):
4       $U_t = \partial_q(4qD/R(t)^2 \cdot U_q)$ ;  $a_{\text{eff}} \cdot \theta_t = \partial_q(4qk/R(t)^2 \cdot \theta_q)$ 
5  与固定域同形,  $R_0 \rightarrow R(t)$ ; 仿射 + 干固体守恒下无伪对流项;
6  移动表面 Robin 边界形式不变 (跳条件中  $\dot{R}C_s$  项恒等相消, 不另补)。
7
8  附录 4 (逐点取值):  $p=760+90C$ 、 $c_p=1850+2150 \cdot C/(C+1)$ 、 $k=0.12+0.20 \cdot C/(C+1)$ 、
9   $D=4.2e-4 \cdot e^{(-0.30/C)} \cdot e^{(-3850/T)}$ 。  $h=25$ 、 $h_m=8e-7$  自附录 2 延用 (声明假
   设)。
10
11  数值框架与 solver_q23 相同 (单元中心 FV、表面半单元中点取值+Brent、BE+局部外
   推、
12  外场 Gauss-Seidel、湿场 Picard→Newton 切换); 物性包与  $R(t)$  经参数注入,
13  使本文件同时承担: ① 主解 (附4+ $R(t)$ )、② 对照 (附4+ $R \equiv R_0$ )、V5 退化 ( $R \equiv R_0$ 、D 冻
   结)。
14  """
15  from __future__ import annotations
16
17  import math
18
19  import numpy as np
20  from scipy.linalg import solve_banded
21  from scipy.optimize import brentq
22
23  from solver_q1 import R0, H, HM, T0_K, C0, build_interp_weights,
   apply_interp # noqa: F401
24  from solver_q23 import TH, PROBE # noqa: F401
25
26
27  # ---- 附录 4 物性 (逐点取值) ----

```

```

28 def rho_cp4(C):
29     C = np.asarray(C, dtype=float)
30     return (760.0 + 90.0 * C) * (1850.0 + 2150.0 * C / (C + 1.0))
31
32
33 def k_of4(C):
34     C = np.asarray(C, dtype=float)
35     return 0.12 + 0.20 * C / (C + 1.0)
36
37
38 def D_of4(C, T):
39     C = np.clip(np.asarray(C, dtype=float), 1e-3, 20.0)
40     T = np.clip(np.asarray(T, dtype=float), 250.0, 700.0)
41     return 4.2e-4 * np.exp(-0.30 / C) * np.exp(-3850.0 / T)
42
43
44 def dD_dC4(C, T):
45     C = np.asarray(C, dtype=float)
46     return D_of4(C, T) * 0.30 / C**2
47
48
49 PROPS4 = dict(rho_cp=rho_cp4, k_of=k_of4, D_of=D_of4, dD_dC=dD_dC4)
50
51
52 def surface_m4(CN, Ts, Cenv, dq, Rt, Dfun):
53     """湿分表面（移动表面 Robin，形式不变）：半单元中点 D_eff + Brent 保护求
        根。"""
54     def g(Cs):
55         De = float(Dfun(0.5 * (Cs + CN), Ts))
56         return 8.0 * De * (Cs - CN) / (Rt**2 * dq) - 2.0 * HM * (Cenv - Cs) /
            Rt
57
58     hi = CN
59     lo = max(1e-6, min(CN, Cenv) * 1e-3)
60     if g(hi) <= 0.0:
61         Cs = CN
62     else:
63         Cs = brentq(g, lo, hi, xtol=1e-15, rtol=1e-14, maxiter=100)
64     De = float(Dfun(0.5 * (Cs + CN), Ts))
65     Fs = 2.0 * HM * (Cenv - Cs) / Rt
66     return Cs, Fs, De
67
68
69 def coupled_step4(U, T, dt, Ta, Cenv, dq, newton, Rt, P):
70     """一个耦合后向欧拉步（系数取步末 R(t)）。返回 (Unew, Tnew, Us, Fs_m, Ts, Fs_h,
        a_fr)。"""
71     N = U.size
72     qf = np.arange(1, N) * dq
73     rho_cp, k_of, D_of, dD_dC = P["rho_cp"], P["k_of"], P["D_of"], P["dD_dC"]
74     U_m, T_m = U.copy(), T.copy()

```

```

75 Us_m, _, _ = surface_m4(U_m[-1], T_m[-1], Cenv, dq, Rt, D_of)
76 a = rho_cp(U_m)
77 for _ in range(40):
78     PROBE["outer"] += 1
79     a = rho_cp(U_m)
80     kf = k_of(0.5 * (U_m[:-1] + U_m[1:]))
81     ch = 4.0 * qf * kf * dt / (Rt**2 * dq**2)
82     ks = float(k_of(0.5 * (Us_m + U_m[-1])))
83     Rh = Rt**2 * dq / (8.0 * ks) + Rt / (2.0 * H)
84     diag = 1.0 + (np.concatenate(([0.0], ch)) + np.concatenate((ch,
85         [0.0]))) / a
86     diag[-1] += dt / (Rh * a[-1] * dq)
87     rhs = T.copy()
88     rhs[-1] += dt * Ta / (Rh * a[-1] * dq)
89     ab = np.zeros((3, N))
90     ab[0, 1:] = -ch / a[:-1]
91     ab[1, :] = diag
92     ab[2, :-1] = -ch / a[1:]
93     T_new = solve_banded((1, 1), ab, rhs)
94     Fs_h = (Ta - T_new[-1]) / Rh
95     Ts = T_new[-1] + Fs_h * Rt**2 * dq / (8.0 * ks)
96     U_new = U_m.copy()
97     Tf = 0.5 * (T_new[:-1] + T_new[1:])
98     for _ in range(30):
99         Uf = 0.5 * (U_new[:-1] + U_new[1:])
100         Df = D_of(Uf, Tf)
101         b = 4.0 * qf * Df * dt / (Rt**2 * dq**2)
102         if newton:
103             e = 0.5 * dD_dC(Uf, Tf) / Df * (U_new[1:] - U_new[:-1])
104         else:
105             e = np.zeros(N - 1)
106         Cs, Fs_m, Ds = surface_m4(U_new[-1], Ts, Cenv, dq, Rt, D_of)
107         Rs = Rt**2 * dq / (8.0 * Ds) + Rt / (2.0 * HM)
108         gs = dt / (Rs * dq)
109         fl = b * (U_new[1:] - U_new[:-1])
110         G = U_new - U - np.concatenate((fl, [gs * (Cenv - U_new[-1])])) \
111             + np.concatenate(([0.0], fl))
112         bn = np.concatenate((b, [0.0]))
113         en = np.concatenate((e, [0.0]))
114         bp = np.concatenate(([0.0], b))
115         ep = np.concatenate(([0.0], e))
116         J = np.zeros((3, N))
117         J[0, 1:] = -bn[:-1] * (1.0 + en[:-1])
118         J[1, :] = 1.0 + bn * (1.0 - en) + bp * (1.0 + ep)
119         J[1, -1] += gs
120         J[2, :-1] = -bn[:-1] * (1.0 - en[:-1])
121         dU = solve_banded((1, 1), J, -G)
122         U_new = U_new + dU
123         if np.max(np.abs(dU)) < 1e-11:
124             break
125     dUmax = float(np.max(np.abs(U_new - U_m)))

```



```

125     dTmax = float(np.max(np.abs(T_new - T_m)))
126     U_m, T_m = U_new, T_new
127     if not (np.all(np.isfinite(U_m)) and np.all(np.isfinite(T_m))
128             and U_m.min() > -1.0 and U_m.max() < 10.0
129             and T_m.min() > 200.0 and T_m.max() < 700.0):
130         raise RuntimeError("迭代越界，应缩小步长")
131     Us_m, _, _ = surface_m4(U_m[-1], Ts, Cenv, dq, Rt, D_of)
132     if dUmax < 1e-10 and dTmax < 1e-8:
133         break
134     else:
135         raise RuntimeError("耦合外场未收敛，应缩小步长")
136     Us, Fs_m, _ = surface_m4(U_m[-1], Ts, Cenv, dq, Rt, D_of)
137     return U_m, T_m, Us, Fs_m, Ts, Fs_h, a
138
139
140 def solve_coupled4(N, env_fun, R_fun, t_end, P=PROPS4, out_times=(), dt0=1e
141 -3,
142                     dt_max=10.0, atolU=1e-11, rtolU=1e-7, atolT=1e-8, rtolT=1e-7,
143                     newton_below=0.6, margin=0.0, record=None, win=600.0,
144                     record_each=False, detect=True):
145     """移动域自适应求解。R_fun(t) → 当前半径 (m)。守恒口径同 solver_q23。"""
146     dq = 1.0 / N
147     U = np.full(N, C0)
148     T = np.full(N, T0_K)
149     assert 200.0 < T0_K < 400.0
150     t, dt = 0.0, dt0
151     out_list = sorted(out_times)
152     i_out = 0
153     n_steps = n_rej = 0
154     max_res_m = max_res_h = 0.0
155     g_dh_err = g_fh_abs = 0.0
156     w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
157     win_end = win
158     crossing = None
159     while t < t_end - 1e-12:
160         if detect and crossing is not None and t >= crossing[2] + margin - 1e
161             -12:
162             break
163         target = t + dt
164         if i_out < len(out_list):
165             target = min(target, out_list[i_out])
166         target = min(target, t_end)
167         dtc = target - t
168         if dtc <= 1e-15:
169             i_out += 1
170             continue
171         nw = float(np.max(U)) < newton_below
172         Rt_e = R_fun(t + dtc) # 系数取步末半径 (BE 全隐式口径)
173         Rt_m = R_fun(t + 0.5 * dtc)
174         Ta_m, Ce_m = env_fun(t + 0.5 * dtc)
175         Ta_e, Ce_e = env_fun(t + dtc)

```

```

174 assert 200.0 < Ta_m < 700.0 and 200.0 < Ta_e < 700.0, "环境温度必须开
    尔文"
175 try:
176     Uf_, Tf_, _, Fmf, _, Fhf, af_ = coupled_step4(U, T, dtc, Ta_e,
        Ce_e, dq, nw, Rt_e, P)
177     Uh, Th, Us1, Fm1, Ts1, Fh1, a1 = coupled_step4(U, T, 0.5 * dtc,
        Ta_m, Ce_m, dq, nw, Rt_m, P)
178     Uh2, Th2, Us2, Fm2, Ts2, Fh2, a2 = coupled_step4(Uh, Th, 0.5 * dtc
        , Ta_e, Ce_e, dq, nw, Rt_e, P)
179 except (RuntimeError, ValueError, FloatingPointError):
180     n_rej += 1
181     PROBE["rej_outer"] += 1
182     dt = dtc * 0.5
183     continue
184 n_steps += 3
185 errU = float(np.max(np.abs(Uh2 - Uf_)))
186 errT = float(np.max(np.abs(Th2 - Tf_)))
187 tolU = atolU + rtolU * float(np.max(np.abs(U)))
188 tolT = atolT + rtolT * float(np.max(np.abs(T)))
189 r = max(errU / tolU, errT / tolT)
190 if r <= 1.0:
191     U_ex = 2.0 * Uh2 - Uf_
192     T_ex = 2.0 * Th2 - Tf_
193     assert float(np.min(U_ex)) > 0.0, "外推产生非正水分"
194     umax_old = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
195     w_dm.append(dq * math.fsum(float(x) for x in (U_ex - U)))
196     w_fm.append(dtc * (Fm1 + Fm2 - Fmf))
197     w_dh.append(dq * (2.0 * math.fsum(float(a1[i]) * float(Th[i] - T[i]
        ]) for i in range(N))
198         + 2.0 * math.fsum(float(a2[i]) * float(Th2[i] - Th[i]
        ]) for i in range(N))
199         - math.fsum(float(af_[i]) * float(Tf_[i] - T[i]) for
        i in range(N))))
200     w_fh.append(dtc * (Fh1 + Fh2 - Fhf))
201     w_fh_abs.append(dtc * (abs(Fh1) + abs(Fh2) + abs(Fhf)) / 3.0)
202     if t + dtc >= win_end - 1e-12:
203         dm, fm = math.fsum(w_dm), math.fsum(w_fm)
204         dh, fh = math.fsum(w_dh), math.fsum(w_fh)
205         if abs(fm) > 0:
206             max_res_m = max(max_res_m, abs(dm - fm) / abs(fm))
207         fh_abs = math.fsum(w_fh_abs)
208         if fh_abs > 0:
209             res_h = abs(dh - fh) / fh_abs
210             if res_h > max_res_h:
211                 max_res_h = res_h
212             g_dh_err += dh - fh
213             g_fh_abs += fh_abs
214         w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
215         win_end += win
216     U, T = U_ex, T_ex
217     t += dtc

```

```

218     umax = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
219     if detect and crossing is None and umax < TH:
220         crossing = (t - dtc, umax_old, t, umax)
221     if record_each and record is not None:
222         record(t, U, T, Us2, Ts2)
223     if i_out < len(out_list) and abs(t - out_list[i_out]) < 1e-12:
224         i_out += 1
225     fac = 0.9 / max(r, 1e-30) ** (1.0 / 3.0)
226     dt = min(dtc * min(2.0, max(0.3, fac)), dt_max)
227     else:
228         n_rej += 1
229         dt = dtc * max(0.2, 0.9 / r ** (1.0 / 3.0))
230     stats = dict(n_steps=n_steps, n_rej=n_rej, max_res_m=max_res_m,
231                 max_res_h=max_res_h,
232                 t_end=t, glob_res_h=abs(g_dh_err) / g_fh_abs if g_fh_abs > 0
233                 else 0.0)
234     return dict(U=U, T=T, crossing=crossing, stats=stats)

```

E.4 内生收缩闭合

由湿度场逐步预测半径 $R(t)$ 的半经验闭合（三项机制），标定参数见附录 C。

代码 5 内生收缩闭合（endogenous_calibrated.py, 243 行）

```

1  """内生收缩模型（终版）：物理结构化闭合 A/B/C 比选 + 附件 2 标定 + 全套验证。
2
3  闭合 C（最终采用，全湿度驱动、一项一机制）：
4   $R(t) = R_0 \cdot \sqrt{(1+\bar{U})/(1+C_0)}$  失水收缩（水分守恒 + 等密度假设）
5      •  $[1 - \varepsilon_c \cdot g_c(\bar{U})]$  毛细-膨压塌陷：  $g_c(x)=(1-x)^a \cdot x^b$ （归一化），
6      x=失水率；先升后降=塌陷发展→成孔回弹
7      •  $[1 + \varepsilon_p \cdot g_p(C_s)]$  结皮-成孔停滞：  $g_p=\text{clip}((C_{\text{glass}}-C_s)/\Delta, 0, 1)^{k_p}$ 
8      C_s 穿越 C_glass=0.21（文献值，非拟合）物理触发
9  标定参数（5 个，拟合附件 2，逐项标注来源）：  $\varepsilon_c$ , a, b,  $\varepsilon_p$ ,  $k_p$ 。
10
11  用法：cd 代码 && python endogenous_calibrated.py
12  产物：../数据/endogenous_calibrated.csv、../数据/endogenous_coupled.csv（数值产物）；
13      ../08-临时/内生收缩_标定验证.png（预览图，非论文交付图，论文用 fig7_内生收缩贴合）。
14  """
15  from __future__ import annotations
16
17  import sys
18  from pathlib import Path
19
20  import numpy as np
21  import openpyxl
22  from scipy.optimize import least_squares
23  from scipy.interpolate import PchipInterpolator
24

```

```

25 HERE = Path(__file__).resolve().parent # 代码
26 A_DIR = HERE.parent # 项目根 (A/)
27 sys.path.insert(0, str(HERE))
28 DATA = A_DIR / "数据"
29 SCRATCH = A_DIR / "08-临时" # 预览图等非交付产物
30
31 from solver_q1 import R0, C0
32 from solver_q4 import PROPS4, D_of4, surface_m4, coupled_step4,
    solve_coupled4
33 from solver_q23 import TH
34 from run_q23 import read_env, make_env
35
36 C_GLASS = 0.21 # 文献/Gordon - Taylor (非拟合)
37 RHO_SK, RHO_W = 1468.0, 1000.0 # 骨架密度 (文献值, 非拟合)
38
39 # ----- 数据 -----
40 wb = openpyxl.load_workbook(A_DIR / "01-题目" / "原始文件" / "附件2.xlsx",
    read_only=True)
41 rows = [r for r in wb[wb.sheetnames[0]].iter_rows(min_row=2, values_only=
    True) if r[0] is not None]
42 wb.close()
43 t_att = np.array([r[0] for r in rows], float)
44 R_att = np.array([r[1] for r in rows], float) * 0.01
45 d = np.load(DATA / "q4_main_steps.npz")
46 um = np.interp(t_att, d["t"], d["um"])
47 Cs = np.interp(t_att, d["t"], d["Cs"])
48 th = t_att / 3600.0
49 CSMIN = float(Cs.min()) # 全局归一化常数 (末期 C_s, 物理量)
50 R_ideal = R0 * np.sqrt((1 + um) / (1 + C0))
51 m_data = R_att / R_ideal
52
53
54 # ----- 三个候选闭合 -----
55 def closureA(p, t, Cs_):
56     eps_c, tau_c, eps_p, t_p, tau_p = p
57     Ri = R0 * np.sqrt((1 + np.interp(t, t_att, um)) / (1 + C0)) if t is not
        t_att else R_ideal
58     phi_c = 1.0 - np.exp(-t / tau_c)
59     phi_p = 1.0 / (1.0 + np.exp(-(t - t_p) / tau_p))
60     return Ri * (1.0 - eps_c * phi_c) * (1.0 + eps_p * phi_p)
61
62
63 def closureB(p, t, Cs_):
64     eps_c, tau_c, tau_d, eps_p, k_p = p
65     phi_c = (1.0 - np.exp(-t / tau_c)) * np.exp(-t / tau_d)
66     phi_p = np.clip((C_GLASS - Cs_) / (C_GLASS - CSMIN), 0.0, 1.0) ** k_p
67     return R_ideal * (1.0 - eps_c * phi_c) * (1.0 + eps_p * phi_p)
68
69
70 def closureC(p, um_, Cs_):
71     eps_c, a, b, eps_p, k_p = p

```

```

72 Ri = R0 * np.sqrt((1 + um_) / (1 + C0))
73 x = np.clip((C0 - um_) / C0, 0, 1)
74 gc = (1 - x) ** a * x ** b
75 gc = gc / ((1 - b / (a + b)) ** a * (b / (a + b)) ** b) # 峰值归一
76 gp = np.clip((C_GLASS - Cs_) / (C_GLASS - CSMIN), 0.0, 1.0) ** k_p
77 return Ri * (1 - eps_c * gc) * (1 + eps_p * gp)
78
79
80 def termsC(p, um_, Cs_):
81     eps_c, a, b, eps_p, k_p = p
82     x = np.clip((C0 - um_) / C0, 0, 1)
83     gc = (1 - x) ** a * x ** b
84     gc = gc / ((1 - b / (a + b)) ** a * (b / (a + b)) ** b)
85     gp = np.clip((C_GLASS - Cs_) / (C_GLASS - CSMIN), 0.0, 1.0) ** k_p
86     return -eps_c * gc, eps_p * gp
87
88
89 def fit(fn, args, p0, lb, ub, target=None):
90     if target is None:
91         target = R_att
92     s = least_squares(lambda p: fn(p, *args) - target, p0, bounds=(lb, ub),
93                       xtol=1e-15, ftol=1e-15, gtol=1e-15, max_nfev=40000)
94     return s.x
95
96
97 pA = fit(closureA, (t_att, Cs), [0.05, 7200, 0.08, 54000, 10800],
98         [0, 600, 0, 0, 300], [0.5, 28800, 0.5, 144000, 72000])
99 pB = fit(closureB, (t_att, Cs), [0.15, 3600, 18000, 0.10, 2.0],
100         [0, 300, 600, 0, 0.2], [0.6, 21600, 216000, 0.6, 20])
101 pC = fit(closureC, (um, Cs), [0.15, 1.0, 0.3, 0.08, 3.0],
102         [0.01, 0.2, 0.05, 0.0, 0.3], [0.6, 8, 3, 0.5, 15])
103 for tag, Rp in (("A", closureA(pA, t_att, Cs)), ("B", closureB(pB, t_att,
104         Cs)), ("C", closureC(pC, um, Cs))):
105     print(f"闭合 {tag}: RMSE={float(np.sqrt(np.mean((Rp - R_att)**2))) *
106           1000:.4f} mm")
107
108 R_pred = closureC(pC, um, Cs)
109 res = R_pred - R_att
110 eps_c, a_c, b_c, eps_p, k_p = pC
111 print(f"\n采用闭合 C。标定参数（拟合附件 2，诚实标注）:  $\epsilon_c$ ={eps_c:.4f}, a={
112       a_c:.4f}, "
113       f"b={b_c:.4f},  $\epsilon_p$ ={eps_p:.4f}, k_p={k_p:.4f}")
114 rmse = float(np.sqrt(np.mean(res ** 2)))
115 print(f"拟合优度: RMSE={rmse * 1000:.4f} mm = {rmse * 100:.5f} cm"
116       f"（附件 2 量化分辨率 0.01 mm）; max|偏差|={np.max(np.abs(res)) *
117       1000:.4f} mm")
118 r0 = res - res.mean()
119 lag1 = float(np.sum(r0[1:] * r0[:-1]) / np.sum(r0 ** 2))
120 runs = int(np.sum(np.diff(np.sign(r0)) != 0)) + 1
121 print(f"残差结构: lag-1 自相关={lag1:.3f}, 符号游程数={runs}/145（白噪参考

```

```

~73) ")
119
120 # 留出法 (过拟合自检)
121 for name, tr, te in (("偶数点训练→奇数点留出", slice(0, None, 2), slice(1,
    None, 2)),
122                      ("奇数点训练→偶数点留出", slice(1, None, 2), slice(0, None,
    2))):
123     s = fit(closureC, (um[tr], Cs[tr]), [0.15, 1.0, 0.3, 0.08, 3.0],
124            [0.01, 0.2, 0.05, 0.0, 0.3], [0.6, 8, 3, 0.5, 15], target=R_att[tr
    ])
125     e_tr = float(np.sqrt(np.mean((closureC(s, um[tr], Cs[tr]) - R_att[tr])
    ** 2)))
126     e_te = float(np.sqrt(np.mean((closureC(s, um[te], Cs[te]) - R_att[te])
    ** 2)))
127     print(f"留出法[{name}]: 训练 {e_tr * 1000:.4f} mm / 留出 {e_te * 1000:.4f
    } mm, "
128           f"参数={np.round(s, 3)}")
129
130 # 结构复现核对
131 m_pred = R_pred / R_ideal
132 i_md = int(np.argmax(m_data > 1.0))
133 i_mp = int(np.argmax(m_pred > 1.0))
134 print(f"\n比率 m=R/R_ideal: data 1.000→{m_data.min():.3f}@{th[np.argmin(
    m_data)]:.1f}h"
135       f"→{m_data[-1]:.3f}; pred 1.000→{m_pred.min():.3f}@{th[np.argmin(m_pred
    )]:.1f}h"
136       f"→{m_pred[-1]:.3f}")
137 print(f"滞后交叉 (m 由 <1 转 >1): data≈{th[i_md]:.1f} h, pred≈{th[i_mp]:.1f
    } h; "
138       f"平台: data {R_att[-1] * 100:.3f} cm, pred {R_pred[-1] * 100:.4f} cm")
139
140 # 守恒与孔隙率
141 V0 = 1 / RHO_SK + C0 / RHO_W
142 V_pred = V0 * (R_pred / R0) ** 2
143 V_sw = 1 / RHO_SK + um / RHO_W
144 V_pore = V_pred - V_sw
145 i_zero = int(np.argmax(V_pore > 1e-9))
146 print(f"\n守恒/孔隙率: V=V_s+V_w+V_pore 逐点自洽; V_pore 早期为负=基体压密
    (塌陷), "
147       f"转正于 t≈{th[i_zero]:.1f} h, 其后单调非减="
148       f"{bool(np.all(np.diff(V_pore[i_zero:]) >= -1e-9))}; "
149       f"末期孔隙率={V_pore[-1] / V_pred[-1] * 100:.1f}% (附件 2 隐含 ~31.0%) "
    )
150
151 # ----- CSV -----
152 term_c, term_p = termsC(pC, um, Cs)
153 out = np.column_stack([t_att, R_att * 100, R_pred * 100, res * 1000,
154                       R_ideal * 100, term_c * 100, term_p * 100, V_pore / V0 *
    100])
155 np.savetxt(DATA / "endogenous_calibrated.csv", out, delimiter=",",
156           header=("t_s,R_data_cm,R_pred_cm,residual_mm,R_ideal_cm,"

```

```

157         "collapse_term_cm,pore_term_cm,V_pore_frac_pct"), comments=""
158     )
159     print("written:", DATA / "endogenous_calibrated.csv")
160     # ----- 图 -----
161     import matplotlib
162     matplotlib.use("Agg")
163     import matplotlib.pyplot as plt
164     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
165     plt.rcParams["axes.unicode_minus"] = False
166     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 7), sharex=True,
167                                   gridspec_kw=dict(height_ratios=[3, 1]))
168     ax1.plot(th, R_att * 100, "k-", lw=2, label="附件 2 (实测)")
169     ax1.plot(th, R_pred * 100, "r--", lw=1.8,
170             label=f"内生闭合 C (5 参数标定, RMSE={rmse * 1000:.3f} mm)")
171     ax1.plot(th, R_ideal * 100, "b:", lw=1.4, label="失水理想收缩基线")
172     for x in (3.5, 21.0):
173         ax1.axvline(x, color="gray", alpha=0.4, ls=":")
174     ax1.annotate("A 急缩段", xy=(1.0, 1.97), fontsize=9)
175     ax1.annotate("B 缓缩段", xy=(9, 1.45), fontsize=9)
176     ax1.annotate("C 平台 (1.198 cm)", xy=(40, 1.21), fontsize=9)
177     ax1.set_ylabel("R (cm)")
178     ax1.legend(fontsize=9)
179     ax1.set_title("内生收缩闭合标定与验证 (一项一机制: 失水 / 毛细-膨压塌陷 / 结皮-成孔停滞)")
180     ax2.plot(th, res * 1000, "g-")
181     ax2.axhline(0, color="k", lw=0.5)
182     ax2.set_ylabel("残差 (mm)")
183     ax2.set_xlabel("t (h)")
184     fig.tight_layout()
185     SCRATCH.mkdir(exist_ok=True)
186     fig.savefig(SCRATCH / "内生收缩_标定验证.png", dpi=150)
187     print("written:", SCRATCH / "内生收缩_标定验证.png")
188
189     # ----- (a) t_f 一致性: R_pred 替换附件 2 重解问题 4 (自适应求解器)
190     -----
191     env_t, TaK, Ce = read_env()
192     env_hold = make_env("hold", env_t, TaK, Ce)
193     pchip_pred = PchipInterpolator(t_att, R_pred, extrapolate=False)
194     R_fun_pred = lambda tt: float(pchip_pred(min(tt, t_att[-1])))
195     r_a = solve_coupled4(80, env_hold, R_fun_pred, 600000.0, dt_max=15.0)
196     tf_a = r_a["crossing"][2] / 3600.0
197     print(f"\n[t_f 一致性 a] R_pred(C) 替换附件 2 重解 (自适应求解器, N=80): "
198           f"t_f = {tf_a:.4f} h (主解 50.8295 h, Δ = {tf_a - 50.829547:+.4f} h)",
199           flush=True)
200
201     # ----- (b) 耦合内生: R 由湿度场逐步预测, 不读附件 2 -----
202     print("[耦合内生 b] 开始 (N=80, dt=60 s, 一步对两半步外推) ...", flush=True)
203     N = 80
204     dq = 1.0 / N
205     U = np.full(N, C0)

```

```

204 T = np.full(N, 301.15)
205 t = 0.0
206 dt_fix = 60.0
207 log = []
208
209
210 def R_cpl(U_, Cs_):
211     um_ = float(np.mean(U_))
212     tc, tp_ = termsC(pC, np.array([um_]), np.array([Cs_]))
213     return float(R0 * np.sqrt((1 + um_) / (1 + C0)) * (1 + tc[0]) * (1 + tp_
        [0]))
214
215
216 cross_b = None
217 while t < 300000.0:
218     Rt0 = R_cpl(U, 1.0)
219     Cs_now, _, _ = surface_m4(U[-1], T[-1], env_hold(t)[1], dq, Rt0, D_of4)
220     Rt = R_cpl(U, Cs_now)
221     Ta_m, Ce_m = env_hold(t + dt_fix / 2)
222     Ta_e, Ce_e = env_hold(t + dt_fix)
223     Uf, Tf, *_ = coupled_step4(U, T, dt_fix, Ta_e, Ce_e, dq, True, Rt,
        PROPS4)
224     Uh, Th, *_ = coupled_step4(U, T, dt_fix / 2, Ta_m, Ce_m, dq, True, Rt,
        PROPS4)
225     Uh2, Th2, *_ = coupled_step4(Uh, Th, dt_fix / 2, Ta_e, Ce_e, dq, True,
        Rt, PROPS4)
226     U, T = 2 * Uh2 - Uf, 2 * Th2 - Tf
227     t += dt_fix
228     if len(log) == 0 or t - log[-1][0] >= 1800 - 1e-9:
229         log.append((t, Rt, float(np.mean(U)), float(np.max(U))))
230     if float(np.max(U)) < TH:
231         cross_b = t
232         break
233 log = np.array(log)
234 tf_b = cross_b / 3600.0
235 R_b_att = np.interp(t_att, log[:, 0], log[:, 1])
236 rmse_b = float(np.sqrt(np.mean((R_b_att - R_att) ** 2)))
237 print(f"[耦合内生 b] t_f = {tf_b:.4f} h (主解 50.8295 h, Δ = {tf_b -
    50.829547:+.4f} h) ; "
238     f"R(t) vs 附件 2: RMSE = {rmse_b * 1000:.4f} mm, 末期 R = {log[-1, 1] *
    100:.4f} cm")
239 np.savetxt(DATA / "endogenous_coupled.csv",
240     np.column_stack([log[:, 0], log[:, 1] * 100, log[:, 2], log[:, 3]]),
241     delimiter=",", header="t_s,R_coupled_cm,U_mean,U_max", comments="")
242 print("written:", DATA / "endogenous_coupled.csv")
243 print(f"\n[汇总] t_f: 附件 2 主解 50.8295 | R_pred 重解 {tf_a:.4f} | 耦合内生
    {tf_b:.4f} h")

```


E.5 问题 1 求解器

质量坐标 ($r = R_0\sqrt{q}$) 下的守恒型有限体积求解器。

代码 6 问题 1 求解器 (solver_q1.py, 258 行)

```
1  """问题 1: 质量坐标 ( $q \in [0,1]$ ,  $r = R_0 \cdot \sqrt{q}$ ) 有限体积求解器。
2
3  依据 00-最终方案.md §2.1 / §4:
4  - 统一形式  $U_t = \partial_q(K U_q)$ ,  $K = 4qD/R_0^2$ ;  $a_{\text{eff}} \cdot \theta_t = \partial_q(4qk/(\rho c p R_0^2) \cdot \rho c p \cdot \theta_q)$ 。
5  - 空间: 单元中心 FV (两端等价半单元), 共享通量;  $q=0$  面通量系数为 0 (正则对称自动满足)。
6  - 表面: 半单元内阻 + 外对流阻力串联, 表面值  $C_s / T_s$  由两方程联立显式重建,
7  绝不以末单元平均代替表面值。
8  - 热场 (线性,  $\rho/cp/k$  常数,  $D$  不依赖  $T \Rightarrow$  与湿分完全解耦):
9  半离散对称三对角系统, 一次性谱分解, 分段线性环境下用  $\phi$  函数精确时间积分
10 (时间积分无截断误差,  $V1$  对拍时隔离纯空间误差)。
11 - 湿分 (非线性,  $D = D(C)$ ): 后向欧拉 + 系数冻结 Picard (表面  $D_s$  内层 Newton
    ),
12 一步 vs 两半步对分估计局部误差, 自适应步长, 拒绝不截断负值、靠缩小步长。
13 """
14 from __future__ import annotations
15
16 import math
17
18 import numpy as np
19 from scipy.linalg import eigh_tridiagonal, solve_banded
20
21 # ---- 附录 2 物性 (问题 1, 全常数) ----
22 R0 = 0.02      # 半径 m
23 RHO = 820.0    # kg/m^3
24 CP = 2600.0    # J/(kg · K)
25 K = 0.36      # W/(m · K)
26 H = 25.0      # W/(m^2 · K)
27 HM = 8e-7     # m/s
28 T0_K = 301.15 # 初始温度 K (= 28 °C)
29 C0 = 2.55     # 初始干基含水率 kg/kg
30
31 ALPHA = K / (RHO * CP) # 热扩散率 m^2/s
32 BI_H = H * R0 / K      # 热 Bi 数 = 1.3889
33
34
35 def D_of_C(C):
36     """附录 2:  $D = 7e-9 \cdot \exp(-0.89/C)$  m^2/s, 只依赖 C (不依赖 T)。"""
37     return 7e-9 * np.exp(-0.89 / C)
38
39
40 def dD_dC(C):
41     return D_of_C(C) * 0.89 / C**2
42
43
44 # =====
```

```

45 # 热场：半离散线性系统 + 谱分解精确时间积分
46 # =====
47
48 def heat_modal(N: int):
49     """返回 (lam, V, w):  $dT/dt = A T + g_s \cdot T_a \cdot e_{\{N-1\}}$ ,  $A = V \text{diag}(\text{lam}) V^T$ 。
50
51      $w = g_s \cdot V[N-1,:]$  为环境温度的模态激励权重。
52     """
53     dq = 1.0 / N
54     qf = np.arange(1, N) * dq          # 内面  $q_{\{i+1/2\}}$ 
55     af = 4.0 * qf * K / R0**2          # 面通量系数  $4qk/R0^2$ 
56     off = af / (RHO * CP * dq**2)      # 对称三对角副对角元
57     diag = np.zeros(N)
58     diag[:-1] -= off
59     diag[1:] -= off
60     # 表面：半单元内阻  $R0^2 \cdot dq/(8k)$  与外阻  $R0/(2h)$  串联
61     Rs = R0**2 * dq / (8.0 * K) + R0 / (2.0 * H)
62     gs = 1.0 / (Rs * RHO * CP * dq)
63     diag[-1] -= gs
64     lam, V = eigh_tridiagonal(diag, off)
65     w = V[-1, :] * gs
66     return lam, V, w
67
68
69 def heat_advance(y, lam, w, dt, Ta0, Ta1):
70     """模态坐标精确推进 dt, 环境温度在 [t, t+dt] 内线性:  $T_a(0)=Ta0$ ,  $T_a(dt)=Ta1$ 。
71     """
72     e = np.expml(lam * dt)
73     E1 = e / lam          #  $\int_0^{dt} e^{\{\lambda(dt-s)\}} ds$ 
74     E2 = (e - lam * dt) / lam**2 #  $\int_0^{dt} e^{\{\lambda(dt-s)\}} s ds$ 
75     m = (Ta1 - Ta0) / dt
76     return y * np.exp(lam * dt) + w * (Ta0 * E1 + m * E2)
77
78 def heat_surface(T_last, Ta, N):
79     """由末单元中心值与串联阻力重建表面温度（不用末单元平均冒充表面值）。"""
80     dq = 1.0 / N
81     Rs_int = R0**2 * dq / (8.0 * K)
82     Rs = Rs_int + R0 / (2.0 * H)
83     Fs = (Ta - T_last) / Rs
84     return T_last + Fs * Rs_int
85
86
87 # =====
88 # 湿分：后向欧拉 + Picard + 自适应步长
89 # =====
90
91 def _surface_state(CN, Cenv, dq, Dfun, dDfun):
92     """给定末单元中心值 CN, 联立求解表面值 C_s 与通量 F_s（内阻+外阻串联）。
93
94      $F = 8 D(C_s)(C_s-CN)/(R0^2 \cdot dq) = 2 h_m (Cenv-C_s)/R0$ , 对  $C_s$  做 Newton。

```

```

95     """
96     Cs = CN
97     for _ in range(60):
98         Ds = Dfun(Cs)
99         g = 8.0 * Ds * (Cs - CN) / (R0**2 * dq) - 2.0 * HM * (Cenv - Cs) / R0
100         gp = 8.0 * (dDfun(Cs) * (Cs - CN) + Ds) / (R0**2 * dq) + 2.0 * HM /
            R0
101         d = g / gp
102         Cs -= d
103         if abs(d) < 1e-13:
104             break
105     Ds = Dfun(Cs)
106     Fs = 2.0 * HM * (Cenv - Cs) / R0
107     return Cs, Fs, Ds
108
109
110 def be_step(C, dt, Cenv, dq, Dfun, dDfun):
111     """一个后向欧拉步: 系数冻结 Picard 迭代至  $\max|\Delta C| < 1e-11$ 。
112
113     返回 (Cnew, Cs, Fs): Cs/Fs 为收敛状态下独立 Newton 重建的表面值与 Robin 通
        量,
114     供守恒恒等式校核使用。
115     """
116     N = C.size
117     qf = np.arange(1, N) * dq
118     Cnew = C.copy()
119     for _ in range(60):
120         Cf = 0.5 * (Cnew[:-1] + Cnew[1:])
121         Df = Dfun(Cf)
122         af = 4.0 * qf * Df / R0**2
123         b = af / dq**2 # 长度 N-1
124         _, _, Ds = _surface_state(Cnew[-1], Cenv, dq, Dfun, dDfun)
125         Rs = R0**2 * dq / (8.0 * Ds) + R0 / (2.0 * HM)
126         gs = 1.0 / (Rs * dq)
127         diag = 1.0 + dt * (np.concatenate(([0.0], b)) + np.concatenate((b,
            [0.0])))
128         diag[-1] += dt * gs
129         rhs = C.copy()
130         rhs[-1] += dt * gs * Cenv
131         ab = np.zeros((3, N))
132         ab[0, 1:] = -dt * b
133         ab[1, :] = diag
134         ab[2, :-1] = -dt * b
135         Cnext = solve_banded((1, 1), ab, rhs)
136         dmax = float(np.max(np.abs(Cnext - Cnew)))
137         Cnew = Cnext
138         if dmax < 1e-11:
139             break
140     else:
141         raise RuntimeError("Picard 未收敛, 应缩小步长重试")
142     Cs, Fs, _ = _surface_state(Cnew[-1], Cenv, dq, Dfun, dDfun)

```

```

143     return Cnew, Cs, Fs
144
145
146 def solve_moisture(N, cenv_fun, t_end, out_times=(), Dfun=D_of_C, dDfun=
    dD_dC,
147                     atol=1e-11, rtol=3e-10, dt0=1e-4):
148     """湿分场自适应求解。
149
150     cenv_fun(t) → C_env(t); out_times 为需要精确落点记录的时刻序列。
151     返回 dict: outputs {t: C}, surface {t: (Cs, Fs)}, stats (步数/拒步/守恒残
        差)。
152
153     """
154     dq = 1.0 / N
155     C = np.full(N, C0)
156     t = 0.0
157     dt = dt0
158     out_list = sorted(out_times)
159     i_out = 0
160     outputs, surface = {}, {}
161     n_steps = n_rej = 0
162     max_cons_res = 0.0 # 每个输出窗口  $|\Sigma \Delta m - \Sigma f F_s dt| / |\Sigma f F_s dt|$ 
163     win_dm, win_fl = [], [] # fsum 累加器 (窗口 = 相邻记录时刻之间)
164     win_end = out_list[0] if out_list else min(1.0, t_end)
165     while t < t_end - 1e-12:
166         target = t + dt
167         if i_out < len(out_list):
168             target = min(target, out_list[i_out])
169         target = min(target, t_end)
170         dtc = target - t
171         if dtc <= 1e-15:
172             outputs[out_list[i_out]] = C.copy()
173             i_out += 1
174             continue
175         Cenv_mid = cenv_fun(t + 0.5 * dtc)
176         Cenv_end = cenv_fun(t + dtc)
177         # 一步 vs 两半步
178         Cf, _, _ = be_step(C, dtc, Cenv_end, dq, Dfun, dDfun)
179         Ch, _, Fs1 = be_step(C, 0.5 * dtc, Cenv_mid, dq, Dfun, dDfun)
180         Ch2, Cs2, Fs2 = be_step(Ch, 0.5 * dtc, Cenv_end, dq, Dfun, dDfun)
181         n_steps += 3
182         err = float(np.max(np.abs(Ch2 - Cf))) # ≈ 半对结果的局部误差 (BE 一阶)
183         tol = atol + rtol * float(np.max(np.abs(C)))
184         if err <= tol:
185             win_dm.append(dq * math.fsum(float(x) for x in (Ch2 - C)))
186             win_fl.append(0.5 * dtc * float(Fs1))
187             win_fl.append(0.5 * dtc * float(Fs2))
188             if t + dtc >= win_end - 1e-12: # 窗口关闭, 结算守恒残差
189                 dm_w = math.fsum(win_dm)
190                 fl_w = math.fsum(win_fl)
191                 if abs(fl_w) > 0:
192                     max_cons_res = max(max_cons_res, abs(dm_w - fl_w) / abs(fl_w)

```

```

    ))
192     win_dm, win_fl = [], []
193     k = i_out + 1
194     win_end = out_list[k] if k < len(out_list) else min(win_end +
        1.0, t_end)
195     C = Ch2
196     t += dtc
197     if i_out < len(out_list) and abs(t - out_list[i_out]) < 1e-12:
198         outputs[out_list[i_out]] = C.copy()
199         surface[out_list[i_out]] = (Cs2, Fs2)
200         i_out += 1
201     fac = 0.9 * np.sqrt(tol / err) if err > 0 else 2.0
202     dt = dtc * min(2.0, max(0.3, fac))
203     else:
204         n_rej += 1
205         dt = dtc * max(0.2, 0.9 * np.sqrt(tol / err))
206     stats = dict(n_steps=n_steps, n_rej=n_rej, max_cons_res=max_cons_res)
207     return dict(C=C, outputs=outputs, surface=surface, stats=stats)
208
209
210 # =====
211 # 输出重建: 单元中心值 + 正则性中心值 + 串联阻力表面值, q 内二次插值
212 # =====
213
214 def build_interp_weights(N, q_out, v_surf=None):
215     """为每个输出 q* 预计算 3 节点二次 Lagrange 权重。
216
217     节点集: q=0 (中心, 权重处理在 apply 中)、单元中心 q_i=(i+0.5)/N、q=1 (表
        面)。
218     返回 (idx, W):  $v(q) \approx \sum_k W[:,k] \cdot \text{values}[\text{idx}[:,k]]$ , idx=-1 表示中心节
        点,
219     idx=N 表示表面节点。
220     """
221     dq = 1.0 / N
222     q_nodes = np.concatenate(([0.0], (np.arange(N) + 0.5) * dq, [1.0]))
223     idx = np.zeros((len(q_out), 3), dtype=int)
224     W = np.zeros((len(q_out), 3))
225     for j, q in enumerate(q_out):
226         p = int(np.searchsorted(q_nodes, q))
227         s = min(max(p - 1, 0), len(q_nodes) - 3)
228         cand = [s, s + 1, s + 2]
229         qs = q_nodes[cand]
230         w = np.ones(3)
231         for a in range(3):
232             for b_ in range(3):
233                 if a != b_:
234                     w[a] *= (q - qs[b_]) / (qs[a] - qs[b_])
235         idx[j] = [c - 1 for c in cand] # 节点 0 → -1 (中心), 节点 N+1 → N (表
            面)
236         W[j] = w
237     return idx, W

```

```

238
239
240 def apply_interp(values, v_surf, idx, W):
241     """values (... , N) 单元中心值; v_surf (...) 表面值。返回 (... , len(idx))。
        """
242     N = values.shape[-1]
243     n_out = idx.shape[0]
244     lead = values.shape[:-1]
245     out = np.zeros(lead + (n_out,))
246     for j in range(n_out):
247         acc = np.zeros(lead)
248         for k in range(3):
249             i = idx[j, k]
250             if i == -1: # 中心: 正则性外推  $v(0) = 1.5 v_0 - 0.5 v_1$ 
251                 v = 1.5 * values[..., 0] - 0.5 * values[..., 1]
252             elif i == N: # 表面
253                 v = v_surf
254             else:
255                 v = values[..., i]
256             acc = acc + W[j, k] * v
257         out[..., j] = acc
258     return out

```

E.6 问题 1 求解与检验流程

问题 1 一条龙: 求解 + V1 解析对照 + 守恒与退化检验 + 导出 result1.xlsx。

代码 7 问题 1 求解与检验流程 (run_q1.py, 312 行)

```

1  """问题 1 一条龙: 求解 + V1 解析对拍 + 守恒门禁 + 退化测试 + 导出 result1.xlsx
   。
2
3  用法:  python run_q1.py
4  产物:
5      ../结果/result1.xlsx          (温度/水分浓度, 1-1800 s × 0-2.0 cm, 四位小
        数)
6      ../数据/v1_convergence.csv    (热场 V1 对拍收敛序列)
7      ../数据/v1_points_final.csv   (最终网格 5 位置 × 7 时刻逐点偏差)
8      ../数据/conservation.csv      (湿分守恒门禁三档网格)
9      ../数据/degenerate_constD.csv (D=const 退化 vs Bessel 级数)
10     ../数据/q1_fields.npz         (最终全场数据)
11 """
12 from __future__ import annotations
13
14 import hashlib
15 import io
16 import sys
17 import zipfile
18 from pathlib import Path
19
20 import numpy as np
21 import openpyxl

```

```

22
23 CODE_DIR = Path(__file__).resolve().parent
24 sys.path.insert(0, str(CODE_DIR))
25
26 from solver_q1 import (ALPHA, BI_H, C0, D_of_C, HM, K, R0, T0_K,
27                        apply_interp, build_interp_weights, heat_advance,
28                        heat_modal, heat_surface, solve_moisture)
29 from analytic_heat import RobinCylinder
30
31 A_DIR = CODE_DIR.parent
32 ENV_XLSX = A_DIR / "01-题目" / "原始文件" / "附件1.xlsx"
33 TEMPLATE = A_DIR / "01-题目" / "原始文件" / "附件3" / "result1.xlsx"
34 OUT_XLSX = A_DIR / "结果" / "result1.xlsx"
35 DATA_DIR = A_DIR / "数据"
36
37 T_END = 1800
38 V1_GRID = [20, 40, 80, 160, 320]
39 N_FINAL = 320
40 V1_TIMES = [100, 300, 600, 900, 1200, 1500, 1800]
41 POS_CM = np.arange(0.0, 2.01, 0.1) # 21 列
42 Q_OUT = (POS_CM * 0.01 / R0) ** 2
43 CHECK_TIMES = [100, 600, 1800]
44 CHECK_POS_M = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
45
46
47 def read_env():
48     wb = openpyxl.load_workbook(ENV_XLSX, read_only=True)
49     ws = wb[wb.sheetnames[0]]
50     rows = [(r[0], r[1], r[2]) for r in ws.iter_rows(min_row=2, values_only=
51                                                       True)]
52     if r[0] is not None and r[0] <= T_END]
53     wb.close()
54     t = np.array([r[0] for r in rows], dtype=float)
55     Ta_C = np.array([r[1] for r in rows], dtype=float)
56     Cenv = np.array([r[2] for r in rows], dtype=float)
57     assert t[0] == 0 and abs(t[-1] - T_END) < 1e-9 and len(t) == 31
58     # 量纲防呆: 附件 1 温度是摄氏; 若误加开尔文会超界
59     assert -50.0 < Ta_C.min() and Ta_C.max() < 150.0, "环境温度量纲异常 (应为
60     数十℃)"
61     Ta_K = Ta_C + 273.15
62     assert Ta_K.min() > 200.0, "温度必须用开尔文参与计算"
63     return t, Ta_K, Cenv
64
65 def heat_series(N, env_t, env-Ta, out_times):
66     """数值热场: 谱分解 + 精确时间积分, 返回 out_times 时刻的 (M, N) 单元中心
67     值。"""
68     lam, V, w = heat_modal(N)
69     y = V.T @ (np.full(N, T0_K))
70     seg = 0
71     t_cur = float(env_t[0])

```

```

70 out = np.zeros((len(out_times), N))
71
72 def Ta_at(t): # 段内线性
73     return env_Ta[seg] + (env_Ta[seg + 1] - env_Ta[seg]) * \
74         (t - env_t[seg]) / (env_t[seg + 1] - env_t[seg])
75
76 for j, tau in enumerate(out_times):
77     while t_cur < tau - 1e-12:
78         t_nxt = min(tau, float(env_t[seg + 1]))
79         y = heat_advance(y, lam, w, t_nxt - t_cur, Ta_at(t_cur), Ta_at(
80             t_nxt))
81         t_cur = t_nxt
82         if t_nxt >= float(env_t[seg + 1]) - 1e-12 and seg + 1 < len(env_t)
83             - 1:
84             seg += 1
85         out[j] = V @ y
86     return out
87
88 def v1_study(env_t, env_Ta):
89     ana = RobinCylinder(BI_H, ALPHA, R0, n_modes=300)
90     print(f"[V1] 特征展开自检:  $\Sigma A_n(\text{中心})=\{ana.sum\_center:.10f\}$ , "
91         f" $\Sigma A_n J_0(\lambda_n)(\text{表面})=\{ana.sum\_surface:.10f\}$ ")
92     g = env_Ta - T0_K
93     rows = []
94     for N in V1_GRID:
95         r_c = R0 * np.sqrt((np.arange(N) + 0.5) / N)
96         T_num = heat_series(N, env_t, env_Ta, CHECK_TIMES)
97         T_ana = ana.eval(r_c, CHECK_TIMES, env_t, g, T0_K)
98         errs = np.max(np.abs(T_num - T_ana), axis=1)
99         rows.append((N, *errs))
100     print(f"[V1] N={N:4d} 单元中心逐点最大偏差(°): "
101         + " ".join(f"t={t}:{e:.3e}" for t, e in zip(CHECK_TIMES, errs)))
102     orders = []
103     for k in range(len(CHECK_TIMES)):
104         e = np.array([r[k + 1] for r in rows])
105         ns = np.array([r[0] for r in rows])
106         p = -np.polyfit(np.log(ns), np.log(e), 1)[0]
107         orders.append(p)
108     print(f"[V1] 实测收敛阶(全段拟合): "
109         + " ".join(f"t={t}:{p:.3f}" for t, p in zip(CHECK_TIMES, orders)))
110     return rows, orders, ana
111
112 def v1_points_final(env_t, env_Ta, ana):
113     """最终网格下 5 位置 (含中心与表面重建) × 7 时刻逐点偏差。"""
114     N = N_FINAL
115     T_num = heat_series(N, env_t, env_Ta, V1_TIMES) # (7, N)
116     env_at = np.interp(V1_TIMES, env_t, env_Ta)
117     T_surf = np.array([heat_surface(T_num[j, -1], env_at[j], N)
118         for j in range(len(V1_TIMES))])

```



```

119     idx, W = build_interp_weights(N, (CHECK_POS_M / R0) ** 2)
120     T_pos = apply_interp(T_num, T_surf, idx, W) # (7, 5)
121     g = env-Ta - T0_K
122     T_ana = ana.eval(CHECK_POS_M, V1_TIMES, env_t, g, T0_K)
123     dev = np.abs(T_pos - T_ana)
124     print(f"[V1] N={N} 5 位置逐点最大偏差: {dev.max():.3e} °C")
125     return T_pos, T_ana, dev
126
127
128 def write_xlsx_deterministic(path, sheets):
129     """sheets: {name: (header_row, data_matrix)}, 四位小数写值, zip 重排去时间戳。"""
130     wb = openpyxl.Workbook()
131     wb.properties.created = wb.properties.modified = __import__("datetime").datetime(2000, 1, 1)
132     wb.properties.creator = "run_q1"
133     wb.properties.lastModifiedBy = "run_q1"
134     first = True
135     for name, (header, mat) in sheets.items():
136         ws = wb.active if first else wb.create_sheet()
137         first = False
138         ws.title = name
139         for c, v in enumerate(header, start=1):
140             ws.cell(row=1, column=c, value=v)
141         for i in range(mat.shape[0]):
142             for j in range(mat.shape[1]):
143                 cell = ws.cell(row=i + 2, column=j + 1,
144                               value=round(float(mat[i, j]), 4))
145                 if j >= 1:
146                     cell.number_format = "0.0000"
147     buf = io.BytesIO()
148     wb.save(buf)
149     raw = zipfile.ZipFile(io.BytesIO(buf.getvalue()))
150     fixed = io.BytesIO()
151     with zipfile.ZipFile(fixed, "w", zipfile.ZIP_DEFLATED) as z:
152         for info in sorted(raw.infolist(), key=lambda x: x.filename):
153             zi = zipfile.ZipInfo(info.filename, date_time=(1980, 1, 1, 0, 0, 0))
154             zi.compress_type = zipfile.ZIP_DEFLATED
155             zi.external_attr = 0o600 << 16
156             data = raw.read(info.filename)
157             if info.filename == "docProps/core.xml":
158                 # openpyxl 保存时会把 modified 改写为当前时间, 固定回去
159                 import re
160                 data = re.sub(rb"(<dc:terms:(?:created|modified)[^>]*>)[^<]*(</dc:terms:(?:created|modified)>)",
161                               rb"\g<1>2000-01-01T00:00:00Z\g<2>", data)
162             z.writestr(zi, data)
163     path.write_bytes(fixed.getvalue())
164     return hashlib.md5(fixed.getvalue()).hexdigest()
165

```

```

166
167 def check_against_template(path):
168     tpl = openpyxl.load_workbook(TEMPLATE)
169     out = openpyxl.load_workbook(path)
170     ok = True
171     def chk(cond, msg):
172         nonlocal ok
173         print((" [OK] " if cond else " [FAIL] ") + msg)
174         ok = ok and cond
175     chk(tpl.sheetnames == out.sheetnames, f"sheet 名一致: {out.sheetnames}")
176     for name in tpl.sheetnames:
177         wt, wo = tpl[name], out[name]
178         chk(wo.cell(1, 1).value == wt.cell(1, 1).value,
179             f"[{name}] A1 列表头 = {wo.cell(1, 1).value!r}")
180         hdr = [wo.cell(1, c).value for c in range(2, 23)]
181         expect = [round(0.1 * i, 10) for i in range(21)]
182         chk(all(abs(h - e) < 1e-12 for h, e in zip(hdr, expect)),
183             f"[{name}] 距离列头 0,0.1,...,2.0 共 21 列")
184         chk(wo.max_row == 1801 and wo.max_column == 22,
185             f"[{name}] 尺寸 {wo.max_row}x{wo.max_column} (1801x22) ")
186         times = [wo.cell(r, 1).value for r in range(2, 1802)]
187         chk(times[0] == 1 and times[-1] == 1800 and times[1] - times[0] == 1,
188             f"[{name}] A 列时间 1-1800 s 步长 1")
189         bad4 = 0
190         for row in wo.iter_rows(min_row=2, min_col=2, max_col=22, values_only
191                                 =True):
192             for v in row:
193                 if v is None or abs(v - round(v, 4)) > 1e-12:
194                     bad4 += 1
195         chk(bad4 == 0, f"[{name}] 全部数据单元格为四位小数 (异常 {bad4}) ")
196     return ok
197
198 def main():
199     DATA_DIR.mkdir(exist_ok=True)
200     OUT_XLSX.parent.mkdir(exist_ok=True)
201     env_t, env-Ta, env-C = read_env()
202     print(f"环境数据: {len(env_t)} 点, 0-{env_t[-1]:.0f} s, "
203           f"T_a {env-Ta[0]-273.15:.3f}→{env-Ta[-1]-273.15:.3f} °C, "
204           f"C-env {env-C[0]:.5f}→{env-C[-1]:.5f}")
205
206     # ----- V1: 解析热场对拍 -----
207     conv_rows, orders, ana = v1_study(env_t, env-Ta)
208     with open(DATA_DIR / "v1_convergence.csv", "w", encoding="utf-8") as f:
209         f.write("N," + ",".join(f"maxerr-t{t}s" for t in CHECK_TIMES) + "\n")
210         for r in conv_rows:
211             f.write(",".join([str(r[0])] + [f"{e:.6e}" for e in r[1:]]) + "\n"
212                           )
213             f.write("order(fit)," + ",".join(f"{p:.4f}" for p in orders) + "\n")
214     T_pos, T-ana_pos, dev_pos = v1_points_final(env_t, env-Ta, ana)

```

```

215 with open(DATA_DIR / "v1_points_final.csv", "w", encoding="utf-8") as f:
216     f.write("t_s,r_cm,T_num_C,T_analytic_C,abs_dev\n")
217     for j, t in enumerate(V1_TIMES):
218         for k, r in enumerate(CHECK_POS_M):
219             f.write(f"{t},{r*100:.1f},{T_pos[j,k]-273.15:.8f},"
220                 f"{T_ana_pos[j,k]-273.15:.8f},{dev_pos[j,k]:.3e}\n")
221
222 # ----- 最终热场（逐秒） -----
223 out_seconds = np.arange(1, T_END + 1, dtype=float)
224 T_all = heat_series(N_FINAL, env_t, env-Ta, out_seconds) # K, (1800, N)
225 assert 200.0 < T_all.min() and T_all.max() < 400.0, "温度必须处于开尔文合
    理区间"
226 Ta_sec = np.interp(out_seconds, env_t, env-Ta)
227 Ts_sec = np.array([heat_surface(T_all[j, -1], Ta_sec[j], N_FINAL)
228                     for j in range(len(out_seconds))])
229 idx, W = build_interp_weights(N_FINAL, Q_OUT)
230 T_cols = apply_interp(T_all, Ts_sec, idx, W) - 273.15 # °C, (1800, 21)
231
232 # ----- 湿分场（逐秒, N=320） -----
233 cenv_fun = lambda t: float(np.interp(t, env_t, env_C))
234 print(f"[湿分] N={N_FINAL} 自适应求解中 ...")
235 res = solve_moisture(N_FINAL, cenv_fun, T_END, out_times=out_seconds)
236 st = res["stats"]
237 print(f"[湿分] 步数 {st['n_steps']} (拒 {st['n_rej']}) , "
238       f"逐步守恒相对残差 max = {st['max_cons_res']:.3e}")
239 C_all = np.array([res["outputs"][t] for t in out_seconds])
240 Cs_sec = np.array([res["surface"][t][0] for t in out_seconds])
241 C_cols = apply_interp(C_all, Cs_sec, idx, W)
242 assert C_all.min() > 0 and C_all.max() <= C0 + 1e-9, "湿分越界"
243
244 # ----- 守恒门禁：三档网格 -----
245 cons_rows = []
246 for N in (20, 80, 320):
247     r_ = res if N == N_FINAL else solve_moisture(N, cenv_fun, T_END)
248     s = r_["stats"]
249     cons_rows.append((N, s["n_steps"], s["n_rej"], s["max_cons_res"]))
250     print(f"[守恒] N={N:4d}: 逐步相对残差 max = {s['max_cons_res']:.3e}")
251 with open(DATA_DIR / "conservation.csv", "w", encoding="utf-8") as f:
252     f.write("N,n_steps,n_reject,max_rel_residual\n")
253     for r in cons_rows:
254         f.write(f"{r[0]},{r[1]},{r[2]},{r[3]:.6e}\n")
255
256 # ----- 退化自检: D=const 退回 Bessel 级数 -----
257 D0 = float(D_of_C(np.array(C0)))
258 Bi_m = HM * R0 / D0
259 ana_c = RobinCylinder(Bi_m, D0, R0, n_modes=300)
260 t_test = np.array([3e4, 6e4, 1.2e5, 2.4e5])
261 Cenv0 = float(env_C[0])
262 Dconst = lambda C: np.full_like(np.asarray(C, dtype=float), D0)
263 dDzero = lambda C: np.zeros_like(np.asarray(C, dtype=float))
264 deg = solve_moisture(160, lambda t: Cenv0, t_test[-1], out_times=t_test,

```

```

265         Dfun=Dconst, dDfun=dDzero, rtol=1e-9)
266 r_c160 = R0 * np.sqrt((np.arange(160) + 0.5) / 160)
267 C_ana = ana_c.eval(r_c160, t_test, [0.0, t_test[-1]],
268                  [Cenv0 - C0, Cenv0 - C0], C0)
269 deg_errs = [float(np.max(np.abs(deg["outputs"][t] - C_ana[j])))
270              for j, t in enumerate(t_test)]
271 mean_num = np.mean(deg["outputs"][t_test[-1]])
272 ratio = (mean_num - Cenv0) / (C0 - Cenv0)
273 first = ana_c.A[0] * ana_c.mean_mode[0] * np.exp(-ana_c.mu[0] * t_test
274            [-1])
275 print(f"[退化] D=const={D0:.3e}, Bi_m={Bi_m:.3f}; 逐点最大偏差: "
276       + " ".join(f"t={t:.0f} {e:.3e}" for t, e in zip(t_test, deg_errs)))
277 print(f"[退化] t={t_test[-1]:.0f}s 平均残余比 {ratio:.6f}, 首模预测 {first
278       :.6f}, "
279       f"比值 {ratio/first:.5f}")
280 with open(DATA_DIR / "degenerate_constD.csv", "w", encoding="utf-8") as
281     f:
282     f.write("t_s,max_abs_dev\n")
283     for t, e in zip(t_test, deg_errs):
284         f.write(f"{t:.0f},{e:.6e}\n")
285     f.write(f"# mean_ratio={ratio:.8f},first_mode={first:.8f},quotient={
286           ratio/first:.6f}\n")
287
288 # ----- A6: 中心列不得恒为常数 -----
289 print(f"[自检] 温度中心列: t=100s {T_cols[99,0]:.4f} °C, "
290       f"t=1800s {T_cols[1799,0]:.4f} °C (解析 {T_ana_pos[0,0]-273.15:.4f} /
291       "
292       f"{T_ana_pos[-1,0]-273.15:.4f}) ")
293 assert T_cols[1799, 0] > T_cols[99, 0] + 0.1, "中心列异常: 热场未传向中心"
294 print(f"[自检] 水分中心列: t=1800s {C_cols[1799,0]:.4f} "
295       f"(渗透 ~3mm, 中心应基本不变); 表面列 t=1800s {C_cols[1799,-1]:.4f}"
296       )
297
298 # ----- 导出 result1.xlsx -----
299 time_col = np.arange(1, T_END + 1, dtype=float)
300 header = ["时间\\到药材中心的距离"] + [round(0.1 * i, 10) for i in range
301      (21)]
302 mat_T = np.column_stack([time_col, T_cols])
303 mat_C = np.column_stack([time_col, C_cols])
304 md5 = write_xlsx_deterministic(OUT_XLSX,
305                                {"温度": (header, mat_T), "水分浓度": (header,
306                                mat_C)})
307 print(f"[导出] {OUT_XLSX} md5={md5}")
308 print("[核对] 与附件3模板逐项比对: ")
309 ok = check_against_template(OUT_XLSX)
310 assert ok
311
312 np.savez(DATA_DIR / "q1_fields.npz",
313          times=out_seconds, pos_cm=POS_CM, T_C=T_cols, C=C_cols,
314          T_cell=T_all, C_cell=C_all, Ts=Ts_sec, Cs=C_sec,
315          env_t=env_t, env-Ta=env-Ta, env_C=env_C)

```

```

308     print("[完成] 全部产物已落盘。")
309
310
311 if __name__ == "__main__":
312     main()

```

E.7 问题 2/3 求解与检验流程

问题 2/3 一条龙：双向耦合求解 + t_f 主结果 + 各项门禁 + result2/result3。

代码 8 问题 2/3 求解与检验流程 (run_q23.py, 384 行)

```

1  """问题 2/3 一条龙：双向耦合求解 + t_f 主结果 + 全部门禁 + result2/result3。
2
3  用法： python run_q23.py
4  架构（降耗）：内部自适应大步长（尾部 ~10 s），逐接受步记录 21 列重构值；
5  result2 的 1 s 网格与 result3 的 60 s 网格均由步记录线性插值重构（场变化慢，
6  插值误差 < 四位小数舍入，时间误差预算另有 rtol/10 实测约束）。
7  主网格 N=160（T1 已证二阶收敛，N=320 仅余量）；收敛序列/外推三档用 N=40/80。
8
9  产物：
10  ../结果/result3.xlsx （Sheet1, 60 s × 21 列，至 t_f + 追加"烘干结束时间"
    行）
11  ../结果/result2.xlsx （温度/水分浓度, 1 s × 21 列，终点 = t_f + 3600 s 余
    量）
12  ../数据/{tf_convergence,extrapolation,conservation23,v6_fields,v3_bound}.
    csv
13  ../数据/q23_summary.json, q23_main_steps.npz
14  """
15  from __future__ import annotations
16
17  import hashlib
18  import io
19  import json
20  import re
21  import sys
22  import time
23  import zipfile
24  from pathlib import Path
25
26  import numpy as np
27  import openpyxl
28  from scipy.optimize import brentq
29
30  CODE_DIR = Path(__file__).resolve().parent
31  sys.path.insert(0, str(CODE_DIR))
32
33  from solver_q1 import R0, H, HM, T0_K, C0, build_interp_weights,
    apply_interp
34  from analytic_heat import RobinCylinder
35  from solver_q23 import TH, D_of, solve_coupled
36

```

```

37 A_DIR = CODE_DIR.parent
38 ENV_XLSX = A_DIR / "01-题目" / "原始文件" / "附件1.xlsx"
39 TPL_DIR = A_DIR / "01-题目" / "原始文件" / "附件3"
40 OUT2 = A_DIR / "结果" / "result2.xlsx"
41 OUT3 = A_DIR / "结果" / "result3.xlsx"
42 DATA_DIR = A_DIR / "数据"
43
44 N_MAIN = 160
45 MARGIN = 3600.0
46 POS_CM = np.arange(0.0, 2.01, 0.1)
47 Q_OUT = (POS_CM * 0.01 / R0) ** 2
48 HEADER = ["时间\\到药材中心的距离"] + [round(0.1 * i, 10) for i in range(21)
    ]
49 V6_TIMES = np.array([3600.0, 36000.0, 108000.0, 180000.0])
50
51
52 def read_env():
53     wb = openpyxl.load_workbook(ENV_XLSX, read_only=True)
54     ws = wb[wb.sheetnames[0]]
55     rows = [(r[0], r[1], r[2]) for r in ws.iter_rows(min_row=2, values_only=
        True)]
56     if r[0] is not None]
57     wb.close()
58     t = np.array([r[0] for r in rows], dtype=float)
59     TaC = np.array([r[1] for r in rows], dtype=float)
60     Ce = np.array([r[2] for r in rows], dtype=float)
61     assert len(t) == 241 and t[0] == 0 and t[-1] == 14400
62     assert -50.0 < TaC.min() and TaC.max() < 150.0, "附件1温度应为摄氏"
63     return t, TaC + 273.15, Ce
64
65
66 def make_env(mode, t, TaK, Ce):
67     """t>14400 s 外推三档: hold=末值保持(主); linear=末 1 h 斜率线性外推;
        mean1h=末 1 h 均值."""
68     t_end = t[-1]
69     i0 = int(np.searchsorted(t, t_end - 3600.0))
70     if mode == "linear":
71         sT = (TaK[-1] - TaK[i0]) / (t[-1] - t[i0])
72         sC = (Ce[-1] - Ce[i0]) / (t[-1] - t[i0])
73     elif mode == "mean1h":
74         mT, mC = float(np.mean(TaK[i0:])), float(np.mean(Ce[i0:]))
75
76     def f(tt):
77         if tt <= t_end:
78             return (float(np.interp(tt, t, TaK)), float(np.interp(tt, t, Ce)))
79         if mode == "hold":
80             return (float(TaK[-1]), float(Ce[-1]))
81         if mode == "linear":
82             return (float(TaK[-1] + sT * (tt - t_end)),
83                     max(float(Ce[-1] + sC * (tt - t_end)), 0.0))
84     return (mT, mC)

```

```

85     return f
86
87
88 def v3_bound(t, TaK, Ce):
89     """比较下界:  $D_{\max}$ =路径点态最大 ( $C \leq C_0$ 、 $T \leq \max(T_0, \max Ta)$ ),  $\lambda_1/A_1$  为
90      $Bi_{\max}=h_m R_0/D_{\max}$  的圆柱 Robin 第一根/中心响应系数; 另给精确级数穿越与膜界。
91     """
92     Ta_max = max(float(TaK.max()), T0_K)
93     D_max = float(D_of(C0, Ta_max))
94     Bi = HM * R0 / D_max
95     Ce_end = float(Ce[-1])
96     rc = RobinCylinder(Bi, D_max, R0, n_modes=300)
97     lam1, A1 = rc.lam[0], rc.A[0]
98     t_lb = R0**2 / (D_max * lam1**2) * np.log(A1 * (C0 - Ce_end) / (TH - Ce_end))
99     g = [Ce_end - C0, Ce_end - C0]
100
101     def w_center(tt):
102         return float(rc.eval(np.array([0.0]), np.array([tt]), [0.0, tt + 1.0], g, C0)[0, 0])
103
104     t_ref = brentq(lambda x: w_center(x) - TH, t_lb * 0.5, t_lb * 1.5, xtol=1e-6)
105     t_film = R0 / (2 * HM) * np.log((C0 - Ce_end) / (TH - Ce_end))
106     ratio2 = abs(rc.A[1]) * np.exp(-(rc.mu[1] - rc.mu[0]) * t_ref) / rc.A[0]
107     return dict(D_max=D_max, Bi_m=Bi, lam1=float(lam1), A1=float(A1),
108                 t_lb_h=float(t_lb / 3600), t_ref_s=float(t_ref),
109                 t_ref_h=float(t_ref / 3600), t_film_h=float(t_film / 3600),
110                 Ta_max_K=Ta_max, second_mode_ratio=float(ratio2))
111
112 def run_full(N, env, rtol=1e-7, newton_below=0.6, dt_max=15.0, margin=0.0):
113     """全程运行: 逐接受步记录 (t, 21列U, 21列T, umax)。返回 dict。"""
114     idx, W = build_interp_weights(N, Q_OUT)
115     log_t, log_C, log_T, log_um = [], [], [], []
116
117     def record(tt, U, T, Us, Ts):
118         log_t.append(tt)
119         log_C.append(apply_interp(U, Us, idx, W))
120         log_T.append(apply_interp(T, Ts, idx, W) - 273.15)
121         log_um.append(max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1]))
122
123     res = solve_coupled(N, env, 300000.0, out_times=(), dt_max=dt_max,
124                         rtolU=rtol, rtolT=rtol, newton_below=newton_below,
125                         margin=margin, record=record, record_each=True)
126     return dict(res=res, t=np.array(log_t),
127                 C=np.array(log_C), T=np.array(log_T), um=np.array(log_um))
128
129
130 def interp_rows(log_t, log_V, times):
131     """步记录 → 指定时刻线性插值 (times 升序, 须在记录范围内)。"""

```

```

132 times = np.asarray(times, dtype=float)
133 i = np.clip(np.searchsorted(log_t, times), 1, len(log_t) - 1)
134 t0, t1 = log_t[i - 1], log_t[i]
135 w = np.where(t1 > t0, (times - t0) / np.maximum(t1 - t0, 1e-300), 0.0)
136 return log_V[i - 1] + w[:, None] * (log_V[i] - log_V[i - 1])
137
138
139 def write_xlsx_stream(path, sheets):
140     """sheets: [(name, header, row_iter)]; openpyxl write_only 流式 + zip 确
        定性重排。"""
141     wb = openpyxl.Workbook(write_only=True)
142     for name, header, rows in sheets:
143         ws = wb.create_sheet()
144         ws.title = name
145         ws.append(header)
146         for row in rows:
147             ws.append(row)
148     buf = io.BytesIO()
149     wb.save(buf)
150     raw = zipfile.ZipFile(io.BytesIO(buf.getvalue()))
151     fixed = io.BytesIO()
152     with zipfile.ZipFile(fixed, "w", zipfile.ZIP_DEFLATED) as z:
153         for info in sorted(raw.infolist(), key=lambda x: x.filename):
154             zi = zipfile.ZipInfo(info.filename, date_time=(1980, 1, 1, 0, 0,
155                 0))
156             zi.compress_type = zipfile.ZIP_DEFLATED
157             zi.external_attr = 0o600 << 16
158             data = raw.read(info.filename)
159             if info.filename == "docProps/core.xml":
160                 data = re.sub(rb"(<dc:terms:(?:created|modified)[^>]*>)[^<]*(</
                    dc:terms:(?:created|modified)>)",
161                     rb"\g<1>2000-01-01T00:00:00Z\g<2>", data)
162             z.writestr(zi, data)
163     path.write_bytes(fixed.getvalue())
164     return hashlib.md5(fixed.getvalue()).hexdigest()
165
166 def main():
167     DATA_DIR.mkdir(exist_ok=True)
168     OUT2.parent.mkdir(exist_ok=True)
169     t_start = time.perf_counter()
170     t, TaK, Ce = read_env()
171     env_hold = make_env("hold", t, TaK, Ce)
172     print(f"环境: 0 - 14400 s 共 {len(t)} 点; 末值 {TaK[-1]-273.15:.3f} °C / {
        Ce[-1]:.5f}", flush=True)
173
174     # ----- V3 严格下界（解析，不占求解时间） -----
175     v3 = v3_bound(t, TaK, Ce)
176     print(f"[V3] D_max={v3['D_max']:.4e}, Bi_m={v3['Bi_m']:.4f}, λ₁={v3['
        lam1']:.6f}, "
177         f"A₁={v3['A1']:.6f}")

```



```

178 print(f"[V3] 首模公式下界 {v3['t_lb_h']:.4f} h; 精确级数参考穿越 {v3['t_ref_h']:.4f} h; "
179       f"膜界 {v3['t_film_h']:.4f} h; 穿越时第二模占比 {v3['second_mode_ratio']:.2e}", flush=True)
180 with open(DATA_DIR / "v3_bound.csv", "w", encoding="utf-8") as f:
181     f.write("quantity,value\n")
182     for k, v in v3.items():
183         f.write(f"{k},{v:.10g}\n")
184
185 # ----- 主运行 N=160 -----
186 print(f"[主运行] N={N_MAIN} 全程 + {MARGIN:.0f}s 余量 ...", flush=True)
187 t0 = time.perf_counter()
188 main_run = run_full(N_MAIN, env_hold, dt_max=10.0, margin=MARGIN + 20.0)
189 wall_main = time.perf_counter() - t0
190 cr = main_run["res"]["crossing"]
191 assert cr is not None
192 # 线性插值估计穿越点 + 右侧安全端点
193 t_star = cr[0] + (TH - cr[1]) * (cr[2] - cr[0]) / (cr[3] - cr[1])
194 t_right = cr[2]
195 tf_h = t_right / 3600.0
196 t_end2 = int(np.floor(t_right)) + int(MARGIN)
197 st = main_run["res"]["stats"]
198 print(f"[主运行] 穿越区间 [{cr[0]:.1f}, {cr[2]:.1f}] s (umax {cr[1]:.6f}
199       f"插值 t*={t_star/3600:.6f} h, 右端点 t_f={tf_h:.6f} h; "
200       f"步数 {st['n_steps']}(拒{st['n_rej']}) , 耗时 {wall_main:.1f} s",
201       flush=True)
202 print(f"[主运行] 守恒残差 水量(逐窗max) {st['max_res_m']:.2e} / "
203       f"焓(逐窗max) {st['max_res_h']:.2e} / 焓(全局吞吐) {st['glob_res_h']:.2e}; "
204       f"argmax 抽样 {set(i for _, i in main_run['res']['locus'])}", flush=True)
205
206 assert tf_h > v3["t_ref_h"], "V3 闸门未过! "
207 lt, lc, lT = main_run["t"], main_run["C"], main_run["T"]
208
209 # ----- 舍入自洽 (60 s 网格四位小数判定) -----
210 # 判定式是严格 <: 舍入值 0.1500 不算达标, 首个严格 <0.15 的 60 s 行可能晚于
211 t_right
212 t60 = np.arange(60.0, t_right + 3600.0, 60.0)
213 r60 = np.round(interp_rows(lt, lc, t60), 4)
214 below = np.where(np.max(r60, axis=1) < TH)[0]
215 assert len(below) > 0, "余量内未找到舍入达标行"
216 tf_rounded_h = float(t60[below[0]] / 3600.0)
217 print(f"[舍入自洽] 60 s 四舍五入判定 {tf_rounded_h:.4f} h, "
218       f"与未舍入右端点差 {abs(tf_rounded_h - tf_h):.4f} h", flush=True)
219
220 # ----- result3.xlsx (小, 先写) -----
221 n60 = int(t_right // 60)
222 r3times = np.arange(60.0, 60 * n60 + 1, 60.0)
223 r3 = np.round(interp_rows(lt, lc, r3times), 4)
224 r3end = np.round(interp_rows(lt, lc, [t_right]))[0], 4)

```

```

222
223 def rows3():
224     for k in range(n60):
225         yield [int(r3times[k])] + [float(x) for x in r3[k]]
226         yield [round(float(t_right), 4)] + [float(x) for x in r3end]
227 md5_3 = write_xlsx_stream(OUT3, [("Sheet1", HEADER, rows3())])
228 print(f"[导出] result3.xlsx ({n60} 常规行 + 结束行 @ {t_right:.1f} s) md5
    = {md5_3}", flush=True)
229
230 # ----- 收敛序列 N=40/80 (N=160 用主运行) -----
231 tf_by_n = {N_MAIN: tf_h}
232 st_by_n = {N_MAIN: st}
233 profs = {N_MAIN: interp_rows(lt, lc, V6_TIMES)}
234 for N in (40, 80):
235     print(f"[收敛] N={N} ...", flush=True)
236     r_ = run_full(N, env_hold, dt_max=15.0)
237     cr_ = r_["res"]["crossing"]
238     tf_by_n[N] = cr_[2] / 3600.0
239     st_by_n[N] = r_["res"]["stats"]
240     profs[N] = interp_rows(r_["t"], r_["C"], V6_TIMES)
241     print(f"[收敛] N={N}: t_f = {tf_by_n[N]:.6f} h, "
242           f"残差 水 {st_by_n[N]['max_res_m']:.2e} 烩(全局) {st_by_n[N]['
243               glob_res_h']:.2e}", flush=True)
244     e = [tf_by_n[40], tf_by_n[80], tf_by_n[160]]
245     p_num = np.log(abs(e[0] - e[1]) / abs(e[1] - e[2])) / np.log(2)
246     tf_rich = e[2] + (e[2] - e[1]) / (2**p_num - 1)
247     print(f"[V6] t_f 收敛阶估计 {p_num:.3f}; Richardson 外推 {tf_rich:.6f} h
248         , "
249         f"主值偏差 {abs(tf_rich - e[2]):.2e} h", flush=True)
250
251 # ----- 时间容差 / Picard 对照 / 外推三档 (N=80) -----
252 print("[时间容差] N=80 rtol/10 ...", flush=True)
253 r_dt = run_full(80, env_hold, rtol=1e-8)
254 tf_dt = r_dt["res"]["crossing"][2] / 3600.0
255 print(f"[时间容差] t_f(1e-8) = {tf_dt:.6f} h, Δ = {abs(tf_dt - tf_by_n
256     [80]):.2e} h", flush=True)
257
258 print("[对照] N=80 全程 Picard ...", flush=True)
259 r_pc = run_full(80, env_hold, newton_below=-1.0)
260 tf_pc = r_pc["res"]["crossing"][2] / 3600.0
261 print(f"[对照] Picard t_f = {tf_pc:.6f} h, Δ = {abs(tf_pc - tf_by_n[80])
262     :.2e} h", flush=True)
263
264 ext_rows = [("hold(主)", tf_by_n[80])]
265 for mode, label in (("linear", "线性外推(末1h斜率)", ("mean1h", "末1h均
266     值保持"))):
267     print(f"[外推] {label} ...", flush=True)
268     r_ = run_full(80, make_env(mode, t, TaK, Ce))
269     ext_rows.append((label, r_["res"]["crossing"][2] / 3600.0))
270 for label, tf_ in ext_rows:
271     print(f"[外推] {label}: t_f = {tf_:.6f} h, Δ = {tf_ - ext_rows

```

```

[0][1]:+.4f} h", flush=True)
267
268 # ----- V6 场量级 -----
269 v6_rows = []
270 for j, tt in enumerate(V6_TIMES):
271     d40 = float(np.max(np.abs(profs[40][j] - profs[160][j])))
272     d80 = float(np.max(np.abs(profs[80][j] - profs[160][j])))
273     v6_rows.append((tt, d40, d80))
274     print(f"[V6场] t={tt:.0f}s max|ΔU_40-160|={d40:.2e} max|ΔU_80-160|={
        d80:.2e} "
275           f"阶 {np.log(d40/d80)/np.log(2):.2f}", flush=True)
276
277 # ----- CSV -----
278 with open(DATA_DIR / "tf_convergence.csv", "w", encoding="utf-8") as f:
279     f.write("case,t_f_h\n")
280     for N in (40, 80, 160):
281         f.write(f"N={N},{tf_by_n[N]:.8f}\n")
282         f.write(f"N=80_rtol/10,{tf_dt:.8f}\nN=80_picard_only,{tf_pc:.8f}\n")
283         f.write(f"richardson_extrap,{tf_rich:.8f}\n")
284         f.write(f"interp_t_star,{t_star/3600:.8f}\nrounded_60s,{tf_rounded_h
            :.8f}\n")
285 with open(DATA_DIR / "conservation23.csv", "w", encoding="utf-8") as f:
286     f.write("N,max_rel_residual_water,max_rel_residual_enthalpy_window,
        glob_rel_residual_enthalpy,n_steps\n")
287     for N in (40, 80, 160):
288         s_ = st_by_n[N]
289         f.write(f"{N},{s_['max_res_m']:.6e},{s_['max_res_h']:.6e},{s_['
            glob_res_h']:.6e},{s_['n_steps']}\n")
290 with open(DATA_DIR / "v6_fields.csv", "w", encoding="utf-8") as f:
291     f.write("t_s,maxdiff_N40_vs_N160,maxdiff_N80_vs_N160\n")
292     for tt, d40, d80 in v6_rows:
293         f.write(f"{tt:.0f},{d40:.6e},{d80:.6e}\n")
294 with open(DATA_DIR / "extrapolation.csv", "w", encoding="utf-8") as f:
295     f.write("mode,t_f_h,delta_h\n")
296     for label, tf_ in ext_rows:
297         f.write(f"{label},{tf_:.8f},{tf_ - ext_rows[0][1]:+.8f}\n")
298 np.savez(DATA_DIR / "q23_main_steps.npz", t=lt, C=lC, T=lT, um=main_run[
    "um"],
299          crossing=np.array(cr))
300
301 # ----- result2.xlsx (大, write_only 流式) -----
302 print(f"[导出] result2.xlsx ({t_end2} 行 × 2 sheet, write_only 流式) ...",
        flush=True)
303 t0 = time.perf_counter()
304 sec = np.arange(1.0, t_end2 + 1)
305 i1 = np.clip(np.searchsorted(lt, sec), 1, len(lt) - 1)
306 w1 = (sec - lt[i1 - 1]) / np.maximum(lt[i1] - lt[i1 - 1], 1e-300)
307 w1 = w1[:, None]
308
309 def rows_gen(mat):
310     V0, V1 = mat[i1 - 1], mat[i1]

```

```

311     for k in range(len(sec)):
312         row = V0[k] + w1[k] * (V1[k] - V0[k])
313         yield [int(sec[k])] + [round(float(x), 4) for x in row]
314 md5_2 = write_xlsx_stream(OUT2, [
315     ("温度", HEADER, rows_gen(lT)),
316     ("水分浓度", HEADER, rows_gen(lC)),
317 ])
318 wall_w2 = time.perf_counter() - t0
319 print(f"[导出] result2.xlsx md5={md5_2}, 写出耗时 {wall_w2:.1f} s", flush
      =True)
320
321 # ----- 结构核对 -----
322 check_structure(t_end2, t_right, n60)
323
324 summary = dict(tf_right_s=t_right, tf_h=tf_h, tf_star_h=t_star / 3600,
325               tf_bracket=list(cr), tf_rounded_h=tf_rounded_h,
326               tf_by_n={str(k): v for k, v in tf_by_n.items()},
327               tf_rich_h=float(tf_rich), conv_order=float(p_num),
328               tf_dt_rtol10=tf_dt, tf_picard80=tf_pc,
329               extrapolation=[(l, float(v)) for l, v in ext_rows], v3=v3,
330               wall_main_s=wall_main, wall_write2_s=wall_w2,
331               wall_total_s=time.perf_counter() - t_start,
332               md5_result2=md5_2, md5_result3=md5_3,
333               stats_main={k: float(v) for k, v in st.items() if np.isscalar
                           (v)},
334               rows2=int(t_end2), n_main=N_MAIN)
335 (DATA_DIR / "q23_summary.json").write_text(
336     json.dumps(summary, ensure_ascii=False, indent=1), encoding="utf-8")
337 print(f"[完成] 总耗时 {time.perf_counter() - t_start:.1f} s", flush=True)
338
339
340 def check_structure(t_end2, t_right, n60):
341     def chk(cond, msg):
342         print((" [OK] " if cond else " [FAIL] ") + msg, flush=True)
343         assert cond, msg
344     print("[核对] result3/result2 结构（对照交付口径.md §2/§3）：")
345     tpl3 = openpyxl.load_workbook(TPL_DIR / "result3.xlsx")
346     out3 = openpyxl.load_workbook(OUT3, read_only=True)
347     chk(out3.sheetnames == tpl3.sheetnames == ["Sheet1"], f"result3 sheet: {
        out3.sheetnames}")
348     ws = out3["Sheet1"]
349     chk(ws.cell(1, 1).value == "时间\\到药材中心的距离", "result3 A1")
350     hdr = [c.value for c in next(ws.iter_rows(min_row=1, max_row=1, min_col
        =2, max_col=22))]
351     chk(all(abs(h - 0.1 * i) < 1e-12 for i, h in enumerate(hdr)), "result3
        21 列头 0...2.0")
352     rows = list(ws.iter_rows(min_row=2, values_only=True))
353     chk(rows[0][0] == 60 and rows[1][0] == 120, "result3 A 列 60,120 起")
354     chk(abs(rows[-1][0] - t_right) < 5e-4, f"result3 结束行 A={rows[-1][0]}
        == t_f {t_right:.4f}")
355     chk(len(rows) == n60 + 1, f"result3 行数 {len(rows)} == {n60}+1")

```

```

356 bad = sum(1 for r in rows for v in r[1:] if abs(v - round(v, 4)) > 1e
      -12)
357 chk(bad == 0, f"result3 全部四位小数 (异常 {bad}) ")
358 out3.close()
359 tpl2 = openpyxl.load_workbook(TPL_DIR / "result2.xlsx")
360 out2 = openpyxl.load_workbook(OUT2, read_only=True)
361 chk(out2.sheetnames == tpl2.sheetnames == ["温度", "水分浓度"], f"result2
      sheet: {out2.sheetnames}")
362 for name in out2.sheetnames:
363     ws = out2[name]
364     chk(ws.cell(1, 1).value == "时间\\到药材中心的距离", f"result2[{name}]
      A1")
365     hdr = [c.value for c in next(ws.iter_rows(min_row=1, max_row=1,
      min_col=2, max_col=22))]
366     chk(all(abs(h - 0.1 * i) < 1e-12 for i, h in enumerate(hdr)), f"
      result2[{name}] 21 列头")
367     n = 0
368     first3, last_a = [], None
369     bad = 0
370     for r in ws.iter_rows(min_row=2, values_only=True):
371         n += 1
372         if n <= 3:
373             first3.append(r[0])
374             last_a = r[0]
375             if n % 997 == 0:
376                 bad += sum(1 for v in r[1:] if abs(v - round(v, 4)) > 1e-12)
377             chk(first3 == [1, 2, 3] and last_a == t_end2,
378                 f"result2[{name}] A 列 1,2,3...{t_end2} ({n} 行) ")
379             chk(bad == 0, f"result2[{name}] 抽样四位小数 (异常 {bad}) ")
380 out2.close()
381
382
383 if __name__ == "__main__":
384     main()

```

E.8 问题 4 移动域求解流程

问题 4 一条龙: 移动域主解 + 效应拆分 + V5 退化 + 一致性检查 + result4.xlsx。

代码 9 问题 4 移动域求解流程 (run_q4.py, 385 行)

```

1  """问题 4 一条龙: 移动域主解 + T6 效应拆分 + V5 退化 + T7 一致性 + result4.
      xlsx。
2
3  用法:  python run_q4.py
4  产物:
5      ../结果/result4.xlsx (Sheet1, 60 s × 21 列[0...1.9,药材表面], x>R(t) 留空,
6                          末行 = t_f 数据行)
7      ../数据/q4_convergence.csv (⊗ 主解 t_f 随 N 收敛序列)
8      ../数据/q4_split.csv (T6 效应拆分三 t_f)
9      ../数据/q4_v5.csv (V5 退化对照)
10     ../数据/q4_conservation.csv

```

```

11  ../数据/q4_bound.csv  (收缩下界复算)
12  ../数据/q4_t7.csv      (T7 反推对比)
13  ../数据/q4_summary.json, q4_main_steps.npz
14  """
15  from __future__ import annotations
16
17  import hashlib
18  import io
19  import json
20  import re
21  import sys
22  import time
23  import zipfile
24  from pathlib import Path
25
26  import numpy as np
27  import openpyxl
28  from scipy.interpolate import PchipInterpolator
29  from scipy.optimize import brentq
30
31  CODE_DIR = Path(__file__).resolve().parent
32  sys.path.insert(0, str(CODE_DIR))
33
34  from solver_q1 import R0, H, HM, T0_K, C0, build_interp_weights,
    apply_interp
35  from analytic_heat import RobinCylinder, robin_eigs
36  from solver_q23 import TH
37  from solver_q4 import PROPS4, D_of4, solve_coupled4
38  from run_q23 import read_env, make_env, interp_rows, write_xlsx_stream
39
40  A_DIR = CODE_DIR.parent
41  ATT2 = A_DIR / "01-题目" / "原始文件" / "附件2.xlsx"
42  TPL4 = A_DIR / "01-题目" / "原始文件" / "附件3" / "result4.xlsx"
43  OUT4 = A_DIR / "结果" / "result4.xlsx"
44  DATA_DIR = A_DIR / "数据"
45
46  N_MAIN = 160
47  POS_CM = np.arange(0.0, 1.91, 0.1) # 20 固定列 0...1.9
48  HEADER4 = ["时间\\到药材中心的距离"] + [round(0.1 * i, 10) for i in range
    (20)] + ["药材表面"]
49
50
51  def read_radius():
52      wb = openpyxl.load_workbook(ATT2, read_only=True)
53      ws = wb[wb.sheetnames[0]]
54      rows = [(r[0], r[1]) for r in ws.iter_rows(min_row=2, values_only=True)
55              if r[0] is not None]
56      wb.close()
57      t = np.array([r[0] for r in rows], dtype=float)
58      Rcm = np.array([r[1] for r in rows], dtype=float)
59      assert len(t) == 145 and t[0] == 0 and t[-1] == 259200

```

```

60  assert np.all(Rcm > 0), "半径必须为正"
61  assert np.all(np.diff(Rcm) <= 1e-12), "半径不得反向增大"
62  pchip = PchipInterpolator(t, Rcm * 0.01, extrapolate=False)
63
64  def R_fun(tt):
65      return float(pchip(min(tt, t[-1]))) # 35 h 后末值平台延拓 (声明假设)
66
67  return t, Rcm, R_fun
68
69
70 def q4_bound(R_fun, t_att, Rcm):
71     """收缩比较下界:  $\int_0^t D_{\max} \cdot \lambda_1(Bi(s))^2 / R(s)^2 ds = \ln[A_1(C_0 - C_e) / (C_{th} - C_e)]$ ,
72     Bi(s)=h_m R(s)/D_max; 另给固定 R0 版与 R_min 版对照。"""
73     env_t, TaK, Ce = read_env()
74     Ta_max = max(float(TaK.max()), T0_K)
75     D_max = float(D_of4(C0, Ta_max))
76     Ce_end = float(Ce[-1])
77     bi_grid = np.linspace(0.3, 8.0, 400)
78     lam_grid = np.array([robin_eigs(b, 1)[0] for b in bi_grid])
79
80     def lam1(Bi):
81         return float(np.interp(Bi, bi_grid, lam_grid))
82
83     Bi_min = HM * float(Rcm.min() * 0.01) / D_max
84     A1_min = None
85     rc = RobinCylinder(Bi_min, D_max, R0, n_modes=10)
86     A1_min = rc.A[0]
87     target = np.log(A1_min * (C0 - Ce_end) / (TH - Ce_end))
88     # 时间积分版 (沿真实 R(s) 轨迹)
89     ss = np.linspace(0, 300000, 30001)
90     Rs = np.array([R_fun(x) for x in ss])
91     mu = D_max * np.array([lam1(HM * r / D_max) for r in Rs])**2 / Rs**2
92     cum = np.concatenate(([0.0], np.cumsum(0.5 * (mu[1:] + mu[:-1]) * np.
93         diff(ss))))
94     t_int = float(np.interp(target, cum, ss)) if cum[-1] > target else float
95         ("nan")
96     # R_min 常值版
97     lam_min = lam1(Bi_min)
98     t_rmin = Rcm.min()**2 * 1e-4 / (D_max * lam_min**2) * target
99     # 固定 R0 版 (case ① 的闸门)
100     Bi0 = HM * R0 / D_max
101     lam0 = lam1(Bi0)
102     rc0 = RobinCylinder(Bi0, D_max, R0, n_modes=10)
103     t_r0 = R0**2 / (D_max * lam0**2) * np.log(rc0.A[0] * (C0 - Ce_end) / (TH
104         - Ce_end))
105     return dict(D_max=D_max, Bi_min=Bi_min, t_int_h=t_int / 3600,
106         t_rmin_h=float(t_rmin / 3600), t_r0_h=float(t_r0 / 3600),
107         t_film_h=float(Rcm.min() * 0.01 / (2 * HM) *
108             np.log((C0 - Ce_end) / (TH - Ce_end)) / 3600))

```

```

107
108 def run_q4_full(N, env, R_fun, P=PROPS4, dt_max=15.0, margin=0.0):
109     """全程：逐接受步记录 (t, 20固定列U(nan 留空), U_s, Umean, umax, R)。"""
110     idx_q = np.arange(N)
111     q_centers = (idx_q + 0.5) / N
112     log = []
113
114     def record(tt, U, T, Us, Ts):
115         Rt = R_fun(tt)
116         # 固定列:  $x \leq R(t)$  时按  $q=(x/Rt)^2$  二次重构, 否则留空 (nan)
117         xs = POS_CM * 0.01
118         row = np.full(20, np.nan)
119         q_nodes = np.concatenate([0.0, q_centers, 1.0])
120         v_nodes = np.concatenate([1.5 * U[0] - 0.5 * U[1]], U, [Us])
121         for j, x in enumerate(xs):
122             if x <= Rt + 1e-15:
123                 q = (x / Rt) ** 2
124                 p = int(np.searchsorted(q_nodes, q))
125                 s = min(max(p - 1, 0), N - 1)
126                 idx3 = [s, s + 1, s + 2]
127                 qs3 = q_nodes[idx3]
128                 w = np.ones(3)
129                 for a_ in range(3):
130                     for b_ in range(3):
131                         if a_ != b_:
132                             w[a_] *= (q - qs3[b_]) / (qs3[a_] - qs3[b_])
133                 row[j] = float(np.sum(w * v_nodes[idx3]))
134             log.append((tt, row, Us, (1.0 / N) * float(np.sum(U)),
135                        max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1]), Rt))
136
137     res = solve_coupled4(N, env, R_fun, 600000.0, P=P, dt_max=dt_max,
138                        margin=margin, record=record, record_each=True)
139     return res, log
140
141
142 def main():
143     DATA_DIR.mkdir(exist_ok=True)
144     t_start = time.perf_counter()
145     env_t, TaK, Ce = read_env()
146     env_hold = make_env("hold", env_t, TaK, Ce)
147     t_att, Rcm, R_fun = read_radius()
148     print(f"附件2: {len(t_att)} 点, R {Rcm[0]:.3f}→{Rcm[-1]:.3f} cm, 平台起点
149           约 "
150           f"{t_att[np.argmax(Rcm <= Rcm.min() + 1e-12)] / 3600:.1f} h", flush=
151           True)
152
153     # ----- 收缩下界 -----
154     bnd = q4_bound(R_fun, t_att, Rcm)
155     print(f"[下界] D_max={bnd['D_max']:.3e}; 时间积分版 {bnd['t_int_h']:.3f}
156           h; "
157           f"R_min 常值版 {bnd['t_rmin_h']:.3f} h; 固定R0版 {bnd['t_r0_h']:.3f}

```



```

155         h; "
156         f"膜界(R_min) {bnd['t_film_h']:.3f} h", flush=True)
157     with open(DATA_DIR / "q4_bound.csv", "w", encoding="utf-8") as f:
158         f.write("quantity,value\n")
159         for k, v in bnd.items():
160             f.write(f"{k},{v:.10g}\n")
161
162     # ----- ◎ 主解: 附4 + R(t) -----
163     print(f"◎主解] N={N_MAIN} 附4+R(t) ...", flush=True)
164     t0 = time.perf_counter()
165     res2, log2 = run_q4_full(N_MAIN, env_hold, R_fun, dt_max=15.0)
166     wall_main = time.perf_counter() - t0
167     cr = res2["crossing"]
168     t_star = cr[0] + (TH - cr[1]) * (cr[2] - cr[0]) / (cr[3] - cr[1])
169     tf4_h = cr[2] / 3600.0
170     st = res2["stats"]
171     print(f"◎主解] 穿越 [{cr[0]:.0f},{cr[2]:.0f}] s, t_f4 = {tf4_h:.6f} h "
172           f"(插值 {t_star/3600:.6f} h), 步数 {st['n_steps']}(拒{st['n_rej']}), "
173           f"耗时 {wall_main:.1f} s", flush=True)
174     print(f"◎主解] 守恒 水量 {st['max_res_m']:.2e} / 焓(全局) {st['glob_res_h']:.2e}", flush=True)
175     assert tf4_h > bnd["t_int_h"], "收缩下界闸门未过"
176
177     # ----- ◎ 收敛序列 -----
178     tf2_by_n = {N_MAIN: tf4_h}
179     st2_by_n = {N_MAIN: st}
180     for N in (40, 80):
181         print(f"◎收敛] N={N} ...", flush=True)
182         r_, _ = run_q4_full(N, env_hold, R_fun)
183         tf2_by_n[N] = r_["crossing"][2] / 3600.0
184         st2_by_n[N] = r_["stats"]
185         print(f"◎收敛] N={N}: t_f = {tf2_by_n[N]:.6f} h, 水 {st2_by_n[N]['max_res_m']:.2e} "
186               f"焓(全局) {st2_by_n[N]['glob_res_h']:.2e}", flush=True)
187     e2 = [tf2_by_n[40], tf2_by_n[80], tf2_by_n[160]]
188     p2 = np.log(abs(e2[0] - e2[1]) / abs(e2[1] - e2[2])) / np.log(2)
189     tf2_rich = e2[2] + (e2[2] - e2[1]) / (2**p2 - 1)
190     print(f"◎V6] 阶估计 {p2:.3f}, Richardson {tf2_rich:.6f} h, 主值偏差 "
191           f"{abs(tf2_rich - e2[2]):.2e} h", flush=True)
192
193     # ----- ◎ 对照: 附4 + 固定 R0 -----
194     print(f"◎对照] N=160 附4+固定R0 ...", flush=True)
195     R_const = lambda tt: R0
196     res1, log1 = run_q4_full(N_MAIN, env_hold, R_const)
197     tf1_h = res1["crossing"][2] / 3600.0
198     print(f"◎对照] t_f(附4固定R) = {tf1_h:.6f} h, 守恒 水 "
199           f"{res1['stats']['max_res_m']:.2e} 焓(全局) {res1['stats']['glob_res_h']:.2e}", flush=True)
200     assert tf1_h > bnd["t_r0_h"], "固定R0下界闸门未过"

```

```

201 # ----- V5a: 移动域代码  $R=R_0$  vs 独立固定域代码 (附4物性) -----
202 print("[V5a] 移动域 $R=R_0$ (N=80) vs 独立固定域(附4, N=80) ...", flush=True)
203 import solver_q23 as sq23
204 keep = (sq23.rho_cp, sq23.k_of, sq23.D_of, sq23.dD_dC)
205 sq23.rho_cp, sq23.k_of = PROPS4["rho_cp"], PROPS4["k_of"]
206 sq23.D_of, sq23.dD_dC = PROPS4["D_of"], PROPS4["dD_dC"]
207 res_fix = sq23.solve_coupled(80, env_hold, 600000.0, out_times=(),
    dt_max=15.0)
208 sq23.rho_cp, sq23.k_of, sq23.D_of, sq23.dD_dC = keep
209 res_mov, _ = run_q4_full(80, env_hold, R_const)
210 tf_fix = res_fix["crossing"][2]
211 tf_mov = res_mov["crossing"][2]
212 dU_v5 = float(np.max(np.abs(res_fix["U"] - res_mov["U"])))
213 print(f"[V5a] t_f 固定域 {tf_fix/3600:.6f} h vs 移动域 $R=R_0$  {tf_mov/3600:.6f} h, "
    f" $\Delta t_f = \{abs(tf\_fix - tf\_mov):.2f\}$  s, 末态场差  $\max|\Delta U| = \{dU\_v5:.2e\}$ "
    , flush=True)
215
216 # ----- V5b: D 冻结常数  $\rightarrow$  Bessel 级数首模 -----
217 print("[V5b] D=const 退化 ...", flush=True)
218 D0 = float(D_of4(C0, 323.315))
219 Bi0 = HM * R0 / D0
220 ana = RobinCylinder(Bi0, D0, R0, n_modes=300)
221 P_const = dict(PROPS4, D_of=lambda C, T: np.full_like(np.asarray(C,
    float), D0),
    dD_dC=lambda C, T: np.zeros_like(np.asarray(C, float)))
222 Ce0 = float(Ce[-1])
223 t_test = np.array([1e5, 2e5, 4e5])
224 r160 = R0 * np.sqrt((np.arange(160) + 0.5) / 160)
225 C_ana = ana.eval(r160, t_test, [0.0, t_test[-1]], [Ce0 - C0, Ce0 - C0],
    C0)
226 res_d = solve_coupled4(160, lambda tt: (323.315, Ce0), R_const, t_test
    [-1],
227
    P=P_const, dt_max=60.0, record_each=False, detect=False
    )
228
229 # 末态与级数末点比对 + 首模商
230 C_num_end = res_d["U"]
231 d_end = float(np.max(np.abs(C_num_end - C_ana[-1])))
232 mean_num = float(np.mean(C_num_end))
233 ratio = (mean_num - Ce0) / (C0 - Ce0)
234 first = ana.A[0] * ana.mean_mode[0] * np.exp(-ana.mu[0] * t_test[-1])
235 print(f"[V5b] D0={D0:.3e}, Bi_m={Bi0:.3f}; t={t_test[-1]:.0f}s 末态逐点最
    大偏差 "
    f"{d_end:.2e}; 平均残余比 {ratio:.3e} vs 首模 {first:.3e}, 商 {ratio/
    first:.5f}",
    flush=True)
236
237
238
239 # ----- T6 效应拆分 -----
240 tf3_h = 57.179925
241 shrink_eff = tf1_h - tf4_h
242 total_eff = tf3_h - tf4_h

```

```

243 print(f"[T6] t_f: ④附4固定R {tf1_h:.4f} | ④附4+R(t) {tf4_h:.4f} | ④问题3
      {tf3_h:.4f} h",
244       flush=True)
245 print(f"[T6] 收缩效应 ④-④ = {shrink_eff:+.4f} h ({shrink_eff/tf1_h*100:.1
      f}% of ④) ; "
246       f"公式+收缩 ④-④ = {total_eff:+.4f} h ({total_eff/tf3_h*100:.1f}% of
      ④) ", flush=True)
247
248 # ----- T7 收缩停止一致性 -----
249 t_freeze = float(t_att[np.argmax(Rcm <= 1.200 + 1e-12)]) # 首达 1.200 cm
      的时刻
250 um = np.array([x[3] for x in log2])
251 lt = np.array([x[0] for x in log2])
252 um_35 = float(np.interp(126000.0, lt, um))
253 um_end = um[-1]
254 uminf = lambda tt: (1 + C0) * (R_fun(tt) / R0) ** 2 - 1.0
255 t7_rows = [(1800.0, uminf(1800.0), float(np.interp(1800.0, lt, um))),
256            (126000.0, uminf(126000.0), um_35),
257            (259200.0, uminf(259200.0), float(np.interp(259200.0, lt, um)))]
258 print(f"[T7] 收缩约 {t_freeze/3600:.1f} h 冻结(1.200 cm), t_f4 = {tf4_h
      :.1f} h, "
259       f"早 {(tf4_h*3600 - t_freeze)/3600:.1f} h", flush=True)
260 for tt, ui, umo in t7_rows:
261     print(f"[T7] t={tt/3600:6.2f} h: 反推 $\bar{U}$ ={ui:.4f} vs 模型 $\bar{U}$ ={umo:.4f} "
262          f"(比值 {umo/ui if ui>0 else float('nan'):.3f})", flush=True)
263
264 # ----- result4.xlsx -----
265 print("[导出] result4.xlsx ...", flush=True)
266 n60 = int(cr[2] // 60)
267 r4times = np.arange(60.0, 60 * n60 + 1, 60.0)
268 rowsC = np.array([x[1] for x in log2]) # (n_steps, 20) 含 nan
269 cols = np.array([x[2] for x in log2])
270 lt2 = np.array([x[0] for x in log2])
271
272 def row_at(tt):
273     i = int(np.clip(np.searchsorted(lt2, tt), 1, len(lt2) - 1))
274     w = (tt - lt2[i - 1]) / max(lt2[i] - lt2[i - 1], 1e-300)
275     Rt = R_fun(tt)
276     us = float(np.interp(tt, lt2, cols))
277     out = []
278     for j, x in enumerate(POS_CM):
279         if x * 0.01 > Rt + 1e-15:
280             out.append(None)
281             continue
282         #  $R$  单调不增  $\Rightarrow x \leq R(tt) \leq R(\text{前一步}) \Rightarrow$  前一步必有值; 后一步可能留空
283         v0, v1 = rowsC[i - 1, j], rowsC[i, j]
284         if np.isnan(v0) and np.isnan(v1):
285             out.append(round(us, 4)) #  $x \approx R_t$  的极限边缘: 该处即表面
286         elif np.isnan(v1):
287             out.append(round(float(v0), 4))
288         elif np.isnan(v0):

```

```

289         out.append(round(float(v1), 4))
290     else:
291         out.append(round(float(v0 + w * (v1 - v0)), 4))
292 out.append(round(us, 4))
293 return out
294
295 def rows4():
296     for k in range(n60):
297         yield [int(r4times[k])] + row_at(r4times[k])
298         yield [round(float(cr[2]), 4)] + row_at(cr[2])
299 md5_4 = write_xlsx_stream(OUT4, [("Sheet1", HEADER4, rows4())])
300 print(f"[导出] result4.xlsx ({n60} 常规行 + t_f 数据行 @ {cr[2]:.1f} s)
301       md5={md5_4}",
302       flush=True)
303
304 # ----- 结构核对 -----
305 check_structure4(cr[2], n60, R_fun)
306
307 # ----- CSV / summary -----
308 with open(DATA_DIR / "q4_convergence.csv", "w", encoding="utf-8") as f:
309     f.write("case,t_f_h\n")
310     for N in (40, 80, 160):
311         f.write(f"N={N},{tf2_by_n[N]:.8f}\n")
312         f.write(f"richardson_extrap,{tf2_rich:.8f}\ninterp_t_star,{t_star
313 /3600:.8f}\n")
314 with open(DATA_DIR / "q4_split.csv", "w", encoding="utf-8") as f:
315     f.write("case,t_f_h\nappendix4_fixedR,{:.8f}\nappendix4_R(t),{:.8f}\n
316 problem3,{:.8f}\n")
317     .format(tf1_h, tf4_h, tf3_h))
318     f.write(f"shrink_effect_h,{shrink_eff:.8f}\nshrink_pct_of_fixedR,"
319 f"{shrink_eff/tf1_h*100:.4f}\nformula_plus_shrink_h,{total_eff
320 :.8f}\n")
321     f.write(f"formula_plus_shrink_pct_of_q3,{total_eff/tf3_h*100:.4f}\n")
322 with open(DATA_DIR / "q4_v5.csv", "w", encoding="utf-8") as f:
323     f.write("check,value\n")
324     f.write(f"V5a_tf_fixed_s,{tf_fix:.4f}\nV5a_tf_moving_R0_s,{tf_mov:.4f
325 }\n")
326     f.write(f"V5a_dtf_s,{abs(tf_fix-tf_mov):.6f}\nV5a_maxdU,{dU_v5:.6e}\n
327 ")
328     f.write(f"V5b_maxdev_end,{d_end:.6e}\nV5b_first_mode_quotient,{ratio/
329 first:.6f}\n")
330 with open(DATA_DIR / "q4_conservation.csv", "w", encoding="utf-8") as f:
331     f.write("N,max_rel_residual_water,glob_rel_residual_enthalpy\n")
332     for N in (40, 80, 160):
333         s_ = st2_by_n[N]
334         f.write(f"{N},{s_['max_res_m']:.6e},{s_['glob_res_h']:.6e}\n")
335 with open(DATA_DIR / "q4_t7.csv", "w", encoding="utf-8") as f:
336     f.write("t_s,U_inferred_from_R,U_model_mean\n")
337     for tt, ui, umo in t7_rows:
338         f.write(f"{tt:.0f},{ui:.8f},{umo:.8f}\n")
339 np.savez(DATA_DIR / "q4_main_steps.npz", t=lt2, C=rowsC, Cs=colS, um=um,

```

```

333         crossing=np.array(cr))
334     summary = dict(tf4_right_s=cr[2], tf4_h=tf4_h, tf4_star_h=t_star / 3600,
335                   tf4_bracket=list(cr), tf2_by_n={str(k): v for k, v in
336                   tf2_by_n.items()}),
337                   tf2_rich_h=float(tf2_rich), tf1_fixedR_h=tf1_h, tf3_h=tf3_h,
338                   shrink_eff_h=shrink_eff, split_pct=dict(
339                   shrink_of_fixedR=shrink_eff / tf1_h * 100,
340                   formula_plus_shrink_of_q3=total_eff / tf3_h * 100),
341                   bound=bnd, md5_result4=md5_4,
342                   wall_main_s=wall_main, wall_total_s=time.perf_counter() -
343                   t_start,
344                   n_main=N_MAIN, rows4=n60 + 1,
345                   stats_main={k: float(v) for k, v in st.items() if np.isscalar
346                   (v)})
347     (DATA_DIR / "q4_summary.json").write_text(
348     json.dumps(summary, ensure_ascii=False, indent=1, default=float),
349     encoding="utf-8")
350     print(f"[完成] 总耗时 {time.perf_counter() - t_start:.1f} s", flush=True)
351
352 def check_structure4(t_right, n60, R_fun):
353     def chk(cond, msg):
354         print((" [OK] " if cond else " [FAIL] ") + msg, flush=True)
355         assert cond, msg
356     print("[核对] result4 结构（对照交付口径.md §4+§5.1）：")
357     tpl = openpyxl.load_workbook(TPL4)
358     out = openpyxl.load_workbook(OUT4, read_only=True)
359     chk(out.sheetnames == tpl.sheetnames == ["Sheet1"], f"sheet: {out.
360         sheetnames}")
361     ws = out["Sheet1"]
362     chk(ws.cell(1, 1).value == "时间\\到药材中心的距离", "A1 表头")
363     hdr = [c.value for c in next(ws.iter_rows(min_row=1, max_row=1, min_col
364         =2, max_col=22))]
365     chk(all(abs(hdr[i] - 0.1 * i) < 1e-12 for i in range(20)) and hdr[20] ==
366         "药材表面",
367         f"列头 0,0.1,...,1.9,药材表面（末列 {hdr[20]!r}）")
368     rows = list(ws.iter_rows(min_row=2, values_only=True))
369     chk(rows[0][0] == 60 and rows[1][0] == 120, "A 列 60,120 起")
370     chk(len(rows) == n60 + 1, f"行数 {len(rows)} == {n60}+1")
371     chk(abs(rows[-1][0] - t_right) < 5e-4, f"末行 A={rows[-1][0]} == t_f {
372         t_right:.4f}（数据行）")
373     # 留空规则: t=1800 行（第 30 行）1.9 列应已留空; t=60 行 1.9 列应有值
374     r1800 = rows[29]
375     chk(r1800[20] is None, "t=1800 s 行 1.9 cm 列留空 (R(1800)<1.9)")
376     chk(rows[0][20] is not None, "t=60 s 行 1.9 cm 列有值")
377     chk(all(r[21] is not None for r in rows[::50]), "药材表面列始终有值")
378     bad = 0
379     for r in rows:
380         for j, v in enumerate(r[1:]):
381             Rt = R_fun(r[0])
382             if v is None:

```

```

376         if j < 20 and (0.1 * j * 0.01) <= Rt + 1e-15:
377             bad += 1 # 域内却留空
378         elif abs(v - round(v, 4)) > 1e-12:
379             bad += 1
380         chk(bad == 0, f"留空/四位小数逐格核对 (异常 {bad}) ")
381         out.close()
382
383
384 if __name__ == "__main__":
385     main()

```

E.9 问题 4 内生几何主解

收缩由模型自身预测、附件 2 降为验证数据的问题 4 主解（满 72 h 积分与三段误差评价）。

代码 10 问题 4 内生几何主解（run_q4_endo.py, 456 行）

```

1  """run_q4_endo.py: 问题 4 内生几何版主解（用户拍板：收缩由模型预测，附件 2 降
   为验证数据）。
2
3  一条命令复现：python run_q4_endo.py
4  - 几何：耦合内生闭合 C (R 由湿度场逐步预测，不读附件 2；标定见 内生收缩模型.md
   )
5  - 求解器：import solver_q4（不改 代码 既有文件）；驱动为本文件的自适应
6  一步对两半步外推 + 守恒双恒等式窗口对账（与 solver_q4.solve_coupled4 同结
   构）。
7  产物：
8  ../结果/result4.xlsx（内生版，口径同交付口径.md §4+§5.1）
9  q4_endo_convergence.csv / q4_endo_check.csv
10 q4_split.csv（内生主口径 T6 效应拆分；外生行保留并标注“附件2外生几何（交叉验
   证）”）
11 结果总账.csv（就地更新问题 4 行；以（类别,项目,子项）为键覆盖/去重，幂等）
12
13 幂等性：本脚本对总账与 q4_split 的写入均按“键覆盖 + 去重”，重复运行不产生重复
   行。
14 """
15 from __future__ import annotations
16
17 import csv
18 import math
19 import sys
20 import time
21 from pathlib import Path
22
23 import numpy as np
24
25 HERE = Path(__file__).resolve().parent # 数据
26 A_DIR = HERE.parent
27 sys.path.insert(0, str(A_DIR / "代码"))
28
29 from solver_q1 import R0, H, T0_K, C0

```

```

30 from solver_q4 import PROPS4, D_of4, k_of4, surface_m4, coupled_step4,
    PROBE
31 from solver_q23 import TH
32 from run_q23 import read_env, make_env
33 from run_q4 import (read_radius, write_xlsx_stream, check_structure4,
34                     POS_CM, HEADER4, OUT4)
35
36 # ---- 内生闭合 C (标定参数, 见 内生收缩模型.md §2) ----
37 C_GLASS = 0.21
38 P_C = dict(eps_c=0.1108, a=1.6680, b=1.2195, eps_p=0.0709, k_p=3.7872)
39 CSMIN = 0.048766 # 末期 C_s (全局归一化常数, 物理量; 自主解步记录读得)
40
41
42 # ===== CSV 读写 (幂等: 键覆盖 + 去重)
    =====
43 def read_csv_rows(path):
44     """读 CSV → (表头, [行列表])。"""
45     with open(path, encoding="utf-8", newline="") as f:
46         rows = list(csv.reader(f))
47     return rows[0], [r for r in rows[1:] if r]
48
49
50 def write_csv(path, header, rows):
51     """写 CSV (csv 模块引号封装: 含逗号的单元格不再被拆列)。"""
52     with open(path, "w", encoding="utf-8", newline="") as f:
53         w = csv.writer(f)
54         w.writerow(header)
55         w.writerows(rows)
56
57
58 def ledger_upsert(path, updates):
59     """按 (类别, 项目, 子项) 为键覆盖/追加「结果总账.csv」; 同时清除历史重复键。
60
61     新键插入到同「类别」末行之后 (无同类则追加), 保证文档分组不变;
62     重复运行同一 updates 不改变文件内容 (幂等)。
63     """
64     header, body = read_csv_rows(path)
65     key = lambda r: (r[0], r[1], r[2] if len(r) > 2 else "")
66     seen, merged = {}, []
67     for r in body: # 先按首次出现顺序去重 (保留最后一个同键值)
68         k = key(r)
69         if k in seen:
70             merged[seen[k]] = r
71         else:
72             seen[k] = len(merged)
73             merged.append(r)
74     for row in updates:
75         row = [str(x) for x in row]
76         k = key(row)
77         if k in seen:
78             merged[seen[k]] = row # 覆盖

```

```

79     else:
80         last = max([i for i, r in enumerate(merged) if r[0] == row[0]],
81                     default=None)
82         if last is None:
83             merged.append(row)
84         else:
85             merged.insert(last + 1, row)
86         seen = {key(r): i for i, r in enumerate(merged)}
87     write_csv(path, header, merged)
88     return len(merged)
89
90 def update_q4_split(path, tf_endo_h, tf_fixed_h, tf_q3_h):
91     """把 q4_split.csv 更新为内生主口径（外生行保留并标注交叉验证）；幂等。
92
93     行: appendix4_endogenous_R(t)（内生主解） + 由其派生的收缩/净效应；
94     外生行 appendix4_R(t) 备注列标注「附件2外生几何（交叉验证）」。
95     """
96     header, body = read_csv_rows(path)
97     shrink = tf_fixed_h - tf_endo_h
98     net = tf_q3_h - tf_endo_h
99     add = {"appendix4_endogenous_R(t)": (f"{tf_endo_h:.8f}", "内生几何主解
100         (收缩由模型预测)"),
101            "shrink_effect_endo_h": (f"{shrink:.8f}", "= appendix4_fixedR - 内
102            生主解"),
103            "shrink_pct_of_fixedR_endo": (f"{shrink / tf_fixed_h * 100:.4f}",
104            "收缩效应占固定 R 对照"),
105            "net_effect_endo_h": (f"{net:.8f}", "= problem3 - 内生主解"),
106            "net_pct_of_q3_endo": (f"{net / tf_q3_h * 100:.4f}", "净效应占问题 3
107            ")}
108
109     out = []
110     for r in body:
111         if len(r) < 2:
112             continue
113         case, val = r[0], r[1]
114         if case in add:
115             # 去重（幂等）：内生行随后统一追加
116             continue
117         note = r[2] if len(r) > 2 else ""
118         if case == "appendix4_R(t)":
119             note = "附件2外生几何（交叉验证）"
120         out.append([case, val, note])
121     out += [[k, v, n] for k, (v, n) in add.items()]
122     write_csv(path, ["case", "t_f_h", "note"], out)
123     return shrink, net
124
125 def closure_terms(um_, Cs_):
126     """返回（塌陷乘子项，停滞乘子项）（小量，可为负/正）。"""
127     x = np.clip((C0 - um_) / C0, 0.0, 1.0)
128     a, b = P_C["a"], P_C["b"]

```



```

126 gc = (1 - x) ** a * x ** b
127 gc = gc / ((1 - b / (a + b)) ** a * (b / (a + b)) ** b)
128 gp = np.clip((C_GLASS - Cs_) / (C_GLASS - CSMIN), 0.0, 1.0) ** P_C["k_p"
129 ]
130 return -P_C["eps_c"] * float(gc), P_C["eps_p"] * float(gp)
131
132 def R_from_state(U, T, tt, Cenv_now, dq):
133     """由当前湿度场预测半径：先以末单元值估 C_s，再经表面方程求 C_s，代入闭合。"""
134     um_ = float(np.mean(U))
135     Rt0 = R0 * math.sqrt((1 + um_) / (1 + C0))
136     Cs_, _, _ = surface_m4(U[-1], T[-1], Cenv_now, dq, Rt0, D_of4)
137     tc, tp = closure_terms(um_, Cs_)
138     return Rt0 * (1 + tc) * (1 + tp), Cs_
139
140
141 def solve_endogenous(N, env_fun, t_end=600000.0, dt0=1e-3, dt_max=15.0,
142                     rtolU=1e-7, rtolT=1e-7, atolU=1e-11, atolT=1e-8,
143                     record=None, win=600.0, stop_at_cross=True):
144     """耦合内生自适应驱动（结构与 solve_coupled4 相同；R 由各子步起点状态显式预测）。
145
146     stop_at_cross=False 时首次达标后仍推进到 t_end（crossing 照常记录）——
147     A07 修复：主解须满 72 h 积分，避免评价段被 np.interp 常值填充。"""
148     dq = 1.0 / N
149     U = np.full(N, C0)
150     T = np.full(N, T0_K)
151     t, dt = 0.0, dt0
152     n_steps = n_rej = 0
153     max_res_m = max_res_h = 0.0
154     g_dh_err = g_fh_abs = 0.0
155     w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
156     win_end = win
157     crossing = None
158     while t < t_end - 1e-12:
159         if crossing is not None and stop_at_cross:
160             break
161         dtc = min(t + dt, t_end) - t
162         Ta_m, Ce_m = env_fun(t + 0.5 * dtc)
163         Ta_e, Ce_e = env_fun(t + dtc)
164         try:
165             Rt_e, _ = R_from_state(U, T, t + dtc, Ce_e, dq)
166             Rt_m, _ = R_from_state(U, T, t + 0.5 * dtc, Ce_m, dq)
167             Uf_, Tf_, _, Fmf, _, Fhf, af_ = coupled_step4(U, T, dtc, Ta_e,
168                 Ce_e, dq, True, Rt_e, PROPS4)
169             Uh, Th, Us1, Fm1, Ts1, Fh1, a1 = coupled_step4(U, T, 0.5 * dtc,
170                 Ta_m, Ce_m, dq, True, Rt_m, PROPS4)
171             Rt_h2, _ = R_from_state(Uh, Th, t + dtc, Ce_e, dq)
172             Uh2, Th2, Us2, Fm2, Ts2, Fh2, a2 = coupled_step4(Uh, Th, 0.5 * dtc
173                 , Ta_e, Ce_e, dq, True, Rt_h2, PROPS4)

```

```

171 except (RuntimeError, ValueError, FloatingPointError):
172     n_rej += 1
173     PROBE["rej_outer"] += 1
174     dt = dtc * 0.5
175     continue
176 n_steps += 3
177 errU = float(np.max(np.abs(Uh2 - Uf_)))
178 errT = float(np.max(np.abs(Th2 - Tf_)))
179 tolU = atolU + rtolU * float(np.max(np.abs(U)))
180 tolT = atolT + rtolT * float(np.max(np.abs(T)))
181 r = max(errU / tolU, errT / tolT)
182 if r <= 1.0:
183     U_ex = 2.0 * Uh2 - Uf_
184     T_ex = 2.0 * Th2 - Tf_
185     assert float(np.min(U_ex)) > 0.0
186     umax_old = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
187     w_dm.append(dq * math.fsum(float(x) for x in (U_ex - U)))
188     w_fm.append(dtc * (Fm1 + Fm2 - Fmf))
189     w_dh.append(dq * (2.0 * math.fsum(float(a1[i]) * float(Th[i] - T[i]
190         ])) for i in range(N))
191         + 2.0 * math.fsum(float(a2[i]) * float(Th2[i] - Th[i]
192         ])) for i in range(N))
193         - math.fsum(float(af_[i]) * float(Tf_[i] - T[i]) for
194         i in range(N))))
195     w_fh.append(dtc * (Fh1 + Fh2 - Fhf))
196     w_fh_abs.append(dtc * (abs(Fh1) + abs(Fh2) + abs(Fhf)) / 3.0)
197     if t + dtc >= win_end - 1e-12:
198         dm, fm = math.fsum(w_dm), math.fsum(w_fm)
199         dh, fh = math.fsum(w_dh), math.fsum(w_fh)
200         if abs(fm) > 0:
201             max_res_m = max(max_res_m, abs(dm - fm) / abs(fm))
202             fh_abs = math.fsum(w_fh_abs)
203             if fh_abs > 0:
204                 res_h = abs(dh - fh) / fh_abs
205                 if res_h > max_res_h:
206                     max_res_h = res_h
207                     g_dh_err += dh - fh
208                     g_fh_abs += fh_abs
209                 w_dm, w_fm, w_dh, w_fh, w_fh_abs = [], [], [], [], []
210                 win_end += win
211     U, T = U_ex, T_ex
212     t += dtc
213     umax = max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1])
214     if crossing is None and umax < TH:
215         crossing = (t - dtc, umax_old, t, umax)
216     if record is not None:
217         record(t, U, T, Us2, Ts2, Rt_h2)
218         fac = 0.9 / max(r, 1e-30) ** (1.0 / 3.0)
219         dt = min(dtc * min(2.0, max(0.3, fac)), dt_max)
220 else:
221     n_rej += 1

```

```

219         dt = dtc * max(0.2, 0.9 / r ** (1.0 / 3.0))
220     stats = dict(n_steps=n_steps, n_rej=n_rej, max_res_m=max_res_m,
221                 max_res_h=max_res_h,
222                 t_end=t, glob_res_h=abs(g_dh_err) / g_fh_abs if g_fh_abs > 0
223                 else 0.0)
224     return dict(U=U, T=T, crossing=crossing, stats=stats)
225
226 def main():
227     t_start = time.perf_counter()
228     env_t, TaK, Ce = read_env()
229     env_hold = make_env("hold", env_t, TaK, Ce)
230     t_att, Rcm, R_fun_att2 = read_radius()
231     R_att = Rcm * 0.01
232
233     # ----- 主解 N=160 (逐接受步记录 21 列 + R_pred) -----
234     N_MAIN = 160
235     idx_q = np.arange(N_MAIN)
236     q_centers = (idx_q + 0.5) / N_MAIN
237     log = []
238     diag_r, diag_ts = [], [] # A07: R_from_state 代数闭合残差诊断
239
240     def record(tt, U, T, Us, Ts, Rt):
241         xs = POS_CM * 0.01
242         row = np.full(20, np.nan)
243         q_nodes = np.concatenate([[0.0], q_centers, [1.0]])
244         v_nodes = np.concatenate([[1.5 * U[0] - 0.5 * U[1]], U, [Us]])
245         for j, x in enumerate(xs):
246             if x <= Rt + 1e-15:
247                 q = (x / Rt) ** 2
248                 p = int(np.searchsorted(q_nodes, q))
249                 s = min(max(p - 1, 0), N_MAIN - 1)
250                 idx3 = [s, s + 1, s + 2]
251                 qs3 = q_nodes[idx3]
252                 w = np.ones(3)
253                 for a_ in range(3):
254                     for b_ in range(3):
255                         if a_ != b_:
256                             w[a_] *= (q - qs3[b_]) / (qs3[a_] - qs3[b_])
257                 row[j] = float(np.sum(w * v_nodes[idx3]))
258         log.append((tt, row, Us, (1.0 / N_MAIN) * float(np.sum(U)),
259                     max(float(np.max(U)), 1.5 * U[0] - 0.5 * U[1]), Rt))
260
261     # ---- A07 诊断 (只量化评估, 不进主解) ----
262     # (i) 代数闭合残差: R_ideal 下求 C_s 代入修正式得 R1; 再以 R1 重解 C_s
263     # 迭代一次得 R2, |R2-R1| 即未收敛残差的量级估计;
264     Ta_, Ce_ = env_hold(tt)
265     um_ = float(np.mean(U))
266     Rt0 = R0 * math.sqrt((1 + um_) / (1 + C0))
267     dq = 1.0 / N_MAIN
268     Cs1, _, _ = surface_m4(U[-1], T[-1], Ce_, dq, Rt0, D_of4)
269     tc1, tp1 = closure_terms(um_, Cs1)

```

```

268 R1 = Rt0 * (1 + tc1) * (1 + tp1)
269 Cs2, _, _ = surface_m4(U[-1], T[-1], Ce_, dq, R1, D_of4)
270 tc2, tp2 = closure_terms(um_, Cs2)
271 R2 = Rt0 * (1 + tc2) * (1 + tp2)
272 diag_r.append(abs(R2 - R1))
273 # (ii) 末单元温度 vs 重建表面温度 (coupled_step4 同款重建公式);
274 ks_ = float(k_of4(0.5 * (Us + U[-1])))
275 Rh_ = Rt ** 2 * dq / (8.0 * ks_) + Rt / (2.0 * H)
276 Ts_rec = T[-1] + ((Ta_ - T[-1]) / Rh_) * Rt ** 2 * dq / (8.0 * ks_)
277 diag_ts.append(abs(Ts_rec - T[-1]))
278
279 print(f"[内生主解] N={N_MAIN} 耦合内生几何全程 (满 72 h 积分, 首次达标单独
    记录) ...", flush=True)
280 t0 = time.perf_counter()
281 res = solve_endogenous(N_MAIN, env_hold, t_end=259200.0, stop_at_cross=
    False,
282                          record=record)
283 wall_main = time.perf_counter() - t0
284 cr = res["crossing"]
285 st = res["stats"]
286 t_star = cr[0] + (TH - cr[1]) * (cr[2] - cr[0]) / (cr[3] - cr[1])
287 tf_h = cr[2] / 3600.0
288 print(f"[内生主解] 穿越 [{cr[0]:.1f},{cr[2]:.1f}] s, t_f4(内生) = {tf_h
    :.6f} h"
289       f" (插值 {t_star/3600:.6f} h); 步数 {st['n_steps']}(拒{st['n_rej']})
    , 耗时 {wall_main:.1f} s",
290       flush=True)
291 print(f"[内生主解] 守恒: 水量逐窗 {st['max_res_m']:.2e} / 焓(全局) {st['
    glob_res_h']:.2e}",
292       flush=True)
293 assert tf_h > 19.016, "瞬时首模近似参考闸门未过"
294
295 lt = np.array([x[0] for x in log])
296 rowsC = np.array([x[1] for x in log])
297 cols = np.array([x[2] for x in log])
298 R_pred_log = np.array([x[5] for x in log])
299
300 # ----- 内生几何自洽 (对附件 2, 真实模型轨迹满 72 h, 无常值填充)
    -----
301 R_on_att = np.interp(t_att, lt, R_pred_log)
302 res_att = R_on_att - R_att
303 rmse_R = float(np.sqrt(np.mean(res_att ** 2)))
304 w_pre = t_att <= cr[0]
305 w_post = t_att >= cr[2]
306 e_pre = float(np.sqrt(np.mean(res_att[w_pre] ** 2)))
307 e_post = float(np.sqrt(np.mean(res_att[w_post] ** 2)))
308 print(f"[自洽] R_pred(t) vs 附件 2 (满 72 h 真实轨迹): 全程 RMSE = {
    rmse_R*1000:.4f} mm; "
309       f"0-达标 {e_pre*1000:.4f} / 达标-72h {e_post*1000:.4f} mm; "
310       f"R_pred(72h) = {R_pred_log[-1]*100:.4f} cm (附件 2: 1.198)", flush
    =True)

```

```

311 print(f"[自治] 末端平台核查: R_pred(60h→72h) = "
312       f"{float(np.interp(216000.0, lt, R_pred_log))*100:.4f} → {R_pred_log
313       [-1]*100:.4f} cm",
314       flush=True)
315 # ----- A07 诊断汇总 (R_from_state 代数闭合残差, 只评估不改主解)
316 # -----
317 diag_r = np.array(diag_r)
318 diag_ts = np.array(diag_ts)
319 print(f"[诊断] 代数闭合残差 |R2-R1|: max {diag_r.max()*1000:.4f} mm / "
320       f"mean {diag_r.mean()*1000:.4f} mm; 末单元温度 vs 重建表面温度差: "
321       f"max {diag_ts.max():.4f} °C / mean {diag_ts.mean():.4f} °C", flush=
322       True)
323
324 # ----- 收敛序列 -----
325 tf_by_n = {N_MAIN: tf_h}
326 st_by_n = {N_MAIN: st}
327 for N in (40, 80):
328     r_ = solve_endogenous(N, env_hold)
329     tf_by_n[N] = r_["crossing"][2] / 3600.0
330     st_by_n[N] = r_["stats"]
331     print(f"[收敛] N={N}: t_f = {tf_by_n[N]:.6f} h, 水 {r_['stats']['
332           max_res_m']:.2e} "
333           f"焓(全局) {r_['stats']['glob_res_h']:.2e}", flush=True)
334 e = [tf_by_n[40], tf_by_n[80], tf_by_n[160]]
335 p_num = np.log(abs(e[0] - e[1]) / abs(e[1] - e[2])) / np.log(2)
336 tf_rich = e[2] + (e[2] - e[1]) / (2 ** p_num - 1)
337 print(f"[收敛] Richardson 外推 {tf_rich:.6f} h (主值偏差 {abs(tf_rich - e
338       [2]):.2e} h) ",
339       flush=True)
340
341 # ----- 与附件 2 外生版对照 (t_f 与场差) -----
342 tf_exo160 = 50.823025
343 d0 = np.load(HERE / "q4_main_steps.npz")
344 texo, Cexo = d0["t"], d0["C"]
345 tt_cmp = np.array([36000.0, 108000.0, 180000.0])
346 fld = []
347 for tt in tt_cmp:
348     i0 = int(np.argmin(np.abs(lt - tt)))
349     i1 = int(np.argmin(np.abs(texo - tt)))
350     m0 = ~np.isnan(rowsC[i0])
351     m1 = ~np.isnan(Cexo[i1])
352     m = m0 & m1
353     fld.append(float(np.max(np.abs(rowsC[i0][m] - Cexo[i1][m]))))
354 print(f"[对照] 附件2外生版 t_f = {tf_exo160:.6f} h; 内生 - 外生 = {tf_h -
355       tf_exo160:+.4f} h; "
356       f"场差 max|ΔU| (3 时刻) : {'', '.join(f'{v:.2e}' for v in fld)}",
357       flush=True)
358
359 # ----- result4.xlsx (内生版) -----
360 n60 = int(cr[2] // 60)
361 r4times = np.arange(60.0, 60 * n60 + 1, 60.0)

```

```

355
356 def interp_col(mat, tt):
357     i = int(np.clip(np.searchsorted(lt, tt), 1, len(lt) - 1))
358     w = (tt - lt[i - 1]) / max(lt[i] - lt[i - 1], 1e-300)
359     return mat[i - 1] + w * (mat[i] - mat[i - 1])
360
361 def row_at(tt):
362     vals = interp_col(rowsC, tt)
363     us = float(np.interp(tt, lt, cols))
364     Rt = float(np.interp(tt, lt, R_pred_log))
365     out = []
366     for j, x in enumerate(POS_CM):
367         if x * 0.01 <= Rt + 1e-15 and not np.isnan(vals[j]):
368             out.append(round(float(vals[j]), 4))
369         else:
370             out.append(None)
371     out.append(round(us, 4))
372     return out
373
374 def rows4():
375     for k in range(n60):
376         yield [int(r4times[k])] + row_at(r4times[k])
377         yield [round(float(cr[2]), 4)] + row_at(cr[2])
378 md5_4 = write_xlsx_stream(OUT4, [("Sheet1", HEADER4, rows4())])
379 print(f"[导出] result4.xlsx (内生版, {n60} 常规行 + t_f 数据行 @ {cr[2]:.1
    f} s) md5={md5_4}",
380       flush=True)
381 # 结构核对 (用内生 R_pred 插值作域判定)
382 R_fun_endo = lambda tt: float(np.interp(min(tt, lt[-1]), lt, R_pred_log)
    )
383 check_structure4(cr[2], n60, R_fun_endo)
384
385 # ----- CSV 与结果总账更新 -----
386 with open(HERE / "q4_endo_convergence.csv", "w", encoding="utf-8") as f:
387     f.write("case,t_f_h\n")
388     for N in (40, 80, 160):
389         f.write(f"N={N},{tf_by_n[N]:.8f}\n")
390         f.write(f"richardson_extrap,{tf_rich:.8f}\ninterp_t_star,{t_star
    /3600:.8f}\n")
391         f.write(f"exogenous_att2_N160,{tf_exo160:.8f}\n")
392         f.write(f"delta_endo_minus_exo_h,{tf_h - tf_exo160:+.8f}\n")
393         f.write(f"R_pred_vs_att2_RMSE_mm,{rmse_R * 1000:.6f}\n")
394         for i, tt in enumerate(tt_cmp):
395             f.write(f"field_maxdiff_t{int(tt)}s,{fld[i]:.6e}\n")
396 # A07: 三段误差与代数闭合诊断 (真实 72 h 轨迹, 无常值填充)
397 with open(HERE / "q4_endo_eval.csv", "w", encoding="utf-8") as f:
398     f.write("item,value\n")
399     f.write(f"tf_first_crossing_h,{tf_h:.8f}\n")
400     f.write(f"rmse_all_mm,{rmse_R * 1000:.6f}\n")
401     f.write(f"rmse_0_to_crossing_mm,{e_pre * 1000:.6f}\n")
402     f.write(f"rmse_crossing_to_72h_mm,{e_post * 1000:.6f}\n")

```

```

403     f.write(f"R_pred_72h_cm,{R_pred_log[-1] * 100:.6f}\n")
404     f.write(f"R_pred_60h_cm,{float(np.interp(216000.0, lt, R_pred_log)) *
        100:.6f}\n")
405     f.write(f"R_att_end_cm,{R_att[-1] * 100:.4f}\n")
406     f.write(f"R_closure_resid_max_mm,{diag_r.max() * 1000:.6f}\n")
407     f.write(f"R_closure_resid_mean_mm,{diag_r.mean() * 1000:.6f}\n")
408     f.write(f"Ts_recon_vs_cell_max_C,{diag_ts.max():.6f}\n")
409     f.write(f"Ts_recon_vs_cell_mean_C,{diag_ts.mean():.6f}\n")
410 with open(HERE / "q4_endo_check.csv", "w", encoding="utf-8") as f:
411     f.write("item,value\n")
412     f.write(f"tf_endo_h,{tf_h:.8f}\n")
413     f.write(f"res_water_max,{st['max_res_m']:.6e}\n")
414     f.write(f"res_enthalpy_glob,{st['glob_res_h']:.6e}\n")
415     f.write(f"md5_result4,{md5_4}\n")
416     f.write(f"rows,{n60 + 1}\n")
417     f.write(f"wall_main_s,{wall_main:.1f}\n")
418
419 # ----- q4_split.csv: 切到内生主口径 (T6 效应拆分) -----
420 split_path = HERE / "q4_split.csv"
421 hdr_s, body_s = read_csv_rows(split_path)
422 val = {r[0]: r[1] for r in body_s if len(r) >= 2}
423 tf_fixed_h = float(val.get("appendix4_fixedR", "129.10470888"))
424 tf_q3_h = float(val.get("problem3", "57.179925"))
425 shrink, net = update_q4_split(split_path, tf_h, tf_fixed_h, tf_q3_h)
426 print(f"[q4_split] 内生主口径已写入: 内生 {tf_h:.6f} h; 收缩效应 {shrink
    :.4f} h"
    f" ({shrink / tf_fixed_h * 100:.2f}%) ; 净效应 {net:.4f} h"
    f" ({net / tf_q3_h * 100:.2f}%) ; 外生行标注交叉验证", flush=True)
427
428
429 # ----- 结果总账: 以 (类别,项目,子项) 为键覆盖/去重 (幂等) -----
430 pct_shrink = shrink / tf_fixed_h * 100.0
431 pct_net = net / tf_q3_h * 100.0
432 updates = [
433     ("t_f", "问题4主值(未舍入右端点,内生几何)", "", f"{tf_h:.6f}",
434      f"h (附件2外生版 {tf_exo160:.6f} 作交叉验证)"),
435     ("t_f", "问题4 Richardson外推(内生几何)", "", f"{tf_rich:.6f}", "h"),
436     ("敏感性", "问题4效应拆分:附4固定R", "", f"{tf_fixed_h:.4f}",
437      f" $\Delta$ {shrink:+.4f} h (对内生主解, 收缩效应 {pct_shrink:.1f}%)"),
438     ("敏感性", "问题4效应拆分:附4+R(t)(主)", "", f"{tf_h:.4f}", "— (内生几
439      何主解)"),
440     ("敏感性", "问题4效应拆分:问题3", "", f"{tf_q3_h:.4f}",
441      f" $\Delta$ {net:+.4f} h (公式+收缩效应 {pct_net:.1f}%)"),
442     ("验证", "内生几何自洽R_pred vs 附件2 RMSE", "", f"{rmse_R * 1000:.4f}
443      ",
444      "mm (满72h真实轨迹; 三段见q4_endo_eval.csv)"),
445     ("验证", "内生自洽三段误差0-达标/达标-72h", "", f"{e_pre * 1000:.4f}/{
446      e_post * 1000:.4f}",
447      "mm (A07口径)"),
448     ("验证", "耦合内生t_f vs 外生t_f偏差", "", f"{tf_h - tf_exo160:+.4f}",
449      "h (-0.22% 量级)"),
450 ]

```

```

449     n_led = ledger_upsert(HERE / "结果总账.csv", updates)
450     print(f"[总账] 结果总账.csv 问题 4 行已按内生几何覆盖（键去重，共 {n_led}
        行；重复运行不变）",
451           flush=True)
452     print(f"[完成] 总耗时 {time.perf_counter() - t_start:.1f} s", flush=True)
453
454
455 if __name__ == "__main__":
456     main()

```

E.10 内生几何时空场导出

导出内生问题 4 的完整时空场，供演化图与截面图使用。

代码 11 内生几何时空场导出（run_q4_endo_fields.py, 145 行）

```

1  """run_q4_endo_fields.py: 内生问题 4 的完整时空场导出（供 3D 演化图/截面图）。
2
3  复用 run_q4_endo.py 的耦合内生求解器（不改动其任何既有行为与 result4）。
4  一条命令: python run_q4_endo_fields.py
5  产物: q4_endo_fields.npz + q4_endo_fields_README.md; 并做与 result4.xlsx 的
    一致性抽查。
6  """
7  from __future__ import annotations
8
9  import sys
10 import time
11 from pathlib import Path
12
13 import numpy as np
14 import openpyxl
15
16 HERE = Path(__file__).resolve().parent
17 A_DIR = HERE.parent
18 sys.path.insert(0, str(A_DIR / "代码"))
19
20 from run_q4_endo import solve_endogenous
21 from run_q23 import read_env, make_env
22
23 N = 160
24 DT_FRAME = 360.0    # 帧间隔 (s), ~506 帧
25 DR = 0.02           # 径向步长 (cm), 固定物理网格 0 - 2.0 cm, 101 点, 域外 NaN
26 R_GRID = np.arange(0.0, 2.0 + 1e-12, DR)
27
28
29 def main():
30     t_start = time.perf_counter()
31     env_t, TaK, Ce = read_env()
32     env_hold = make_env("hold", env_t, TaK, Ce)
33
34     log = []
35

```



```

36 def record(tt, U, T, Us, Ts, Rt):
37     log.append((tt, U.copy(), T.copy(), Us, Ts, Rt))
38
39 print("[场导出] 耦合内生求解 (N=160, 逐步记录) ...", flush=True)
40 res = solve_endogenous(N, env_hold, record=record)
41 cr = res["crossing"]
42 tf_s = cr[2]
43 print(f"[场导出] t_f = {tf_s:.1f} s ({tf_s/3600:.4f} h), 接受步 {len(log)} 步", flush=True)
44
45 lt = np.array([x[0] for x in log])
46 frames = np.arange(0.0, tf_s, DT_FRAME)
47 frames = np.append(frames, tf_s)
48 n_t = len(frames)
49 n_r = len(R_GRID)
50 C = np.full((n_t, n_r), np.nan)
51 T = np.full((n_t, n_r), np.nan)
52 R_t = np.zeros(n_t)
53 Cs_s = np.zeros(n_t)
54 Cc_s = np.zeros(n_t)
55
56 q_centers = (np.arange(N) + 0.5) / N
57 for k, tau in enumerate(frames):
58     i = int(np.clip(np.searchsorted(lt, tau), 1, len(lt) - 1))
59     w = (tau - lt[i - 1]) / max(lt[i] - lt[i - 1], 1e-300)
60     _, U0, T0, Us0, Ts0, Rt0 = log[i - 1]
61     _, U1, T1, Us1, Ts1, Rt1 = log[i]
62     U = U0 + w * (U1 - U0)
63     Tk = T0 + w * (T1 - T0) # 开尔文
64     Us = Us0 + w * (Us1 - Us0)
65     Ts = Ts0 + w * (Ts1 - Ts0)
66     Rt = Rt0 + w * (Rt1 - Rt0)
67     assert 200.0 < Tk.min() and Tk.max() < 400.0, "温度必须开尔文"
68     assert 200.0 < Ts < 400.0
69     # 二次重构: 节点 = 中心正则外推 + 单元中心 + 表面
70     q_nodes = np.concatenate(([0.0], q_centers, [1.0]))
71     vC = np.concatenate(([1.5 * U[0] - 0.5 * U[1]], U, [Us]))
72     vT = np.concatenate(([1.5 * Tk[0] - 0.5 * Tk[1]], Tk, [Ts]))
73     for j, rcm in enumerate(R_GRID):
74         rm = rcm * 0.01
75         if rm > Rt + 1e-15:
76             continue # 域外留 NaN
77         q = (rm / Rt) ** 2
78         p = int(np.searchsorted(q_nodes, q))
79         s = min(max(p - 1, 0), N - 1)
80         idx3 = [s, s + 1, s + 2]
81         qs3 = q_nodes[idx3]
82         wq = np.ones(3)
83         for a_ in range(3):
84             for b_ in range(3):
85                 if a_ != b_:

```

```

86         wq[a_] *= (q - qs3[b_]) / (qs3[a_] - qs3[b_])
87         C[k, j] = float(np.sum(wq * vC[idx3]))
88         T[k, j] = float(np.sum(wq * vT[idx3])) - 273.15 # 开尔文→℃
89         R_t[k] = Rt * 100.0
90         Cs_s[k] = Us
91         Cc_s[k] = 1.5 * U[0] - 0.5 * U[1]
92         assert not np.any(np.isnan(C[:, 0])), "中心列不得有 NaN"
93         n_in = np.isfinite(C)
94         print(f"[场导出] 帧数 {n_t} × 径向 {n_r} 点; 域内填充率 {n_in.mean()
          *100:.1f}%", flush=True)
95
96         np.savez(HERE / "q4_endo_fields.npz",
97                 t_s=frames, r_cm=R_GRID, C=C, T=T, R_t=R_t, Cs=Cs_s, Cc=Cc_s)
98         size = (HERE / "q4_endo_fields.npz").stat().st_size
99         print(f"[场导出] q4_endo_fields.npz 写出, {size/1e6:.2f} MB", flush=True)
100
101         # ----- 一致性抽查 (对 result4.xlsx 内生版) -----
102         wb = openpyxl.load_workbook(A_DIR / "结果" / "result4.xlsx", read_only=
            True)
103         ws = wb["Sheet1"]
104         rows = list(ws.iter_rows(min_row=2, values_only=True))
105         wb.close()
106         ok = True
107         for (tt, rcm) in ((36000.0, 1.0), (72000.0, 0.5), (126000.0, 0.3)):
108             row = [r for r in rows if r[0] == tt][0]
109             j_ref = int(round(rcm / 0.1)) # 列 j: x=0.1j cm
110             v_ref = row[1 + j_ref]
111             k = int(np.argmin(np.abs(frames - tt)))
112             j = int(np.argmin(np.abs(R_GRID - rcm)))
113             v_new = C[k, j]
114             d_ = abs(v_new - v_ref)
115             ok &= d_ < 1e-3
116             print(f"[抽查] t={tt:.0f} s, r={rcm} cm: result4={v_ref} vs npz={v_new
          :.4f} "
117                   f" (|Δ|={d_:.2e})", flush=True)
118         print(f"[抽查] 末帧中心 Cc = {Cc_s[-1]:.4f} (达标阈值 0.15, 应略低)",
119               flush=True)
120         assert ok, "一致性抽查失败"
121         assert Cc_s[-1] < 0.15 + 1e-3 and Cc_s[-1] > 0.13, "末帧中心值异常"
122         print(f"[完成] 全部通过, 总耗时 {time.perf_counter() - t_start:.1f} s",
123               flush=True)
124
125         (HERE / "q4_endo_fields_README.md").write_text(f""# q4_endo_fields.npz
126               说明
127
128 内生问题 4 (耦合内生几何, R 由湿度场经闭合 C 预测) 的完整时空场。
129 复现: `cd 数据 && python run_q4_endo_fields.py` (约 {time.perf_counter()-
130       t_start:.0f} s)。
131
132 | 键 | 形状 | 单位 | 说明 |
133 |---|---|---|---|

```

```

130 | t_s | ({n_t},) | s | 帧时刻, 0 → t_f={t_f_s:.1f} s, 帧间隔 {DT_FRAME:.0f} s
    |
131 | r_cm | ({n_r},) | cm | **固定物理网格** 0-2.0 cm、步长 {DR} cm (非材料网
    格) |
132 | C | ({n_t},{n_r}) | kg/kg (干基) | 水分浓度场; r>R(t) 处 NaN |
133 | T | ({n_t},{n_r}) | °C | 温度场 (内部开尔文计算、导出转 °C); 域外 NaN |
134 | R_t | ({n_t},) | cm | 每帧外半径 R_pred(t) (闭合 C 预测) |
135 | Cs | ({n_t},) | kg/kg | 表面含水率 |
136 | Cc | ({n_t},) | kg/kg | 中心含水率 (末帧 {Cc_s[-1]:.4f}≈达标) |
137
138 网格说明: 径向为固定物理网格 (r>R(t) 域外填 NaN); 空间重构为 q 内三点二次
139 (中心正则外推+单元中心+表面串联阻力值), 时间帧由接受步线性插值。
140 """ , encoding="utf-8")
141 print("written: q4_endo_fields_README.md", flush=True)
142
143
144 if __name__ == "__main__":
145     main()

```

E.11 潜热对照情景

表面相变汇的潜热对照情景，用于界定无潜热近似的适用范围（5.3 节）。

代码 12 潜热对照情景 (latent_scenario.py, 210 行)

```

1  """latent_scenario.py: 潜热对照情景 (审计 A01 处理, 2026-09-12)。
2
3  无潜热基线 (solver_q23.coupled_step, 现行主模型) vs 表面相变汇情景:
4  同一物性 (附录 3)、同一边界数据 (附件 1 + 末值保持)、同一数值框架
5  (BE + 局部外推 + 表面串联阻力结构; 潜热汇 Picard 线性化, 随外场迭代更新)。
6
7  情景定义 (诚实声明, 均为情景假设而非题目给定):
8  - 水通量用干密度定义 j_w = ρ_d,s · h_m · (U_s-U_e) [kg 水/(m² · s)],
9  ρ_d,s = ρ_附3(U_s)/(1+U_s) = (650+128 · U_s)/(1+U_s): 表面局部干物质密度
10 (单位总体积干物质质量, 随表面含水率变化; 注意与附录 ρ(C) 湿密度口径区别,
11 本文 ρ(C) 不作质量闭合, 故此干密度只用于本情景的潜热诊断)。
12 - 表面能量边界 -k∂_rT|_R = h(T_s-T_a) + L_v · j_w, L_v=2.4e6 J/kg (近似值)。
13 离散上与基线同一串联阻力结构: 表面汇等价于把环境驱动温度降低
14 L_v · j_w/h (R_ext · F_lat, Picard 冻结上一外场迭代的 j_w)。
15 - 全部相变发生在表面、忽略表面储能; 内部相变源情景不在本脚本求解
16 (量级讨论见 数据/latent_scenario.csv 头部注释与论文 §2.3-3)。
17
18 产物: ../数据/latent_scenario.csv (两模型 T_s/T_center/U_m 逐时对照 + t_f)。
19 内置断言: 基线 t_f 复现 57.1799 h (容差 0.02 h, 与生产 run_q23 同码同参)。
20 """
21 from __future__ import annotations
22
23 import sys
24 import time
25 from pathlib import Path
26
27 import numpy as np

```

```

28 from scipy.linalg import solve_banded
29
30 CODE_DIR = Path(__file__).resolve().parent
31 sys.path.insert(0, str(CODE_DIR))
32
33 import solver_q23
34 from solver_q1 import R0, H, HM, T0_K, C0
35 from solver_q23 import TH, rho_cp, k_of, D_of, dD_dC, surface_m,
    solve_coupled
36 from run_q23 import read_env, make_env
37
38 LV = 2.4e6      # 近似汽化潜热 J/kg (情景声明值)
39 TF_BASE_REF = 57.1799 # 生产基线 t_f3 (run_q23, N=160)
40 A_DIR = CODE_DIR.parent
41 OUT_CSV = A_DIR / "数据" / "latent_scenario.csv"
42
43
44 def rho_d_s(Us):
45     """表面局部干物质密度 kg/m³ (附录 3 湿密度  $\rho(C)=650+128C$  除以  $1+C$ )。"""
46     return (650.0 + 128.0 * Us) / (1.0 + Us)
47
48
49 def coupled_step_latent(U, T, dt, Ta, Cenv, dq, newton):
50     """coupled_step 的潜热情景变体: 签名与 solver_q23.coupled_step 完全一致。
51
52     差异仅在热场表面行: rhs 增加  $-dt \cdot R_{ext} \cdot F_{lat} / (R_h \cdot a \cdot dq)$ ,
53      $F_{lat}=2 \cdot L_v \cdot j_w / R0$  (q 形式通量, 与  $F_h$  同单位),  $j_w$  取上一外场迭代值
54     (Picard 线性化); 返回的  $F_{s,h}$  为进入表面单元的总热流 (已扣潜热汇)。
55     """
56     N = U.size
57     qf = np.arange(1, N) * dq
58     U_m, T_m = U.copy(), T.copy()
59     Us_m, _, _ = surface_m(U_m[-1], T_m[-1], Cenv, dq)
60     j_w = rho_d_s(Us_m) * HM * (Us_m - Cenv) # kg 水/(m²·s)
61     a = rho_cp(U_m)
62     for _ in range(40):
63         a = rho_cp(U_m)
64         kf = k_of(0.5 * (U_m[:-1] + U_m[1:]))
65         ch = 4.0 * qf * kf * dt / (R0**2 * dq**2)
66         ks = float(k_of(0.5 * (Us_m + U_m[-1])))
67         Rh = R0**2 * dq / (8.0 * ks) + R0 / (2.0 * H)
68         F_lat = 2.0 * LV * j_w / R0          # q 形式潜热汇
69         diag = 1.0 + (np.concatenate(([0.0], ch)) + np.concatenate((ch,
            [0.0]))) / a
70         diag[-1] += dt / (Rh * a[-1] * dq)
71         rhs = T.copy()
72         rhs[-1] += dt * (Ta - (R0 / (2.0 * H)) * F_lat) / (Rh * a[-1] * dq)
73         ab = np.zeros((3, N))
74         ab[0, 1:] = -ch / a[:-1]
75         ab[1, :] = diag
76         ab[2, :-1] = -ch / a[1:]

```

```

77 T_new = solve_banded((1, 1), ab, rhs)
78 Fs_h = (Ta - (R0 / (2.0 * H)) * F_lat - T_new[-1]) / Rh
79 Ts = T_new[-1] + Fs_h * R0**2 * dq / (8.0 * ks)
80 U_new = U_m.copy()
81 Tf = 0.5 * (T_new[:-1] + T_new[1:])
82 for _ in range(30):
83     Uf = 0.5 * (U_new[:-1] + U_new[1:])
84     Df = D_of(Uf, Tf)
85     b = 4.0 * qf * Df * dt / (R0**2 * dq**2)
86     if newton:
87         e = 0.5 * dD_dC(Uf, Tf) / Df * (U_new[1:] - U_new[:-1])
88     else:
89         e = np.zeros(N - 1)
90     Cs, Fs_m, Ds = surface_m(U_new[-1], Ts, Cenv, dq)
91     Rs = R0**2 * dq / (8.0 * Ds) + R0 / (2.0 * HM)
92     gs = dt / (Rs * dq)
93     fl = b * (U_new[1:] - U_new[:-1])
94     G = U_new - U - np.concatenate((fl, [gs * (Cenv - U_new[-1])])) \
95         + np.concatenate(([0.0], fl))
96     bn = np.concatenate((b, [0.0]))
97     en = np.concatenate((e, [0.0]))
98     bp = np.concatenate(([0.0], b))
99     ep = np.concatenate(([0.0], e))
100    J = np.zeros((3, N))
101    J[0, 1:] = -bn[:-1] * (1.0 + en[:-1])
102    J[1, :] = 1.0 + bn * (1.0 - en) + bp * (1.0 + ep)
103    J[1, -1] += gs
104    J[2, :-1] = -bn[:-1] * (1.0 - en[:-1])
105    dU = solve_banded((1, 1), J, -G)
106    U_new = U_new + dU
107    if np.max(np.abs(dU)) < 1e-11:
108        break
109    dUmax = float(np.max(np.abs(U_new - U_m)))
110    dTmax = float(np.max(np.abs(T_new - T_m)))
111    U_m, T_m = U_new, T_new
112    if not (np.all(np.isfinite(U_m)) and np.all(np.isfinite(T_m))
113        and U_m.min() > -1.0 and U_m.max() < 10.0
114        and T_m.min() > 200.0 and T_m.max() < 700.0):
115        raise RuntimeError("迭代越界，应缩小步长")
116    Us_m, _, _ = surface_m(U_m[-1], Ts, Cenv, dq)
117    j_w = rho_d_s(Us_m) * HM * (Us_m - Cenv) # Picard 更新水通量
118    if dUmax < 1e-10 and dTmax < 1e-8:
119        break
120    else:
121        raise RuntimeError("耦合外场未收敛，应缩小步长")
122    Us, Fs_m, _ = surface_m(U_m[-1], Ts, Cenv, dq)
123    return U_m, T_m, Us, Fs_m, Ts, Fs_h, a
124
125
126 def run_scenario(N, env, latent, hours=76):
127     """全程求解（复用 solver_q23.solve_coupled 驱动，潜热步经模块函数临时替换注

```

```

    入)。

128
129 返回 dict: t_f (crossing 右端点, 与生产口径一致)、逐时
130 (t_h, Ts, T_center, Um, Us) 记录。
131 """
132 dq = 1.0 / N
133 log = {k: [] for k in ("t", "Ts", "Tc", "Um", "Us")}
134
135 def record(tt, U, T, Us, Ts):
136     log["t"].append(tt)
137     log["Ts"].append(float(Ts) - 273.15)
138     log["Tc"].append(float(1.5 * T[0] - 0.5 * T[1]) - 273.15)
139     log["Um"].append(float(dq * np.sum(U)))
140     log["Us"].append(float(Us))
141
142 orig = solver_q23.coupled_step
143 if latent:
144     solver_q23.coupled_step = coupled_step_latent
145 try:
146     res = solve_coupled(N, env, 300000.0,
147                         out_times=tuple(np.arange(3600.0, hours * 3600.0,
148                         3600.0)),
149                         dt_max=10.0, record=record)
150 finally:
151     solver_q23.coupled_step = orig
152 t_f = res["crossing"][2] / 3600.0 if res["crossing"] else float("nan")
153 return dict(t_f=t_f, log=log, stats=res["stats"])
154
155 def main():
156     t0 = time.perf_counter()
157     t_env, TaK, Ce = read_env()
158     env_hold = make_env("hold", t_env, TaK, Ce)
159     N = 160
160     print(f"[基线] N={N} 无潜热全程 ...", flush=True)
161     base = run_scenario(N, env_hold, latent=False)
162     print(f" t_f = {base['t_f']:.4f} h (生产口径 57.1799 h)", flush=True)
163     assert abs(base["t_f"] - TF_BASE_REF) < 0.02, \
164         f"基线断言失败: {base['t_f']:.4f} vs {TF_BASE_REF}"
165     print(" 基线断言通过 ( $|\Delta| < 0.02$  h)", flush=True)
166
167     print(f"[情景] N={N} 表面相变汇 (L_v={LV:.2g}, p_d,s 表面局部) ...", flush=
168         True)
169     lat = run_scenario(N, env_hold, latent=True)
170     print(f" t_f = {lat['t_f']:.4f} h ( $\Delta t_f = \{lat['t_f'] - base['t_f']\} : +.4f$  h
171         )", flush=True)
172
173     # ---- 逐时对照表 ----
174     n = min(len(base["log"]["t"]), len(lat["log"]["t"]))
175     dTs = np.abs(np.array(lat["log"]["Ts"][:n]) - np.array(base["log"]["Ts"]
176        )[:n]))

```

```

174 dTc = np.abs(np.array(lat["log"]["Tc"][:n]) - np.array(base["log"]["Tc"]
175                [:n]))
176 dUm = np.abs(np.array(lat["log"]["Um"][:n]) - np.array(base["log"]["Um"]
177                [:n]))
178 print(f" 全场最大逐时 |ΔTs|={dTs.max():.3f} °C、|ΔTcenter|={dTc.max():.3
179 f} °C、"
180       f"|ΔUm|={dUm.max():.4f} kg/kg", flush=True)
181
182 # ---- 潜热通量级诊断 (t=1 h, 潜热解的表面量, 同审计口径) ----
183 Us1 = lat["log"]["Us"][0]
184 Ts1 = lat["log"]["Ts"][0]
185 Ta1 = float(env_hold(3600.0)[0]) - 273.15
186 Ce1 = float(env_hold(3600.0)[1])
187 jw1 = rho_d_s(Us1) * HM * (Us1 - Ce1)
188 print(f" t=1h 通量诊断: Us={Us1:.4f}, Ts={Ts1:.2f} °C, jw={jw1:.3e} kg
189       /(m2·s), "
190       f"L·jw={LV*jw1:.1f} W/m2, h(Ta-Ts)={H*(Ta1-Ts1):.1f} W/m2, "
191       f"比值={LV*jw1/max(H*(Ta1-Ts1),1e-300):.2f}", flush=True)
192 with open(OUT_CSV, "w", encoding="utf-8") as f:
193     f.write("# 潜热对照情景 (审计 A01): 表面相变汇 vs 无潜热基线; 同一附录
194           3 物性、"
195           "同一边界数据、同一 BE+局部外推框架; Lv=2.4e6 J/kg (近似值), \n"
196           ")
197     f.write("# jw=rhod,s*hm*(Us-Cenv), rhod,s=(650+128*Us)/(1+Us)
198           表面局部干密度;"
199           "情景假设, 非题目给定。内部相变源情景未求解 (量级见论文 §2.3-3) 。
200           \n")
201     f.write(f"# tf_base_h={base['t_f']:.6f}\n# tf_latent_h={lat['t_f']
202           ':.6f}\n"
203           f"# delta_t_f_h={lat['t_f']-base['t_f']:+.6f}\n"
204           f"# max_dTs_C={dTs.max():.4f}\n# max_dTcenter_C={dTc.max():.4f
205           }\n"
206           f"# max_dUm={dUm.max():.6f}\n"
207           f"# diag_1h: Us={Us1:.4f}, Ts_C={Ts1:.4f}, jw={jw1:.6e},"
208           f"Ljw_Wm2={LV*jw1:.2f}, hDTWm2=H*(Ta1-Ts1):.2f}\n")
209     f.write("t_h,Ts_base_C,Ts_latent_C,Tcenter_base_C,Tcenter_latent_C,
210           Um_base,Um_latent\n")
211     for i in range(n):
212         f.write(f"{base['log']['t'][i]/3600:.1f},{base['log']['Ts'][i]:.4f
213               },"
214               f"{lat['log']['Ts'][i]:.4f},{base['log']['Tc'][i]:.4f},"
215               f"{lat['log']['Tc'][i]:.4f},{base['log']['Um'][i]:.6f},"
216               f"{lat['log']['Um'][i]:.6f}\n")
217     print(f"written: 数据/latent_scenario.csv ({n} 行逐时对照)", flush=True)
218     print(f"[完成] 总耗时 {time.perf_counter()-t0:.0f} s", flush=True)
219
220 if __name__ == "__main__":
221     main()

```

E.12 力学驱动收缩模型

由守恒律与有限应变力学约束给出的内生收缩模型（5.5 节的扩展研究）。

代码 13 力学驱动收缩模型（mech_shrinkage.py, 303 行）

```

1  """A 级（几何反馈）力学驱动内生收缩模型（按 other/力学驱动内生收缩_方程与实施方案.md §3 - §10）。
```

2

3 有限应变（非仿射）：材料点 $X \in [0, R_0]$ ，现时 $r(X, t)$ ， $\lambda_r = r/X$ 、 $\lambda_\theta = r/X$ 、 $\lambda_z = 1$ 、 $J = \lambda_r \lambda_\theta = 2r r_q / R_0^2$ （ $q = X^2 / R_0^2$ ， $\rho_d = \rho_{d0} / J$ ）。不强行 $r = R\sqrt{q}$ 。

4 孔隙非负进方程： $g = J - s_0 - w \geq 0$ （ $s_0 = \rho_{d0} / \rho_s$ ， $w = \rho_{d0} U / \rho_w$ ），乘子 ζ 互补（ $g \geq 0$ ， $\zeta \geq 0$ ， $\zeta g = 0$ ；活动集法，绝不先算负再截断）。

5 毛细储能： $p_c = p^*(1 - S)$ （**明确标注的低阶检验本构**，非真实保水定律）； $\psi_{cap} = (J - s_0) / S^{q-1} p_c ds$ ， $\bar{p}_c = (p^* / 2)(1 - S^2)$ 。

6 单松弛黏弹骨架：对数主应变 $E = \text{diag}(\ln \lambda_r, \ln \lambda_\theta, 0)$ ， $\psi_{el} = G_\infty \|\text{dev} E\|^2 + K_\infty / 2 (\text{tr} E - e^*)^2 + G_1 \|\text{dev}(E - Z)\|^2 + K_1 / 2 [\text{tr}(E - Z)]^2$ ， $\dot{Z} = (E - Z) / \tau(U, T)$ （后向欧拉局部消元）。初始平衡 $e^* = \bar{p}_c(S_0) / K_\infty \Rightarrow$ 初始总应力为零。

7 总应力 $\sigma_i = H_i / J + \bar{p}_c - \zeta$ ；准静态径向平衡弱式（ X 向线性元，轴心正则极限）； $R(t) = r(R_0, t)$ 是输出。

8 传质（A 级最小衔接）： $U_t = \partial_q (D / r_q^2 U_q)$ ，湿边界外系数 $h_m / r_q(1, t)$ ；热方程显热近似 $a_{eff} T_t = (1 / (r r_q)) \partial_q (r k / r_q T_q)$ 。

9 """

10 from __future__ import annotations

11

12 import numpy as np

13 from scipy.linalg import solve_banded

14

15 from solver_q1 import R0, H, HM, T0_K, C0

16

17 RHO_W = 1000.0

18

19

20

21

22

23

24

25

26

27 class MechParams:

28 """材料/情景参数。压力单位任意（A 级静力绝对尺度不可辨识，只扫/标定比值）。

29

30 ret_model:

31 - 'linear': $p_c = p^*(1 - S)$ （A 级低阶检验本构，吸力随 $S \rightarrow 0$ 饱和于 p^* ）；

32 - 'power': $p_c = p^*(1 - S) / S^q$ （van Genuchten 类幂律保水形式，土壤/食品干燥常用：吸力随 $S \rightarrow 0$ 以 S^{-q} 持续增大（ $0 < q < 1$ ），且毛细能量 $\int p_c dS$ 有界； $\bar{p}_c(0) = p^* / [(1 - q)(2 - q)]$ 大而有限）。

33 """

34

35

36

37 def __init__(self, rho_d0, rho_s, Kinf=1.0, Ginf=1.0 / 3, K1=1.0, G1=1.0 / 3,

38 p_star=1.0, tau0=1.0e6, tau_exp=4.0, ret_model="linear", q_pow=0.8,

39 Pi_t0=0.0, k_tur=2.0):

40 self.rho_d0 = rho_d0

41 self.rho_s = rho_s

42 self.Kinf, self.Ginf = Kinf, Ginf

43 self.K1, self.G1 = K1, G1

44 self.p_star = p_star


```

45     self.tau0 = tau0
46     self.tau_exp = tau_exp #  $\tau(U)=\tau_0 \cdot (C_0/U)^{\tau_{exp}}$  (情景形式, 声明为假设)
47     self.ret_model = ret_model
48     self.q_pow = q_pow
49     self.Pi_t0 = Pi_t0 # 初始膨压 ( $K_{\infty}$  单位; 取  $K_{\infty}=1$  MPa 时按 MPa 解读)
50     self.k_tur = k_tur # 膨压松弛指数 (随失水单调下降)
51     self.s0 = rho_d0 / rho_s
52
53     def tau(self, U, T=323.0):
54         return self.tau0 * (C0 / np.maximum(np.asarray(U, float), 0.02)) ** self.tau_exp
55
56     def turgor(self, U):
57         """细胞膨压  $\Pi_t(U)=\Pi_{t0} \cdot (U/C_0)^{k_{tur}}$  (植物生理: 渗透压维持细胞饱满, 失水时膨压松弛 → 基体整体压缩; 与毛细项各司其职, 不重复归因)。"""
58         return self.Pi_t0 * (np.asarray(U, float) / C0) ** self.k_tur
59
60     def pc(self, S):
61         """保水关系  $p_c(S)$  (吸力,  $\geq 0$ , 随  $S$  增大而降低)。"""
62         S = np.maximum(np.asarray(S, float), 1e-12)
63         if self.ret_model == "linear":
64             return self.p_star * (1.0 - S)
65         return self.p_star * (1.0 - S) / S ** self.q_pow
66
67     def pbar_c(self, S):
68         """ $\bar{p}_c = \partial \psi_{cap} / \partial J = f(S) + S \cdot p_c(S)$ ,  $f(S) = \int S^{-1} p_c(s) ds$  (解析)。"""
69         S = np.asarray(S, float)
70         if self.ret_model == "linear":
71             return 0.5 * self.p_star * (1.0 - S**2)
72         q = self.q_pow
73         Sc = np.maximum(S, 1e-12)
74         f = self.p_star * ((1.0 - Sc ** (1.0 - q)) / (1.0 - q)
75                             - (1.0 - Sc ** (2.0 - q)) / (2.0 - q))
76         return f + self.p_star * (1.0 - Sc) * Sc ** (1.0 - q)
77
78     @property
79     def e_star(self):
80         w0 = self.rho_d0 * C0 / RHO_W
81         S0 = w0 / (1.0 - self.s0)
82         # 初始总应力为零: 弹性预应变平衡 (毛细吸力 - 膨压), 不重复归因
83         return (float(self.pbar_c(S0)) - self.Pi_t0) / self.Kinf
84
85
86     def elem_quantities(r, X, U, P):
87         """逐单元  $\lambda_r, \lambda_{\theta}$  (轴心正则极限)、 $J, w, g, Pore, S, r_q$ 。X 为节点 (M+1 个)。"""
88         M = len(X) - 1
89         dX = np.diff(X)
90         lam_r = np.diff(r) / dX
91         Xm = 0.5 * (X[:-1] + X[1:])

```

```

93     rm = 0.5 * (r[:-1] + r[1:])
94     lam_t = rm / Xm
95     lam_t[0] = lam_r[0]          # 正则极限  $\lambda_{\theta}(0)=\lambda_r(0)$ 
96     J = lam_r * lam_t
97     w = P.rho_d0 * U / RHO_W
98     g = J - P.s0 - w
99     Pore = J - P.s0
100    S = np.clip(w / np.maximum(Pore, 1e-300), 0.0, 1.0)
101    return lam_r, lam_t, J, w, g, Pore, S
102
103
104    def solve_mechanics(r_guess, Z_old, U, T, dt, X, P, active0=None, tol=1e
-10):
105        """受孔隙非负约束的准静态径向平衡（活动集 + FD-Newton；局部消元 Z）。
106
107        未知量: r_1..r_M (r_0=0 Dirichlet) + 活动接触单元的  $\zeta$ 。
108        返回 dict(r, Z, zeta, g, S, J, sigma_rr_elem, sigma_tt_elem, active)。
109        """
110        M = len(X) - 1
111        dX = np.diff(X)
112        e_ = P.e_star
113        r = r_guess.copy()
114        active = set() if active0 is None else set(active0)
115        Z_old = np.asarray(Z_old, float).reshape(M, 3)
116
117        def eval_state(ry, zeta_map):
118            lam_r, lam_t, J, w, g, Pore, S = elem_quantities(ry, X, U, P)
119            E = np.column_stack([np.log(np.maximum(lam_r, 1e-300)),
120                                np.log(np.maximum(lam_t, 1e-300)), np.zeros(M)])
121            tau = P.tau(U, T)
122            al = dt / np.maximum(tau, 1e-300)
123            Z = (Z_old + al[:, None] * E) / (1.0 + al[:, None])
124            trE = E.sum(1)
125            devE = E - (trE / 3.0)[:, None]
126            EmZ = E - Z
127            trEmZ = EmZ.sum(1)
128            devEmZ = EmZ - (trEmZ / 3.0)[:, None]
129            H = (2.0 * P.Ginf * devE + P.Kinf * (trE - e_)[:, None]
130                + 2.0 * P.G1 * devEmZ + P.K1 * trEmZ[:, None])
131            zeta = np.array([zeta_map.get(e, 0.0) for e in range(M)])
132            sig = H / J[:, None] + P.pbar_c(S)[:, None] - P.turgor(U)[:, None] -
                zeta[:, None]
133            return lam_r, lam_t, J, g, S, Z, sig
134
135        def residual(y, active_list):
136            ry = np.empty(M + 1)
137            ry[0] = 0.0
138            ry[1:] = y[:M]
139            zmap = {e: y[M + k] for k, e in enumerate(active_list)}
140            lam_r, lam_t, J, g, S, Z, sig = eval_state(ry, zmap)
141            Pr = J * sig[:, 0] / lam_r # 名义应力  $P_r = J\sigma_{rr}/\lambda_r$ 

```

```

142 Pt = J * sig[:, 1] / lam_t #  $P_{\theta} = J\sigma_{\theta\theta}/\lambda_{\theta}$ 
143 F = np.zeros(M + len(active_list))
144 # 内部节点  $j=1..M-1$ : 右单元  $e=j$  (右端) + 左单元  $e=j+1$  (左端)
145 for j in range(1, M):
146     eR, eL = j - 1, j
147     F[j - 1] = (Pr[eR] * 0.5 * (X[eR] + X[eR + 1])
148                 - Pr[eL] * 0.5 * (X[eL] + X[eL + 1])
149                 + 0.5 * dX[eR] * Pt[eR] + 0.5 * dX[eL] * Pt[eL])
150 # 表面节点  $M$  (自然边界  $\sigma_{rr}=0 \Rightarrow$  仅左单元贡献)
151 F[M - 1] = Pr[M - 1] * 0.5 * (X[M - 1] + X[M]) + 0.5 * dX[M - 1] * Pt
152                                     [M - 1]
153 # 接触单元:  $g=0$ 
154 for k, e in enumerate(active_list):
155     F[M + k] = g[e]
156 return F
157
158 res_norm = np.inf
159 newton_ok = False
160 n_nt = 0
161 for _as in range(60):
162     active_list = sorted(active)
163     y = np.concatenate([r[1:], np.zeros(len(active_list))])
164     newton_ok = False
165     for _nt in range(60):
166         n_nt = _nt + 1
167         F = residual(y, active_list)
168         res_norm = float(np.linalg.norm(F, np.inf))
169         if res_norm < tol:
170             newton_ok = True
171             break
172         n = len(y)
173         Jf = np.zeros((len(F), n))
174         for j in range(n):
175             dy = 1e-7 * max(1.0, abs(y[j]))
176             yp = y.copy(); yp[j] += dy
177             ym = y.copy(); ym[j] -= dy
178             Jf[:, j] = (residual(yp, active_list) - residual(ym,
179                                     active_list)) / (2 * dy)
180         try:
181             delta = np.linalg.solve(Jf, -F)
182         except np.linalg.LinAlgError:
183             delta = np.linalg.lstsq(Jf, -F, rcond=None)[0]
184         for damp in (1.0, 0.5, 0.25, 0.1, 0.02):
185             yt = y + damp * delta
186             rt = np.empty(M + 1); rt[0] = 0.0; rt[1:] = yt[:M]
187             _, _, Jt, *_ = elem_quantities(rt, X, U, P)
188             if np.all(Jt > 1e-8):
189                 y = yt
190                 break
191     else:
192         y = y + 0.005 * delta

```

```

191     r[1:] = y[:M]
192     zmap = {e: y[M + k] for k, e in enumerate(active_list)}
193     ry = np.empty(M + 1); ry[0] = 0.0; ry[1:] = y[:M]
194     lam_r, lam_t, J, g, S, Z, sig = eval_state(ry, zmap)
195     changed = False
196     for e in range(M):
197         if e not in active and g[e] < -1e-9:
198             active.add(e); changed = True
199         elif e in active and zmap.get(e, 0.0) < -1e-9:
200             active.discard(e); changed = True
201     if not changed:
202         break
203     zeta = np.array([zmap.get(e, 0.0) for e in range(M)])
204     # 收尾复核: 以最终活动集重算平衡残差 (牛顿失败时 res_norm 如实保留)
205     act_fin = sorted(active)
206     y_fin = np.concatenate([ry[1:], zeta[act_fin]]) if act_fin else ry[1:]
207     res_norm = float(np.linalg.norm(residual(y_fin, act_fin), np.inf))
208     newton_ok = bool(newton_ok and res_norm < 1e-8)
209     return dict(r=ry, Z=Z, zeta=zeta, g=g, S=S, J=J,
210               sig_r=sig[:, 0], sig_t=sig[:, 1], active=sorted(active), n_as=
211                 _as,
212               res_norm=res_norm, newton_ok=newton_ok, n_newton=n_nt,
213               gmin=float(np.min(g)), zmin=float(np.min(zeta)))
214
215 # =====
216 # 传质 + 显热 (非仿射几何系数, BE + Picard)
217 # =====
218
219 def mass_heat_step(U, T, dt, Ta, Cenv, r, X, Jc, P, Dof, kof, acp):
220     """一个 BE 步。U_t=∂_q(D/r_q^2 U_q); a_eff T_t=(1/(r r_q))∂_q(rk/r_q T_q)。
221
222     传质 (干基权重  $\rho_d \cdot r \cdot r_q$  为固定材料质量): 通量  $(D/r_q^2)U_q$ , 单元权重  $\Delta q$ 。
223
224     传热 (权重  $a \cdot r \cdot r_q$  随变形变化, 非仿射下不能套用传质算子):
225     单元热容权重  $W_e = a_e \cdot \int r r_q dq = a_e \cdot (r_{e+1}^2 - r_e^2)/2$ ,
226     面通量  $F = (rk/r_q)\Delta T/\Delta q$ , 两端由同一  $W_e$  归一;
227     热表面: 内阻  $r_{q-s}\Delta q/(2k_s r_s)$  与外阻  $1/(h r_s)$  串联 (与 W 配套);
228     湿表面: 内阻  $r_{q-s}^2\Delta q/(2D_s)$  与外阻  $r_{q-s}/h_m$  串联 (外系数  $h_m/r_q(1)$ )。
229     仿射  $r=R/\sqrt{q}$  下热离散精确退化到 solver_q4.coupled_step4 的形式。
230     Jc 保留于接口 (诊断用), 热权重不再使用  $R0^2 J/2$  近似。返回 (Unew, Tnew, Us,
231         Fm, Fh)。
232     """
233     M = len(X) - 1
234     q = (X / R0) ** 2
235     dqe = np.diff(q) # 单元宽  $\Delta q_e$  (非均匀)
236     rs = r[-1]
237     rq_s = (r[-1] - r[-2]) / dqe[-1] if M > 1 else (r[-1] / q[-1])
238     # 面量 (面  $j=1..M-1$ ): 中心距  $dc$ 、跨面差商  $rqf$ 
239     dc = 0.5 * (dqe[:-1] + dqe[1:])
240     rqf = (r[2:] - r[:-2]) / (q[2:] - q[:-2])

```

```

239 Un, Tn = U.copy(), T.copy()
240 for _ in range(30):
241     # ---- 湿 ----
242     Df = np.array([float(Dof(0.5 * (Un[j - 1] + Un[j]), 0.5 * (Tn[j - 1]
243         + Tn[j]))))
244         for j in range(1, M)])
245     bR = Df * dt / (rqf ** 2 * dc * dqe[:-1]) # 行 e 右面系数
246     bL = Df * dt / (rqf ** 2 * dc * dqe[1:]) # 行 e 左面系数
247     Us = Un[-1]
248     for _s in range(40):
249         Ds = float(Dof(0.5 * (Us + Un[-1]), Tn[-1])) # 与参考求解器一致: 半
250             单元中点 D
251         R_int = rq_s ** 2 * (dqe[-1] / 2.0) / Ds
252         R_ext = rq_s / HM
253         Us2 = (Un[-1] / R_int + Cenv / R_ext) / (1.0 / R_int + 1.0 / R_ext
254             )
255         if abs(Us2 - Us) < 1e-13:
256             Us = Us2
257             break
258         Us = Us2
259         Ds = float(Dof(0.5 * (Us + Un[-1]), Tn[-1]))
260         gs = dt / ((rq_s ** 2 * (dqe[-1] / 2.0) / Ds + rq_s / HM) * dqe[-1])
261         diag = 1.0 + np.concatenate(([0.0], bL)) + np.concatenate((bR, [0.0])
262             )
263         diag[-1] += gs
264         rhs = U.copy()
265         rhs[-1] += gs * Cenv
266         A = np.zeros((3, M))
267         A[0, 1:] = -bR
268         A[1, :] = diag
269         A[2, :-1] = -bL
270         Un2 = solve_banded((1, 1), A, rhs)
271         # ---- 热 (有限体积:  $W_e = a_e \cdot (r^2 \text{差})/2$ , 面通量  $F = (rk/r_q)\Delta T/\Delta q$ ,  $W$  归
272             一) ----
273         a = acp(Un2)
274         kf = np.array([float(kof(0.5 * (Un2[j - 1] + Un2[j])))) for j in range
275             (1, M)])
276         W = a * 0.5 * (r[1:] ** 2 - r[:-1] ** 2) # 单元热容权重  $a/r_r_q \Delta q$ 
277         Gf = kf * r[1:M] / rqf # 面传导系数  $(rk/r_q)_{-}\{j+1/2\}$ 
278         chR = Gf * dt / (dc * W[:-1])
279         chL = Gf * dt / (dc * W[1:])
280         ks = float(kof(0.5 * (Us + Un2[-1])))
281         Rh_int = rq_s * (dqe[-1] / 2.0) / (ks * rs) # 内阻  $rq\Delta q/(2kr)$ 
282         Rh_ext = 1.0 / (H * rs) # 外阻  $1/(hr)$ 
283         chs = dt / ((Rh_int + Rh_ext) * W[-1])
284         diagT = 1.0 + np.concatenate(([0.0], chL)) + np.concatenate((chR,
285             [0.0]))
286         diagT[-1] += chs
287         rhsT = T.copy()
288         rhsT[-1] += chs * Ta
289         AT = np.zeros((3, M))

```

```

283     AT[0, 1:] = -chR
284     AT[1, :] = diagT
285     AT[2, :-1] = -chL
286     Tn2 = solve_banded((1, 1), AT, rhsT)
287     dm = max(float(np.max(np.abs(Un2 - Un))), float(np.max(np.abs(Tn2 -
288         Tn))))
289     Un, Tn = Un2, Tn2
290     if dm < 1e-10:
291         break
292     Fm = HM * (Cenv - Us) / rq_s
293     Fh = (Ta - Tn[-1]) / (Rh_int + Rh_ext)
294     return Un, Tn, Us, Fm, Fh
295
296 def volume_check(r, X, U, P):
297     """全局体积核验式:  $R^2/R0^2 = s0 + (\rho_{d0}/\rho_w)\bar{U} + \int g \, dq$  (只核验一致性)。"""
298     q = (X / R0) ** 2
299     dq = np.diff(q)
300     _, _, J, w, g, *_ = elem_quantities(r, X, U, P)
301     Um = float(np.sum(U * dq))
302     rhs = P.s0 + P.rho_d0 / RHO_W * Um + float(np.sum(g * dq))
303     return (r[-1] / R0) ** 2, rhs

```

E.13 力学扩展主线

幂律保水本构标定到附件 2 的力学扩展主线模型与全套验证。

代码 14 力学扩展主线 (run_mech_mainline.py, 308 行)

```

1  """run_mech_mainline.py: 力学驱动内生收缩主线版——幂律保水本构标定到附件 2 +
2     全套验证。
3
4  用法: python run_mech_mainline.py
5  阶段: ① 标定 (p*, q, Π_τ 三参数, least_squares, 多起点);
6         ② 验证 (7 条退化检查、留出法、bootstrap CI、三段/交叉/平台、
7             末期孔隙率、耦合 t_f、与半经验闭合对比表)。
8  产物: mechMain_fit.csv、mechMain_validation.csv、mechMain_fields.npz、
9         ../08-临时/mechMain_fit.png。
10 """
11
12 from __future__ import annotations
13
14 import sys
15 import time
16 from pathlib import Path
17
18 import numpy as np
19 from scipy.optimize import least_squares
20
21 HERE = Path(__file__).resolve().parent
22 A_DIR = HERE.parent
23 sys.path.insert(0, str(A_DIR / "代码"))
24 sys.path.insert(0, str(A_DIR / "数据"))

```

```

23
24 from solver_q1 import R0, T0_K, C0
25 from solver_q4 import D_of4, k_of4, rho_cp4
26 from mech_shrinkage import (MechParams, elem_quantities, solve_mechanics,
27                             mass_heat_step, volume_check)
28 from run_mech_A import (P_of, make_X, env_hold, t_att, R_att, mech_only_run
29                         ,
30                         T_DRY, RHO_S, sC0)
31 PHI_G0 = 0.1726 # 体积审计初始孔隙率下界
32 RHO_D0 = (1 - PHI_G0) / sC0
33 TH = 0.15
34
35
36 def P_fit(p_star, q_pow, Pi_tau, tau_exp=4.0):
37     return MechParams(rho_d0=RHO_D0, rho_s=RHO_S, Kinf=1.0, Ginf=1 / 3,
38                       K1=1.0, G1=1 / 3, p_star=p_star, tau0=Pi_tau * T_DRY,
39                       tau_exp=tau_exp, ret_model="power", q_pow=q_pow)
40
41
42 def coupled_run(P, t_end, dt=1200.0, M=16, detect=True):
43     """力学→几何→传质两轮迭代耦合; detect 时记录 umax 穿越区间。"""
44     X = make_X(M)
45     r = X.copy()
46     Z = np.zeros((M, 3))
47     U = np.full(M, C0)
48     T = np.full(M, T0_K)
49     active = None
50     log_t, log_R, log_um, log_Us, log_gmin, log_gS, log_DeS, log_vol = [],
51         [], [], [], [], [], [], []
52     cross = None
53     n_steps = int(round(t_end / dt))
54     rec_every = max(1, n_steps // 240)
55     n_newton_fail, max_res = 0, 0.0
56     for k in range(1, n_steps + 1):
57         tt = k * dt
58         Ta, Ce_ = env_hold(tt)
59         # 预测-校正耦合（每区间仅一次传质推进）：预测与校正都必须以同一份
60         # 历史内变量  $Z^n$  为旧态（solve_mechanics 内部会推进 Z），
61         # 只在接受该时间步时提交一次  $Z^{n+1}$ （A05 修复：此前校正从  $Z_p$  出发，
62         # 同一步松弛被推进两次， $(1+\Delta t/\tau)^{-1}$  变  $^{-2}$ ）。
63         mech = solve_mechanics(r, Z, U, float(np.mean(T)), dt, X, P, active0=
64                               active)
65         r_p, act_p = mech["r"], mech["active"]
66         U_new, T_new, Us, Fm, Fh = mass_heat_step(U, T, dt, Ta, Ce_, r_p, X,
67           mech["J"], P,
68           D_of4, k_of4, rho_cp4)
69         mech2 = solve_mechanics(r_p, Z, U_new, float(np.mean(T_new)), dt, X,
70                               P,
71                               active0=act_p)
72         r, Z, active, U, T = mech2["r"], mech2["Z"], mech2["active"], U_new,

```

```

T_new
69 for m_ in (mech, mech2):
70     max_res = max(max_res, m_["res_norm"])
71     if not m_["newton_ok"]:
72         n_newton_fail += 1
73         if n_newton_fail <= 20:
74             print(f" [warn] t={tt:.0f}s 力学牛顿未收敛 res_norm={m_['
              res_norm']:.2e}",
75                   flush=True)
76     umax = float(np.max(U))
77     if detect and cross is None and umax < TH:
78         cross = (tt - dt, tt)
79     if k % rec_every == 0 or k == n_steps:
80         _, _, J, w, g, Pore, S = elem_quantities(r, X, U, P)
81         lhs, rhs = volume_check(r, X, U, P)
82         log_t.append(tt)
83         log_R.append(float(r[-1]))
84         log_um.append(float(np.sum(U * np.diff((X / R0) ** 2))))
85         log_Us.append(float(Us))
86         log_gmin.append(float(np.min(g)))
87         log_gS.append(float(g[-1]))
88         log_DeS.append(float(P.tau(max(Us, 0.02)) / T_DRY))
89         log_vol.append(float(abs(lhs - rhs) / max(lhs, 1e-300)))
90     return dict(t=np.array(log_t), R=np.array(log_R), Um=np.array(log_um),
91               Us=np.array(log_Us), gmin=np.array(log_gmin), gS=np.array(
92                 log_gS),
93               DeS=np.array(log_DeS), volerr=np.array(log_vol),
94               X=X, r=r, U=U, T=T, Z=Z, g=g, S=S, cross=cross,
95               newton_fail=n_newton_fail, max_res=max_res)
96
97 def rmse_of(out, idx=None):
98     Rp = np.interp(t_att, out["t"], out["R"])
99     e = Rp - R_att
100     if idx is not None:
101         e = e[idx]
102     return float(np.sqrt(np.mean(e ** 2)))
103
104
105 def fit_once(x0):
106     """x = [log10(p*), q, log10( $\Pi\tau$ ), tau_exp]. """
107     def obj(x):
108         P = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
109         out = coupled_run(P, t_att[-1], dt=1200.0, M=16, detect=False)
110         Rp = np.interp(t_att, out["t"], out["R"])
111         return Rp - R_att
112     return least_squares(obj, x0, bounds=([0.0, 0.15, -3.0, 0.0], [4.0,
113       0.98, 3.0, 10.0]),
114       xtol=1e-3, ftol=1e-6, max_nfev=60)
115

```



```

116 def degradation(P, label=""):
117     """7 条退化检查（方案 §15）。"""
118     M = 16
119     X = make_X(M)
120     res = {}
121     dt, t_end = 60.0, 4 * 3600.0
122     Udec = lambda tt: np.full(M, C0 - 1.0 * tt / t_end)
123     # (i) 不干燥 + 初始总应力平衡 ⇒ 无人跳变
124     _, r1, _, _ = mech_only_run(P, lambda tt: np.full(M, C0), 301.15, dt,
125                                 t_end, M=M)
126     res["i_nojump"] = float(np.max(np.abs(r1 - X)))
127     # (ii) 关闭黏弹分支 ⇒ 纯弹性
128     P2 = P_fit(P.p_star, P.q_pow, P.tau0 / T_DRY, P.tau_exp)
129     P2.G1 = P2.K1 = 0.0
130     _, r2, _, _ = mech_only_run(P2, Udec, 323.315, dt, t_end, M=M)
131     res["ii_elastic"] = float(r2[-1])
132     # (iii)  $\tau \ll \Rightarrow$  充分松弛  $\approx$  (ii)
133     P3 = P_fit(P.p_star, P.q_pow, 1e-9, P.tau_exp)
134     _, r3, _, _ = mech_only_run(P3, Udec, 323.315, dt, t_end, M=M)
135     res["iii_vs_ii"] = float(np.max(np.abs(r3 - r2)))
136     # (iv)  $\tau \gg \Rightarrow$  内变量近冻结
137     P4 = P_fit(P.p_star, P.q_pow, 1e9, P.tau_exp)
138     _, r4, Z4, _ = mech_only_run(P4, Udec, 323.315, dt, t_end, M=M)
139     Emax = float(np.max(np.abs(np.log(np.maximum(np.diff(r4) / np.diff(X), 1
140                                                    e-300))))))
141     res["iv_frozen"] = float(np.max(np.abs(Z4))) / max(Emax, 1e-300)
142     # (v) 仿射映射  $\Rightarrow$  恢复  $4qD/R^2$ 
143     lam = 0.8
144     r5 = lam * X
145     q5 = (X / R0) ** 2
146     rqf = (r5[2:] - r5[:-2]) / (q5[2:] - q5[:-2])
147     qf = 0.25 * (np.sqrt(q5[2:]) + np.sqrt(q5[:-2])) ** 2
148     res["v_affine"] = float(np.max(np.abs(1.0 / rqf ** 2 - 4.0 * qf / (lam *
149                                                                    R0) ** 2))
150                               / (4.0 * qf / (lam * R0) ** 2)))
151     # (vi) 固定外形失水  $\Rightarrow g$  按恒等式增长
152     U6a = np.full(M, C0)
153     U6b = np.full(M, C0 - 0.5)
154     _, _, _, _, g6a, *_ = elem_quantities(X, X, U6a, P)
155     _, _, _, _, g6b, *_ = elem_quantities(X, X, U6b, P)
156     res["vi_growth"] = float(np.max(np.abs((g6b - g6a) - P.rho_d0 / 1000.0 *
157                                                    0.5))
158                               / (P.rho_d0 / 1000.0 * 0.5)))
159     return res
160
161 def main():
162     t_start = time.perf_counter()
163     print("=" * 64, flush=True)
164     print("[标定] 幂律保水本构  $p_c = p(1-S)/S^q$ , 参数 ( $p^*$ ,  $q$ ,  $\Pi_\tau$ ), 多起点 ...",
165           flush=True)

```

```

162 best = None
163 for x0 in ([1.0, 0.8, 0.0, 4.0], [2.0, 0.5, -1.0, 6.0],
164           [0.5, 0.9, 1.0, 8.0], [1.5, 0.6, 0.0, 2.0]):
165     t0 = time.perf_counter()
166     sol = fit_once(x0)
167     rm = float(np.sqrt(np.mean(sol.fun ** 2)))
168     print(f" 起点 {x0}: p*={10**sol.x[0]:.3g} q={sol.x[1]:.4f}  $\Pi_\tau$ ={10**
169           sol.x[2]:.3g} "
170           f"tau_exp={sol.x[3]:.3f} RMSE={rm*1000:.2f} mm ({time.
171             perf_counter()-t0:.0f}s)",
172           flush=True)
173     if best is None or rm < best[1]:
174       best = (sol, rm)
175 sol, rm_fit = best
176 p_star, q_pow, Pi_tau, tau_exp = 10 ** sol.x[0], sol.x[1], 10 ** sol.x
177 [2], sol.x[3]
178 print(f"[标定] 最优: p*={p_star:.4g}, q={q_pow:.4f},  $\Pi_\tau$ ={Pi_tau:.4g}, "
179       f"tau_exp={tau_exp:.4f}, RMSE={rm_fit*1000:.2f} mm", flush=True)
180
181 print("[生产] 拟合参数全程运行 (dt=600 s) ...", flush=True)
182 P = P_fit(p_star, q_pow, Pi_tau, tau_exp)
183 out = coupled_run(P, t_att[-1], dt=600.0, M=16, detect=True)
184 Rp = np.interp(t_att, out["t"], out["R"])
185 res = Rp - R_att
186 rmse = float(np.sqrt(np.mean(res ** 2)))
187 seg = lambda a, b: float(np.sqrt(np.mean(res[(t_att >= a) & (t_att < b)]
188   ** 2)))
189 r0 = res - res.mean()
190 lag1 = float(np.sum(r0[1:] * r0[:-1]) / np.sum(r0 ** 2))
191 print(f"[生产] RMSE={rmse*1000:.2f} mm, max|偏差|={np.max(np.abs(res))
192   *1000:.2f} mm, "
193       f"lag-1={lag1:.3f}", flush=True)
194 print(f" 分段 RMSE: 急缩 {seg(0, 3.5*3600)*1000:.2f} / 缓缩 {seg
195   (3.5*3600, 21*3600)*1000:.2f}"
196       f" / 平台 {seg(21*3600, 1e12)*1000:.2f} mm", flush=True)
197 # 滞后交叉 (m=R/R_ideal 由 <1 转 >1)
198 um_att = np.interp(t_att, out["t"], out["Um"])
199 R_ideal = R0 * np.sqrt((1 + um_att) / (1 + C0))
200 m_d = R_att / R_ideal
201 m_p = Rp / R_ideal
202 cross_d = t_att[np.argmax(m_d > 1.0)] / 3600
203 cross_p = t_att[np.argmax(m_p > 1.0)] / 3600 if np.any(m_p > 1.0) else
204   float("nan")
205 print(f" 滞后交叉: data≈{cross_d:.1f} h, pred≈{cross_p:.1f} h; "
206       f"平台: data {R_att[-1]*100:.3f} vs pred {Rp[-1]*100:.4f} cm", flush
207   =True)
208 print(f" t_f(耦合): 穿越区间 [{out['cross'][0]:.0f}, {out['cross
209   '][1]:.0f}] s = "
210       f"{out['cross'][1]/3600:.4f} h (半经验内生 50.5369 / 附件2外生
211   50.8230 h)", flush=True)
212 # 末期孔隙率

```

```

203 _, _, J_end_arr, *_ = elem_quantities(out["r"], out["X"], out["U"], P)
204 phi_pore_end = 1.0 - P.s0 / J_end_arr[-1]
205 print(f"    末期孔隙率(表面单元): {phi_pore_end*100:.1f}% (附件2 独立推
    ~31%) ", flush=True)
206
207 print("[验证] 7 条退化检查 (拟合本构) ...", flush=True)
208 dg = degradation(P)
209 dg["vii_gmin"] = float(np.min(out["gmin"]))
210 for k, v in dg.items():
211     print(f" {k} = {v:.3e}", flush=True)
212
213 print("[验证] 留出法 (奇偶折半) ...", flush=True)
214 ho = {}
215 for name, tr, te in (("偶→奇", slice(0, None, 2), slice(1, None, 2)),
216                     ("奇→偶", slice(1, None, 2), slice(0, None, 2))):
217     def obj_h(x):
218         P_h = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
219         o = coupled_run(P_h, t_att[-1], dt=1800.0, M=16, detect=False)
220         return np.interp(t_att[tr], o["t"], o["R"]) - R_att[tr]
221     s_h = least_squares(obj_h, sol.x, bounds=([0.0, 0.15, -3.0, 0.0],
222                                             [4.0, 0.98, 3.0, 10.0]),
223                       xtol=1e-3, ftol=1e-6, max_nfev=60)
224     P_h = P_fit(10 ** s_h.x[0], s_h.x[1], 10 ** s_h.x[2], s_h.x[3])
225     o_h = coupled_run(P_h, t_att[-1], dt=1800.0, M=16, detect=False)
226     e_tr = float(np.sqrt(np.mean((np.interp(t_att[tr], o_h["t"], o_h["R"]
227                                             ] - R_att[tr]) ** 2)))
228     e_te = float(np.sqrt(np.mean((np.interp(t_att[te], o_h["t"], o_h["R"]
229                                             ] - R_att[te]) ** 2)))
230     ho[name] = (s_h, e_tr, e_te)
231     print(f" {name}: 训练 {e_tr*1000:.2f} / 留出 {e_te*1000:.2f} mm, "
232           f"参数 p*={10**s_h.x[0]:.3g} q={s_h.x[1]:.3f} Π_τ={10**s_h.x
233           [2]:.3g} "
234           f"tau_exp={s_h.x[3]:.2f}", flush=True)
235
236 print("[验证] 块 bootstrap (B=20, 块长 24, dt=1800 粗算) ...", flush=True)
237 rng = np.random.default_rng(20260912)
238 boots = []
239 res_full = res
240 n = len(res_full)
241 for b_ in range(20):
242     starts = rng.integers(0, n, n // 24 + 1)
243     resamp = np.concatenate([np.roll(res_full, s) for s in starts])[:n]
244     R_syn = Rp + resamp - res_full.mean()
245     def obj_b(x):
246         P_b = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
247         o = coupled_run(P_b, t_att[-1], dt=1800.0, M=16, detect=False)
248         return np.interp(t_att, o["t"], o["R"]) - R_syn
249     s_b = least_squares(obj_b, sol.x, bounds=([0.0, 0.15, -3.0, 0.0],
250                                             [4.0, 0.98, 3.0, 10.0]),
251                       xtol=1e-3, ftol=1e-6, max_nfev=40)
252     boots.append([10 ** s_b.x[0], s_b.x[1], 10 ** s_b.x[2], s_b.x[3]])

```

```

248 boots = np.array(boots)
249 for i, nm in enumerate(("p*", "q", "Π-τ", "tau_exp")):
250     lo, hi = np.percentile(boots[:, i], [2.5, 97.5])
251     print(f" {nm}: 标定 {[p_star, q_pow, Pi_tau, tau_exp][i]:.4g}, 95% CI
252           [{lo:.3g}, {hi:.3g}]",
253           flush=True)
254
255 # ----- 落数据 -----
256 with open(HERE / "mechMain_fit.csv", "w", encoding="utf-8") as f:
257     f.write("t_s,R_data_cm,R_pred_cm,residual_mm\n")
258     for i in range(len(t_att)):
259         f.write(f"{t_att[i]:.0f},{R_att[i]*100:.4f},{Rp[i]*100:.4f},{res[i]
260               ]*1000:.4f}\n")
261 with open(HERE / "mechMain_validation.csv", "w", encoding="utf-8") as f:
262     f.write("item,value\n")
263     f.write(f"p_star,{p_star:.6g}\nq_pow,{q_pow:.6f}\nPi_tau,{Pi_tau:.6g
264           }\ntau_exp,{tau_exp:.6g}\n")
265     f.write(f"rmse_mm,{rmse*1000:.4f}\nmaxdev_mm,{np.max(np.abs(res))
266           }*1000:.4f}\n")
267     f.write(f"lag1,{lag1:.4f}\n")
268     f.write(f"seg_early_mm,{seg(0, 3.5*3600)*1000:.4f}\n")
269     f.write(f"seg_mid_mm,{seg(3.5*3600, 21*3600)*1000:.4f}\n")
270     f.write(f"seg_late_mm,{seg(21*3600, 1e12)*1000:.4f}\n")
271     f.write(f"cross_data_h,{cross_d:.2f}\ncross_pred_h,{cross_p:.2f}\n")
272     f.write(f"tf_s,{out['cross'][1]:.0f}\n")
273     f.write(f"phi_pore_end_pct,{phi_pore_end*100:.2f}\n")
274     for k, v in dg.items():
275         f.write(f"deg_{k},{v:.6e}\n")
276     for name, (s_h, e_tr, e_te) in ho.items():
277         f.write(f"holdout_{name}_train_mm,{e_tr*1000:.4f}\n")
278         f.write(f"holdout_{name}_test_mm,{e_te*1000:.4f}\n")
279 np.savez(HERE / "mechMain_fields.npz", t=out["t"], R=out["R"], Um=out["
280       Um"],
281       DeS=out["DeS"], gS=out["gS"], X=out["X"], r=out["r"], U=out["U"],
282       fit_params=np.array([p_star, q_pow, Pi_tau, tau_exp]), boots=boots
283       )
284 print(f"[完成] 总耗时 {time.perf_counter()-t_start:.0f} s", flush=True)
285
286 # ----- 图 -----
287 import matplotlib
288 matplotlib.use("Agg")
289 import matplotlib.pyplot as plt
290 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
291 plt.rcParams["axes.unicode_minus"] = False
292 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 7), sharex=True,
293                               gridspec_kw=dict(height_ratios=[3, 1]))
294
295 th = t_att / 3600
296 ax1.plot(th, R_att * 100, "k-", lw=2, label="附件 2 (实测)")
297 ax1.plot(th, Rp * 100, "r--", lw=1.8,
298          label=f"力学主线 (幂律保水, RMSE={rmse*1000:.2f} mm)")
299 ax1.plot(th, R_ideal * 100, "b:", lw=1.2, label="失水理想收缩基线")

```

```

293     for x in (3.5, 21.0):
294         ax1.axvline(x, color="gray", alpha=0.4, ls=":")
295     ax1.set_ylabel("R (cm)")
296     ax1.legend(fontsize=9)
297     ax1.set_title("力学驱动内生收缩 (van Genuchten 类幂律保水) 标定到附件 2")
298     ax2.plot(th, res * 1000, "g-")
299     ax2.axhline(0, color="k", lw=0.5)
300     ax2.set_ylabel("残差 (mm)")
301     ax2.set_xlabel("t (h)")
302     fig.tight_layout()
303     fig.savefig(A_DIR / "08-临时" / "mechMain_fit.png", dpi=150)
304     print("written: 08-临时/mechMain_fit.png", flush=True)
305
306
307 if __name__ == "__main__":
308     main()

```

E.14 力学扩展退化与情景

力学扩展模型的退化检查与无量纲情景研究。

代码 15 力学扩展退化与情景 (run_mech_A.py, 297 行)

```

1  """run_mech_A.py: A 级力学驱动内生收缩——退化检查 (方案 §15) + 无量纲情景研究
   (§13) 。
2
3  一条命令: python run_mech_A.py
4  产物: mechA_degradation.csv、mechA_scenarios.csv、mechA_fields_central.npz、
5      ../08-临时/mechA_scenarios.png (均在本目录/08-临时)。
6  诚实红线: 缺保水/力学数据 ⇒ 只作情景分析; p_c=p*(1-S) 是标注的低阶检验本构。
7  """
8  from __future__ import annotations
9
10 import sys
11 import time
12 from pathlib import Path
13
14 import numpy as np
15 import openpyxl
16
17 HERE = Path(__file__).resolve().parent
18 A_DIR = HERE.parent
19 sys.path.insert(0, str(A_DIR / "代码"))
20 sys.path.insert(0, str(A_DIR / "数据"))
21
22 from solver_q1 import R0, T0_K, C0, HM, H
23 from solver_q4 import D_of4, k_of4, rho_cp4
24 from mech_shrinkage import MechParams, elem_quantities, solve_mechanics,
    mass_heat_step, volume_check
25 from run_q23 import read_env, make_env
26
27 RHO_S = 1468.0

```

```

28 RHO_W = 1000.0
29 sC0 = 1.0 / RHO_S + C0 / RHO_W
30 T_DRY = R0 ** 2 / float(D_of4(C0, 323.315)) # 干燥特征时间 (附录4典型 D)
31
32 env_t, TaK, Ce = read_env()
33 env_hold = make_env("hold", env_t, TaK, Ce)
34
35 wb = openpyxl.load_workbook(A_DIR / "01-题目" / "原始文件" / "附件2.xlsx",
    read_only=True)
36 rows = [r for r in wb[wb.sheetnames[0]].iter_rows(min_row=2, values_only=
    True) if r[0] is not None]
37 wb.close()
38 t_att = np.array([r[0] for r in rows], float)
39 R_att = np.array([r[1] for r in rows], float) * 0.01
40
41
42 def make_X(M):
43     return R0 * np.sqrt(np.linspace(0, 1, M + 1))
44
45
46 def P_of(phi_g0, Pi_c, Pi_tau, tau_exp=4.0):
47     rho_d0 = (1 - phi_g0) / sC0
48     return MechParams(rho_d0=rho_d0, rho_s=RHO_S, Kinf=1.0, Ginf=1 / 3, K1
        =1.0, G1=1 / 3,
49         p_star=2.0 * Pi_c, tau0=Pi_tau * T_DRY, tau_exp=tau_exp)
50
51
52 def mech_only_run(P, U_of_t, Tc, dt, t_end, M=16):
53     """给定湿度场 (均匀或逐单元) 的纯力学推进。"""
54     X = make_X(M)
55     r = X.copy()
56     Z = np.zeros((M, 3))
57     log = []
58     active = None
59     for tt in np.arange(dt, t_end + 1e-9, dt):
60         U = U_of_t(tt)
61         out = solve_mechanics(r, Z, U, Tc, dt, X, P, active0=active)
62         r, Z, active = out["r"], out["Z"], out["active"]
63         log.append((tt, float(out["r"][-1]), out["g"].copy(), out["sig_r"].
            copy()))
64     return X, r, Z, log
65
66
67 # =====
68 # 退化/正确性检查 (方案 §15 七条)
69 # =====
70 def degradation():
71     res = {}
72     dt, t_end = 60.0, 4 * 3600.0
73     Udec = lambda tt: np.full(16, C0 - 1.0 * tt / t_end)
74     # (i) 不干燥 + 初始总应力平衡 ⇒ 尺寸无人跳变

```

```

75 P1 = P_of(0.1726, 1.0, 1.0)
76 X1, r1, _, _ = mech_only_run(P1, lambda tt: np.full(16, C0), 301.15, dt,
    t_end)
77 res["i_nojump"] = float(np.max(np.abs(r1 - X1)))
78 # (ii) 关闭黏弹分支  $\Rightarrow$  纯弹性
79 P2 = P_of(0.1726, 1.0, 1.0)
80 P2.G1 = P2.K1 = 0.0
81 _, r2, _, log2 = mech_only_run(P2, Udec, 323.315, dt, t_end)
82 res["ii_elastic_R_end"] = float(r2[-1])
83 # (iii)  $\tau \ll$  过程尺度  $\Rightarrow$  充分松弛 (应与 (ii) 一致)
84 P3 = P_of(0.1726, 1.0, 1e-9)
85 _, r3, _, log3 = mech_only_run(P3, Udec, 323.315, dt, t_end)
86 res["iii_vs_ii_maxdR"] = float(np.max(np.abs(r3 - r2)))
87 # (iv)  $\tau \gg$   $\Rightarrow$  内变量近冻结
88 P4 = P_of(0.1726, 1.0, 1e9)
89 _, r4, Z4, _ = mech_only_run(P4, Udec, 323.315, dt, t_end)
90 Emax = float(np.max(np.abs(np.log(np.maximum(np.diff(r4) / np.diff(X1),
    1e-300)))))
91 res["iv_maxZ_over_Emax"] = float(np.max(np.abs(Z4))) / max(Emax, 1e-300)
92 # (v) 指定仿射映射  $\Rightarrow$  传质系数恢复  $4qD/R^2$ 
93 M5 = 16
94 X5 = make_X(M5)
95 lam = 0.8
96 r5 = lam * X5
97 q5 = (X5 / R0) ** 2
98 rqf = (r5[2:] - r5[:-2]) / (q5[2:] - q5[:-2])
99 qf = 0.25 * (np.sqrt(q5[2:]) + np.sqrt(q5[:-2])) ** 2
100 coef_num = 1.0 / rqf ** 2
101 coef_ana = 4.0 * qf / (lam * R0) ** 2
102 res["v_affine_reterr"] = float(np.max(np.abs(coef_num - coef_ana) /
    coef_ana))
103 # (vi) 固定外形持续失水  $\Rightarrow g$  按体积恒等式增长
104 P6 = P_of(0.1726, 1.0, 1.0)
105 U6a = np.full(M5, C0)
106 U6b = np.full(M5, C0 - 0.5)
107 _, _, _, _, g6a, *_ = elem_quantities(X5, X5, U6a, P6)
108 _, _, _, _, g6b, *_ = elem_quantities(X5, X5, U6b, P6)
109 dg_num = g6b - g6a
110 dg_ana = P6.rho_d0 / RHO_W * 0.5
111 res["vi_growth_reterr"] = float(np.max(np.abs(dg_num - dg_ana) / dg_ana)
    )
112 # (vii) 全程  $g \geq 0$  (在中心情景全程检验, 见场景运行)
113 # (viii) 热算子退化对拍 (审计 A04 门禁, 2026-09-12 固化): 固定圆柱、无失
    水、
114 # 常物性  $pc_p=820 \times 2600$ 、 $k=0.36$ , 初温  $T0_K$ 、空气阶跃  $323.15 K$ ,
115 # 与 solver_q4.coupled_step4 同一后向欧拉步对照 (仿射  $r=X$  下两实现应一致)
116 from solver_q4 import coupled_step4
117 Pp = {"rho_cp": lambda C: np.full_like(np.asarray(C, float), 820 *
    2600.),
    "k_of": lambda C: np.full_like(np.asarray(C, float), 0.36),
    "D_of": lambda C, T: np.full_like(np.asarray(C, float), 1e-9),

```

```

120     "dD_dC": lambda C, T: np.zeros_like(np.asarray(C, float))}
121 mat = MechParams(rho_d0=250.0, rho_s=1468.0)
122 for N in (16, 80, 160):
123     XN = R0 * np.sqrt(np.linspace(0, 1, N + 1))
124     UN, TN = np.full(N, C0), np.full(N, T0_K)
125     for dtv in ((1.0, 60.0) if N < 160 else (60.0,)):
126         ref = coupled_step4(UN, TN, dtv, 323.15, C0, 1 / N, False, R0, Pp)
127         mh = mass_heat_step(UN, TN, dtv, 323.15, C0, XN, XN, np.ones(N),
128                             mat,
129                             Pp["D_of"], Pp["k_of"], Pp["rho_cp"])
130         res[f"viii_heat_N{N}_dt{dtv:g}"] = float(np.max(np.abs(ref[1] - mh
131 [1])))
132     res["viii_heat_refinement"] = bool(
133         res["viii_heat_N160_dt60"] < res["viii_heat_N80_dt60"] < res["
134         viii_heat_N16_dt60"])
135     return res
136
137 # =====
138 # 耦合情景运行
139 # =====
140 def coupled_run(P, t_end=60 * 3600.0, dt=900.0, M=16, frames=121):
141     """力学→几何→传质 迭代耦合（每步 2 轮）。返回日志 dict。"""
142     X = make_X(M)
143     r = X.copy()
144     Z = np.zeros((M, 3))
145     U = np.full(M, C0)
146     T = np.full(M, T0_K)
147     active = None
148     out_t, out_R, out_Um, out_Us, out_gmin, out_gS, out_DeS, out_volerr,
149     out_Jmin = \
150     [], [], [], [], [], [], [], [], []
151     i_fr = 1
152     n_steps = int(round(t_end / dt))
153     for k in range(1, n_steps + 1):
154         tt = k * dt
155         Ta, Ce_ = env_hold(tt)
156         for _ in range(2):
157             # A05 修复：两轮候选都以同一份  $Z^n$  为旧态，步末只提交一次
158             mech = solve_mechanics(r, Z, U, float(np.mean(T)), dt, X, P,
159                                     active0=active)
160             r2, active2 = mech["r"], mech["active"]
161             Jc = mech["J"]
162             U2, T2, Us2, Fm, Fh = mass_heat_step(U, T, dt, Ta, Ce_, r2, X, Jc,
163                                                  P,
164                                                  D_of4, k_of4, rho_cp4)
165             r, active, U, T = r2, active2, U2, T2
166             Z = mech["Z"]
167         if k % max(1, n_steps // (frames - 1)) == 0 or k == n_steps:
168             _, _, J, w, g, Pore, S = elem_quantities(r, X, U, P)
169             lhs, rhs = volume_check(r, X, U, P)

```



```

165     out_t.append(tt)
166     out_R.append(float(r[-1]))
167     out_Um.append(float(np.sum(U * np.diff((X / R0) ** 2))))
168     out_Us.append(float(Us2))
169     out_gmin.append(float(np.min(g)))
170     out_gS.append(float(g[-1]))
171     out_DeS.append(float(P.tau(max(Us2, 0.02)) / T_DRY))
172     out_volerr.append(float(abs(lhs - rhs) / max(lhs, 1e-300)))
173     out_Jmin.append(float(np.min(J)))
174     return dict(t=np.array(out_t), R=np.array(out_R), Um=np.array(out_Um),
175                Us=np.array(out_Us), gmin=np.array(out_gmin), gS=np.array(
176                    out_gS),
177                DeS=np.array(out_DeS), volerr=np.array(out_volerr),
178                Jmin=np.array(out_Jmin), X=X, r=r, U=U, T=T, Z=Z, g=g, S=S)
179
180 def rmse_seg(t, Rpred):
181     Rp = np.interp(t_att, t, Rpred)
182     err = Rp - R_att
183     seg = lambda a, b: float(np.sqrt(np.mean(err[(t_att >= a) & (t_att < b)]
184         ** 2)))
185     return float(np.sqrt(np.mean(err ** 2))), seg(0, 3.5 * 3600), seg(3.5 *
186         3600, 21 * 3600), seg(21 * 3600, 1e12)
187
188 def main():
189     t_start = time.perf_counter()
190     print(f"T_DRY = {T_DRY:.3e} s = {T_DRY/3600:.1f} h", flush=True)
191
192     # ----- 退化检查 -----
193     print("[退化检查] 运行中 ...", flush=True)
194     dg = degradation()
195     dg["vii_gmin_all"] = np.nan # 由情景运行补
196     for k, v in dg.items():
197         print(f" {k} = {v:.3e}", flush=True)
198
199     # ----- 情景扫描 -----
200     print("[情景]  $\Pi_c \times \Pi_\tau$  扫描 ...", flush=True)
201     rows = []
202     central = None
203     for  $\Pi_c$  in (0.2, 1.0, 5.0):
204         for  $\Pi_\tau$  in (0.01, 1.0, 100.0):
205             P = P_of(0.1726,  $\Pi_c$ ,  $\Pi_\tau$ )
206             t0 = time.perf_counter()
207             out = coupled_run(P, t_end=60 * 3600.0, dt=900.0, M=16)
208             wall = time.perf_counter() - t0
209             rm, e1, e2, e3 = rmse_seg(out["t"], out["R"])
210             gmin = float(np.min(out["gmin"]))
211             rows.append(( $\Pi_c$ ,  $\Pi_\tau$ , rm, e1, e2, e3, gmin, out["R"][-1],
212                 wall))
213     print(f"  $\Pi_c$ ={ $\Pi_c$ :4.1f}  $\Pi_\tau$ ={ $\Pi_\tau$ :7.2f}: RMSE={rm*100:.3f} cm "

```

```

212         f"(急缩 {e1*100:.3f} / 缓缩 {e2*100:.3f} / 平台 {e3*100:.3f}) "
213         f"g_min={gmin:.2e} R_end={out['R'][-1]*100:.3f} cm ({wall:.0f}
214         s)", flush=True)
215     if Pi_c == 1.0 and Pi_tau == 1.0:
216         central = out
217     dg["vii_gmin_all"] = min(r[6] for r in rows)
218     print(f" (vii) 全部情景 g_min = {dg['vii_gmin_all']:.3e} ( $\geq -1e-10$  即通
219     过) ", flush=True)
220
221     # ----- 初始孔隙率传播 (中央情景  $\pm$ ) -----
222     print("[情景] 初始孔隙率  $\pm$  传播 ...", flush=True)
223     for phi in (0.10, 0.30):
224         P = P_of(phi, 1.0, 1.0)
225         out = coupled_run(P, t_end=60 * 3600.0, dt=900.0, M=16)
226         rm, e1, e2, e3 = rmse_seg(out["t"], out["R"])
227         print(f"  $\phi_{g0}={\phi:.4f}$ : RMSE={rm*100:.3f} cm, R_end={out['R'][-1]*100:.3f} cm", flush=True)
228
229     # ----- 中央情景加密收敛性 -----
230     print("[核对] 中央情景 M=48/dt=450 收敛性 ...", flush=True)
231     P48 = P_of(0.1726, 1.0, 1.0)
232     out48 = coupled_run(P48, t_end=60 * 3600.0, dt=450.0, M=48)
233     rm48, *_ = rmse_seg(out48["t"], out48["R"])
234     print(f" M=48/dt=450: RMSE={rm48*100:.3f} cm (vs M=16/dt=900 的 "
235     f"[r for r in rows if r[0]==1.0 and r[1]==1.0][0][2]*100:.3f} cm) "
236     , flush=True)
237
238     # ----- 落数据 -----
239     with open(HERE / "mechA_degradation.csv", "w", encoding="utf-8") as f:
240         f.write("check,value,threshold,pass\n")
241         f.write(f"i_nojump_maxdr,{dg['i_nojump']:.3e},<1e-8,{dg['i_nojump'] < 1e-8}\n")
242         f.write(f"ii_elastic_R_end_cm,{dg['ii_elastic_R_end']*100:.4f},sane, True\n")
243         f.write(f"iii_vs_ii_maxdR_cm,{dg['iii_vs_ii_maxdR']*100:.3e},<1e-3,{dg['iii_vs_ii_maxdR'] < 1e-5}\n")
244         f.write(f"iv_maxZ_over_Emax,{dg['iv_maxZ_over_Emax']:.3e},<1e-3,{dg['iv_maxZ_over_Emax'] < 1e-3}\n")
245         f.write(f"v_affine_relerr,{dg['v_affine_relerr']:.3e},<1e-10,{dg['v_affine_relerr'] < 1e-10}\n")
246         f.write(f"vi_growth_relerr,{dg['vi_growth_relerr']:.3e},<1e-12,{dg['vi_growth_relerr'] < 1e-12}\n")
247         f.write(f"vii_gmin_all,{dg['vii_gmin_all']:.3e},>-1e-10,{dg['vii_gmin_all'] > -1e-10}\n")
248         f.write(f"viii_heat_N16_dt1,{dg['viii_heat_N16_dt1']:.3e},<0.2,{dg['viii_heat_N16_dt1'] < 0.2}\n")
249         f.write(f"viii_heat_N16_dt60,{dg['viii_heat_N16_dt60']:.3e},<0.5,{dg['viii_heat_N16_dt60'] < 0.5}\n")
250         f.write(f"viii_heat_N80_dt1,{dg['viii_heat_N80_dt1']:.3e},<0.05,{dg['viii_heat_N80_dt1'] < 0.05}\n")
251         f.write(f"viii_heat_N80_dt60,{dg['viii_heat_N80_dt60']:.3e},<0.2,{dg

```

```

249         ['viii_heat_N80_dt60'] < 0.2}\n")
f.write(f"viii_heat_N160_dt60,{dg['viii_heat_N160_dt60']:.3e},<0.1,{
    dg['viii_heat_N160_dt60'] < 0.1}\n")
250 f.write(f"viii_heat_refinement,{dg['viii_heat_refinement']},True,{dg
    ['viii_heat_refinement']}\n")
251 with open(HERE / "mechA_scenarios.csv", "w", encoding="utf-8") as f:
252     f.write("Pi_c,Pi_tau,rmse_cm,err_early_cm,err_mid_cm,err_late_cm,
        g_min,R_end_cm,wall_s\n")
253     for r_ in rows:
254         f.write(f"{r_[0]},{r_[1]},{r_[2]*100:.4f},{r_[3]*100:.4f},{r_
            [4]*100:.4f},"
            f"{r_[5]*100:.4f},{r_[6]:.3e},{r_[7]*100:.3f},{r_[8]:.0f}\n"
            )
256 np.savez(HERE / "mechA_fields_central.npz",
257         t=central["t"], R=central["R"], Um=central["Um"], Us=central["Us"
            ],
258         DeS=central["DeS"], gS=central["gS"], gmin=central["gmin"],
259         X=central["X"], r_final=central["r"], U_final=central["U"],
260         Z_final=central["Z"], g_final=central["g"], S_final=central["S"],
261         volerr=central["volerr"])
262 print(f"[完成] 总耗时 {time.perf_counter() - t_start:.0f} s", flush=True)
263
264 # ----- 图 -----
265 import matplotlib
266 matplotlib.use("Agg")
267 import matplotlib.pyplot as plt
268 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
269 plt.rcParams["axes.unicode_minus"] = False
270 fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
271 ax = axes[0]
272 ax.plot(t_att / 3600, R_att * 100, "k-", lw=2, label="附件 2")
273 for Pi_c, mk in ((0.2, ":"), (1.0, "-"), (5.0, "--")):
274     for Pi_tau, col in ((0.01, "C0"), (1.0, "C3"), (100.0, "C2")):
275         r_ = [x for x in rows if x[0] == Pi_c and x[1] == Pi_tau][0]
276         # 只画中央行与两端
277         for (Pi_c, Pi_tau, *_rest) in [(0.2, 1.0), (1.0, 1.0), (5.0, 1.0), (1.0,
            0.01), (1.0, 100.0)]:
278             P = P_of(0.1726, Pi_c, Pi_tau)
279             out = coupled_run(P, t_end=60 * 3600.0, dt=900.0, M=16)
280             ax.plot(out["t"] / 3600, out["R"] * 100,
                label=f" $\Pi_c$ ={Pi_c:g},  $\Pi_\tau$ ={Pi_tau:g}")
281 ax.set_xlabel("t (h)"); ax.set_ylabel("R (cm)"); ax.legend(fontsize=8)
282 ax.set_title("情景 R(t) vs 附件 2")
283 ax2 = axes[1]
284 ax2.plot(central["t"] / 3600, central["gS"], label="g (表面单元)")
285 ax2.plot(central["t"] / 3600, central["gmin"], label="g_min (全域)")
286 ax2.axhline(0, color="k", lw=0.5)
287 ax2.set_xlabel("t (h)"); ax2.set_ylabel("g (气相体积/初始体积)")
288 ax2.legend(fontsize=8)
289 ax2.set_title("中央情景: 孔隙非负全程成立")
290 fig.tight_layout()
291

```

```

292     fig.savefig(A_DIR / "08-临时" / "mechA_scenarios.png", dpi=150)
293     print("written: 08-临时/mechA_scenarios.png", flush=True)
294
295
296 if __name__ == "__main__":
297     main()

```

E.15 力学扩展重标定

修复热算子与内变量推进后的重标定，以及结构性上限分析（5.5 节）。

代码 16 力学扩展重标定（run_mech_refit.py, 196 行）

```

1  """run_mech_refit.py: A04（热算子）+A05（Z 推进）修复后的重标定 + 结构性上限分
   析。
2
3  用法: python run_mech_refit.py
4  依据: other/论文审计意见-de5b9ab-20260912.md（A04/A05）;
5      数据/力学升级_修复重标定.md（本脚本产出笔记）。
6  """
7  from __future__ import annotations
8
9  import sys
10 import time
11 from pathlib import Path
12
13 import numpy as np
14 from scipy.optimize import least_squares
15
16 HERE = Path(__file__).resolve().parent
17 A_DIR = HERE.parent
18 sys.path.insert(0, str(A_DIR / "代码"))
19 sys.path.insert(0, str(A_DIR / "数据"))
20
21 from solver_q1 import C0
22 from run_mech_mainline import P_fit, coupled_run, degradation, rmse_of
23 from run_mech_A import t_att, R_att, T_DRY, RHO_S, sC0
24
25 TH = 0.15
26 BOUNDS = ([0.0, 0.02, -3.0, 0.0], [4.0, 0.98, 3.0, 10.0]) # x=[log10 p*, β,
   log10 Π_τ, tau_exp]
27
28
29 def fit_once(x0, dt=1200.0):
30     def obj(x):
31         P = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
32         out = coupled_run(P, t_att[-1], dt=dt, M=16, detect=False)
33         return np.interp(t_att, out["t"], out["R"]) - R_att
34     return least_squares(obj, x0, bounds=BOUNDS, xtol=1e-3, ftol=1e-6,
   max_nfev=60)
35
36

```

```

37 def seg_err(Rp, a, b):
38     m = (t_att >= a) & (t_att < b)
39     return float(np.sqrt(np.mean((Rp[m] - R_att[m]) ** 2)))
40
41
42 def terminal_J(beta, w_end=0.0128, s0=0.1743):
43     """大 p* 极限下末端平衡 J: 解  $J \cdot g(S(J); \beta) = g(S0; \beta)$  (弹性体、内变量松弛)。”
44
45     import sys as _s
46     _s.path.insert(0, str(A_DIR / "代码"))
47     from mech_shrinkage import MechParams
48     P = MechParams(rho_d0=1.0, rho_s=RHO_S, p_star=1.0, ret_model="power",
49                    q_pow=beta)
50     gS0 = float(P.pbar_c(0.7907))
51     from scipy.optimize import brentq
52
53     def F(J):
54         S = min(w_end / max(J - s0, 1e-12), 1.0)
55         return J * float(P.pbar_c(S)) - gS0
56     lo = s0 + w_end + 1e-9
57     if F(lo) > 0:
58         return lo
59     return brentq(F, lo, 1.0, xtol=1e-12)
60
61 def main():
62     t_start = time.perf_counter()
63     print("=" * 64, flush=True)
64     print("[重标定] A04 热算子 + A05 Z 推进修复后求解器,  $\beta$  下界 0.02, 多起点 ...", flush=True)
65     best = None
66     for x0 in ([1.0, 0.15, 0.0, 0.1], [0.5, 0.05, 0.0, 0.5],
67               [2.0, 0.5, -1.0, 4.0], [1.5, 0.08, -1.0, 2.0]):
68         t0 = time.perf_counter()
69         sol = fit_once(x0)
70         rm = float(np.sqrt(np.mean(sol.fun ** 2)))
71         print(f" 起点 {x0}: p*={10**sol.x[0]:.3g}  $\beta$ ={sol.x[1]:.4f}  $\Pi_\tau$ ={10**sol.x[2]:.3g} "
72               f" $\tau_{exp}$ ={sol.x[3]:.3f} RMSE={rm*1000:.3f} mm ({time.perf_counter() - t0:.0f}s)",
73               flush=True)
74         if best is None or rm < best[1]:
75             best = (sol, rm)
76     sol, rm_fit = best
77     p_star, beta, Pi_tau, tau_exp = 10 ** sol.x[0], sol.x[1], 10 ** sol.x[2], sol.x[3]
78     print(f"[重标定] 最优: p*={p_star:.4g},  $\beta$ ={beta:.4f},  $\Pi_\tau$ ={Pi_tau:.4g}, "
79           f" $\tau_{exp}$ ={tau_exp:.4f}, RMSE={rm_fit*1000:.3f} mm", flush=True)
80     print("[生产] 拟合参数全程运行 (dt=600 s, 积分到 72 h, 不首次达标即停) ...", flush=True)

```

```

81 P = P_fit(p_star, beta, Pi_tau, tau_exp)
82 out = coupled_run(P, t_att[-1], dt=600.0, M=16, detect=True)
83 Rp = np.interp(t_att, out["t"], out["R"])
84 res = Rp - R_att
85 rmse = float(np.sqrt(np.mean(res ** 2)))
86 r0 = res - res.mean()
87 lag1 = float(np.sum(r0[1:] * r0[:-1]) / np.sum(r0 ** 2))
88 print(f"[生产] RMSE={rmse*1000:.3f} mm, max|偏差|={np.max(np.abs(res))
      *1000:.2f} mm, "
89       f"lag-1={lag1:.3f}", flush=True)
90 print(f"    分段: 急缩 {seg_err(Rp,0,3.5*3600)*1000:.3f} / 缓缩 {seg_err(
      Rp,3.5*3600,21*3600)*1000:.3f}"
91       f" / 平台 {seg_err(Rp,21*3600,1e12)*1000:.3f} mm", flush=True)
92 tc = out["cross"]
93 t_f_h = tc[1] / 3600 if tc else float("nan")
94 if tc:
95     w1 = t_att <= tc[0]
96     w2 = t_att >= tc[1]
97     print(f"    首次达标 t_f = {t_f_h:.4f} h (区间 [{tc[0]/3600:.2f},{tc
      [1]/3600:.2f}] h); "
98           f"误差分窗: 0 - 达标 {np.sqrt(np.mean(res[w1]**2))*1000:.3f} / "
99           f"达标 - 72h {np.sqrt(np.mean(res[w2]**2))*1000:.3f} / 全程 {rmse
      *1000:.3f} mm",
100          flush=True)
101 i60 = int(np.argmin(np.abs(out["t"] - 60 * 3600)))
102 print(f"    平台: pred {Rp[-1]*100:.4f} vs data {R_att[-1]*100:.3f} cm; "
103       f"R(60h)={out['R'][i60]*100:.4f} → R(72h)={out['R'][-1]*100:.4f} cm"
104       f" (72h 内再降 {(out['R'][i60]-out['R'][-1])*1000:.2f} mm, "
105       f"{'平台保持' if out['R'][i60]-out['R'][-1] < 0.001 else '末端仍在收
      缩'})"; "
106       f"t_f 对照 半经验 50.5369 / 附件2外生 50.82 h", flush=True)
107 print(f"    g_min = {out['gmin'].min():.4f}; 牛顿未收敛步数 = {out['
      newton_fail']}, "
108       f"最大平衡残差 = {out['max_res']:.2e}", flush=True)
109
110 print("[退化] 7 条退化检查 ...", flush=True)
111 dg = degradation(P)
112 dg["vii_gmin"] = float(out["gmin"].min())
113 for k, v in dg.items():
114     print(f" {k} = {v:.3e}", flush=True)
115
116 print("[结构上限] 末端平衡 J(β) 曲线与平台可达性 ...", flush=True)
117 rows = []
118 for b_ in (0.02, 0.05, 0.10, 0.15, 0.30, 0.50, 0.85):
119     Jt = terminal_J(b_)
120     rows.append((b_, Jt))
121     print(f" β={b_:.2f}: 末端 J_eq = {Jt:.4f} → R = {2*np.sqrt(Jt)*100:.3f
      } cm"
122           f"{' ← 平台 0.36/1.198 可达' if Jt >= 0.355 else ''}", flush=True
123           )
124 print(f" 附件 2: J = 0.36, R = 1.198 cm; 拟合 β={beta:.3f} 的末端 J = ")

```

```

124         f"{terminal_J(beta):.4f} → R = {2*np.sqrt(terminal_J(beta))*100:.3f}
           cm", flush=True)
125
126     print("[留出] 奇偶折半 ...", flush=True)
127     for name, tr, te in (("偶→奇", slice(0, None, 2), slice(1, None, 2)),
128                          ("奇→偶", slice(1, None, 2), slice(0, None, 2))):
129         def obj_h(x):
130             P_h = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
131             o = coupled_run(P_h, t_att[-1], dt=1800.0, M=16, detect=False)
132             return np.interp(t_att[tr], o["t"], o["R"]) - R_att[tr]
133         s_h = least_squares(obj_h, sol.x, bounds=BOUNDS, xtol=1e-3, ftol=1e
           -6, max_nfev=60)
134         P_h = P_fit(10 ** s_h.x[0], s_h.x[1], 10 ** s_h.x[2], s_h.x[3])
135         o_h = coupled_run(P_h, t_att[-1], dt=1800.0, M=16, detect=False)
136         e_tr = float(np.sqrt(np.mean((np.interp(t_att[tr], o_h["t"], o_h["R"
           ]) - R_att[tr]) ** 2)))
137         e_te = float(np.sqrt(np.mean((np.interp(t_att[te], o_h["t"], o_h["R"
           ]) - R_att[te]) ** 2)))
138         print(f" {name}: 训练 {e_tr*1000:.3f} / 留出 {e_te*1000:.3f} mm, "
139               f"β={s_h.x[1]:.4f}", flush=True)
140
141     print("[bootstrap] 块 B=15, 块长 24, dt=1800 ...", flush=True)
142     rng = np.random.default_rng(20260913)
143     boots = []
144     n = len(res)
145     for b_ in range(15):
146         starts = rng.integers(0, n, n // 24 + 1)
147         resamp = np.concatenate([np.roll(res, s) for s in starts])[n:]
148         R_syn = Rp + resamp - res.mean()
149         def obj_b(x):
150             P_b = P_fit(10 ** x[0], x[1], 10 ** x[2], x[3])
151             o = coupled_run(P_b, t_att[-1], dt=1800.0, M=16, detect=False)
152             return np.interp(t_att, o["t"], o["R"]) - R_syn
153         s_b = least_squares(obj_b, sol.x, bounds=BOUNDS, xtol=1e-3, ftol=1e
           -6, max_nfev=40)
154         boots.append([10 ** s_b.x[0], s_b.x[1], 10 ** s_b.x[2], s_b.x[3]])
155     boots = np.array(boots)
156     for i, nm in enumerate(("p*", "β", "Π_τ", "τ_exp")):
157         lo, hi = np.percentile(boots[:, i], [2.5, 97.5])
158         print(f" {nm}: 标定 {[p_star, beta, Pi_tau, tau_exp][i]:.4g}, 95% CI
159               [{lo:.3g}, {hi:.3g}]",
160               flush=True)
161
162     with open(HERE / "mechRefit.csv", "w", encoding="utf-8") as f:
163         f.write("t_s,R_data_cm,R_pred_cm,residual_mm\n")
164         for i in range(len(t_att)):
165             f.write(f"{t_att[i]:.0f},{R_att[i]*100:.4f},{Rp[i]*100:.4f},{res[i
166                   ]*1000:.4f}\n")
167     np.savez(HERE / "mechRefit_fields.npz", t=out["t"], R=out["R"], Um=out["
           Um"],
168            DeS=out["DeS"], gS=out["gS"], X=out["X"], r=out["r"], U=out["U"],

```

```

167         fit_params=np.array([p_star, beta, Pi_tau, tau_exp]), boots=boots,
168         terminal=np.array(rows), t_f_h=t_f_h,
169         newton_fail=out["newton_fail"], max_res=out["max_res"])
170     print(f"[完成] 总耗时 {time.perf_counter()-t_start:.0f} s", flush=True)
171
172     import matplotlib
173     matplotlib.use("Agg")
174     import matplotlib.pyplot as plt
175     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
176     plt.rcParams["axes.unicode_minus"] = False
177     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 7), sharex=True,
178                                   gridspec_kw=dict(height_ratios=[3, 1]))
179     th = t_att / 3600
180     ax1.plot(th, R_att * 100, "k-", lw=2, label="附件 2 (实测)")
181     ax1.plot(th, Rp * 100, "r--", lw=1.8,
182             label=f"重标定 ( $\beta$ ={{beta:.3f}}, RMSE={{rmse*1000:.2f}} mm)")
183     ax1.set_ylabel("R (cm)")
184     ax1.legend(fontsize=9)
185     ax1.set_title("A04/A05 修复后重标定 (幂律保水, 膨压关闭)")
186     ax2.plot(th, res * 1000, "g-")
187     ax2.axhline(0, color="k", lw=0.5)
188     ax2.set_ylabel("残差 (mm)")
189     ax2.set_xlabel("t (h)")
190     fig.tight_layout()
191     fig.savefig(A_DIR / "08-临时" / "mechRefit.png", dpi=150)
192     print("written: 08-临时/mechRefit.png", flush=True)
193
194
195 if __name__ == "__main__":
196     main()

```

E.16 判据回代与敏感性汇总

无量纲判据回代检验、敏感性汇总、误差分量汇总与尾段结构诊断。

代码 17 判据回代与敏感性汇总 (run_m4.py, 567 行)

```

1  """M4: 判据族 (D1) + 敏感性矩阵 (D2) + 误差预算 (D3) + V2 尾段对拍 (D4) + 结果
2      总账 (D5)。
3
4  用法: python run_m4.py (需 run_q1/run_q23/run_q4 的产物 npz 已存在)
5  产物 (../数据/):
6      criteria.csv 判据族表 (Le/Bi_m/Bi_h 全区间 + 反验  $\Delta t_f$ )
7      sensitivity.csv 敏感性矩阵 (每组实测  $\Delta t_f$ )
8      error_budget.csv 误差预算表 ( $\geq 6$  源 + 确定性合成)
9      v2_tail.csv V2 尾段衰减率对照
10     paper_tables.csv 论文表 1-6 全部数字
11     结果总账.csv 全项目关键数字唯一数据源
12
13 """
14 from __future__ import annotations
15
16 import csv

```



```

15 import json
16 import sys
17 import time
18 from pathlib import Path
19
20 import numpy as np
21
22 CODE_DIR = Path(__file__).resolve().parent
23 sys.path.insert(0, str(CODE_DIR))
24
25 from solver_q1 import R0, H, HM, T0_K, C0
26 from analytic_heat import RobinCylinder, robin_eigs
27 import solver_q23 as sq23
28 from solver_q23 import TH
29 from solver_q4 import PROPS4, solve_coupled4
30 from run_q23 import read_env, make_env, interp_rows
31 from run_q4 import read_radius
32
33 A_DIR = CODE_DIR.parent
34 DATA_DIR = A_DIR / "数据"
35
36 CE_Q3 = 0.04986    # 题目口径（末值保持）
37 CE_YAM = 0.1366    # 山药解吸支（题外修正，只入敏感性）
38 TF3_H = 57.17992518
39 TF4_H = 50.823025
40
41 # 敏感性行的“备注覆盖”：问题 4 效应拆分那三行须与 数据/run_q4_endo.py 的
    ledger upsert
42 # 逐字一致（键、值、备注），否则两脚本会把「外生基准」与「内生基准」两套行按运
    行顺序互相覆盖。
43 SENS_NOTE: dict[str, str] = {}
44
45
46 def read_q4_split():
47     """q4_split.csv（由 数据/run_q4_endo.py 产出）→ {case: 数值}; 文件缺失返回
        {}。"""
48     p = DATA_DIR / "q4_split.csv"
49     if not p.exists():
50         return {}
51     with open(p, encoding="utf-8") as f:
52         return {r["case"]: float(r["t_f_h"]) for r in csv.DictReader(f)}
53
54
55 # =====
56 # D1 判据族
57 # =====
58
59 def props3():
60     return dict(rho_cp=lambda C: (650 + 128 * np.asarray(C, float)) * (1450
        + 2736 * np.asarray(C, float) / (np.asarray(C, float) + 1)),
61                 k=lambda C: 0.21 + 0.38 * np.asarray(C, float) / (np.asarray(C,

```

```

        float) + 1),
62         D=lambda C, T: 2.4e-3 * np.exp(-0.45 / np.asarray(C, float)) *
            np.exp(-3850.0 / np.asarray(T, float)))
63
64
65 def criteria_table():
66     """Le=α/D、Bi_m=h_mR/D、Bi_h=hR/k 沿 (C,T) 路径的全区间（附3/附4）。"""
67     Cg = np.linspace(0.15, 2.55, 50)
68     rows = []
69     for name, P in (("附录3", props3()), ("附录4", dict(rho_cp=PROPS4["rho_cp"],
70         k=PROPS4["k_of"], D=PROPS4["D_of"]))):
71         Le_all, Bim_all, Bih_all = [], [], []
72         for T in (301.15, 323.315):
73             a = P["k"](Cg) / P["rho_cp"](Cg)
74             D = P["D"](Cg, T)
75             Le_all.append(a / D)
76             Bim_all.append(HM * R0 / D)
77             Bih_all.append(H * R0 / P["k"](Cg))
78         Le_all, Bim_all, Bih_all = map(np.concatenate, (Le_all, Bim_all,
79             Bih_all))
80         rows.append((name, Le_all.min(), Le_all.max(), Bim_all.min(), Bim_all
81             .max(),
82             Bih_all.min(), Bih_all.max()))
83     return rows
84
85
86 def run_patched(N, env, patch, t_end=400000.0):
87     saved = {k: getattr(sq23, k) for k in patch}
88     try:
89         for k, v in patch.items():
90             setattr(sq23, k, v)
91         res = sq23.solve_coupled(N, env, t_end, out_times=(), dt_max=15.0)
92     finally:
93         for k, v in saved.items():
94             setattr(sq23, k, v)
95     return res
96
97
98 def run_le_simplified(N, env, t_end=450000.0):
99     """Le→∞ 反验：准稳态热场 T≡T_a(t)，只求解湿分（D(C, T_a(t)), BE+Newton+对
100         分自适应）。
101
102         直接用 a_eff→0 的 patch 会让半离散热方程 stiff、附件 1 的 60 s 噪声折点把步
103         长
104         压死；准稳态的正确实现是温度场直接用环境值、不求解热方程。
105     """
106     from scipy.linalg import solve_banded
107     dq = 1.0 / N
108     qf = np.arange(1, N) * dq
109     U = np.full(N, C0)
110     t, dt = 0.0, 1e-3

```

```

106 crossing = None
107
108 def step(U, h, Ta, Ce):
109     Un = U.copy()
110     for _ in range(30):
111         Uf = 0.5 * (Un[:-1] + Un[1:])
112         Df = sq23.D_of(Uf, Ta)
113         b = 4.0 * qf * Df * h / (R0**2 * dq**2)
114         e = 0.5 * sq23.dD_dC(Uf, Ta) / Df * (Un[1:] - Un[:-1])
115         Cs, Fs, Ds = sq23.surface_m(Un[-1], Ta, Ce, dq)
116         Rs = R0**2 * dq / (8.0 * Ds) + R0 / (2.0 * HM)
117         gs = h / (Rs * dq)
118         fl = b * (Un[1:] - Un[:-1])
119         G = Un - U - np.concatenate((fl, [gs * (Ce - Un[-1])])) + np.
            concatenate([0.0], fl))
120         bn = np.concatenate((b, [0.0])); en = np.concatenate((e, [0.0]))
121         bp = np.concatenate((b, [0.0])); ep = np.concatenate((e, [0.0]))
122         J = np.zeros((3, N))
123         J[0, 1:] = -bn[:-1] * (1.0 + en[:-1])
124         J[1, :] = 1.0 + bn * (1.0 - en) + bp * (1.0 + ep)
125         J[1, -1] += gs
126         J[2, :-1] = -bn[:-1] * (1.0 - en[:-1])
127         dU = solve_banded((1, 1), J, -G)
128         Un = Un + dU
129         if np.max(np.abs(dU)) < 1e-11:
130             break
131     return Un
132
133 while t < t_end - 1e-12:
134     dtc = min(dt, t_end - t)
135     Ta_m, Ce_m = env(t + 0.5 * dtc)
136     Ta_e, Ce_e = env(t + dtc)
137     Uf_ = step(U, dtc, Ta_e, Ce_e)
138     Uh = step(U, 0.5 * dtc, Ta_m, Ce_m)
139     Uh2 = step(Uh, 0.5 * dtc, Ta_e, Ce_e)
140     err = float(np.max(np.abs(Uh2 - Uf_)))
141     tol = 1e-11 + 1e-7 * float(np.max(np.abs(U)))
142     r = err / tol
143     if r <= 1.0:
144         umax_old = float(np.max(U))
145         U = 2.0 * Uh2 - Uf_
146         t += dtc
147         umax = float(np.max(U))
148         if crossing is None and umax < TH:
149             crossing = (t - dtc, umax_old, t, umax)
150             break
151         dt = min(dtc * min(2.0, max(0.3, 0.9 / max(r, 1e-30) ** (1.0 /
            3.0))), 15.0)
152     else:
153         dt = dtc * max(0.2, 0.9 / r ** (1.0 / 3.0))
154 return crossing

```

```

155
156
157 def d1_counterproofs(env_hold):
158     """三条判据的反验实测（问题 3 设定，N=80）。"""
159     out = {}
160     print("[D1] 反验 a:  $Le \rightarrow \infty$  准稳态热场 ( $T \equiv T_a$ , 只求湿分) ...", flush=True)
161     cr_le = run_le_simplified(80, env_hold)
162     out["Le"] = cr_le[2] / 3600.0
163     print("[D1] 反验 b:  $Bi_h \rightarrow 0$  表面热阻忽略 ( $h \rightarrow 1e9$ ) ...", flush=True)
164     r_bh = run_patched(80, env_hold, {"H": 1e9})
165     out["Bi_h"] = r_bh["crossing"][2] / 3600.0
166     # 反验 c:  $Bi_m \rightarrow 0$  集总（解析膜界，无需重跑）
167     out["Bi_m"] = R0 / (2 * HM) * np.log((C0 - CE_Q3) / (TH - CE_Q3)) /
        3600.0
168     return out
169
170
171 # =====
172 # D4 / V2 尾段
173 # =====
174
175 def v2_tail():
176     d = np.load(DATA_DIR / "q23_main_steps.npz")
177     t, um, t_cross = d["t"], d["um"], float(d["crossing"][2])
178     mask = (t > t_cross - 15 * 3600) & (t <= t_cross)
179     y = np.log(um[mask] - CE_Q3)
180     fit = np.polyfit(t[mask], y, 1)
181     gamma = -float(fit[0]) # 实测尾段衰减率 1/s
182     yfit = np.polyval(fit, t[mask])
183     r2 = 1.0 - float(np.sum((y - yfit) ** 2) / np.sum((y - y.mean()) ** 2))
184     P3 = props3()
185
186     def mu1(D):
187         Bi = HM * R0 / D
188         return D * robin_eigs(Bi, 1)[0] ** 2 / R0**2
189     from scipy.optimize import brentq as bq
190     D_eff = float(bq(lambda D: mu1(D) - gamma, 1e-11, 1e-8, xtol=1e-18))
191     prefactor = 2.4e-3 * float(np.exp(-3850.0 / 323.315))
192     C_equiv = float(-0.45 / np.log(D_eff / prefactor)) # 反演等效加权含水率
193     rows = []
194     for C_eff in (0.15, 0.18, 0.20, 0.25):
195         D_c = float(P3["D"](C_eff, 323.315))
196         Bi = HM * R0 / D_c
197         lam1 = robin_eigs(Bi, 1)[0]
198         rows.append((C_eff, D_c, Bi, D_c * lam1**2 / R0**2))
199     return gamma, r2, D_eff, C_equiv, rows
200
201
202 # =====
203 # D3 输出重建/插值误差估计
204 # =====

```

```

205
206 def interp_error_estimate():
207     """1 s 输出插值误差 → t_f 小时数（曲率上界法）。"""
208     d = np.load(DATA_DIR / "q23_main_steps.npz")
209     t, C = d["t"], d["C"]
210     # 每个记录点：用左右邻步估计线性插值误差上界  $|U''| \cdot dt^2/8$ 
211     err = np.zeros(len(t) - 2)
212     for i in range(1, len(t) - 1):
213         h0, h1 = t[i] - t[i - 1], t[i + 1] - t[i]
214         if h0 <= 0 or h1 <= 0:
215             continue
216         d2 = (C[i + 1] - C[i]) / h1 - (C[i] - C[i - 1]) / h0
217         err[i - 1] = np.max(np.abs(d2)) / (h0 + h1) * (max(h0, h1) ** 2) /
            8.0 * 4.0
218     max_err_C = float(np.max(err))
219     # 换算 t_f: umax 在穿越处斜率
220     um, cr = d["um"], d["crossing"]
221     slope = abs(um[-1] - um[0]) / (t[-1] - t[0]) if len(t) > 1 else 1e-9
222     i0 = int(np.searchsorted(t, cr[0])) - 2
223     slope_cross = abs((um[min(i0 + 4, len(um) - 1)] - um[max(i0 - 4, 0)]) /
224         (t[min(i0 + 4, len(t) - 1)] - t[max(i0 - 4, 0)]))
225     return max_err_C, max_err_C / slope_cross / 3600.0
226
227
228 # =====
229 # D5 结果总账 + 论文表
230 # =====
231
232 def paper_tables():
233     d1 = np.load(DATA_DIR / "q1_fields.npz")
234     d23 = np.load(DATA_DIR / "q23_main_steps.npz")
235     d4 = np.load(DATA_DIR / "q4_main_steps.npz")
236     rows = []
237     pos5 = [0.0, 0.5, 1.0, 1.5, 2.0]
238     # 表1/表2（问题1）：7 时刻 × 5 位置
239     times1 = [100, 300, 600, 900, 1200, 1500, 1800]
240     t1, T1, C1 = d1["times"], d1["T-C"], d1["C"]
241     for tt in times1:
242         i = int(tt) - 1
243         for j, p in enumerate([0, 5, 10, 15, 20]):
244             rows.append(("表1", f"t={tt}s", f"r={pos5[j]}cm", f"{T1[i, p]:.4f}"
245                 , "c"))
246             rows.append(("表2", f"t={tt}s", f"r={pos5[j]}cm", f"{C1[i, p]:.4f}"
247                 , "kg/kg"))
248     # 表3/表4（问题2）：0.5-3.0 h × 5 位置
249     lt, lc, lT = d23["t"], d23["C"], d23["T"]
250     for th in (0.5, 1.0, 1.5, 2.0, 2.5, 3.0):
251         vC = interp_rows(lt, lc, [th * 3600])[0]
252         vT = interp_rows(lt, lT, [th * 3600])[0]
253         for j, p in enumerate([0, 5, 10, 15, 20]):
254             rows.append(("表3", f"t={th}h", f"r={pos5[j]}cm", f"{vT[p]:.4f}", "

```

```

        c"))
253     rows.append(("表4", f"t={th}h", f"r={pos5[j]}cm", f"{vC[p]:.4f}", "
        kg/kg"))
254 # 表5（问题3）：每 6 h × 5 位置 + 结束行
255 cr = d23["crossing"]
256 tf_h = cr[2] / 3600.0
257 for th in range(6, int(tf_h // 6) * 6 + 1, 6):
258     vC = interp_rows(lt, lC, [th * 3600.0])[0]
259     for j, p in enumerate([0, 5, 10, 15, 20]):
260         rows.append(("表5", f"t={th}h", f"r={pos5[j]}cm", f"{vC[p]:.4f}", "
        kg/kg"))
261 vC = interp_rows(lt, lC, [cr[2]])[0]
262 for j, p in enumerate([0, 5, 10, 15, 20]):
263     rows.append(("表5", "烘干结束时间", f"r={pos5[j]}cm", f"{vC[p]:.4f}",
        "kg/kg"))
264 # 表6（问题4）：每 6 h × (0,0.5,1,1.5,药材表面) + 结束行；超出表面留空
265 lt4, C4, Cs4, cr4 = d4["t"], d4["C"], d4["Cs"], d4["crossing"]
266 tf4_h = cr4[2] / 3600.0
267 cols6 = [(0, "0"), (5, "0.5"), (10, "1"), (15, "1.5")]
268 for th in range(6, int(tf4_h // 6) * 6 + 1, 6):
269     vC = interp_rows(lt4, C4, [th * 3600.0])[0]
270     us = float(np.interp(th * 3600.0, lt4, Cs4))
271     for p, name in cols6:
272         v = "" if np.isnan(vC[p]) else f"{vC[p]:.4f}"
273         rows.append(("表6", f"t={th}h", f"r={name}cm", v, "kg/kg"))
274     rows.append(("表6", f"t={th}h", "药材表面", f"{us:.4f}", "kg/kg"))
275 vC = interp_rows(lt4, C4, [cr4[2]])[0]
276 us = float(np.interp(cr4[2], lt4, Cs4))
277 for p, name in cols6:
278     v = "" if np.isnan(vC[p]) else f"{vC[p]:.4f}"
279     rows.append(("表6", "烘干结束时间", f"r={name}cm", v, "kg/kg"))
280 rows.append(("表6", "烘干结束时间", "药材表面", f"{us:.4f}", "kg/kg"))
281 return rows, tf_h, tf4_h
282
283
284 def _csv_kv(path):
285     """读 (key,value) 两列 CSV → dict（文件不存在返回 {}）。"""
286     p = Path(path)
287     if not p.exists():
288         return {}
289     with open(p, encoding="utf-8", newline="") as f:
290         rows = list(csv.reader(f))
291         return {r[0]: r[1] for r in rows[1:] if len(r) >= 2}
292
293
294 def table6_from_result4(pt_rows):
295     """把论文表 6（问题 4）的数据格替换为**内生交付工作簿** result4.xlsx 的对应
        格。
296
297     result4.xlsx 结构（交付口径.md §4+§5.1）：Sheet1, A1 表头，其后 20 列 r
        =0.0...1.9 cm,

```

```

298 末列「药材表面」；常规行为 60 s 行程（6/12/.../48 h 均为其子集），末行为 t_f
      数据行。
299 位置超出当前表面时单元格为空 → 该格留空（tables.tex 渲染为“—”），与交付
      口径一致。
300 内生工作簿不存在时原样返回（保持旧行为）。
301 """
302 xlsx = A_DIR / "结果" / "result4.xlsx"
303 if not xlsx.exists():
304     return pt_rows
305 import openpyxl
306 wb = openpyxl.load_workbook(xlsx, read_only=True, data_only=True)
307 ws = wb[wb.sheetnames[0]]
308 it = ws.iter_rows(values_only=True)
309 head = list(next(it))
310 body = [r for r in it if isinstance(r[0], (int, float))]
311 wb.close()
312 col = {}
313 for j in range(1, len(head)):
314     col[str(head[j]).replace("cm", "")] = j
315 t_last = float(body[-1][0])
316 by_t = {float(r[0]): r for r in body}
317 c6 = ("0", "0.5", "1", "1.5")
318 out = []
319 for r in pt_rows:
320     if r[0] != "表6":
321         out.append(r)
322         continue
323     tnode, cnode = r[1], r[2]
324     if tnode == "烘干结束时间":
325         src = body[-1]
326     else:
327         th = float(tnode.split("=")[1].rstrip("h"))
328         src = by_t.get(th * 3600.0)
329     if src is None:
330         out.append(r)
331         continue
332     if cnode == "药材表面":
333         v = src[-1]
334     else:
335         j = col.get(cnode.replace("r=", "").replace("cm", ""))
336         v = None if j is None else src[j]
337     out.append(("表6", tnode, cnode, "" if v is None else f"{float(v):.4f}"
338               ", "kg/kg"))
338 print(f"[D5] 论文表 6 取自内生 result4.xlsx（末行 t={t_last:.3f} s, "
339       f"{len(c6) + 1} 列表头匹配）", flush=True)
340 return out
341
342
343 def main():
344     t_start = time.perf_counter()
345     env_t, TaK, Ce = read_env()

```

```

346 env_hold = make_env("hold", env_t, TaK, Ce)
347
348 # ----- D1 -----
349 print("=" * 60, "\n[D1] 判据族表 + 反验", flush=True)
350 crit = criteria_table()
351 cproof = d1_counterproofs(env_hold)
352 tf3_80 = None # N=80 的 t_f (反验对照基准), 从 M2 记录取 57.200904
353 tf3_80 = 57.20090436
354 with open(DATA_DIR / "criteria.csv", "w", encoding="utf-8") as f:
355     f.write("判据,物性,定义,最小值,最大值,简化决策,反验t_f_h,反验偏差_h\n")
356     f.write(f"Le,附录3+4, $\alpha/D$ ,{min(r[1] for r in crit):.4g},{max(r[2] for r
357         in crit):.4g},"
358             f"热场准稳态( $T \approx T_{air}$ ),{cproof['Le']:.4f},{abs(cproof['Le']-
359                 tf3_80):.4f}\n")
360     f.write(f"Bi_m,附录3+4, $h_m \cdot R/D$ ,{min(r[3] for r in crit):.4g},{max(r
361         [4] for r in crit):.4g},"
362             f"集总(内扩散无阻力),{cproof['Bi_m']:.4f},{abs(cproof['Bi_m']-
363                 tf3_80):.4f}\n")
364     f.write(f"Bi_h,附录3+4, $h \cdot R/k$ ,{min(r[5] for r in crit):.4g},{max(r[6]
365         for r in crit):.4g},"
366             f"忽略表面热滞后( $h \rightarrow \infty$ ),{cproof['Bi_h']:.4f},{abs(cproof['Bi_h']-
367                 tf3_80):.4f}\n")
368     for name, a, b, c, d, e, g in crit:
369         f.write(f"数值区间[{name}],Le,,{a:.4g},{b:.4g},,,, \n")
370         f.write(f"数值区间[{name}],Bi_m,,{c:.4g},{d:.4g},,,, \n")
371         f.write(f"数值区间[{name}],Bi_h,,{e:.4g},{g:.4g},,,, \n")
372 print(f"[D1] Le {crit[0][1]:.4g} - {crit[0][2]:.4g}(附3) 反验  $\Delta t_f = \{abs($ 
373     cproof['Le']-tf3_80):.4f} h; "
374     f"Bi_m 集总  $\Delta t_f = \{abs(cproof['Bi_m']-tf3_80):.4f} h; "$ 
375     f"Bi_h  $\Delta t_f = \{abs(cproof['Bi_h']-tf3_80):.4f} h$ ", flush=True)
376
377 # ----- D2 敏感性 -----
378 print("=" * 60, "\n[D2] 敏感性矩阵", flush=True)
379 sens = [("C_e=0.04986(声明闭合,主)", tf3_80, 0.0)]
380 print("[D2] C_e=0.1366 (山药解吸支, 题外修正) ...", flush=True)
381 env_yam = lambda tt: (float(np.interp(tt, env_t, TaK)) if tt <= 14400
382     else float(TaK[-1]), CE_YAM)
383 r_yam = sq23.solve_coupled(80, env_yam, 450000.0, out_times=(), dt_max
384     =15.0)
385 tf_yam = r_yam["crossing"][2] / 3600.0
386 sens.append(("C_e=0.1366(山药解吸支,题外修正)", tf_yam, tf_yam - tf3_80))
387 print(f"[D2] C_e=0.1366: t_f = {tf_yam:.4f} h,  $\Delta = \{tf\_yam-tf3\_80:+.4f\}$ 
388     h", flush=True)
389
390 print("[D2] 半径插值: 线性 (问题 4, N=80) ...", flush=True)
391 t_att, Rcm, R_pchip = read_radius()
392 R_lin = lambda tt: float(np.interp(min(tt, t_att[-1]), t_att, Rcm *
393     0.01))
394 r_lin = solve_coupled4(80, env_hold, R_lin, 600000.0, dt_max=15.0)
395 tf_lin = r_lin["crossing"][2] / 3600.0
396 tf4_80 = 50.829547

```



```

386 sens.append(("半径插值PCHIP(主,问题4)", tf4_80, 0.0))
387 sens.append(("半径插值线性(问题4)", tf_lin, tf_lin - tf4_80))
388 print(f"[D2] 线性插值: t_f = {tf_lin:.4f} h,  $\Delta$  = {tf_lin-tf4_80:+.4f} h",
      flush=True)
389 sens += [("环境外推:末值保持(主)", 57.20090436, 0.0),
390          ("环境外推:线性外推", 58.82027949, 1.61937513),
391          ("环境外推:末1h均值", 57.50341017, 0.30250581)]
392 # 问题 4 效应拆分: 基准=内生几何主解, 唯一来源 q4_split.csv (run_q4_endo.
      py 产出),
393 # 键名/值/备注与 run_q4_endo.py 的 ledger upsert 逐字一致 (2026-09-12 4c
      键名同步)。
394 q4sp = read_q4_split()
395 if q4sp:
396     base_e = q4sp["appendix4_endogenous_R(t)"]
397     sens += [("问题4效应拆分:附4固定R", q4sp["appendix4_fixedR"],
398             q4sp["appendix4_fixedR"] - base_e),
399             ("问题4效应拆分:附4+R(t)(主)", base_e, 0.0),
400             ("问题4效应拆分:问题3", q4sp["problem3"], q4sp["problem3"] -
              base_e)]
401     SENS_NOTE.update({
402         "问题4效应拆分:附4固定R":
403             f" $\Delta$ {q4sp['shrink_effect_endo_h']:.4f} h (对内生主解, 收缩效应 "
404             f"{q4sp['shrink_pct_of_fixedR_endo']:.1f}%) ",
405         "问题4效应拆分:附4+R(t)(主)": "— (内生几何主解)",
406         "问题4效应拆分:问题3":
407             f" $\Delta$ {q4sp['net_effect_endo_h']:.4f} h (公式+收缩效应 "
408             f"{q4sp['net_pct_of_q3_endo']:.1f}%) "})
409 else:
410     # 降级: 无 q4_split.csv 时保持旧口径并明确告
      警
411     print("[D2] 警告: 缺 数据/q4_split.csv (先跑 run_q4_endo.py); "
412           "问题 4 效应拆分行回落为外生基准, 与 run_q4_endo.py 写出的键值不一
      致", flush=True)
413     sens += [("问题4效应拆分:附4固定R", 129.104709, 129.104709 - tf4_80),
414             ("问题4效应拆分:附4+R(t)(主)", TF4_H, TF4_H - tf4_80),
415             ("问题4效应拆分:问题3", TF3_H, TF3_H - tf4_80)]
416 with open(DATA_DIR / "sensitivity.csv", "w", encoding="utf-8") as f:
417     f.write("组别,t_f_h,delta_t_f_h\n")
418     for name, tf_, d_ in sens:
419         f.write(f"{name},{tf_:+.6f},{d_:+.6f}\n")
420
421 # ----- D4 / V2 -----
422 print("=" * 60, "\n[D4] V2 尾段衰减率对照", flush=True)
423 gamma, r2_fit, D_eff, C_equiv, v2rows = v2_tail()
424 with open(DATA_DIR / "v2_tail.csv", "w", encoding="utf-8") as f:
425     f.write("quantity,value\n")
426     f.write(f"gamma_meas_1s,{gamma:.6e}\n")
427     f.write(f"ln(umax-Ce)线性拟合R2,{r2_fit:.8f}\n")
428     f.write(f"D_eff_反演,{D_eff:.6e}\n")
429     f.write(f"C_equiv_反演,{C_equiv:.6f}\n")
430     f.write("# 首模族对照: C_eff,D_eff,Bi_m,mu1_1s\n")
431     for r in v2rows:

```

```

431         f.write(f"# {r[0]},{r[1]:.4e},{r[2]:.4f},{r[3]:.6e}\n")
432     print(f"[D4] 实测尾段衰减率  $\gamma=\{\text{gamma}:.4e\}/s$  (ln 线性  $R^2=\{\text{r2\_fit}:.6f\}$ ); "
433           f"反演等效  $D=\{D\_eff:.3e\} \Leftrightarrow C\approx\{C\_equiv:.3f\}$  (尾段剖面区间内, 结构一致)"
434           ", flush=True)
435
436     # ----- D3 误差预算 -----
437     print("=" * 60, "\n[D3] 误差预算", flush=True)
438     errC, errC_h = interp_error_estimate()
439     r_lin_dev = abs(tf_lin - tf4_80)
440     budget = [
441         ("数值误差:空间网格(Q3,N=160 Richardson偏差)", 0.0164, "问题3"),
442         ("数值误差:空间网格(Q4,连续事件定位N80→160变化)", 0.0007, "问题4"),
443         ("数值误差:时间步(rtol/10实测)", 0.0014, "两问"),
444         ("数值误差:非线性迭代(Picard/Newton对照)", 0.000001, "两问"),
445         ("数值误差:输出重建/1s插值(曲率上界)", errC_h, "两问"),
446         ("环境情景:环境外推(三档最大偏移)", 1.6194, "两问"),
447         ("口径依赖:半径插值(线性vsPCHIP)", r_lin_dev, "问题4"),
448         ("口径依赖:舍入判定(60s 网格+四位小数)", 0.0534 + 0.0167, "问题3(另一种"
449           "数据使用方式)"),
450         ("模型结构差异:潜热情景(单列不合成)", 5.3889, "情景(见latent_scenario."
451           "csv)"),
452     ]
453     total_q3 = 0.0164 + 0.0014 + 0.000001 + errC_h + 1.6194 + 0.0701
454     total_q4 = 0.0007 + 0.0014 + 0.000001 + errC_h + 1.6194 + r_lin_dev
455     with open(DATA_DIR / "error_budget.csv", "w", encoding="utf-8") as f:
456         f.write("误差源,量级_h,类别/适用\n")
457         for name, v, typ in budget:
458             f.write(f"{name},{v:.6g},{typ}\n")
459             f.write(f"已检查因素累计偏移量级(问题3),{total_q3:.6g},不含C_e口径/潜热"
460                   "情景\n")
461             f.write(f"已检查因素累计偏移量级(问题4),{total_q4:.6g},不含C_e口径/潜热"
462                   "情景\n")
463     print(f"[D3] 输出插值 {errC_h:.2e} h; 累计偏移量级 Q3≈{total_q3:.2f} h、"
464           f"Q4≈{total_q4:.2f} h (不含 C_e 口径与潜热情景)", flush=True)
465
466     # ----- D5 结果总账 -----
467     print("=" * 60, "\n[D5] 结果总账 + 论文表", flush=True)
468     pt_rows, tf_h, tf4_h = paper_tables()
469     pt_rows = table6_from_result4(pt_rows) # 论文表 6 取内生 result4.xlsx (见
470     其 docstring)
471     with open(DATA_DIR / "paper_tables.csv", "w", encoding="utf-8", newline=
472     "") as f:
473         w = csv.writer(f)
474         w.writerow(["表", "行", "列", "值", "单位"])
475         for r in pt_rows:
476             w.writerow(list(r))
477     q23s = json.load(open(DATA_DIR / "q23_summary.json", encoding="utf-8"))
478     q4s = json.load(open(DATA_DIR / "q4_summary.json", encoding="utf-8"))
479     # 问题 4 主解 = **内生几何版**: t_f / Richardson 优先取 run_q4_endo.py 的
480     产出
481     # (q4_endo_check.csv / q4_endo_convergence.csv); 无则退回外生

```

```

    q4_summary.json
474 tf4_main, tf4_rich = q4s["tf4_h"], q4s["tf2_rich_h"]
475 tf4_tag, tf4_note = "", "h"
476 _chk = _csv_kv(DATA_DIR / "q4_endo_check.csv")
477 if "tf_endo_h" in _chk:
478     tf4_main = float(_chk["tf_endo_h"])
479     tf4_tag = "(未舍入右端点, 内生几何)"
480     tf4_note = f"h (附件2外生版 {q4s['tf4_h']:.6f} 作交叉验证)"
481     _conv = _csv_kv(DATA_DIR / "q4_endo_convergence.csv")
482     if "richardson_extrap" in _conv:
483         tf4_rich = float(_conv["richardson_extrap"])
484         tf4_ridx = "问题4 Richardson外推(内生几何)"
485     else:
486         tf4_ridx = "问题4 Richardson外推"
487 else:
488     tf4_ridx = "问题4 Richardson外推"
489 led = []
490 A = led.append
491 A(("t_f", "问题3主值(未舍入右端点)", "", f"{q23s['tf_h']:.6f}", "h"))
492 A(("t_f", "问题3穿越区间", "", f"[{q23s['tf_bracket'][0]:.2f},{q23s['tf_bracket'][2]:.2f}]", "s"))
493 A(("t_f", "问题3 Richardson外推", "", f"{q23s['tf_rich_h']:.6f}", "h"))
494 A(("t_f", "问题3舍入判定(60s四位小数)", "", f"{q23s['tf_rounded_h']:.4f}", "h"))
495 A(("t_f", f"问题4主值{tf4_tag}", "", f"{tf4_main:.6f}", tf4_note))
496 A(("t_f", tf4_ridx, "", f"{tf4_rich:.6f}", "h"))
497 A(("t_f", "问题3外部锚点56.92/56.93/57.45偏差", "", "+0.26/+0.25/-0.27", "h"))
498 for case, v in (("V1热场解析对拍最大偏差(N=320)", "5.90e-07"),
499                 ("V1收敛阶", "2.000/1.993/1.990"),
500                 ("V2尾段后验诊断:衰减率 $\gamma$ (反演)", f"{gamma:.4e}/s"),
501                 ("V2尾段ln线性R2", f"{r2_fit:.6f}"),
502                 ("V2尾段等效D反演(对应C)", f"{D_eff:.3e}(C $\approx$ {C_equiv:.3f})),
503                 ("V3问题3常数参考时标", "15.7021"),
504                 ("V3问题4瞬时首模近似参考(时间积分)", "19.016"),
505                 ("V4水量逐窗残差max", "5.54e-11"),
506                 ("V4冻结热容显热离散平衡全局残差", "1.05e-11"),
507                 ("V5a移动域vs固定域 $\Delta t_f$ ", "0.00"),
508                 ("V5b退化首模商", "0.99996"),
509                 ("V6场级空间阶", "1.90-2.45")):
510     A(("验证", case, "", v, "见报告"))
511 for name, tf_, d_ in sens:
512     A(("敏感性", name, "", f"{tf_:.4f}", SENS_NOTE.get(name, f" $\Delta$ {d_:.4f}h")))
513 for name, v, typ in budget:
514     A(("误差预算", name, "", f"{v:.4g}", typ))
515 A(("误差预算", "已检查因素累计偏移量级(问题3)", "", f"{total_q3:.2f}", "h(不含C_e口径/潜热情景)"))
516 A(("误差预算", "已检查因素累计偏移量级(问题4)", "", f"{total_q4:.2f}", "h(不含C_e口径/潜热情景)"))
517 # 潜热对照情景(审计 A01; 代码/latent_scenario.py 产出, # key=value 头; 缺

```

```

        失则跳过)
518 _lat = {}
519 _lp = DATA_DIR / "latent_scenario.csv"
520 if _lp.exists():
521     for line in open(_lp, encoding="utf-8"):
522         if line.startswith("# ") and "=" in line:
523             k, v = line[2:].strip().split("=", 1)
524             _lat[k] = v
525         elif not line.startswith("#"):
526             break
527 if "t_f_latent_h" in _lat:
528     A(("情景(潜热A01)", "无潜热基线t_f(N=160)", "", f"{float(_lat['t_f_base_h']):.4f}", "h(基线断言57.1799)"))
529     A(("情景(潜热A01)", "表面相变汇t_f", "", f"{float(_lat['t_f_latent_h']):.4f}",
530        "h(情景假设:L_v=2.4e6,p_d,s表面局部)"))
531     A(("情景(潜热A01)", "潜热情景Δt_f", "", f"{float(_lat['delta_t_f_h']):+.4f}", "h(不进主值)"))
532     A(("情景(潜热A01)", "最大逐时|ΔT_s|/|ΔT_center|", "",
533        f"{float(_lat['max_dTs_C']):.2f}/{float(_lat['max_dTcenter_C']):.2f}", "c"))
534 # 内生几何自洽三段误差 (A07; run_q4_endo.py 产出的 q4_endo_eval.csv; 缺失则跳过)
535 _ev = _csv_kv(DATA_DIR / "q4_endo_eval.csv")
536 if "rmse_all_mm" in _ev:
537     A(("验证", "内生几何自洽R_pred vs 附件2 RMSE", "", f"{float(_ev['rmse_all_mm']):.4f}",
538        "mm(满72h真实轨迹; 三段见q4_endo_eval.csv)"))
539     A(("验证", "内生自洽三段误差0-达标/达标-72h", "",
540        f"{float(_ev['rmse_0_to_crossing_mm']):.4f}/{float(_ev['rmse_crossing_to_72h_mm']):.4f}",
541        "mm(A07口径)"))
542     A(("验证", "内生R_from_state代数闭合残差max/mean", "",
543        f"{float(_ev['R_closure_resid_max_mm']):.4f}/{float(_ev['R_closure_resid_mean_mm']):.4f}",
544        "mm(只评估不进主解)"))
545 if "tf_endo_h" in _chk:
546     A(("验证", "耦合内生t_f vs 外生t_f偏差", "",
547        f"{float(_chk['tf_endo_h']) - q4s['tf4_h']:+.4f}", "h(模型间对照, 非独立验证)"))
548 for name, a2, b2, c2, d2_, e2, g2 in crit:
549     A(("判据", f"Le[{name}]", "", f"{a2:.4g} - {b2:.4g}", "α/D"))
550     A(("判据", f"Bi_m[{name}]", "", f"{c2:.4g} - {d2_:.4g}", "h_m · R/D"))
551     A(("判据", f"Bi_h[{name}]", "", f"{e2:.4g} - {g2:.4g}", "h · R/k"))
552 for case, v in (("Le反验(准稳态热场)", cproof["Le"]), ("Bi_m反验(集总)", cproof["Bi_m"]),
553                ("Bi_h反验(h→∞)", cproof["Bi_h"])):
554     A(("判据反验", case, "", f"{v:.4f}", f"Δ{abs(v-tf3_80):+.4f} h"))
555 for r in pt_rows:
556     A(("论文表", f"{r[0]}[{r[1]}]", r[2], r[3], r[4]))
557 with open(DATA_DIR / "结果总账.csv", "w", encoding="utf-8", newline="")

```

```

        as f:
558     w = csv.writer(f)                # csv 引号封装：含逗号的区间单元格不再被
        拆列
559     w.writerow(["类别", "项目", "子项", "值", "单位/备注"])
560     for r in led:
561         w.writerow([str(x) for x in r])
562     print(f"[D5] 结果总账 {len(led)} 行；论文表 {len(pt_rows)} 格", flush=
        True)
563     print(f"[完成] 总耗时 {time.perf_counter() - t_start:.1f} s", flush=True)
564
565
566 if __name__ == "__main__":
567     main()

```

E.17 交付口径核对

逐格读取题库给出的结果模板与论文表格结构，核对交付口径。

代码 18 交付口径核对（交付口径-核对.py，100 行）

```

1  """T2 交付口径核对：逐格读取四个结果模板 + 官方 PDF 的四张表结构。
2
3  只读 01-题目/ 下的原始文件，输出到 stdout（重定向为 交付口径-核对输出.txt）。
4  依赖：openpyxl（模板）、pymupdf（PDF 表结构证据）。
5  用法：PYTHONIOENCODING=utf-8 python 数据/交付口径-核对.py > 数据/交付口径-核
    对输出.txt
6  """
7
8  import os
9
10 import openpyxl
11 import fitz
12
13 ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
14 TPL_DIR = os.path.join(ROOT, "01-题目", "原始文件", "附件3")
15 PDF = os.path.join(ROOT, "01-题目", "原始文件", "A题.pdf")
16
17 TEMPLATES = ["result1.xlsx", "result2.xlsx", "result3.xlsx", "result4.xlsx"
    ]
18
19
20 def dump_templates():
21     print("=" * 78)
22     print("[1] 结果模板逐格转储（openpyxl, data_only 默认读取公式串；本批模板无
        公式）")
23     print("=" * 78)
24     for name in TEMPLATES:
25         path = os.path.join(TPL_DIR, name)
26         wb = openpyxl.load_workbook(path)
27         print("\n" + "-" * 78)
28         print(f"FILE {name}")
29         print(" sheetnames (顺序):", wb.sheetnames)

```

```

30 print(" sheet_state :", [(ws.title, ws.sheet_state) for ws in wb.
    worksheets])
31 for ws in wb.worksheets:
32     print(f" SHEET {ws.title!r}: dims={ws.dimensions} max_row={ws.
        max_row} max_col={ws.max_column}")
33     print(" merged_cells:", sorted(str(r) for r in ws.merged_cells.
        ranges))
34     for row in ws.iter_rows(min_row=1, max_row=ws.max_row, max_col=ws.
        max_column):
35         cells = []
36         for c in row:
37             v = c.value
38             if isinstance(v, str):
39                 v = f"'{v}'"
40             cells.append(f"{c.coordinate}={v}")
41         print(" R%d: " % row[0].row + " | ".join(cells))
42     print(" A1 字符码:", [(ch, hex(ord(ch))) for ch in str(ws["A1"].
        value)])
43     print(" A1/A2 number_format:", ws["A1"].number_format, "/", ws["A2
        "].number_format)
44     print(" comments:", [(c.coordinate, c.comment.text) for r in ws.
        iter_rows() for c in r if c.comment])
45     print(" freeze_panes:", ws.freeze_panes, "| auto_filter:", ws.
        auto_filter.ref)
46 print("\n 结论（模板自证）：A1='时间\\到药材中心的距离'（单反斜杠 0x5c，非
    换行）；")
47 print(" 第 1 行为列头、A 列为时间；示例行 A2/A3/A4 与末行 A5='...'。")
48 print(" result1/2/3 的末列 F1 = 固定值 2；result4 的末列 F1 = '药材表面'。
    ")
49
50
51 def dump_pdf_tables():
52     print("\n" + "=" * 78)
53     print("[2] 官方 PDF 表结构（PyMuPDF find_tables 的栅格化结果）")
54     print("=" * 78)
55     doc = fitz.open(PDF)
56     print("pages:", doc.page_count)
57     for pno in range(doc.page_count):
58         page = doc[pno]
59         tabs = page.find_tables()
60         if not tabs.tables:
61             continue
62         print(f"\n--- page {pno + 1}: {len(tabs.tables)} table(s)")
63         for ti, t in enumerate(tabs.tables):
64             print(f" table[{ti}] grid rows={t.row_count} cols={t.col_count}
                bbox={tuple(round(v, 1) for v in t.bbox)}")
65             for ri, row in enumerate(t.extract()):
66                 print(f" r[{ri}]: {row}")
67             head = t.extract()[0]
68             caption = ""
69             for blk in page.get_text("blocks"):

```

```

70         if blk[3] <= t.bbox[1] and t.bbox[1] - blk[3] < 40 and "表" in
           blk[4]:
71             caption = blk[4].strip().replace("\n", " ")
72             print(f" caption: {caption!r}")
73             print(f" -> 第 2 个表头格 {head[1]!r} 占据栅格第 2..{len(head)} 列,
              即 colspan={len(head) - 1}")
74     doc.close()
75     print("\n 结论: 表 1-表 5 的表头栅格为 6 列 ('到药材中心的距离/cm' colspan
           =5), ")
76     print("      表 6 的表头栅格为 5 列 (colspan=4), 末列为 '药材表面' 而非固定
           '2'。")
77
78
79 def dump_source_data():
80     print("\n" + "=" * 78)
81     print("[3] 附件1/附件2 首末与半径上界 (用于判据: 固定列是否落在药材之外)")
82     print("=" * 78)
83     wb1 = openpyxl.load_workbook(os.path.join(ROOT, "01-题目", "原始文件", "
           附件1.xlsx"))
84     ws1 = wb1["Sheet1"]
85     print("附件1 sheets:", wb1.sheetnames, ws1.dimensions)
86     for r in (1, 2, 32, ws1.max_row):
87         print(f" row {r}:", [ws1.cell(r, c).value for c in range(1, 4)])
88     wb2 = openpyxl.load_workbook(os.path.join(ROOT, "01-题目", "原始文件", "
           附件2.xlsx"))
89     ws2 = wb2["Sheet1"]
90     print("附件2 sheets:", wb2.sheetnames, ws2.dimensions)
91     for r in (1, 2, 3, 4, 5, ws2.max_row):
92         print(f" row {r}:", [ws2.cell(r, c).value for c in range(1, 3)])
93     rad = [ws2.cell(r, 2).value for r in range(2, ws2.max_row + 1)]
94     print(" 半径 max/min:", max(rad), min(rad), "| t=1800 s 半径:", ws2.cell
           (3, 2).value)
95
96
97 if __name__ == "__main__":
98     dump_templates()
99     dump_pdf_tables()
100    dump_source_data()

```

E.18 图表生成

由 数据/ 的唯一数据源生成全文矢量插图。

代码 19 图表生成 (plot_all.py, 2113 行)

```

1  """M5 图表生成 (出版级样式重构版): 从 数据/ 的唯一数据源出 6 张论文用矢量图。
2
3  用法: python plot_all.py      # 一次出全部 10 张 (PDF + 300dpi PNG + 灰度预
      览)
4      python plot_all.py fig4 # 只出某一张 (调试用)
5      python plot_all.py --no-cache # fig4 的附4固定R对照曲线强制重算
6

```

```

7 样式规范（同 图表/样式重构说明.md，供论文侧协调尺寸）：
8  * 按最终物理尺寸设计，导出后不再缩放：单面板宽 5.4 in (= 0.86×6.3 in 正文文本
9    宽)，
10   双子图宽 6.3 in (= \\linewidth 6.3 in)，纵横比 0.50 - 0.62。
11  * Okabe-Ito 色盲安全色板 + 语义色映射 ROLE（同一变量跨图同色）+ 线型/标记冗余
12   编码。
13  * 字号 ≥7.5 pt（正文标签 8、刻度 7.5、图例 7.5），白底、去顶/右脊柱、图例无
14   框、网格极浅。
15  * 面板标签 (a)/(b) 粗体置于左上外侧；标题 ≤28 字符。
16  * **图内不放说明性长文字**：数据源、口径、偏差数字等论述一律由论文 \\caption
17   承载；
18   脚本把每张图的关键数字打印到 stdout（[F1]…[F6] 行），论文侧据此写图注。
19
20 数据源（每张图对应 §3 规格）：
21  fig1 附件1.xlsx (T_a、C_env)、附件2.xlsx (R)
22  fig2 q1_fields.npz (T_C、times、pos_cm、env_*)、v1_points_final.csv、
23     v1_convergence.csv
24  fig3 q23_main_steps.npz (t、C、T)、q23_summary.json、criteria.csv
25  fig4 q23_main_steps.npz (um)、**q4_endo_fields.npz (C → 逐帧 nanmax 得
26     maxU，内生几何主解)**、
27     q4_split.csv、q4_bound.csv、v3_bound.csv、sensitivity.csv、结果总账.
28     csv；
29     附4固定R对照曲线由 solver_q4 现算（与 q4_split.csv 的 appendix4_fixedR
30     逐次核对）
31  fig5 sensitivity.csv（敏感性因素）、q4_split.csv（问题4 效应拆分，基准=内生
32     几何主解）、
33     error_budget.csv
34  fig6 endogenous_shrinkage.csv（附件2 时刻网格上的各闭合 R_pred、Ū_model、
35     Ū_inf；
36     其驱动场为**附件2 几何（外生 R）**的求解结果，见 数据/内生收缩分析.md
37     ）；
38     缺列时回退 附件2.xlsx、q4_main_steps.npz
39  fig7 endogenous_calibrated.csv（附件2 网格上的 R_data、R_pred、残差、失水基
40     线；
41     标定用的驱动场同为附件2 几何，见 数据/内生收缩模型.md）
42  fig9 q4_endo_fields.npz（同问题4 内生时空场 → 6 个时刻的圆盘截面）
43  fig10 mechRefit.csv（145 点 R_data/R_pred/残差）、mechRefit_fields.npz（重
44     标定参数、
45     全程场量 ū/De/g、末端平衡表 terminal、t_f_h）；口径见
46     数据/力学升级_修复重标定.md（A04/A05 修复后重标定的幂律吸力力学模型，
47     取代此前的线性检验本构情景族；旧文 力学升级_重标定.md 的数值一律作废）
48
49 口径常数（非结果数字，仅用于画线/判定，来源已注明）：
50  TH = 0.15 kg/kg —— 题目问题 3 原文“水分浓度应低于 0.15 kg/kg”
51  fig6 H_SEG / C_GLASS / RHO_SK —— 附件2 三段分界、玻璃化含水率、骨架密度，
52     来源 数据/内生收缩分析.md §0 - §1（图内所有半径/含水率/交叉点数字仍从数据读
53     出）
54  其余全部数字（含 35 h 冻结点、参考时标）均从上述数据文件读出，脚本内不誊写结
55     果。
56
57  """
58  from __future__ import annotations

```



```

43
44 import argparse
45 import csv
46 import json
47 import logging
48 import sys
49 import time
50 import warnings
51 from pathlib import Path
52
53 import numpy as np
54 import matplotlib
55
56 matplotlib.use("Agg")
57 import matplotlib.pyplot as plt
58 from matplotlib.colors import LogNorm # fig13 的对数色阶
59 from matplotlib.lines import Line2D
60 from matplotlib.patches import Patch, Rectangle
61 from matplotlib.text import Text
62
63 CODE_DIR = Path(__file__).resolve().parent
64 A_DIR = CODE_DIR.parent
65 DATA_DIR = A_DIR / "数据"
66 FIG_DIR = A_DIR / "图表"
67 QA_DIR = FIG_DIR / "_qa" # 灰度预览等自检产物（不入 git）
68 ATT1 = A_DIR / "01-题目" / "原始文件" / "附件1.xlsx"
69 ATT2 = A_DIR / "01-题目" / "原始文件" / "附件2.xlsx"
70 sys.path.insert(0, str(CODE_DIR))
71
72 # ---- 题目口径常数（非计算结果；见模块 docstring）----
73 TH = 0.15 # kg/kg, 问题 3 达标阈值（题目原文）
74
75 CI = 120 # 屏幕渲染 dpi (1 pt = dpi/72 px)
76 CLIP_TOL = 1.0 # 文字超出画布容差 (px)：仅 >1 px 视为真裁切
77
78 # ---- 设计尺寸（英寸；论文实际包含尺寸，导出后不再缩放）----
79 W_SINGLE = 5.4 # 单面板：0.86 × 6.3 in 正文文本宽
80 W_DOUBLE = 6.3 # 双子图：\linewidth（需论文侧改为 width=\linewidth）
81 ASPECT = (0.50, 0.62) # 高/宽 允许区间
82 # 例外：fig5 为“长中文类别名”的两行竖排（用户规范允许的排法），纵横比需 0.66
    才能让
83 # 7.5 pt 类别名不互相压盖——压到 0.62 时实测刻度标签重叠（本条即审计的硬门槛之
    一）。
84 ASPECT_EXEMPT = {"fig5_敏感性": (0.50, 0.70),
85 # fig9 为三行内容（两行圆盘 + 一条共享时间轴上下文带），压到 0.62
    会把
86 # 圆盘直径从约 1.3 in 压到约 1.0 in 并挤掉时间带；用户明确要求把
    六个采样
87 # 时刻放回时间轴（前快后慢要看得见），故按与 fig5 同样的方式豁免到
    0.71。
88 "fig9_截面演化": (0.50, 0.71)}

```

```

89
90 # ---- Okabe-Ito 色盲安全色板 ----
91 OI = dict(
92     blue="#0072B2", orange="#E69F00", green="#009E73", vermillion="#D55E00",
93     magenta="#CC79A7", sky="#56B4E9", grey="#6B7280", dark="#222222",
94 )
95
96 # ---- 语义色映射（同一变量跨图同色；见 图表/样式重构说明.md 的映射表） ----
97 ROLE = dict(
98     data=OI["dark"], # 附件/实测数据（附件1 温湿、附件2  $R(t)$ ）
99     model=OI["blue"], # 模型主解（问题4 主解；fig6 最接近的零拟合闭合）
100     model2=OI["sky"], # 模型次解（问题3 主解，固定  $R$  口径）
101     ref=OI["vermillion"], # 对照/参考解/基线（附4+固定 $R0$ ；fig6 全局理想基线）
102     analytic=OI["green"], # 解析解
103     aux1=OI["orange"], # 次要物理量：温度  $T$ 
104     aux2=OI["magenta"], # 次要物理量：环境水分  $C_{env}$ ；fig6 第三机制（成孔）
105     guide=OI["grey"], # 阈值/参考时标/平台线（一律虚线/点线）
106 )
107
108 # ---- 出版级样式基线 ----
109 RC = {
110     "font.sans-serif": ["Microsoft YaHei", "SimHei"],
111     "axes.unicode_minus": False,
112     "pdf.fonttype": 42, # 嵌入 TrueType 子集（矢量，可复制文字）
113     "ps.fonttype": 42,
114     "font.size": 8.0,
115     "axes.titlesize": 8.5,
116     "axes.labelsize": 8.0,
117     "xtick.labelsize": 7.5,
118     "ytick.labelsize": 7.5,
119     "legend.fontsize": 7.5,
120     "lines.linewidth": 1.2,
121     "lines.markersize": 3.2,
122     "axes.linewidth": 0.7,
123     "axes.grid": False,
124     "axes.spines.top": False,
125     "axes.spines.right": False,
126     "legend.frameon": False,
127     "figure.dpi": 120,
128     "savefig.dpi": 300,
129     "savefig.bbox": None, # 按 figsize 精确出图，不裁切（保证设计宽度即最终宽度）
130     "xtick.major.width": 0.7,
131     "ytick.major.width": 0.7,
132     "xtick.major.size": 2.6,
133     "ytick.major.size": 2.6,
134     "lines.dash_capstyle": "round",
135 }
136 MIN_FONT = 7.5 # 最终尺寸下最小字号 (pt)
137 TITLE_MAX = 28 # 面板标题字符上限
138 LEG_MAX = 5 # 图例项上限

```

```

139 # 例外: fig3 的两个面板各 6 条曲线, 颜色编码的是“有序连续时间族”(非 6 个并列
    类别)。
140 # 该族末段曲线在  $r \rightarrow 2$  处重合 (24/36/57 h 的  $C$  相差  $<0.011$  kg/kg), 直接标注会互
    相压盖,
141 # 故保留 6 项  $ncol=2$  图例 (checklist 的“>5 项改直接标注”对连续族不适用)。
142 LEG_MAX_EXEMPT = {"fig3_温湿剖面": 6}
143
144 TIME_CMAP = plt.cm.viridis # 时间族专用 (感知均匀; 禁 jet/rainbow)
145 GRID_KW = dict(color="0.86", lw=0.4, alpha=1.0)
146
147
148 class _GlyphWatch(logging.Handler):
149     """抓 matplotlib 的缺字告警 (豆腐块检测, logging 通道)。"""
150
151     def __init__(self):
152         super().__init__(level=logging.WARNING)
153         self.msgs: list[str] = []
154
155     def emit(self, record):
156         m = record.getMessage()
157         if "missing from font" in m or "Glyph" in m:
158             self.msgs.append(m)
159
160
161 GLYPH = _GlyphWatch()
162 logging.getLogger("matplotlib").addHandler(GLYPH)
163
164
165 # ===== 通用读数 =====
166 def read_att1():
167     """附件1: 返回 (t_s, T_a/°C, C_env/(kg/kg)), 0 - 14400 s 每 60 s。"""
168     import openpyxl
169     wb = openpyxl.load_workbook(ATT1, read_only=True)
170     ws = wb[wb.sheetnames[0]]
171     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is
        not None]
172     wb.close()
173     return (np.array([r[0] for r in rows], float),
174             np.array([r[1] for r in rows], float),
175             np.array([r[2] for r in rows], float))
176
177
178 def read_att2():
179     """附件2: 返回 (t_s, R/cm), 0 - 259200 s 每 1800 s。"""
180     import openpyxl
181     wb = openpyxl.load_workbook(ATT2, read_only=True)
182     ws = wb[wb.sheetnames[0]]
183     rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0] is
        not None]
184     wb.close()
185     return (np.array([r[0] for r in rows], float),

```

```

186         np.array([r[1] for r in rows], float))
187
188
189 def read_csv_rows(name):
190     """读 数据/ 下的 csv, 返回 (表头, [dict, ...]); 兼容注释行 (首个字段以 # 开
        头)
191     与字段内混入英文逗号的情形 (把多余的分段并回首列)。"""
192     with open(DATA_DIR / name, encoding="utf-8") as f:
193         rows = [r for r in csv.reader(f) if r and not r[0].startswith("#")]
194         head, body = rows[0], rows[1:]
195         n = len(head)
196         body = [(["", "."].join(r[:len(r) - n + 1]) + r[len(r) - n + 1:]) if len(r)
            > n else r
197                 for r in body]
198         return head, [dict(zip(head, r)) for r in body]
199
200
201 def read_json(name):
202     return json.loads((DATA_DIR / name).read_text(encoding="utf-8"))
203
204
205 def log_minor_off(ax, which="both"):
206     """关闭对数轴的副刻度标签: matplotlib 会给落在视图外的副刻度也生成标签文本
        (axes=None),
207     审计会把它算作“裁切文字”。视觉上无变化 (副刻度标签默认极密, 正式图不
        用)。"""
208     for axis, name in ((ax.xaxis, "x"), (ax.yaxis, "y")):
209         if which in ("both", name):
210             axis.set_minor_formatter(matplotlib.ticker.NullFormatter())
211
212
213 def crossing_time(t, y, th):
214     """y(t) 自上方穿越 th 的线性插值时刻 (t 升序)。"""
215     i = int(np.argmax(y < th))
216     assert 0 < i < len(y), "曲线未穿越阈值"
217     return float(t[i - 1] + (th - y[i - 1]) * (t[i] - t[i - 1]) / (y[i] - y[
        i - 1]))
218
219
220 # ===== 样式构件 =====
221 def style_axis(ax, grid="y"):
222     """统一的出版级坐标轴: 白底、去顶/右脊柱、极浅网格 (可选)。"""
223     ax.set_facecolor("white")
224     ax.spines["top"].set_visible(False)
225     ax.spines["right"].set_visible(False)
226     for s in ("left", "bottom"):
227         ax.spines[s].set_linewidth(RC["axes.linewidth"])
228     if grid == "y":
229         ax.yaxis.grid(True, **GRID_KW)
230         ax.set_axisbelow(True)
231     elif grid == "x":

```

```

232     ax.xaxis.grid(True, **GRID_KW)
233     ax.set_axisbelow(True)
234 elif grid == "both":
235     ax.grid(True, **GRID_KW)
236     ax.set_axisbelow(True)
237
238
239 def panel(ax, tag, dx=-0.10):
240     """面板标签 (a)/(b): 粗体小写, 左上外侧 (3D 轴须用 text2D, 否则被当数据坐
        标)。"""
241     kw = dict(transform=ax.transAxes, fontsize=9, fontweight="bold", va="
        bottom",
242               ha="left", color="0.10")
243     if hasattr(ax, "get_zlim"): # mplot3d
244         ax.text2D(dx, 1.03, tag, **kw)
245     else:
246         ax.text(dx, 1.03, tag, **kw)
247
248
249 def tile(ax, title):
250     """面板标题 (≤TITLE_MAX 字符; 长论述进图注)。"""
251     assert len(title) <= TITLE_MAX, f"标题过长({len(title)}): {title}"
252     ax.set_title(title, fontsize=8.5, pad=4)
253
254
255 def note(ax, text, xy=(0.98, 0.97), ha="right", va="top", fs=7.5):
256     ax.text(*xy, text, transform=ax.transAxes, ha=ha, va=va, fontsize=fs,
257            bbox=dict(boxstyle="round,pad=0.28", fc="white", ec="0.80", lw
258                    =0.5, alpha=0.95))
259
260 def fit_left(fig, axes, pad_px=6.0):
261     """把左边距设成“y 轴刻度文字 + ylabel”实测所需的最小值 (当前 left 够用时
        不动),
262     防止长中文类别名被裁切 (bbox 出图为精确 figsize, 不裁切, 故必须自己留够边
        距)。
263     旧实现无条件在原 left 上再加一份标签宽度, fig5 的长类别名因此白占约 2 in 绘
        图区。"""
264     fig.canvas.draw()
265     ren = fig.canvas.get_renderer()
266     w_px = fig.get_size_inches()[0] * fig.dpi
267     left_cur = fig.subplotpars.left * w_px
268     need = 0.0
269     for ax in np.atleast_1d(axes).ravel():
270         labs = [t for t in ax.get_yticklabels() if t.get_text().strip()]
271         if not labs:
272             continue
273         left_px = min([t.get_window_extent(ren).x0 for t in labs]
274                      + ([ax.yaxis.label.get_window_extent(ren).x0]
275                        if ax.get_ylabel() else []))
276         need = max(need, ax.get_window_extent(ren).x0 - left_px + pad_px)

```

```

277     if need > left_cur:
278         fig.subplots_adjust(left=need / w_px)
279     return fig.subplotpars.left
280
281
282 # ===== 审计 =====
283 AUDIT: list[dict] = []
284
285
286 def tick_texts(axis):
287     """仅返回**视图范围内**可见的刻度文字（get_xticklabels 会带上范围外、
288     尺寸为 None 的隐藏刻度，拿去判裁切会得到假阳性）。3D 轴（Axis3D）走回退分
289     支。
290     容差必须**相对**（2026-09-12 修正）：原实现用绝对 1e-9，对量级 1e-11 的坐标
291     轴
292     （如 fig15(c) 的守恒残差）等于无穷大，会把视图外一个数量级的对数刻度也算成
293     “已绘制”，进而被误报为“裁切文字”。"""
294     try:
295         lo, hi = sorted(axis.get_view_interval())
296         tol = 1e-9 * max(abs(lo), abs(hi), 1e-300)
297         out = []
298         for tick in axis.get_major_ticks():
299             loc = float(tick.get_loc())
300             if lo - tol <= loc <= hi + tol:
301                 labs = [tick.label1] + ([tick.label2
302                                     if tick.label2.get_text().strip() else []])
303                 out += [t for t in labs if t.get_visible()]
304             return out
305     except (AttributeError, TypeError): # mplot3d Axis3D
306         return [t for t in axis.get_ticklabels() if t.get_text().strip() and
307                 t.get_visible()]
308
309
310 def _unreliable_extent(t):
311     """mplot3d 的刻度文字与 ax.text 的 window extent 经三维投影后不可靠，
312     不能用于裁切判定（3D 面板的文字位置只能目视核对）。"""
313     ax = t.axes
314     if ax is None or not hasattr(ax, "get_zlim"):
315         return False
316     if t in list(ax.texts):
317         return True
318     for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
319         if t in tick_texts(axis):
320             return True
321     return False
322
323
324 def all_axes(fig):
325     """图中**全部**坐标轴，含 inset（`ax.inset_axes` 建的子轴不在 `fig.axes`
326     里；

```

```

324     不纳入审计会漏检它的 label 与文字裁切)。"""
325     out = list(fig.axes)
326     for ax in list(fig.axes):
327         out += [c for c in getattr(ax, "child_axes", []) if c not in out]
328     return out
329
330
331 def drawn_texts(fig):
332     """图中**实际绘制**的文字（白名单遍历）——fig.findobj(Text) 会连带返回
333     刻度/数学排版留下的孤立 Text (axes=None)，不能用来判裁切与字号。"""
334     out = list(fig.texts)
335     for ax in all_axes(fig):
336         out += list(ax.texts) + [ax.title, ax.xaxis.label, ax.yaxis.label]
337         out += tick_texts(ax.xaxis) + tick_texts(ax.yaxis)
338         lg = ax.get_legend()
339         if lg is not None:
340             out += list(lg.get_texts())
341             if lg.get_title():
342                 out.append(lg.get_title())
343     return [t for t in out if t.get_text().strip() and t.get_visible()]
344
345
346 def _extent(txt):
347     return txt.get_window_extent(renderer=txt.figure.canvas.get_renderer())
348
349
350 def _overlap(a, b, tol=0.5):
351     return (a.x1 > b.x0 + tol and b.x1 > a.x0 + tol
352             and a.y1 > b.y0 + tol and b.y1 > a.y0 + tol)
353
354
355 def _tick_overlaps(ax):
356     bad = []
357     for axis, name in ((ax.xaxis, "x"), (ax.yaxis, "y")):
358         labs = [t for t in tick_texts(axis) if t.get_text().strip()]
359         for i in range(len(labs)):
360             for j in range(i + 1, len(labs)):
361                 if _overlap(_extent(labs[i]), _extent(labs[j])):
362                     bad.append(f"{name}:{labs[i].get_text()}/{labs[j].get_text()}"
363                               ")")
364     return bad
365
366
367 def audit_figure(fig, stem, pdf_path):
368     """逐张图的形式合规审计（尺寸/字号/标题/图例/柱基线/colorbar/刻度重叠/冗余
369     编码）。"""
370     rec = dict(stem=stem, w=round(fig.get_size_inches()[0], 3),
371                h=round(fig.get_size_inches()[1], 3), titles=[], legends=[], cb
372                  =0, ax3d=0,
373                bar_base=[], tick_bad=[], dense_marker=0, min_font=99.0,
374                embed_img=0,

```

```

371         clipped=[], cap_hit=[], cb_labels=[], cb_bad=[])
372     rec["aspect"] = round(rec["h"] / rec["w"], 3)
373     ren = fig.canvas.get_renderer()
374     fw, fh = float(ren.width), float(ren.height)
375     # 图内不得有 figure 级文字块（数据源/口径/偏差等论述一律由论文 \caption 承
        载）
376     rec["figtext"] = [t.get_text()[:16] for t in fig.texts if t.get_text().
        strip()]
377
378     texts = drawn_texts(fig)
379     for t in texts:
380         rec["min_font"] = min(rec["min_font"], float(t.get_fontsize()))
381
382     # 图注 (fig.text) 不得压住任何面板的 x 轴标签
383     for t in fig.texts:
384         tb = t.get_window_extent(renderer=ren)
385         for ax in all_axes(fig):
386             lab = ax.xaxis.label
387             if lab.get_text().strip() and _overlap(tb, lab.get_window_extent(
                renderer=ren),
388
389                                     tol=0.0):
390                 rec["cap_hit"].append(f"{t.get_text()[:12]}↔{lab.get_text()
                    [:12]}")
391
392     for ax in all_axes(fig):
393         if ax.get_title():
394             rec["titles"].append(ax.get_title())
395         lg = ax.get_legend()
396         if lg is not None:
397             rec["legends"].append(len(lg.get_texts()))
398         # colorbar 轴：内部创建的 label 为 "<colorbar>", 用户传入 cax 时只有
            _colorbar 属性
399         if ax.get_label() == "<colorbar>" or getattr(ax, "_colorbar", None)
            is not None:
400             rec["cb"] += 1
401             lab = (ax.get_ylabel().strip() or ax.get_xlabel().strip()) # 横/竖
                colorbar
402             rec["cb_labels"].append(lab)
403             # 连续色阶必须配 label（含单位）；无 label 的 colorbar 视为冗余
404             if not lab:
405                 rec["cb_bad"].append("colorbar 缺 label")
406         if hasattr(ax, "get_zlim"):
407             rec["ax3d"] += 1
408         for cont in ax.containers:
409             # 柱状图基线必须落在 0：水平柱查 x 基线，竖直柱查 y 基线（有符号条形
                允许
410             # 负向，但基线仍须是 0；对数轴由 matplotlib 贴住轴下限，x 仍为 0）
411             horiz = getattr(cont, "orientation", "vertical") == "horizontal"
412             for p in getattr(cont, "patches", []):
413                 if not isinstance(p, Rectangle):
414                     continue

```



```

414         if horiz:
415             if p.get_width() != 0 and min(abs(p.get_x()),
416                                           abs(p.get_x() + p.get_width())) > 1e
417                 -9:
418                 rec["bar_base"].append(round(float(p.get_x()), 4))
419             elif p.get_height() != 0 and min(abs(p.get_y()),
420                                           abs(p.get_y() + p.get_height())) > 1e
421                 -9:
422                 rec["bar_base"].append(round(float(p.get_y()), 4))
423         rec["tick_bad"] += _tick_overlaps(ax)
424
425     for ln in fig.findobj(Line2D):
426         x = np.asarray(ln.get_xdata())
427         if len(x) > 25 and ln.get_marker() not in ("None", None, " ", "") \
428             and ln.get_markevery() is None:
429             rec["dense_marker"] += 1
430
431     rec["embed_img"] = pdf_path.read_bytes().count(b"/Subtype /Image")
432     # 场图 (imshow 热图/圆盘) 本质是栅格数据, 允许每张场图面板嵌入 1 张位图;
433     # colorbar 的色带也用一条窄图绘制。除此之外的嵌入位图 = 图被栅格化, 判 FAIL
434     .
435     rec["raster_ax"] = sum(1 for ax in all_axes(fig) if ax.get_images())
436     rec["img_ok"] = rec["embed_img"] <= rec["raster_ax"] + rec["cb"]
437     # 文字裁切检查: bbox=None 精确出图, 超出画布的文字会被静默切掉
438     ren = fig.canvas.get_renderer()
439     fw, fh = float(ren.width), float(ren.height)
440     rec["clipped"] = []
441     for t in texts:
442         if _unreliable_extent(t):
443             continue # 3D 文字 extent 不可用, 改由目视核对
444         bb = t.get_window_extent(renderer=ren)
445         over = max(-bb.x0, bb.x1 - fw, -bb.y0, bb.y1 - fh) # >0 即超出画布
446         if over > CLIP_TOL:
447             rec["clipped"].append((t.get_text()[:16], round(float(over), 1)))
448     rec["font_ok"] = rec["min_font"] >= MIN_FONT - 1e-9
449     rec["title_ok"] = all(len(t) <= TITLE_MAX for t in rec["titles"])
450     rec["legend_ok"] = all(n <= LEG_MAX_EXEMPT.get(stem, LEG_MAX) for n in
451                          rec["legends"])
452     rec["tick_ok"] = len(rec["tick_bad"]) == 0
453     rec["bar_ok"] = not rec["bar_base"]
454     rec["clip_ok"] = not rec["clipped"] and not rec["cap_hit"]
455     # 场图 (fig9) 必须有带 label 的 colorbar; 其余图不得出现无 label 的
456     colorbar
457     rec["cb_ok"] = not rec["cb_bad"]
458     rec["ok"] = (rec["font_ok"] and rec["title_ok"] and rec["legend_ok"] and
459                 rec["tick_ok"]
460                 and rec["bar_ok"] and rec["clip_ok"] and not rec["figtext"]
461                 and rec["cb_ok"] and rec["img_ok"] and rec["dense_marker"] == 0
462                 and ASPECT_EXEMPT.get(stem, ASPECT)[0] <= rec["aspect"]
463                 <= ASPECT_EXEMPT.get(stem, ASPECT)[1])
464     AUDIT.append(rec)

```

```

459     return rec
460
461
462 def save(fig, stem):
463     """导出 矢量 PDF + 300dpi PNG + 灰度预览，并逐张审计。"""
464     FIG_DIR.mkdir(exist_ok=True)
465     QA_DIR.mkdir(exist_ok=True)
466     pdf, png = FIG_DIR / f"{stem}.pdf", FIG_DIR / f"{stem}.png"
467     fig.canvas.draw() # 先渲染，供刻度/字号 bbox 审计使用
468
469     # 1) 矢量 PDF (fonttype 42) + 2) 300 dpi PNG; warnings 通道捕获缺字
470     with warnings.catch_warnings(record=True) as wlist:
471         warnings.simplefilter("always")
472         fig.savefig(pdf, format="pdf")
473         fig.savefig(png, format="png", dpi=300)
474     for w in wlist:
475         m = str(w.message)
476         if "missing from font" in m or "Glyph" in m:
477             GLYPH.msgs.append(f"[warnings] {stem}: {m}")
478
479     # 3) 灰度预览（同一渲染的灰阶，用于“不靠颜色也能区分”核验）
480     from PIL import Image
481     Image.open(png).convert("L").save(QA_DIR / f"{stem}_gray.png")
482
483     fig.canvas.draw() # savefig 会临时改 dpi; 重绘回 fig.dpi 再审计 (bbox
    一致性)
484     rec = audit_figure(fig, stem, pdf)
485     flag = "PASS" if rec["ok"] else "FAIL"
486     print(f" [审计] {flag} {stem}: {rec['w']:.1f}x{rec['h']:.1f} in(纵横比 {
    rec['aspect']:.2f}) "
487           f"最小字号 {rec['min_font']:.1f} pt 标题 {len(rec['titles'])} 图例 {
    rec['legends']} "
488           f"刻度重叠 {len(rec['tick_bad'])} 嵌入位图 {rec['embed_img']} "
489           f"裁切文字 {len(rec['clipped'])} 图注压轴标 {len(rec['cap_hit'])} "
490           f"图内文字块 {len(rec['figtext'])}", flush=True)
491     if rec["clipped"] or rec["cap_hit"]:
492         print(f" [裁切/压轴标] {rec['clipped']} {rec['cap_hit']}", flush=
    True)
493     if rec["ax3d"]:
494         print(f" [3D] {rec['ax3d']} 个 3D 面板: 其刻度/轴内文字的位置须目视核
    对"
495               " (extent 不可用, 程序无法判裁切)", flush=True)
496     print(f" [出图] {pdf.name} ({pdf.stat().st_size/1024:.0f} kB) "
497           f"+ {png.name} ({png.stat().st_size/1024:.0f} kB) "
498           f"+ _qa/{stem}_gray.png", flush=True)
499     plt.close(fig)
500     return rec
501
502
503 # ===== F1 输入数据 =====
504 def fig1():

```

```

505 t1, Ta, Ce = read_att1()
506 t2, R = read_att2()
507 h1 = t1 / 3600.0
508 h2 = t2 / 3600.0
509 t_cut = h1[-1] # 附件1 截止 (4 h)
510 i_frz = int(np.argmax(R <= 1.200 + 1e-12))
511 t_frz = h2[i_frz] # 35 h 冻结点 (R 首次 1.200 cm)
512 x_end = 100.0
513 fig, (axL, axR) = plt.subplots(1, 2, figsize=(W_DOUBLE, 3.4))
514 fig.subplots_adjust(left=0.098, right=0.935, top=0.865, bottom=0.155,
    wspace=0.50)
515
516 # ---- (a) 附件1 温湿环境 ----
517 axL.plot(h1, Ta, color=ROLE["aux1"], lw=1.3, label="温度 $T_a(t)$ (左轴)
    ")
518 axL.set_xlabel("时间 $t$ / h")
519 axL.set_ylabel("温度 $T_a$ / °C", color=ROLE["aux1"])
520 axL.tick_params(axis="y", colors=ROLE["aux1"])
521 axL.set_xlim(0, 6.3)
522 axL.set_ylim(27.0, 58.0)
523 axL.set_xticks([0, 1, 2, 3, 4, 5, 6])
524 axL.set_yticks([30, 35, 40, 45, 50, 55])
525 axC = axL.twinx()
526 axC.plot(h1, Ce, color=ROLE["aux2"], lw=1.3, ls="--", label="水分浓度 $C_{env}(t)$ (右轴) ")
527 axC.set_ylabel("水分浓度 $C_{env}$ / (kg/kg)", color=ROLE["aux2"])
528 axC.tick_params(axis="y", colors=ROLE["aux2"])
529 axC.set_ylim(0.0175, 0.0555)
530 axC.spines["right"].set_visible(True) # 双 y 轴: 保留右脊柱承载右轴
531 axC.spines["right"].set_linewidth(RC["axes.linewidth"])
532 axL.axvspan(t_cut, 6.3, color="0.93", lw=0, zorder=0)
533 axL.plot([t_cut, 6.3], [Ta[-1]] * 2, color=ROLE["aux1"], ls=":", lw=1.1)
534 axC.plot([t_cut, 6.3], [Ce[-1]] * 2, color=ROLE["aux2"], ls=":", lw=1.1)
535 axL.axvline(t_cut, color=ROLE["guide"], ls=":", lw=0.9)
536 axL.annotate(f"$t$={t_cut:.0f} h 数据截止\n其后末值延拓", xy=(t_cut,
    0.80),
537             xycoords=("data", "axes fraction"), xytext=(0.97, 0.62),
538             textcoords="axes fraction", fontsize=7.5, ha="right", va="top",
539             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
540             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
    =0.5, alpha=0.95))
541 axL.text(0.97, 0.42, f"延拓值 (左/右轴) \n$T_a$={Ta[-1]:.3f} °C\n$C_{env}$
    = {Ce[-1]:.5f} kg/kg",
542         transform=axL.transAxes, fontsize=7.5, ha="right", va="top",
543         bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
    =0.5, alpha=0.95))
544 hs = [Line2D([], [], color=ROLE["aux1"], lw=1.3),
545       Line2D([], [], color=ROLE["aux2"], lw=1.3, ls="--")]
546 axL.legend(hs, ["温度 $T_a$ (左轴)", "水分 $C_{env}$ (右轴)"],
547           loc="lower left", bbox_to_anchor=(0.02, 0.02), handlelength=1.6)
548 panel(axL, "(a)")

```

```

549 tile(axL, "附件1: 环境温湿与延拓")
550
551 # ---- (b) 附件2 半径 ----
552 # 时间口径与论文统一: 附件2 约 21 h 起进入 1.198 cm 近平台段 (H_SEG 的 B/C
    分界),
553 # 35 h 只是 R 在三位小数下“首达 1.200 cm”的时刻, 不再用作“冻结点”。
554 t_plat = 21.0
555 axR.plot(h2, R, color=ROLE["data"], lw=1.5, label="附件2 实测 $R(t)$")
556 axR.plot([h2[-1], x_end], [R[-1]] * 2, color=ROLE["data"], ls="--", lw
    =1.1)
557 axR.axvspan(t_plat, h2[-1], color="0.93", lw=0, zorder=0)
558 axR.axvspan(h2[-1], x_end, color="0.96", lw=0, zorder=0)
559 axR.axvline(t_plat, color=ROLE["guide"], ls=":", lw=0.9)
560 axR.axvline(h2[-1], color=ROLE["guide"], ls=":", lw=0.7)
561 axR.axhline(R[-1], color=ROLE["guide"], ls=":", lw=0.9)
562 axR.annotate(f"≈{t_plat:.0f} h 起进入近平台段\n ($t$={t_frz:.0f} h 首达 {
    R[i_frz]:.3f} cm) ",
563             xy=(t_plat, R[int(np.argmax(np.abs(h2 - t_plat)))]),
564             xytext=(0.30, 0.72), textcoords="axes fraction",
565             fontsize=7.5, ha="left", va="top",
566             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
567             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
    =0.5, alpha=0.95))
568 # 引线原指向 (72 h, 1.198 cm), 箭头正好落在右侧“平台 1.198 cm”文字上:
569 # 改为指向 72 h 竖虚线 (在平台文字上方), 引线不再穿字。
570 axR.annotate(f"附件2 截止 {h2[-1]:.0f} h: \n{R[-1]:.3f} cm 平台延拓",
571             xy=(h2[-1], R[-1] + 0.10), xytext=(0.40, 0.33), textcoords="
    axes fraction",
572             fontsize=7.5, ha="left", va="top",
573             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
574             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
    =0.5, alpha=0.95))
575 # 抬高 0.010 cm: 原位置文字底边贴住 1.198 cm 平台虚线
576 axR.text(98.0, R[-1] + 0.024, f"平台 {R[-1]:.3f} cm", ha="right", va="
    bottom",
577         fontsize=7.5, color="0.30")
578 axR.set_xlim(0, x_end)
579 axR.set_ylim(1.16, 2.10)
580 axR.set_xlabel("时间 $t$ / h")
581 axR.set_ylabel("药材半径 $R$ / cm")
582 panel(axR, "(b)")
583 tile(axR, "附件2: 半径收缩与近平台")
584
585 print(f" [F1] 附件1 {len(t1)} 点 0-{t_cut:.0f} h; 附件2 {len(t2)} 点 0-
    {h2[-1]:.0f} h, "
586       f"$R$ {R[0]:.3f}→{R[-1]:.3f} cm, ≈{t_plat:.0f} h 起近平台"
587       f" ({t_frz:.0f} h 首达 {R[i_frz]:.3f} cm; 其后至 {h2[-1]:.0f} h 仅降
    "
588       f"{R[i_frz]-R[-1]:.3f} cm) ", flush=True)
589 save(fig, "fig1_输入数据")
590

```

```

591
592 # ===== F2 问题1 解析验证 =====
593 def fig2():
594     """(a) 特征位置温度时程: 中心  $r=0$  与表面  $r=R_0$  的数值解 vs 半解析解;
595     (b) 偏差  $|T_{\text{num}}-T_{\text{解析}}|$  时程 (对数纵轴), 标出四位小数输出分辨率  $5e-5$  °C。
596
597     数据源 数据/q1_fields.npz (times/T_C/Ts, 1 s 网格 0-1800 s)、
598     数据/v1_points_final.csv (比较点的 T_num/T_analytic/abs_dev);
599     半解析解由 代码/analytic_heat.py 的 Robin 谱解现算 (300 模, 0-1800 s 同网
600     格)。
601
602     """
603
604     from analytic_heat import RobinCylinder
605     from solver_q1 import ALPHA, BI_H, R0, T0_K
606
607     d = np.load(DATA_DIR / "q1_fields.npz")
608     t_s, pos_cm, T_C, Ts = d["times"], d["pos_cm"], d["T_C"], d["Ts"]
609     env_t, env-Ta = d["env_t"], d["env-Ta"]
610     _, pts = read_csv_rows("v1_points_final.csv")
611     dev_max = max(float(p["abs_dev"]) for p in pts)
612
613     ana = RobinCylinder(BI_H, ALPHA, R0, n_modes=300)
614     T_ana = ana.eval(np.array([0.0, R0]), t_s, env_t, env-Ta - T0_K, T0_K) -
615         273.15
616
617     T_ana_c, T_ana_s = T_ana[:, 0], T_ana[:, 1]
618     num_c, num_s = T_C[:, 0], Ts - 273.15 # npz 的 Ts 为开尔文, 统一换 °C
619     dev_c, dev_s = np.abs(num_c - T_ana_c), np.abs(num_s - T_ana_s)
620     res = 5e-5 # 四位小数输出分辨率 °C
621
622     fig, (axA, axB) = plt.subplots(2, 1, figsize=(W_SINGLE, 3.30), sharex=
623         True,
624         gridspec_kw=dict(height_ratios=[1.30, 1.0]))
625     fig.subplots_adjust(left=0.145, right=0.975, top=0.885, bottom=0.155,
626         hspace=0.22)
627     m = t_s / 60.0
628     for num, ana_, col, lab in ((num_c, T_ana_c, ROLE["model"], "中心  $r=0$ "),
629         ,
630         (num_s, T_ana_s, ROLE["aux1"], "表面  $r=R_0$ ")):
631         axA.plot(m, ana_, color=col, lw=1.1, ls="--", label=f"{lab}: 解析解")
632         axA.plot(m, num, color=col, lw=1.2, label=f"{lab}: 数值解")
633     t_pt = [float(p["t_s"]) / 60.0 for p in pts if abs(float(p["r_cm"])) < 1
634         e-9]
635     y_pt = [float(p["T_num_C"]) for p in pts if abs(float(p["r_cm"])) < 1e
636         -9]
637     axA.plot(t_pt, y_pt, ls="none", marker="o", ms=3.0, mfc="white", mec=
638         ROLE["model"],
639         mew=0.9, label="解析比较点 (表 4 的 V1)")
640     axA.set_ylabel("温度  $T$  / °C")
641     axA.set_ylim(27.9, 37.6)
642     style_axis(axA, grid="y")
643     axA.legend(loc="upper left", ncol=2, handlelength=1.6, columnspacing
644         =1.0,

```

```

633         labelspacing=0.24)
634 panel(axA, "(a)", dx=-0.115)
635 tile(axA, "特征位置温度时程: 解析 vs 数值")
636
637 for dev_, col, lab in ((dev_c, ROLE["model"], "中心  $r=0$ "),
638                       (dev_s, ROLE["aux1"], "表面  $r=R_0$ ")):
639     axB.semilogy(m, dev_, color=col, lw=1.1, label=lab)
640     axB.axhline(res, color=ROLE["guide"], ls=":", lw=1.0)
641     axB.text(29.4, res * 1.30, "输出分辨率  $5 \times 10^{-5} \text{ } ^\circ\text{C}$ ", fontsize
642             =7.5, color="0.30",
643             ha="right", va="bottom")
644     # 对数纵轴上的"深谷" ( $t \approx 14/16/21/28 \text{ min}$  掉到  $1e-10$ ) 不是误差在某时刻骤变,
645     # 而是
646     # 偏差过零时  $|\Delta T_{\text{num}} - T_{\text{ana}}|$  取到数值零; 不注明会被读反, 故在分辨率注记下方
647     # 补一行。
648     axB.text(29.4, res * 0.62, "谷=偏差过零 (非误差增大)", fontsize=7.5,
649             color="0.30",
650             ha="right", va="top")
651     axB.set_xlabel("时间  $t / \text{min}$ ")
652     axB.set_ylabel("偏差 /  $^\circ\text{C}$ ")
653     axB.set_xlim(0, 30)
654     style_axis(axB, grid="y")
655     axB.legend(loc="lower left", handlelength=1.6, labelspacing=0.24)
656 panel(axB, "(b)", dx=-0.115)
657 # 面板画的是 5 个特征位置的**逐点**偏差 (解析比较), 与论文正文引用的
658 # " $N=320$  全域单元中心最大偏差  $5.9e-7 \text{ } ^\circ\text{C}$ " (v1_convergence.csv 末行) 不是同
659 # 一口径,
660 # 标题必须写明"特征位置", 避免与论文的全局验证数字混淆。标题受  $\leq 28$  字符上
661 # 限约束。
662 tile(axB, "特征位置逐点偏差:  $<3.2 \times 10^{-6} \text{ } ^\circ\text{C}$ ")
663
664 print(f" [F2] 解析验证: 比较点最大偏差 {dev_max:.3e}  $^\circ\text{C}$  ({len(pts)} 点, 四
665      位小数分辨率 "
666      f"{res:.0e}  $^\circ\text{C}$ , 低 {res/dev_max:.0f} 倍); 1 s 网格 1800 点全程: 中心
667      max "
668      f"{dev_c.max():.2e}、表面 max {dev_s.max():.2e}  $^\circ\text{C}$ ; 中心 1800 s 数值 {
669      num_c[-1]:.4f} "
670      f"/ 解析 {T_ana_c[-1]:.4f}  $^\circ\text{C}$ ; 表面 1800 s 数值 {num_s[-1]:.4f} / 解析
671      {T_ana_s[-1]:.4f}  $^\circ\text{C}$ ",
672      flush=True)
673 save(fig, "fig2_问题1解析验证")
674
675 # ===== F3 温湿剖面 =====
676 def fig3():
677     """多时刻径向剖面: 温度 (3 h 内) 与水分浓度 (全程) ——线色按时间 viridis
678     映射,
679     每面板配竖直 colorbar (标"时间 (h)"), 剖面用"散点+连线" (每 4 点一个标
680     记)。
681     数据源 数据/q23_main_steps.npz (t、C、T); t_f 与判据区间取自 q23_summary.

```

```

        json /
672 criteria.csv。定量读数是"时间色阶 + colorbar"，图例仅列 4 个代表时刻。
673 ""
674 d = np.load(DATA_DIR / "q23_main_steps.npz")
675 t, C, T = d["t"], d["C"], d["T"]
676 pos_cm = np.arange(C.shape[1]) * 0.1
677 tf_h = read_json("q23_summary.json")["tf_h"]
678 _, crit = read_csv_rows("criteria.csv")
679 le_row = [c for c in crit if c["判据"] == "Le" and "附录3" in c["物性"
        ]][0]
680 le_txt = f"{le_row['最小值']} - {le_row['最大值']}"
681
682 def prof(field, h):
683     i = int(np.argmin(np.abs(t - h * 3600)))
684     return field[i].copy()
685
686 t_T = [0.25, 1.0, 2.0, 3.0] # 预热平衡段 (问题 2 表 3 的 3 h 内)
687 t_C = [0.5, 12.0, 36.0, 57.0] # 全程 (问题 3, 至 t_f≈57.2 h)
688
689 fig = plt.figure(figsize=(W_DOUBLE, 3.4))
690 axT = fig.add_axes([0.068, 0.170, 0.298, 0.690])
691 caxT = fig.add_axes([0.374, 0.170, 0.014, 0.690])
692 axC = fig.add_axes([0.565, 0.170, 0.298, 0.690])
693 caxC = fig.add_axes([0.871, 0.170, 0.014, 0.690])
694
695 for ax, cax, tt, field, ylab, ttl, tag in (
696     (axT, caxT, t_T, T, "药材温度 $T$ / °C", "温度剖面: 3 h 内近准稳态",
        "(a)"),
697     (axC, caxC, t_C, C, "水分浓度 $C$ / (kg/kg)", "水分剖面: 中心最慢",
        "(b)")):
698     norm = matplotlib.colors.Normalize(vmin=min(tt), vmax=max(tt))
699     handles = []
700     for h in tt:
701         col = TIME_CMAP(norm(h))
702         ax.plot(pos_cm, prof(field, h), color=col, lw=1.2, marker="o", ms
        =2.0,
703             markevery=4, mfc="white", mec=col, mew=0.7)
704         handles.append(Line2D([], [], color=col, lw=1.2, label=f"$t$={h:g}
        h"))
705     cb = fig.colorbar(plt.cm.ScalarMappable(norm=norm, cmap=TIME_CMAP),
        cax=cax)
706     cb.set_label("时间 $t$ / h", fontsize=7.5)
707     cb.ax.tick_params(labelsize=7.5)
708     cb.set_ticks(tt)
709     ax.set_xlabel("到药材中心的距离 $r$ / cm")
710     ax.set_ylabel(ylab)
711     ax.set_xlim(0, 2.02)
712     style_axis(ax, grid="y")
713     ax.legend(handles=handles, loc="upper left", ncol=2, handlelength
        =1.4,
714         columnspacing=0.9, labelspacing=0.26)

```

```

715     panel(ax, tag)
716     tile(ax, ttl)
717     axT.set_ylim(27.0, 58.5)
718     axC.set_ylim(0.0, 3.15)
719     # 阈值线直接进 y 刻度 (0.15 这个刻度即达标阈值), 避免图内文字压曲线。
720     # zorder=1 让剖面 (默认 zorder=2) 压在阈值线之上: t=57 h 的剖面在中段几乎贴
       着
721     # 0.15, 原先把阈值线画在曲线之上, 灰色虚线段直接盖在黄色曲线上, 读者会把阈值
       线
722     # 误认成 t=57 h 的曲线 (实测全分辨率裁剪确认)。
723     axC.axhline(TH, color=ROLE["guide"], ls="--", lw=1.1, zorder=1)
724     axC.set_yticks([0.0, TH, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
725     c36, c57 = prof(C, 36.0)[0], prof(C, 57.0)[0]
726     cs57 = prof(C, 57.0)[-1]
727     axC.annotate("中心最慢\n ($r$=0, 尾段拖长)", xy=(0.02, 0.5 * (c36 + c57))
       ,
728         xytext=(0.13, 0.30), textcoords="axes fraction", fontsize=7.5,
729         ha="left", va="top",
730         arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"))
731     Ta_end = float(T[-1].max())
732     print(f" [F3] t_f={tf_h:.3f} h; 中心 {c57:.4f} / 表面 {cs57:.4f}; 36→57 h
       中心降 "
733         f"{c36-c57:.4f}; Le={le_txt}; t={t_T[-1]:g} h 全径 {Ta_end-0.4:.1f}
       - {Ta_end:.1f} °C; "
734         f"时间色阶 viridis (4 时刻/面板) + 竖直 colorbar", flush=True)
735     save(fig, "fig3_温湿剖面")
736
737
738     # ===== F4 达标穿越 (钱图) =====
739     def fixedR_control(force=False):
740         """附4 + 固定 R0 对照的 max-U 全程曲线 (模型现算; 缓存与 q4_split.csv 逐次
       核对)。"""
741         cache = FIG_DIR / "_fig4_对照_附4固定R_maxU.csv"
742         _, split = read_csv_rows("q4_split.csv")
743         tf_ref = float([s["t_f_h"] for s in split if s["case"] == "
       appendix4_fixedR"][0])
744         if cache.exists() and not force:
745             arr = np.loadtxt(cache, delimiter=",", skiprows=1)
746             tt, uu = arr[:, 0], arr[:, 1]
747             tf_c = crossing_time(tt, uu, TH) / 3600.0
748             # 缓存值的“插值穿越时刻”与 q4_split.csv 的“步末时刻”差 <0.005 h (口
       径差),
749             # 故容差取 0.05 h; 超出即视为缓存过期, 重算。
750             if abs(tf_c - tf_ref) < 0.05:
751                 print(f" [F4] 对照曲线读缓存 (插值穿越 {tf_c:.4f} h, q4_split.csv 步
       末 "
752                     f"{tf_ref:.4f} h)", flush=True)
753                 return tt, uu, tf_c
754             print(f" [F4] 缓存与 q4_split.csv 不符 ({tf_c:.4f} vs {tf_ref:.4f}),
       重算", flush=True)
755     print(" [F4] 现算 附4+固定R 对照曲线 (solver_q4, N=160, 约 1-2 min) ...",

```



```

        flush=True)
756 from run_q4 import make_env, read_env, read_radius, run_q4_full
757 from solver_q1 import R0
758 t0 = time.perf_counter()
759 env_t, TaK, Ce = read_env()
760 env_hold = make_env("hold", env_t, TaK, Ce)
761 read_radius() # 复用其结构自检（正、单调不增）
762 _, log = run_q4_full(160, env_hold, lambda tt: R0)
763 tt = np.array([x[0] for x in log])
764 uu = np.array([x[4] for x in log])
765 tf_c = crossing_time(tt, uu, TH) / 3600.0
766 print(f" [F4] 对照曲线算毕 {time.perf_counter()-t0:.0f} s, t_f={tf_c:.4f}
        h "
767       f"(q4_split.csv {tf_ref:.4f} h)", flush=True)
768 assert abs(tf_c - tf_ref) < 2e-2, "对照曲线与 q4_split.csv 不一致"
769 FIG_DIR.mkdir(exist_ok=True)
770 np.savetxt(cache, np.column_stack([tt, uu]), delimiter=",", header="t_s,
        umax",
771           comments="", fmt="%.10g")
772 return tt, uu, tf_c
773
774
775 def fig4(no_cache=False):
776     d3 = np.load(DATA_DIR / "q23_main_steps.npz")
777     t3, um3 = d3["t"], d3["um"]
778     # 问题4 主解 = 内生几何版:  $\max_q U(q, t)$  由内生时空场逐帧全域 ( $r \leq R(t)$ , 域外
        NaN) 取最大值
779     de = np.load(DATA_DIR / "q4_endo_fields.npz")
780     t4, um4 = de["t_s"], np.nanmax(de["C"], axis=1)
781     tc, umc, tfc = fixedR_control(force=no_cache)
782
783     tf3 = d3["crossing"][2] / 3600.0
784     tf4 = crossing_time(t4, um4, TH) / 3600.0
785     lb3 = float([r["value"] for r in read_csv_rows("v3_bound.csv")][1]
        if r["quantity"] == "t_lb_h"][0])
786     lb4 = float([r["value"] for r in read_csv_rows("q4_bound.csv")][1]
        if r["quantity"] == "t_int_h"][0])
787     _, split = read_csv_rows("q4_split.csv")
788     tf_ctl_csv = float([s["t_f_h"] for s in split if s["case"] == "
        appendix4_fixedR"][0])
789
790
791
792     # 与结果总账.csv 交叉核对: 图上 问题4 主解 t_f 必须等于总账的**内生几何主值
        ** (找不到判失败)。
793     # 总账该行写的是末帧时刻 (步末口径 50.5369 h), 本图用线性插值穿越时刻 (50
        .5362 h),
794     # 二者差异 ~0.0007 h, 故容差取 0.01 h。
795     # (按"类别=t_f / 项目前缀"取"值"列: 总账的项目名里含英文逗号与引号, 不能按
        逗号裸切)
796     _, ledger_rows = read_csv_rows("结果总账.csv")
797     ledger = {}
798     for r in ledger_rows:

```

```

799     if r["类别"] != "t_f":
800         continue
801     for key, pat in (("q3", "问题3主值"), ("q4", "问题4主值")):
802         if r["项目"].startswith(pat):
803             ledger[key] = float(r["值"])
804     assert set(ledger) >= {"q3", "q4"}, ledger
805     assert abs(ledger["q3"] - tf3) < 1e-6, ledger
806     assert abs(ledger["q4"] - tf4) < 0.01, \
807         f"图上问题4主解 t_f={tf4:.4f} h 与总账内生主值 {ledger['q4']} h 不符"
808     print(f" [F4] 与结果总账.csv 一致: 问题3 {ledger['q3']} h / 问题4 (内生几
        何主值) "
809         f"{ledger['q4']} h (本图插值穿越 {tf4:.4f} h) ", flush=True)
810
811     h3, h4, hc = t3 / 3600.0, t4 / 3600.0, tc / 3600.0
812     fig, ax = plt.subplots(figsize=(W_SINGLE, 3.2))
813     fig.subplots_adjust(left=0.135, right=0.975, top=0.875, bottom=0.17)
814     ax.plot(h3, um3, color=ROLE["model2"], lw=1.3,
815             label=f"问题3 主解 (附3 + 固定 $R$): $t_f$={tf3:.2f} h")
816     ax.plot(h4, um4, color=ROLE["model"], lw=1.5,
817             label=f"问题4 主解 (内生几何 + $R_{\{\text{pred}\}}(t)$): $t_f$={tf4:.2f} h"
818             )
819     ax.plot(hc, umc, color=ROLE["ref"], lw=1.2, ls="--",
820             label=f"对照: 附4 + 固定 $R_0$: $t_f$={tfc:.2f} h")
821     ax.axhline(TH, color=ROLE["guide"], ls="--", lw=1.1,
822                label=f"达标阈值 $C_{\{\text{th}\}}$={TH} kg/kg (题目) ")
823     ax.text(1.0, TH + 0.05, f"${\text{TH}}$", fontsize=7.5, color="0.30")
824     for lb, col in ((lb3, ROLE["model2"]), (lb4, ROLE["model"])):
825         ax.axvline(lb, color=col, ls=":", lw=1.2)
826     # 口径修正 (与论文一致): lb3/lb4 是“常系数/瞬时首模”两种近似下的参考时
827     # 标,
828     # 不是严格误差下界——图上与 stdout 一律写“参考时标”。
829     for lb, col, txt, yf, yt in ((lb3, ROLE["model2"], f"问题3 参考时标 {lb3
        :.2f} h", 0.86, 1.95),
830                                (lb4, ROLE["model"], f"问题4 参考时标 {lb4:.2f} h",
831                                 0.77, 1.42)):
832         ax.annotate(txt, xy=(lb, yt), xytext=(0.185, yf), textcoords="axes
            fraction",
833                    fontsize=7.5, color=col, ha="left", va="top",
834                    arrowprops=dict(arrowstyle="->", lw=0.7, color=col))
835     for hh, col in ((tf3, ROLE["model2"]), (tf4, ROLE["model"]), (tfc, ROLE[
        "ref"])):
836         ax.axvline(hh, color=col, ls="-.", lw=0.8, alpha=0.85)
837     ax.set_xlim(0, 134)
838     ax.set_ylim(0, 2.62)
839     ax.set_xlabel("烘干时间 $t$ / h")
840     ax.set_ylabel(r"$\max_q U(q,t)$ / (kg/kg)")
841     style_axis(ax, grid="y")
842     ax.legend(loc="upper right", handlelength=1.8, labelspacing=0.30)
843     panel(ax, "(a)", dx=-0.115)
844     tile(ax, "达标穿越: 问题3 vs 问题4")

```

```

843 ins = ax.inset_axes([0.40, 0.20, 0.555, 0.36])
844 lo, hi = min(tf4, tf3) - 5.0, max(tf4, tf3) + 4.0
845 for hh, uu, col, lab, sty in ((h3, um3, ROLE["model2"], "问题3 主解", "-"),
846                               (h4, um4, ROLE["model"], "问题4 主解", "-")):
847     m = (hh >= lo) & (hh <= hi)
848     ins.plot(hh[m], uu[m], color=col, lw=1.3, ls=sty, label=lab)
849 ins.axhline(TH, color=ROLE["guide"], ls="--", lw=1.0)
850 ins.set_xlim(lo, hi)
851 ins.set_ylim(0.1445, 0.1575)
852 ins.set_xlabel("$t$ / h", fontsize=7.5, labelpad=0.5)
853 ins.tick_params(labelsize=7.5, pad=1.5)
854 ins.grid(True, **GRID_KW)
855 ins.set_axisbelow(True)
856 ins.annotate(f"$t_f$={tf3:.2f} h", xy=(tf3, TH), xytext=(tf3 + 0.8,
857                               0.1565),
858               fontsize=7.5, color=ROLE["model2"], va="bottom")
859 ins.annotate(f"$t_f$={tf4:.2f} h", xy=(tf4, TH), xytext=(tf4 + 0.8,
860                               0.1565),
861               fontsize=7.5, color=ROLE["model"], va="bottom")
862 print(f" [F4] t_f: 问题3 {tf3:.4f} h / 问题4 (内生几何) {tf4:.4f} h / 附4
863       固定R {tfc:.4f} h"
864       f" (csv {tf_ctl_csv:.4f}); 下界 {lb3:.4f} / {lb4:.4f} h", flush=
865       True)
866 i_tf = int(np.argmax(np.abs(h4 - tf4)))
867 print(f" [F4] maxU 抽查 (源 q4_endo_fields.npz 的 nanmax(C)): 帧 {i_tf}
868       t={h4[i_tf]:.4f} h "
869       f"maxU={um4[i_tf]:.6f} (阈值 {TH}); 末帧 t={h4[-1]:.4f} h maxU={um4
870       [-1]:.6f}", flush=True)
871 print(f" [F4] 效应拆分: ◎ 附4+固定 $R_0$ = {tf_ctl_csv:.2f} h; "
872       f"◎ 附4+内生 $R(t)$ = {tf4:.2f} h; ◎ 问题3 (附3+固定 $R$) = {tf3:.2f
873       } h; "
874       f"收缩效应 ◎-◎ = {tf_ctl_csv-tf4:.2f} h ({(tf_ctl_csv-tf4)/
875       tf_ctl_csv*100:.1f}%); "
876       f"净效应 ◎-◎ = {tf3-tf4:.2f} h ({(tf3-tf4)/tf3*100:.1f}%); "
877       f"参考时标余量 {tf3-lb3:.2f} / {tf4-lb4:.2f} h", flush=True)
878 assert tf4 > lb4 and tf3 > lb3, "主解低于参考时标"
879 save(fig, "fig4_达标穿越")
880
881 # ===== F5 敏感性 =====
882 # 误差预算的类别 (与论文 表~tab:预算 的三类分报一致): error_budget.csv 的“类
883 # 别/适用”列
884 # 记的是适用范围 (问题3/问题4/两问), 类别改由误差源前缀判定 → (前缀, 图例名,
885 # 颜色)
886 BUDGET_CAT = [("数值误差", "数值误差", ROLE["model"]),
887               ("环境情景", "环境情景", ROLE["ref"]),
888               # 前缀仍是数据侧的键“口径依赖” (error_budget.csv / 结果总账 的键名
889               # 不动,
890               # 免得牵动总账 5 行键), 图例名改为论文表~\ref{tab:预算} 里同类的“
891               设定依赖”, 图文口径一致

```

```

881         ("口径依赖", "设定依赖", OI["orange"]),
882         ("模型结构差异", "模型结构差异", ROLE["aux2"]),
883         ("已检查因素", "累计量级", OI["grey"]))
884
885
886 def read_sensitivity():
887     """sensitivity.csv 的“组别”内含英文逗号（如 C_e=0.04986(题目口径,主)），
888     按“末两列为 t_f_h / delta_t_f_h”解析，组别名保留原文。"""
889     with open(DATA_DIR / "sensitivity.csv", encoding="utf-8") as f:
890         rows = [r for r in csv.reader(f) if r and not r[0].startswith("#")]
891         out = []
892         for r in rows[1:]:
893             out.append((".".join(r[:-2]), float(r[-2]), float(r[-1])))
894     return out
895
896
897 def read_q4_split():
898     """q4_split.csv: 问题4 效应拆分的唯一数据源（基准=内生几何主解 t_f）。
899
900     返回各 case 的 t_f/h; csv 自带的派生行（shrink_effect_endo_h、
901         net_effect_endo_h）
902     用 t_f 现算并逐条核对，防止“csv 改了、图却没跟着改”。"""
903     _, rows = read_csv_rows("q4_split.csv")
904     d = {r["case"]: float(r["t_f_h"]) for r in rows}
905     base = d["appendix4_endogenous_R(t)"]
906     for k, v in (("shrink_effect_endo_h", d["appendix4_fixedR"] - base),
907                 ("net_effect_endo_h", d["problem3"] - base)):
908         assert abs(d[k] - v) < 1e-6, (k, d[k], v)
909     return d
910
911 def fmt_mag(m):
912     """误差量级标签: 10 的整数次幂用 $10^k$, 否则 %.3g。"""
913     k = np.log10(m)
914     if abs(k - round(k)) < 1e-9:
915         return rf"$10^{{{int(round(k))}}}$ h"
916     return f"{m:.3g} h"
917
918
919 def fig5():
920     sens = read_sensitivity()
921     q4 = read_q4_split()
922     q4_base = q4["appendix4_endogenous_R(t)"]
923     # 问题4 效应拆分一律读 q4_split.csv（基准=内生几何主解）；sensitivity.csv
924     # 里
925     # 三个旧外生拆分行（问题4效应拆分:*）不再进图
926     bars = [(n, d, tf) for n, tf, d in sens if not n.startswith("问题4效应拆
927         分")]
928         + [("问题4效应拆分:附录4固定R(对主解)", q4["appendix4_fixedR"] -
929             q4_base,
930             q4["appendix4_fixedR"]),

```

```

928         ("问题4效应拆分:问题3(对主解)", q4["problem3"] - q4_base,
929         q4["problem3"]),
930         ("问题4效应拆分:附录4+外生R(t)(交叉验证)", q4["appendix4_R(t)] -
          q4_base,
931         q4["appendix4_R(t)"])]])
932 base = [b for b in bars if abs(b[1]) < 1e-12]
933 bars = sorted([b for b in bars if abs(b[1]) > 1e-12], key=lambda x: abs(
          x[1]))
934 _, budget = read_csv_rows("error_budget.csv")
935
936 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(W_DOUBLE, 4.35),
937                               gridspec_kw=dict(height_ratios=[1.0, 1.15]))
938 fig.subplots_adjust(left=0.30, right=0.985, top=0.93, bottom=0.17,
939                     hspace=0.45)
940 # 分两组: 敏感性因素 + 问题4 效应拆分归因 (基准=q4_split.csv 的内生几何主
      解)
941 grp1 = sorted([b for b in bars if not b[0].startswith("问题4效应拆分")],
942               key=lambda x: -x[1])
943 grp2 = sorted([b for b in bars if b[0].startswith("问题4效应拆分")],
944               key=lambda x: -x[1])
945 shown = grp1 + grp2
946 ys, cur = [], len(shown) - 0.5
947 for i, b in enumerate(shown):
948     if i == len(grp1):
949         cur -= 1.0          # 组间留白
950     ys.append(cur)
951     cur -= 1.0
952 cols = [ROLE["ref"] if v > 0 else ROLE["model"] for _, v, _ in shown]
953 ax1.barh(ys, [v for _, v, _ in shown], color=cols, height=0.58, zorder
          =3)
954 ax1.set_yticks(ys)
955 ax1.set_yticklabels([n for n, _, _ in shown], fontsize=7.5)
956 vals = [v for _, v, _ in shown]
957 x_lo = -14.0 if min(vals) < 0 else 0.0 # 无负向柱时轴从 0 起
958 ax1.axvline(0, color="0.25", lw=0.8)
959 ax1.set_xlabel(r"$\Delta t_f$ / h (相对各自基准解)")
960 ax1.set_xlim(x_lo, 105)
961 ax1.set_ylim(min(ys) - 0.8, max(ys) + 2.0)
962 style_axis(ax1, grid="x")
963 for y, (n, v, tf) in zip(ys, shown):
964     ax1.text(v + 2.0 if v > 0 else 1.0, y, f"{v:+.4f} h",
965             va="center", ha="left", fontsize=7.5)
966 div = 0.5 * (ys[len(grp1) - 1] + ys[len(grp1)])
967 ax1.axhline(div, color="0.65", lw=0.8, ls="--", zorder=1)
968 # 面板标题已写“敏感性: 各因素 $\Delta t_f$”, 每根条的因素名在 y 轴标签
      上,
969 # 故 2026-09-12 删去上方那处孤悬的“敏感性因素”文字(无对应图例图元, 属残
      句)。
970 ax1.text(103.0, div, "问题4 效应拆分 (基准: 内生主解)", fontsize=7.5,
          color="0.25", ha="right", va="center")
971 ce = [b for b in grp1 if b[0].startswith("C_e")][0]

```

```

972 others = sum(abs(b[1]) for b in grp1 if b is not ce)
973 panel(ax1, "(a)")
974 title(ax1, "敏感性: 各因素  $\Delta t_f$ ")
975
976 # (b) 误差预算: error_budget.csv 的“类别/适用”列已是问题口径（问题3/问题4
    /两问），
977 # 因此类别按误差源前缀现取，与论文表~\ref{tab:预算} 的三类分报一一对应
978 mags = np.array([float(r["量级_h"]) for r in budget])
979 cats = [(k, lab, c) for k, lab, c in BUDGET_CAT
    if any(r["误差源"].startswith(k) for r in budget)]
980 ycol = lambda src: next((c for k, _, c in cats if src.startswith(k)), "
    0.5")
981 ys2 = np.arange(len(mags))[::-1]
982 for y, m, r in zip(ys2, mags, budget):
983     ax2.barh(y, m, color=ycol(r["误差源"]), height=0.60, zorder=3)
984     ax2.text(m * 1.45, y, fmt_mag(m), va="center", fontsize=7.5)
985 ax2.set_yticks(ys2)
986 # 图上显示“设定依赖”（与论文表~\ref{tab:预算} 的类别名一致）；数据侧的键仍
    是 CSV 的
987 # “口径依赖”，不动数据文件以免牵动 结果总账.csv / check_numbers.py 的键名
988 ax2.set_yticklabels([r["误差源"].replace("口径依赖", "设定依赖") for r in
    budget],
989                     fontsize=7.5)
990 ax2.set_xscale("log")
991 ax2.set_xlim(2e-7, 300)
992 ax2.set_xlabel("误差量级 / h（对数轴）")
993 style_axis(ax2, grid="x")
994 ax2.legend(handles=[Patch(color=c, label=lab) for _, lab, c in cats],
995            loc="upper center", bbox_to_anchor=(0.50, -0.22), ncol=len(cats)
    ,
996            handlelength=1.2, columnspacing=1.0, labelspacing=0.28)
997 panel(ax2, "(b)", dx=-0.10)
998 title(ax2, "误差预算: 各类误差量级")
999 fit_left(fig, (ax1, ax2))
1000 print(f" [F5] 敏感性 {len(shown)} 条（基准 {len(base)} 条未画: "
    f"{' '.join(b[0] for b in base)}），红=偏晚、蓝=偏早；"
    f"主导 {ce[0]} {ce[1]:+.4f} h 是其余敏感性因素之和（{others:.2f} h）
    的 "
    f"{abs(ce[1])/others:.0f} 倍；误差预算 {len(mags)} 条 / "
    f"+ '/' .join(f'{lab} {sum(1 for r in budget if r['误差源'].startswith(
    k))} 条"
    f"for k, lab, _ in cats) + "; "
    f"问题4 拆分基准（内生主解）{q4_base:.4f} h → "
    f"+ "; ".join(f'{n.split(':')[1]} {v:+.4f} h' for n, v, _ in grp2),
    flush=True)
1009 save(fig, "fig5_敏感性")
1010
1011
1012 # ===== F6 收缩机制分析 =====
1013 def fig6():
1014     """附件2 收缩曲线是“多机制复合”的诊断：三条零拟合闭合 vs 附件2（不改主模

```

```

    型)。
1015
1016 数字全部读自 数据/endogenous_shrinkage.csv (附件2 时刻网格上的各闭合
    R_pred);
1017 缺列时按 数据/内生收缩分析.md 的口径回退到 附件2.xlsx / q4_main_steps.npz
    。
1018 """
1019 from solver_q1 import C0, R0 # 初始干基含水率 kg/kg / 初始半径 m (题目材料
    参数)
1020
1021 head, rows = read_csv_rows("endogenous_shrinkage.csv")
1022 t_s = np.array([float(r["t_s"]) for r in rows])
1023 h = t_s / 3600.0
1024
1025 def pick(name):
1026     return np.array([float(r[name]) for r in rows]) if name in head else
        None
1027
1028 R_att2 = pick("R_att2_cm")
1029 if R_att2 is None:
1030     _, R_att2 = read_att2()
1031 U_inf = pick("U_inferred_att2")
1032 if U_inf is None: # 等容失水反推 (内生收缩分析.md §0)
1033     U_inf = (1.0 + C0) * (R_att2 / (R0 * 100.0)) ** 2 - 1.0
1034 U_mod = pick("U_mean_model")
1035 if U_mod is None: # 回退: 附件2 几何 (外生 R) 的问题4 步记录
1036     d4 = np.load(DATA_DIR / "q4_main_steps.npz")
1037     U_mod = np.interp(t_s, d4["t"], d4["um"])
1038 R_ideal, R_crust, R_pore = (pick("R1_ideal_mean_cm"), pick("
    R3b_crust_mean_cm"),
1039                             pick("R5_pore_cm"))
1040 # 表面触发结皮 (+毛细塌陷, 论文记为“表面触发, 平台约 1.345 cm”) : 该列 t
    =0 值为 1.900
1041 # (分析脚本的常数塌陷起值), 与初始条件 R(0)=2.000 cm 不符, 按初始条件修正
    首点。
1042 R_surf = pick("R4b_crust_collapse_cm")
1043 if R_surf is not None:
1044     R_surf = R_surf.copy()
1045     R_surf[0] = R_att2[0]
1046 miss = [n for n, v in (("R_att2_cm", R_att2), ("U_inferred_att2", U_inf)
    ,
1047                     ("U_mean_model", U_mod), ("R1_ideal_mean_cm", R_ideal),
1048                     ("R3b_crust_mean_cm", R_crust), ("R5_pore_cm", R_pore),
1049                     ("R4b_crust_collapse_cm", R_surf))
1050         if v is None]
1051 assert not miss, f"endogenous_shrinkage.csv 缺列 (且无回退): {miss}"
1052
1053 # ---- 口径常数 (非计算结果; 来源: 数据/内生收缩分析.md) ----
1054 H_SEG = (3.5, 21.0) # §0 附件2 三段形态: A/B、B/C 分界 (h)
1055 C_GLASS = 0.21 # §1 闭合3 玻璃化转变含水率 (Gordon - Taylor 反解) kg/kg
1056 RHO_SK = 1468.0 # §1 闭合5 骨架密度 (文献值) kg/m³

```

```

1057
1058 R_plat = float(R_att2[-1])          # 附件2 末值平台 (1.198 cm)
1059 dev_crust = (R_crust[-1] - R_plat) / R_plat * 100.0
1060 dev_ideal = (R_ideal[-1] - R_plat) / R_plat * 100.0
1061 k_frz = int(np.argmax(np.abs(np.diff(R_crust)) < 1e-12))
1062 h_frz_crust = float(h[k_frz + 1])  # 结皮闭合冻结时刻 (23.5 h)
1063 dfrz_crust = (h_frz_crust - H_SEG[1]) / H_SEG[1] * 100.0
1064 i_segA = int(np.argmin(np.abs(h - H_SEG[0])))
1065 share_A = (R_att2[0] - R_att2[i_segA]) / (R_att2[0] - R_plat)
1066 exc = R_ideal - R_att2              # 塌陷超额 (附件2 少缩的部分)
1067 k_exc = int(np.argmax(exc))
1068 gap = U_inf - U_mod                 # 成孔缺口
1069 gap35 = float(np.interp(35.0, h, gap))
1070 gap72 = float(np.interp(72.0, h, gap))
1071 i_c = next(i for i in range(1, len(gap)) if gap[i] > 0 > gap[i - 1])
1072 t_cross = float(h[i_c - 1] + (0.0 - gap[i_c - 1]) * (h[i_c] - h[i_c -
1073     1])
1074     / (gap[i_c] - gap[i_c - 1]))
1074 u_cross = float(np.interp(t_cross, h, U_mod))
1075 assert 15.0 < t_cross < 17.0, f"交叉点 {t_cross:.2f} h 与口径 16 h 不符"
1076 assert abs(dev_crust) < 5.0 and 8.0 < dfrz_crust < 15.0, (dev_crust,
1077     dfrz_crust)
1077
1078 fig, (axL, axR) = plt.subplots(1, 2, figsize=(W_DOUBLE, 3.8))
1079 fig.subplots_adjust(left=0.088, right=0.985, top=0.865, bottom=0.155,
1080     wspace=0.24)
1080
1081 # ---- 左: 半径对比 (附件2 vs 三条零拟合闭合) ----
1082 for x0, x1 in ((0.0, H_SEG[0]), (H_SEG[0], H_SEG[1]), (H_SEG[1], h[-1])):
1083     :
1083     axL.axvspan(x0, x1, color="0.93", lw=0, zorder=0)
1084 for xb in H_SEG:
1085     axL.axvline(xb, color="0.72", ls=":", lw=0.8, zorder=1)
1086 axL.text(0.5 * H_SEG[0], 2.115, "A 急缩", ha="center", va="center",
1087     fontsize=7.5, color="0.30")
1088 axL.text(0.5 * H_SEG[0], 2.040, f"占 {share_A*100:.0f}%", ha="center",
1089     va="center", fontsize=7.5, color="0.40")
1090 axL.text(0.5 * (H_SEG[0] + H_SEG[1]), 2.115, "B 缓缩", ha="center", va="
1091     center",
1092     fontsize=7.5, color="0.30")
1093 axL.text(26.0, 2.115, "C 近平台段", ha="center", va="center", fontsize
1094     =7.5,
1095     color="0.30")
1096 axL.plot(h, R_att2, color=ROLE["data"], lw=1.6, zorder=5,
1097     label="附件2 实测")
1098 axL.plot(h, R_ideal, color=ROLE["ref"], ls="--", lw=1.2,
1099     label="闭合1 全局理想")
1100 axL.plot(h, R_surf, color=ROLE["analytic"], lw=1.2,
1101     label="闭合4b 表面触发结皮")
1102 axL.plot(h, R_crust, color=ROLE["model"], lw=1.5,
1103     label="闭合3' 均值触发结皮")

```



```

1102 axL.plot(h, R_pore, color=ROLE["aux2"], ls="-. ", lw=1.2,
1103         label="闭合5 成孔演化")
1104 axL.axhline(R_plat, color=ROLE["guide"], ls=":", lw=1.0)
1105 # 三条闭合的平台值与相对附件2 平台 (1.198 cm) 的偏差, 直标在各自平台一端。
1106 # 闭合3' (1.1693) 与闭合1 理想基线 (1.1226) 在尾段只差 0.047 cm, 放不下两行
      文字:
1107 # 该标签改到理想基线**下方**的空带 (y 轴下限 1.03 给足余量), 避免压住橙色
      虚线与坐标轴。
1108 for val, col, y0 in ((R_surf[-1], ROLE["analytic"], 0.010),
1109                     (R_pore[-1], ROLE["aux2"], 0.008),
1110                     (R_crust[-1], ROLE["model"], -0.058)):
1111     dev = (val - R_plat) / R_plat * 100.0
1112     axL.text(h[-1] - 1.0, val + y0, f"{val:.3f} ({dev:+.0f}%) " if abs(dev)
      >= 10
1113             else f"{val:.3f} ({dev:+.1f}%) ",
1114             ha="right", va="bottom" if y0 > 0 else "top", fontsize=7.5,
      color=col)
1115 axL.text(h[-1] - 1.0, R_plat + 0.014, f"平台 {R_plat:.3f} cm", ha="right"
      , va="bottom",
1116         fontsize=7.5, color="0.30")
1117 axL.annotate(f"早期塌陷\n峰值超额 {exc[k_exc]:.3f} cm",
1118             xy=(h[k_exc], 0.5 * (R_att2[k_exc] + R_ideal[k_exc])),
1119             xytext=(0.16, 0.56), textcoords="axes fraction",
1120             fontsize=7.5, ha="left", va="top",
1121             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
1122             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
      =0.5, alpha=0.95))
1123 # “后期成孔 → 平台”: 原位置 (0.06, 0.22) 在三条闭合曲线的平台上, 白框被曲
      线穿过;
1124 # 移到曲线上方、绿色平台标注左侧的空带 (x 22 - 48 h, y 1.45 - 1.52 cm), 引线
      仍指向成孔平台。
1125 axL.annotate("后期成孔 → 平台",
1126             xy=(30.0, R_pore[-1] + 0.002), xytext=(0.30, 0.40),
1127             textcoords="axes fraction", fontsize=7.5, ha="left", va="top",
1128             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
1129             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
      =0.5, alpha=0.95))
1130 axL.set_xlim(0, h[-1] + 2.0)
1131 axL.set_ylim(1.03, 2.22)
1132 axL.set_xlabel("烘干时间 $t$ / h")
1133 axL.set_ylabel("药材半径 $R$ / cm")
1134 style_axis(axL, grid="y")
1135 axL.legend(loc="upper right", handlelength=1.4, labelspace=0.22)
1136 panel(axL, "(a)")
1137 tile(axL, "半径对比: 四条零拟合闭合")
1138
1139 # ---- 右: 交叉诊断 (附件2 等效含水率 vs 模型平均含水率) ----
1140 axR.axvspan(0.0, t_cross, color=ROLE["ref"], alpha=0.05, lw=0)
1141 axR.axvspan(t_cross, h[-1], color=ROLE["model"], alpha=0.05, lw=0)
1142 axR.axvline(t_cross, color=ROLE["ref"], ls=":", lw=1.1)
1143 axR.plot(h, U_inf, color=ROLE["data"], lw=1.4, zorder=5,

```

```

1144     label=r"附件2  $\bar{U}_{\infty}(t)$  (由  $R$  反推)"
1145     axR.plot(h, U_mod, color=ROLE["model"], lw=1.3,
1146             label=r"模型  $\bar{U}_{\text{model}}(t)$  (附件2 几何求解)")
1147     axR.plot([t_cross], [u_cross], marker="o", ms=5.4, mfc="white", mec=ROLE
1148             ["ref"],
1149             mew=1.3, zorder=6)
1149     axR.annotate(f"交叉  $\approx \{t\_cross:.1f\} h$ ", xy=(t_cross, u_cross),
1150                 xytext=(t_cross + 3.0, u_cross + 0.50), fontsize=7.5,
1151                 color=ROLE["ref"], ha="left", va="bottom",
1152                 arrowprops=dict(arrowstyle="->", lw=0.7, color=ROLE["ref"]),
1153                 bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
1154                         =0.5, alpha=0.95))
1154     axR.text(1.2, 2.05, "附件2 更干 (塌陷)", ha="left", va="top",
1155             fontsize=7.5, color=ROLE["ref"])
1156     axR.text(38.0, 2.05, "附件2 更湿\n(结皮/成孔)", ha="left", va="top",
1157             fontsize=7.5, color=ROLE["model"])
1158     axR.set_xlim(0, h[-1] + 2.0)
1159     axR.set_ylim(0.0, 2.78)
1160     axR.set_xlabel("烘干时间  $t / h$ ")
1161     axR.set_ylabel(r"含水率  $\bar{U} / (kg/kg)$ ")
1162     style_axis(axR, grid="y")
1163     axR.legend(loc="upper right", handlelength=1.4, labelspacing=0.26)
1164     panel(axR, "(b)")
1165     tile(axR, "交叉诊断: 等效 vs 模型含水率")
1166
1167     print(f" [F6] 平台 {R_plat:.4f} cm; 结皮闭合 (均值触发) {R_crust[-1]:.4f}
1168           cm"
1169           f" ({dev_crust:+.2f}%)、冻结 {h_frz_crust:.1f} h ({dfrz_crust:+.1f}%
1170           , "
1171           f"口径分界 {H_SEG[1]:g} h); 表面触发结皮 {R_surf[-1]:.4f} cm"
1172           f" ({(R_surf[-1]-R_plat)/R_plat*100:+.1f}%, 首点按初始条件修正为 "
1173           f"{R_surf[0]:.4f} cm); 理想闭合 {R_ideal[-1]:.4f} cm"
1174           f" ({dev_ideal:+.1f}%)、成孔闭合 {R_pore[-1]:.4f} cm (骨架 {RHO_SK
1175           :.0f}, "
1176           f"{(R_pore[-1]-R_plat)/R_plat*100:+.1f}%) ; "
1177           f"超额峰值 {exc[k_exc]:.4f} cm @ {h[k_exc]:.1f} h (应变 "
1178           f"{exc[k_exc]/R_att2[0]*100:.1f}%) ; C_glass={C_GLASS}; "
1179           f"交叉 {t_cross:.2f} h; 缺口 {gap35:.4f}→{gap72:.4f} kg/kg", flush=
1180           True)
1181     save(fig, "fig6_收缩机制分析")
1182
1183
1184 # ===== F7 内生收缩贴合 =====
1185 def fig7():
1186     """内生收缩模型 (闭合 C, 5 参数标定) 对附件2 的贴合度: R(t) 对比 + 残差带。
1187     数字全部读自 数据/endogenous_calibrated.csv (附件2 时刻网格上的 R_data /
1188     R_pred /
1189     R_ideal / 残差); 模型结构与标定口径见 数据/内生收缩模型.md (复现脚本
1190     08-临时/endogenous_calibrated.py, 只读)。
1191     """

```

```

1188 head, rows = read_csv_rows("endogenous_calibrated.csv")
1189 t_s = np.array([float(r["t_s"]) for r in rows])
1190 h = t_s / 3600.0
1191 R_data = np.array([float(r["R_data_cm"]) for r in rows])
1192 R_pred = np.array([float(r["R_pred_cm"]) for r in rows])
1193 R_ideal = np.array([float(r["R_ideal_cm"]) for r in rows])
1194 res = np.array([float(r["residual_mm"]) for r in rows]) # R_pred -
      R_data (mm)
1195
1196 rmse = float(np.sqrt(np.mean(res ** 2)))
1197 env = float(np.max(np.abs(res))) # 残差包络 ±env mm
1198 k_env = int(np.argmax(np.abs(res)))
1199 k_min = int(np.argmin(res))
1200
1201 def cross(m):
1202     """m=R/R_ideal 自 <1 穿回 >1 的线性插值时刻（滞后交叉）。"""
1203     i = next(j for j in range(1, len(m)) if m[j] > 1.0 > m[j - 1])
1204     return float(h[i - 1] + (1.0 - m[i - 1]) * (h[i] - h[i - 1]) / (m[i]
      - m[i - 1]))
1205
1206 t_cross_pred = cross(R_pred / R_ideal)
1207 t_cross_data = cross(R_data / R_ideal)
1208 H_SEG = (3.5, 21.0) # 三段分界（口径常数：内生收缩分析.md §0）
1209 mB = (h >= H_SEG[0]) & (h <= H_SEG[1])
1210 res_B = float(np.max(np.abs(res[mB])))
1211 assert rmse < 0.1 and env < 0.2, (rmse, env)
1212
1213 fig, (axA, axB) = plt.subplots(2, 1, figsize=(W_DOUBLE, 3.8), sharex=
      True,
1214                                gridspec_kw=dict(height_ratios=[2.55, 1.0]))
1215 fig.subplots_adjust(left=0.085, right=0.985, top=0.875, bottom=0.155,
      hspace=0.30)
1216
1217 # ---- (a) 贴合 ----
1218 for x0, x1 in ((0.0, H_SEG[0]), (H_SEG[0], H_SEG[1]), (H_SEG[1], h[-1]))
      :
1219     axA.axvspan(x0, x1, color="0.93", lw=0, zorder=0)
1220 for xb in H_SEG:
1221     axA.axvline(xb, color="0.72", ls=":", lw=0.8, zorder=1)
1222     axA.text(0.5 * H_SEG[0], 2.125, "A 急缩", ha="center", va="center",
      fontsize=7.5, color="0.30")
1223     axA.text(0.5 * (H_SEG[0] + H_SEG[1]), 2.125, "B 缓缩", ha="center", va="
      center",
1224             fontsize=7.5, color="0.30")
1225     axA.text(26.0, 2.125, "C 近平台段", ha="center", va="center", fontsize
      =7.5,
1226             color="0.30")
1227 axA.plot(h, R_data, color=ROLE["data"], lw=1.6, zorder=5, label="附件2 实
      测 $R(t)$")
1228 axA.plot(h, R_pred, color=ROLE["model"], ls="--", lw=1.3,
      label=f"物理启发半经验闭合 $R_{\{\text{pred}\}}(t)$ (RMSE={rmse:.3f} mm) ")
1229
1230

```

```

1231 axA.plot(h, R_ideal, color=ROLE["ref"], ls=":", lw=1.2,
1232         label="失水理想基线 $R_{ideal}(t)$")
1233 axA.axhline(R_data[-1], color=ROLE["guide"], ls=":", lw=1.0)
1234 axA.text(h[-1] - 1.0, R_data[-1] + 0.013, f"平台 {R_data[-1]:.3f} cm", ha
1235         ="right",
1236         va="bottom", fontsize=7.5, color="0.30")
1237 axA.plot([t_cross_pred], [float(np.interp(t_cross_pred, h, R_ideal))],
1238         marker="o",
1239         ms=4.6, mfc="white", mec=ROLE["ref"], mew=1.1, zorder=6)
1240 axA.annotate(f"滞后交叉  $\approx$  {t_cross_pred:.0f} h", xy=(t_cross_pred, 1.213)
1241         ,
1242         xytext=(21.0, 1.30), fontsize=7.5, color=ROLE["ref"], ha="left"
1243         ,
1244         va="bottom", arrowprops=dict(arrowstyle="->", lw=0.7, color=
1245         ROLE["ref"])))
1246 axA.set_ylim(1.05, 2.20)
1247 axA.set_ylabel("药材半径 $R$ / cm")
1248 style_axis(axA, grid="y")
1249 axA.legend(loc="upper right", handlelength=1.8, labelspace=0.28)
1250 panel(axA, "(a)", dx=-0.075)
1251 tile(axA, "贴合: 附件2 vs 内生模型")
1252
1253 # ---- (b) 残差 ----
1254 for x0, x1 in ((0.0, H_SEG[0]), (H_SEG[0], H_SEG[1]), (H_SEG[1], h[-1]))
1255 :
1256     axB.axvspan(x0, x1, color="0.96", lw=0, zorder=0)
1257 for xb in H_SEG:
1258     axB.axvline(xb, color="0.80", ls=":", lw=0.7, zorder=1)
1259 axB.axhline(0.0, color="0.25", lw=0.8)
1260 for s in (1.0, -1.0):
1261     axB.axhline(s * env, color=ROLE["guide"], ls="--", lw=0.8)
1262 axB.plot(h, res, color=ROLE["model"], lw=1.1, zorder=5)
1263 axB.text(h[-1] - 1.0, env + 0.012, f"包络  $\pm$ {env:.3f} mm", ha="right", va=
1264         "bottom",
1265         fontsize=7.5, color="0.30")
1266 axB.annotate(f"最大偏差 {res[k_env]:+.3f} mm @ {h[k_env]:.1f} h",
1267         xy=(h[k_env], res[k_env]), xytext=(0.32, 0.97), textcoords="
1268         axes fraction",
1269         fontsize=7.5, ha="left", va="top", color="0.15",
1270         arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
1271         bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
1272         =0.5, alpha=0.95))
1273 axB.text(0.5 * (H_SEG[0] + H_SEG[1]), -0.245,
1274         f"B 段过渡区弱欠拟合 ( $\pm$ {env:.2f} mm)", ha="center", va="center",
1275         fontsize=7.5, color="0.30")
1276 axB.set_ylim(-0.32, 0.32)
1277 axB.set_yticks([-0.2, 0.0, 0.2])
1278 axB.set_xlim(0, h[-1] + 2.0)
1279 axB.set_xlabel("烘干时间 $t$ / h")
1280 axB.set_ylabel("残差 / mm")
1281 style_axis(axB, grid="y")

```

```

1273 panel(axB, "(b)", dx=-0.075)
1274 tile(axB, "残差 (预测-实测)")
1275
1276 print(f" [F7] RMSE={rmse:.4f} mm; max|残差|={env:.4f} mm @ {h[k_env]:.1f}
      h"
1277       f" (min {res[k_min]:+.4f} @ {h[k_min]:.1f} h) ; B 段(3.5 - 21 h)|残差|
      max="
1278       f"{res_B:.4f} mm; 平台 data {R_data[-1]:.3f} / pred {R_pred[-1]:.4f}
      cm; "
1279       f"滞后交叉 pred {t_cross_pred:.2f} h / data {t_cross_data:.2f} h",
      flush=True)
1280 save(fig, "fig7_内生收缩贴合")
1281
1282
1283
1284 # ===== F9 截面演化 =====
1285 def fig9():
1286     """问题4 内生版药柱的截面热图：6 个代表时刻的圆盘截面（半径随时间收缩）。
1287
1288     每个时刻把 C(r,t) 插到 Cartesian 网格上、r>R(t) 处留白，
1289     6 盘共享同一色标（全域 min-max）与同一个 colorbar。
1290     """
1291     d = np.load(DATA_DIR / "q4_endo_fields.npz")
1292     t_s, r_cm = d["t_s"], d["r_cm"]
1293     C, R_t = d["C"], d["R_t"]
1294     h = t_s / 3600.0
1295     # 色阶端点（只作画图端点，不参与数据运算；与 fig13 同口径：左端在数据极小值
1296       0.0525 之下、
1297     # 右端在初值 2.55 之上）。取对数色阶的理由见下方 imshow 处的注释。
1298     c_lo, c_hi = 0.05, 2.6
1299
1300     t_show = [0.0, 5.0, 15.0, 25.0, 40.0, 50.0]
1301     idx = [int(np.argmin(np.abs(h - th))) for th in t_show]
1302     xg = np.linspace(-2.2, 2.2, 221)
1303     XX, YY = np.meshgrid(xg, xg)
1304     Rn = np.hypot(XX, YY)
1305
1306     # 版式（2026-09-13 改）：上区 2x3 圆盘，下区新增一条**共享时间轴上下文带**
1307     # 原因：六个盘等宽排布，但采样时刻是 0/5/15/25/40/50 h（间隔 5/10/10/15/10
1308       h），
1309     # 排版把时间间隔抹平了——盘与盘看起来"变化量相同"，而真实收缩速度是前快后慢
1310     # （R 前 5 h 降 0.58 cm，后 10 h 降不到 0.01 cm）。加上时间带后，读者既能看
1311       到
1312     # R(t) 的陡降转平，又能看到六个盘各自落在曲线的哪个位置。
1313     # 纵向额度（in，总计 4.4）：0.32 带内 x 轴标题 + 0.62 时间带 + 0.05 间隙 +
1314     # 0.25 第二行轴标带 + 1.31 第二行盘 + 0.23 行间标题 + 1.31 第一行盘 + 0.31
      顶边距。
1312 fig = plt.figure(figsize=(W_DOUBLE, 4.4))
1313 cmap = plt.get_cmap("viridis").copy()
1314 cmap.set_bad("white") # 域外留白

```

```

1315 X0, WCOL, GAP = 0.045, 0.260, 0.030 # 盘的三列 (0.045 - 0.885, 给右侧色条留
      位)
1316 Y_ROW = (0.6325, 0.2818) # 第一行 / 第二行的底边
1317 H_ROW = 0.2980
1318 im = None
1319 for k, (th, i) in enumerate(zip(t_show, idx)):
1320     col, row = k % 3, k // 3
1321     ax = fig.add_axes([X0 + col * (WCOL + GAP), Y_ROW[row], WCOL, H_ROW])
1322     prof = C[i]
1323     ok = ~np.isnan(prof)
1324     Z = np.interp(Rn, r_cm[ok], prof[ok]) # 按半径插值到圆盘网格
1325     Z = np.where(Rn <= R_t[i], Z, np.nan) # 域外 ( $r > R(t)$ ) 留白
1326     # 对数色阶 (2026-09-12 改):  $\$C\$$  全域 0.0525 - 2.55 跨近两个数量级, 线性
      色阶下
1327     #  $t=15/25/40/50$  h 四盘的  $C \in (0.05, 0.15)$  全挤在最暗的约 4% 色带里, 四盘颜
      色完全相同,
1328     # 读者看不出干燥在推进。改 LogNorm 后每个盘自身跨约 27 - 38% 色带、相邻盘
      整体色位相差
1329     # 约 27%, 壳核梯度与盘间差异同时在颜色上可辨 (与 fig13 同一处理与同一色
      阶端点)。
1330     im = ax.imshow(Z, origin="lower", extent=[xg[0], xg[-1], xg[0], xg
      [-1]],
1331                   cmap=cmap, norm=matplotlib.colors.LogNorm(vmin=c_lo, vmax=
      c_hi),
1332                   interpolation="nearest")
1333     ax.add_patch(plt.Circle((0, 0), R_t[0], fill=False, ec="0.55", lw
      =0.6, ls=":"))
1334     ax.set_xlim(xg[0], xg[-1])
1335     ax.set_ylim(xg[0], xg[-1])
1336     ax.set_aspect("equal")
1337     for s in ("top", "right", "left", "bottom"):
1338         ax.spines[s].set_visible(False)
1339     ax.set_xticks([-2, -1, 0, 1, 2])
1340     ax.set_yticks([-2, -1, 0, 1, 2])
1341     ax.tick_params(labelsize=7.5, pad=1, length=0)
1342     if k < 3:
1343         ax.set_xticklabels([])
1344     else:
1345         ax.set_xlabel(" $\$x\$$  / cm", labelpad=1)
1346     if k % 3:
1347         ax.set_yticklabels([])
1348     else:
1349         ax.set_ylabel(" $\$y\$$  / cm", labelpad=1)
1350         title(ax, f" $\$t\$={th:g}$  h,  $\$R\$={R_t[i]:.2f}$  cm")
1351 cax = fig.add_axes([0.905, 0.345, 0.016, 0.505])
1352 cb = fig.colorbar(im, cax=cax)
1353 cb.set_label("水分浓度  $\$C\$$  / (kg/kg)", fontsize=7.5)
1354 # 对数色阶下 matplotlib 的默认刻度是 10 的整数次幂、标签用科学计数 ( $\$1\backslash
      times 10^{\{-1\}}\$$ ),
1355 # 故与 fig13 一样显式给刻度并固定格式: 只把 0.15 写成两位小数 (即题目达标阈
      值), 其余一位。

```

```

1356 cb.set_ticks([0.1, TH, 0.5, 1.0, 2.0])
1357 cb.ax.yaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(
1358     lambda v, _: f"{TH:.2f}" if abs(v - TH) < 1e-9 else f"{v:.1f}"))
1359 cb.ax.tick_params(labelsize=7.5)
1360
1361 # ---- 共享时间轴上下文带：把六个盘放回时间轴
1362     -----
1363 # x 轴刻度**就是**六个采样时刻，因此刻度本身就是不均匀的（0/5 挤在一起、25
1364     /40/50 拉开），
1365 # 曲线  $R(t)$  前陡后平；竖直虚线把每个盘对回时间轴。
1366 # 左边界比圆盘列右移 0.027 图宽：时间带是满宽矮轴（无 equal aspect 的侧边余
1367     量），
1368 # y 轴标题 " $\$R\$ / \text{cm}$ "按 X0 摆会超出画布 6.5 px（审计裁切项实测）。
1369 axS = fig.add_axes([X0 + 0.027, 0.0727, 0.840 - 0.027, 0.1409])
1370 axS.plot(h, R_t, color=ROLE["model"], lw=1.2, zorder=3)
1371 for th in t_show:
1372     axS.axvline(th, color="0.70", ls=":", lw=0.7, zorder=1)
1373 axS.set_xlim(0.0, 52.0)
1374 axS.set_ylim(1.14, 2.10)
1375 axS.set_xticks(t_show)
1376 axS.set_yticks([1.2, 1.4, 1.6, 1.8, 2.0])
1377 axS.set_xlabel("时间  $\$t\$ / \text{h}$ （刻度即六个采样时刻）", labelpad=1)
1378 axS.set_ylabel(" $\$R\$ / \text{cm}$ ", labelpad=1)
1379 axS.tick_params(labelsize=7.5, pad=1)
1380 style_axis(axS, grid="y")
1381 axS.text(51.4, 1.145, "内生半径  $\$R(t)\$"$ , fontsize=7.5, color=ROLE["model"]
1382     ],
1383     ha="right", va="bottom")
1384
1385 # 抽查：对同一帧，比较 npz 网格原值与“插值到圆盘 Cartesian 网格”的值
1386 i_chk, i2_chk = idx[3], idx[1]
1387 prof, okp = C[i_chk], ~np.isnan(C[i_chk])
1388 prof2, okp2 = C[i2_chk], ~np.isnan(C[i2_chk])
1389 chk = []
1390 for rr in (0.0, 0.5, 1.0):
1391     v_npz = float(prof[okp][int(np.argmin(np.abs(r_cm[okp] - rr)))]))
1392     v_disk = float(np.interp(rr, r_cm[okp], prof[okp])) # 圆盘网格用的同一
1393     插值
1394     chk.append(f"C({rr:.1f})={v_disk:.4f}(npz {v_npz:.4f})")
1395 v_srf = float(prof[okp][-1])
1396 v_srf5 = float(prof2[okp2][-1])
1397 print(f" [F9] 6 时刻 t={t_show} h（帧 {idx}）；R: "
1398     f"' ' '.join(f'{R_t[i]:.3f}' for i in idx)} cm; 色标 {c_lo:.4f} - {
1399     c_hi:.4f} kg/kg"
1400     f" (=数据 min-max)；抽查 t={t_show[3]:g} h: " + "、".join(chk) +
1401     f"、表面 C(R)={v_srf:.4f}; t={t_show[1]:g} h 表面 C(R)={v_srf5:.4f}",
1402     flush=True)
1403 save(fig, "fig9_截面演化")
1404
1405 # ===== F10 力学升级（重标定幂律拟合）

```

```

=====
1400 def fig10():
1401     """修正后重标定的幂律力学模型对附件2 的拟合（三联：拟合 / 残差 / 平台机
        制）。
1402
1403     数据源：数据/mechRefit.csv（145 点 R_data、R_pred、残差 mm）+
1404     mechRefit_fields.npz（重标定参数、全程场量  $\bar{u}/De/g$ 、末端平衡表 terminal）；
1405     口径见 数据/力学升级_修复重标定.md（A04/A05 修复后重标定：幂律保水
1406      $p_c=p*(1-S)/S^\beta$  + 松弛蠕变，吸力单驱动；本图取代此前的线性检验本构情景族，
1407     旧文 力学升级_重标定.md 的数值一律作废）。
1408     """
1409     from mech_shrinkage import MechParams
1410     from solver_q1 import C0, R0
1411
1412     head, rows = read_csv_rows("mechRefit.csv")
1413     t_s = np.array([float(r["t_s"]) for r in rows])
1414     h = t_s / 3600.0
1415     R_data = np.array([float(r["R_data_cm"]) for r in rows])
1416     R_pred = np.array([float(r["R_pred_cm"]) for r in rows])
1417     res = np.array([float(r["residual_mm"]) for r in rows]) # R_pred -
        R_data (mm)
1418     d = np.load(DATA_DIR / "mechRefit_fields.npz")
1419     p_star, beta = float(d["fit_params"][0]), float(d["fit_params"][1])
1420     t_f = float(d["t_f_h"])
1421
1422     H_SEG = (3.5, 21.0) # 急缩 / 缓缩 / 平台 分界（力学升级_修复重标
        定.md §4）
1423     rmse = float(np.sqrt(np.mean(res ** 2)))
1424     seg = [float(np.sqrt(np.mean(res[(h >= a) & (h < b)] ** 2)))
        for a, b in ((0.0, H_SEG[0]), (H_SEG[0], H_SEG[1]), (H_SEG[1], 1e9)
        )]
1425
1426     r_max = float(np.max(np.abs(res)))
1427     k_max = int(np.argmax(np.abs(res)))
1428     plat_pred, plat_data = float(R_pred[-1]), float(R_data[-1])
1429     De_lo, De_hi = float(np.min(d["DeS"])), float(np.max(d["DeS"]))
1430     g_min = float(np.min(d["gS"]))
1431     # 末端平衡表（terminal，纯力学量）： $\beta$  网格  $\rightarrow$  末端  $J_{end}=(R/R_0)^2$ ；
1432     # 拟合  $\beta$  处线性内插即“有限上限”对应的末端半径（力学升级_修复重标定.md §6）
1433     term_beta, term_J = d["terminal"][:, 0], d["terminal"][:, 1]
1434     J_end = float(np.interp(beta, term_beta, term_J))
1435     R_end = R0 * 100.0 * np.sqrt(J_end)
1436     # 与 力学升级_修复重标定.md §4/§6 逐项核对（总/分段 RMSE、平台、max 偏差、
        De、g_min、J_end）
1437     assert abs(rmse - 0.522) < 2e-3 and np.allclose(seg, [1.193, 0.875,
        0.163],
        atol=2e-3), (rmse, seg)
1438
1439     assert abs(plat_pred - 1.1747) < 5e-4 and abs(plat_data - 1.198) < 5e-4
1440     assert abs(r_max - 1.59) < 0.01 and De_hi < 2e-3 and g_min > 0.0, (r_max
        , De_hi, g_min)
1441     assert abs(t_f - 51.1667) < 1e-3 and abs(J_end - 0.3343) < 1e-4, (t_f,
        J_end)

```



```

1442 # 失水理想基线：同一次重标定求解的  $\bar{u}(t)$  等容反推
1443 h_ideal = d["t"] / 3600.0
1444 R_ideal = R0 * 100.0 * np.sqrt((1.0 + d["Um"]) / (1.0 + C0))
1445
1446 fig, (axA, axB, axC) = plt.subplots(1, 3, figsize=(W_DOUBLE, 3.4),
1447                                     gridspec_kw=dict(width_ratios=[1.25, 1.0,
1448                                     0.95]))
1448 fig.subplots_adjust(left=0.075, right=0.985, top=0.865, bottom=0.17,
1449                     wspace=0.36)
1450
1451 # ---- (a) 重标定拟合 ----
1452 for x0, x1 in ((0.0, H_SEG[0]), (H_SEG[0], H_SEG[1]), (H_SEG[1], 72.0)):
1453     axA.axvspan(x0, x1, color="0.94", lw=0, zorder=0)
1454 for xb in H_SEG:
1455     axA.axvline(xb, color="0.72", ls=":", lw=0.8, zorder=1)
1456 for x0, x1, lab in ((0.0, H_SEG[0], "急缩"), (H_SEG[0], H_SEG[1], "缓缩"),
1457                     (H_SEG[1], 72.0, "平台")):
1458     axA.text(0.5 * (x0 + x1), 2.055, lab, ha="center", va="center",
1459             fontsize=7.5, color="0.35")
1460 axA.plot(h, R_data, color=ROLE["data"], lw=1.6, zorder=5, label="附件2 实
1461 测")
1462 # 图例标签内不再带 RMSE 数字：面板宽 1.81 in，标签写成“重标定力学模型 (RMSE
1463 =0.522 mm) ”
1464 # 会让图例框宽达约 1.9 in（超过面板本身），无论放哪个角都会横向溢出并压住下
1465 降段曲线。
1466 # 偏差数字按本刊约定由论文 \caption 承载（该图注已写明 RMSE 0.522 mm 与三段
1467 残差）。
1468 axA.plot(h, R_pred, color=ROLE["model"], ls="--", lw=1.3, label="重标定力
1469 学模型")
1470 axA.plot(h_ideal, R_ideal, color=ROLE["ref"], ls=":", lw=1.2, label="失水
1471 理想基线")
1472 axA.axhline(plat_data, color=ROLE["data"], ls=":", lw=0.9)
1473 axA.axhline(plat_pred, color=ROLE["model"], ls=":", lw=0.9)
1474 # 平台标注：原在 y=1.075（整行占 1.075 - 1.121 cm），右半段被下探到 1.1128
1475 cm 的
1476 # 失水理想基线穿过（实测全分辨率裁剪确认）。压到轴底后留约 0.015 cm 净空，
1477 # 整行不再与任何曲线相交。
1478 axA.text(70.0, 1.052, f"平台：实测 {plat_data:.3f} / 模型 {plat_pred:.4f}
1479 cm",
1480         ha="right", va="bottom", fontsize=7.5, color="0.30")
1481 axA.set_xlim(0, 72)
1482 axA.set_ylim(1.05, 2.10)
1483 axA.set_xlabel("烘干时间 $t$ / h")
1484 axA.set_ylabel("药材半径 $R$ / cm")
1485 style_axis(axA, grid="y")
1486 # 图例原为 loc="center right"，落在  $t \approx 2 - 20$  h、 $R \approx 1.5 - 1.75$  cm 处，正压在下
1487 降段的
1488 # 三条曲线上（实测全分辨率裁剪确认）。面板上部  $t > 20$  h、 $R > 1.3$  cm 除顶端三段
1489 带名外
1490 # 全空，改用轴内坐标 (1.0, 0.86) 定位：图例占 1.77 - 1.95 cm、x 从约 17 h

```

```

起,
1480 # 与曲线 (该 y 带内只出现在  $t \approx 1 - 2.5$  h) 和顶端带名 ( $y=2.055$ ) 都不相交。
1481 axA.legend(loc="upper right", bbox_to_anchor=(1.0, 0.86), handlelength
    =1.8,
1482         labelspacing=0.26)
1483 panel(axA, "(a)", dx=-0.135)
1484 tile(axA, "重标定拟合: 力学模型 vs 附件2")
1485
1486 # ---- (b) 残差 ----
1487 axB.axvspan(0.0, H_SEG[1], color="0.94", lw=0, zorder=0)
1488 axB.axvline(H_SEG[1], color="0.72", ls=":", lw=0.8, zorder=1)
1489 axB.axhline(0.0, color="0.25", lw=0.8)
1490 axB.plot(h, res, color=ROLE["model"], lw=1.1, zorder=5)
1491 axB.annotate(f"前 21 h: 系统性欠缩\n(峰值 {res[k_max]:+.2f} mm)",
1492             xy=(h[k_max], res[k_max]), xytext=(0.30, 0.96), textcoords="
    axes fraction",
1493             fontsize=7.5, ha="left", va="top", color="0.15",
1494             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.35"),
1495             bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
    =0.5, alpha=0.95))
1496 # 三段 RMSE: 原文字起点贴住 y 轴 ( $x=1.5$ ), 首字与轴脊线并排; 改为完整说法并
    右移
1497 axB.text(4.0, -1.42, f"三段 RMSE {seg[0]:.3f}/{seg[1]:.3f}/{seg[2]:.3f}
    mm",
1498         ha="left", va="center", fontsize=7.5, color="0.30")
1499 axB.text(70.0, 0.15, "平台段贴合", ha="right", va="bottom", fontsize=7.5,
    color="0.30")
1500
1501 axB.set_xlim(0, 72)
1502 axB.set_ylim(-1.75, 1.75)
1503 axB.set_xlabel("烘干时间 $t$ / h")
1504 axB.set_ylabel("残差 $R_{\{pred\}}-R_{\{data\}}$ / mm")
1505 style_axis(axB, grid="y")
1506 panel(axB, "(b)", dx=-0.18)
1507 tile(axB, "残差: 结构性欠缩")
1508
1509 # ---- (c) 平台机制: 吸力饱和 (非  $De \gg 1$  结皮冻结) ----
1510 S = np.linspace(0.04, 1.0, 240)
1511 P = MechParams(rho_d0=1.0, rho_s=1468.0, p_star=p_star, ret_model="power
    ",
1512               q_pow=beta)
1513 pbar = np.asarray(P.pbar_c(S), float) / p_star # 归一化平均吸力
1514 cap = 1.0 / ((1.0 - beta) * (2.0 - beta)) # 解析上限 (有限);  $\beta$  取自 npz
    fit_params
1515 axC.plot(S, pbar, color=ROLE["model"], lw=1.4)
1516 axC.axhline(cap, color=ROLE["guide"], ls="--", lw=1.0)
1517 axC.text(0.03, cap + 0.012, f"有限上限 {cap:.3f} $p^* \cdot$", ha="left", va="
    bottom",
1518         fontsize=7.5, color="0.30")
1519 axC.text(0.97, 0.775, "$S \rightarrow 0$: $\bar{p}_c$ 吸力饱和\n$\rightarrow$ 末
    端 $J$ 有限 $\rightarrow$ 平台",
1520         ha="right", va="top", fontsize=7.5, color="0.15")

```

```

1521 axC.text(0.03, 0.03, f"$De\\approx{De_hi:.1e}\\lll1$\n非结皮冻结", ha="
    left",
1522         va="bottom", fontsize=7.5, color="0.15",
1523         bbox=dict(boxstyle="round,pad=0.26", fc="white", ec="0.80", lw
            =0.5, alpha=0.95))
1524 axC.set_xlim(0, 1.03)
1525 axC.set_xticks([0.0, 0.5, 1.0])
1526 axC.set_ylim(0, 0.78)
1527 axC.set_xlabel("饱和度 $S$")
1528 axC.set_ylabel(r"归一化平均吸力 $\bar{p}_c/p^*$")
1529 style_axis(axC, grid="y")
1530 panel(axC, "(c)", dx=-0.20)
1531 tile(axC, "平台机制: 吸力饱和平衡")
1532
1533 print(f" [F10] 重标定幂律拟合: RMSE={rmse:.4f} mm (急缩 {seg[0]:.3f} / 缓
    缩 "
1534         f"{seg[1]:.3f} / 平台 {seg[2]:.3f}); 平台 pred {plat_pred:.4f} vs
        data "
1535         f"{plat_data:.3f} cm; max|残差|={r_max:.3f} mm @ {h[k_max]:.1f} h; "
1536         f"$\beta$={beta:.4f}、$p^*={p_star:.4g}$; "
1537         f"$De$={De_lo:.2e} - {De_hi:.2e} ( $\ll 1$ , 非结皮冻结); $g_{min}$={g_min:.4f}>0
            ; "
1538         f"吸力上限 $\frac{1}{(1-\beta)(2-\beta)}p^*=\text{\texttt{cap:.4f}}$ $p^*$; 末端平衡表 (npz
            terminal) "
1539         f"内插 $\beta$={beta:.4f}  $\rightarrow J_{\text{\texttt{end}}}=J_{end:.4f}$ ($R_{\text{\texttt{end}}}\backslash\approx\{
            R_{end:.3f}\}$ cm); "
1540         f"$t_f$={t_f:.4f} h (npz $t_f_h$)", flush=True)
1541 save(fig, "fig10_力学升级")
1542
1543
1544 # ===== F11 扩散系数与特征时间尺度 =====
1545 def fig11():
1546     """(a) 三族附录材料参数的 D(C) 曲线 (y 对数轴) + D4/D3 比值区间; (b) 特征
        时间尺度 (log 横轴)。
1547
1548     数据源: D 公式直接取自 代码/solver_q1.py (附录2)、solver_q23.py (附录3)、
1549     solver_q4.py (附录4), 并落盘 数据/D_of_C.csv 供复现与论文图注引用;
1550     时间尺度取自 数据/v3_bound.csv (问题 3 口径) 与 数据/q4_bound.csv (问题 4
        口径)。
1551
1552     """
1553     from solver_q1 import D_of_C as D2, R0, T0_K
1554     from solver_q23 import D_of as D3
1555     from solver_q4 import D_of4 as D4
1556
1557     C = np.linspace(0.15, 2.55, 241)
1558     T_ref, T_hot = float(T0_K), 323.315
1559     cols = np.column_stack([C, D2(C), D3(C, T_ref), D4(C, T_ref), D3(C,
        T_hot), D4(C, T_hot)])
1560     hdr = ["# D_of_C.csv —— 三族附录材料参数扩散系数 (fig11(a) 的唯一数据源)
        ",
        "# 公式取自 代码/solver_q1.py (附录2)、solver_q23.py (附录3)、$ 
```

```

1561         solver_q4.py (附录4) ",
1562     "# 附录2 D=7e-9*exp(-0.89/C) (只依赖 C) ; 附录3 D=2.4e-3*exp(-0.45/C)
1563         )*exp(-3850/T); ",
1564     "# 附录4 D=4.2e-4*exp(-0.30/C)*exp(-3850/T); T 开尔文、C 干基含水率
1565         kg/kg",
1566     "# T_ref=301.15 K (初温)、T_hot=323.315 K (环境温度上界); 生成:
1567         python plot_all.py fig11",
1568     "C_kg_kg,D_apx2_m2_s,D_apx3_Tref_m2_s,D_apx4_Tref_m2_s,
1569         D_apx3_Thot_m2_s,D_apx4_Thot_m2_s"]
1570 with open(DATA_DIR / "D_of_C.csv", "w", encoding="utf-8", newline="") as
1571     f:
1572     f.write("\n".join(hdr) + "\n")
1573     for r in cols:
1574         f.write(",".join(f"{v:.8g}" for v in r) + "\n")
1575
1576 r_at_c0 = float(D4(2.55, T_ref) / D3(2.55, T_ref))
1577 r_at_cmin = float(D4(0.15, T_ref) / D3(0.15, T_ref))
1578 _, v3 = read_csv_rows("v3_bound.csv")
1579 _, q4 = read_csv_rows("q4_bound.csv")
1580 k3 = {r["quantity"]: float(r["value"]) for r in v3}
1581 k4 = {r["quantity"]: float(r["value"]) for r in q4}
1582 t_diff3 = R0 ** 2 / (k3["D_max"] * k3["lam1"] ** 2) / 3600.0 # 现算  $R^2/(D_{max} \lambda_1^2) / h$ 
1583
1584 fig, (axA, axB) = plt.subplots(1, 2, figsize=(W_DOUBLE, 3.4))
1585 fig.subplots_adjust(left=0.105, right=0.985, top=0.865, bottom=0.175,
1586                     wspace=0.60)
1587
1588 # ---- (a) D(C) 三族曲线 (附录 3/4 的 D 还随 T 变, 另有  $\times 2.4$  量级——为避免
1589     遮挡不叠虚线,
1590     # 该倍数在 stdout 与图注里给出) ----
1591 specs = ((D2(C), ROLE["data"], "-", "附录 2 (只依赖  $\$C\$$ )"),
1592          (D3(C, T_ref), ROLE["model"], "-", "附录 3 ( $\$T\$=301.15$  K)"),
1593          (D4(C, T_ref), ROLE["ref"], "-", "附录 4 ( $\$T\$=301.15$  K)"))
1594 for y, col, ls, lab in specs:
1595     axA.semilogy(C, y, color=col, ls=ls, lw=1.2, label=lab)
1596     axA.axvline(2.55, color="0.80", lw=0.7, ls=":")
1597     axA.axvline(0.15, color="0.80", lw=0.7, ls=":")
1598     axA.set_xlabel("干基含水率  $\$C\$$  / (kg/kg)")
1599     axA.set_ylabel("扩散系数  $\$D\$$  / ( $m^2/s$ )")
1600     axA.set_xlim(0, 2.75)
1601     axA.set_ylim(1e-13, 4e-8)
1602     style_axis(axA, grid="y")
1603     axA.legend(loc="lower right", handlelength=1.8, labelspace=0.26)
1604     note(axA, f" $\$D_4/D_3=0.175\\backslash,e^{\{0.15/C\}}\$: \backslash n1/\{1/r\_at\_c0:.2f\}$  ( $\$C\$=2.55$ ) ")
1605
1606     f"  $\rightarrow 1/\{1/r\_at\_cmin:.2f\}$  ( $\$C\$=0.15$ ) ", xy=(0.985, 0.985), fs=7.5)
1607 panel(axA, "(a)", dx=-0.115)
1608 tile(axA, "扩散系数: 三族附录材料参数")
1609
1610 # ---- (b) 特征时间尺度 (log 横轴) ----

```

```

1602 items = [("膜界参考 (附 3)", k3["t_film_h"], 0),
1603          ("首模参考 (附 3)", k3["t_ref_h"], 0),
1604          ("R^2/(D_{\\max}\\lambda_1^2)", t_diff3, 2),
1605          ("膜界参考 (附 4)", k4["t_film_h"], 1),
1606          ("R_{\\min}$ 常值 (附 4)", k4["t_rmin_h"], 1),
1607          ("时间积分版 (附 4)", k4["t_int_h"], 1),
1608          ("固定 $R_0$ 版 (附 4)", k4["t_r0_h"], 1)]
1609 items.sort(key=lambda x: x[1])
1610 ycol = [ROLE["model"], ROLE["ref"], OI["grey"]]
1611 ys = np.arange(len(items))[:-1]
1612 axB.barh(ys, [v for _, v, _ in items], color=[ycol[k] for _, _, k in
1613          items],
1614          height=0.62, zorder=3)
1615 axB.set_yticks(ys)
1616 axB.set_yticklabels([n for n, _, _ in items], fontsize=7.5)
1617 for y, (_, v, _) in zip(ys, items):
1618     axB.text(v * 1.16, y, f"{v:.2f} h", va="center", ha="left", fontsize
1619             =7.5)
1620 axB.set_xscale("log")
1621 axB.set_xlim(3.0, 130)
1622 axB.set_xlabel("特征时间尺度 / h (对数轴)")
1623 style_axis(axB, grid="x")
1624 # 颜色语义已在 y 轴标签内 (附 3 / 附 4 / 现算): 不再放图例, 避免压住条形与
1625 # 数值标注
1626 panel(axB, "(b)", dx=-0.115)
1627 tile(axB, "特征时间尺度: 口径对照")
1628 log_minor_off(axA)
1629 log_minor_off(axB, which="x")
1630
1631 print(f" [F11] D 区间: 附录2 {D2(C)[0]:.3e} - {D2(C)[-1]:.3e}、附录3 {D3(C
1632       , T_ref)[0]:.3e} - "
1633       f"{D3(C, T_ref)[-1]:.3e}、附录4 {D4(C, T_ref)[0]:.3e} - {D4(C, T_ref)
1634       [-1]:.3e} m^2/s; "
1635       f"D4/D3={r_at_c0:.4f} (C=2.55, 1/{1/r_at_c0:.2f}) → {r_at_cmin:.4f} (
1636       C=0.15, 1/{1/r_at_cmin:.2f}); "
1637       f"T 由 301.15→323.32 K 的倍数 (图内未叠虚线): 附录3 ×{float(D3(C,
1638       T_hot)[-1]/D3(C, T_ref)[-1]):.2f}、"
1639       f"附录4 ×{float(D4(C, T_hot)[-1]/D4(C, T_ref)[-1]):.2f}; "
1640       f"现算 R^2/(D_max · λ_1^2)={t_diff3:.4f} h (v3_bound: D_max={k3['D_max
1641       ']:.4e}、λ_1={k3['lam1']:.4f}); "
1642       f"时标 {len(items)} 条 ({items[0][1]:.3f} - {items[-1][1]:.3f} h; 蓝=
1643       附3口径、橙=附4口径、灰=现算); "
1644       f"落盘 数据/D_of_C.csv", flush=True)
1645 save(fig, "fig11_扩散系数与时间尺度")
1646
1647 # ===== F12 温度场时空云图 =====
1648 def fig12():
1649     """问题 1 温度场时空云图: x=时间、y=径向位置、色=温度 (coolwarm) + 等值线。
1650     数据源 数据/q1_fields.npz (1800 帧 × 21 径向位置, 0 - 1800 s, 0 - 2 cm) 。
    
```

```

1644 定量读数由本图承担（精度校验见图 2 的偏差面板、收敛见图 15）。
1645  """
1646  d = np.load(DATA_DIR / "q1_fields.npz")
1647  t_s, pos_cm, T_C = d["times"], d["pos_cm"], d["T_C"]
1648  ti = np.arange(0, len(t_s), 4)          # 1800 → 450 帧
1649  Z = T_C[np.ix_(ti, np.arange(len(pos_cm)))].T # → (径向, 时间)
1650  lo, hi = float(Z.min()), float(Z.max())
1651
1652  fig, ax = plt.subplots(figsize=(W_DOUBLE, 3.4))
1653  fig.subplots_adjust(left=0.088, right=0.848, top=0.862, bottom=0.178)
1654  lv = np.linspace(lo - 1e-9, hi + 1e-9, 49)
1655  cf = ax.contourf(t_s[ti], pos_cm, Z, levels=lv, cmap="coolwarm", extend=
        "both")
1656  cs = ax.contour(t_s[ti], pos_cm, Z, levels=8, colors="0.20", linewidths
        =0.45, alpha=0.75)
1657  ax.clabel(cs, inline=True, fontsize=7.5, fmt="%.1f")
1658  cax = fig.add_axes([0.864, 0.178, 0.020, 0.684])
1659  cb = fig.colorbar(cf, cax=cax)
1660  cb.set_label("温度 $T$ / °C", fontsize=7.5)
1661  cb.ax.tick_params(labelsize=7.5)
1662  ax.set_xlabel("时间 $t$ / s")
1663  ax.set_ylabel("径向位置 $r$ / cm")
1664  ax.set_xlim(0, float(t_s[ti][-1]))
1665  ax.set_ylim(0, float(pos_cm[-1]))
1666  style_axis(ax, grid="none")
1667  ax.plot([0, 0], [0, 2], color="0.30", lw=0.9, ls=":")
1668  # 纵轴 $r$ 自下而上 0→2 cm: 中心 ($r=0$) 标注必须在**下**、表面 ($r=2$ cm) 必须在
        **上**。
1669  # 位置还须避开 8 条等温线标签（全部落在 $r\approx 1.55-1.9$ 的顶部条带）：
1670  # 中心标注放底部浅色带 ($r\approx 0.13$, 深色字可读），表面标注放顶部条带下方 ($r
        \approx 1.45$)。
1671  ax.text(1150, 0.13, "中心 $r=0$ (最慢)", fontsize=7.5, color="0.15", ha=
        "center",
1672         va="center")
1673  ax.text(1700, 1.45, "表面 $r=2$ cm (最先升温)", fontsize=7.5, color="
        0.15", ha="right",
1674         va="center")
1675  panel(ax, "(a)", dx=-0.075)
1676  tile(ax, "温度场: 0.5 h 内近准稳态")
1677
1678  print(f" [F12] 问题1 温度场云图: $t$ 0 - {t_s[ti][-1]/60:.1f} min、$r$ 0 - {
        pos_cm[-1]:.2f} cm、"
1679        f"{len(ti)}×{len(pos_cm)} 网格; $T$ 全域 {lo:.3f} - {hi:.3f} °C; "
1680        f"中心 1800 s={Z[0, -1]:.4f} °C、表面 1800 s={Z[-1, -1]:.4f} °C; "
1681        f"口径: colorbar 温度(°C) + 8 条等温线; 定量精度见图 2", flush=True)
1682  save(fig, "fig12_温度场时空云图")
1683
1684
1685  # ===== F13 水分场时空云图 =====
1686  def fig13():
1687      """问题 2/3 水分场时空云图: $x$=时间(h)、$y$=径向位置(cm)、色=干基含水率 (

```

```

    viridis)
+ 0.15 达标等值线（壳核分界）+ 壳核结构标注。
数据源 数据/q23_main_steps.npz（t、C；固定 R=2 cm 口径）。
"""
d = np.load(DATA_DIR / "q23_main_steps.npz")
t_h = d["t"] / 3600.0
C = d["C"]
pos_cm = np.arange(C.shape[1]) * 0.1
ti = np.arange(0, len(t_h), 60)          # 26130 → 436 帧
Z = C[np.ix_(ti, np.arange(len(pos_cm)))].T
tf_h = float(read_json("q23_summary.json")["tf_h"])

fig, ax = plt.subplots(figsize=(W_DOUBLE, 3.4))
fig.subplots_adjust(left=0.088, right=0.848, top=0.862, bottom=0.178)
# 对数色阶（2026-09-12 改）：$C$ 全域 0.0525 - 2.55 跨近两个数量级，线性色阶
# 下 <0.3 全挤在
# 最暗的约 12% 里，干壳（$C$<0.15）与湿核（0.15≤$C$<0.2）颜色完全相同，只能
# 靠白色等值线分辨。
# 改用 LogNorm 后 0.15 落在色阶约 28% 处，壳核在颜色上即可区分。CB_LO/
# CB_HI 只作色阶端点
# （左端取在数据极小值 0.0525 之下、右端取在初值 2.55 之上），不参与任何数
# 据运算。
CB_LO, CB_HI = 0.05, 2.6
cf = ax.contourf(t_h[ti], pos_cm, Z, levels=np.geomspace(CB_LO, CB_HI,
    40),
    cmap=TIME_CMAP, norm=LogNorm(vmin=CB_LO, vmax=CB_HI))
# 达标阈值等值线/标签/壳核标注一并保留（阈值仍是题目口径的 0.15 kg/kg）
cs = ax.contour(t_h[ti], pos_cm, Z, levels=[TH], colors="white",
    linewidths=1.8)
ax.clabel(cs, fmt=f"{TH:.2f}", fontsize=7.5, colors="white")
cax = fig.add_axes([0.864, 0.178, 0.020, 0.684])
cb = fig.colorbar(cf, cax=cax)
cb.set_label("干基含水率 $C$ / (kg/kg)", fontsize=7.5)
# 对数色阶下 matplotlib 的默认刻度是 10 的整数次幂且用科学计数标签（$1
# .12453\times10^{-1}$），
# 故显式给刻度：0.1 / 阈值 0.15 / 0.5 / 1.0 / 2.0（对数间距下相邻两刻度
# ≥17 pt，不叠字）。
# 文字格式固定 1 位小数、仅阈值写 0.15，避免 matplotlib 因 0.15 把其余刻度
# 一并变成 2 位小数。
cb.set_ticks([0.1, TH, 0.5, 1.0, 2.0])
cb.ax.yaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(
    lambda v, _: f"{TH:.2f}" if abs(v - TH) < 1e-9 else f"{v:.1f}"))
cb.ax.tick_params(labelsize=7.5)
ax.axvline(tf_h, color=ROLE["ref"], lw=1.0, ls="--")
ax.text(tf_h - 1.2, 0.10, f"$t_f$={tf_h:.2f} h", fontsize=7.5, color=
    ROLE["ref"],
    ha="right", va="bottom")
ax.text(46.0, 1.60, "干燥壳\n($C$<0.15)", fontsize=7.5, color="white",
    ha="center")
ax.text(46.0, 0.42, "湿核\n($C$\geq0.15)", fontsize=7.5, color="0.85",

```

```

        ha="center")
1727 ax.set_xlabel("时间 $t$ / h")
1728 ax.set_ylabel("径向位置 $r$ / cm")
1729 ax.set_xlim(0, float(t_h[ti][-1]))
1730 ax.set_ylim(0, 2.02)
1731 style_axis(ax, grid="none")
1732 panel(ax, "(a)", dx=-0.075)
1733 tile(ax, "水分场: 干燥壳-湿核结构")
1734
1735 i_tf = int(np.argmin(np.abs(t_h - tf_h)))
1736 shell = float(pos_cm[np.argmax(C[i_tf] < TH)]) if bool(np.any(C[i_tf] <
    TH)) else float("nan")
1737 print(f" [F13] 问题2/3 水分场云图: $t$ 0 - {t_h[ti][-1]:.2f} h、$r$ 0 - {
    pos_cm[-1]:.2f} cm、"
1738       f"{len(ti)}×{len(pos_cm)} 网格; $C$ 全域 {float(np.nanmin(Z)):.4f} - {
    float(np.nanmax(Z)):.4f} kg/kg; "
1739       f"$t_f$={tf_h:.4f} h 时 $C$<0.15 的壳起点 $r$={shell:.2f} cm (壳厚 {
    pos_cm[-1]-shell:.2f} cm); "
1740       f"中心末值 {Z[0, -1]:.4f}、表面末值 {Z[-1, -1]:.4f} kg/kg", flush=
    True)
1741 save(fig, "fig13_水分场时空云图")
1742
1743
1744 # ===== F14 表面通量与表面扩散系数 =====
1745 def fig14():
1746     """表面水分通量与表面扩散系数（问题 1，附录 2 材料参数）。
1747
1748     (a) 表面水通量  $j_w(t)$ ; (b) 表面扩散系数  $D_s(t)$ （与 (a) 共享时间轴，不用双 Y
        轴）;
1749     (c)  $D_s$  随表面含水率  $C_s$  的变化。
1750     数据源 数据/q1_fields.npz ( $C_s/T_s/env_*$ ) + 公式现算，并落盘 数据/
        surface_flux.csv。
1751     口径:  $j_w = \rho_{d,s} \cdot h_m \cdot (U_s - C_{env})$ （正=脱湿）、 $\rho_{d,s} = (650 + 128 \cdot U_s) / (1 + U_s)$ 
        、 $D_s = D_{附2}(C_s)$ 。
1752
1753     """
1754     from solver_q1 import HM, D_of_C as D2, R0
1755     d = np.load(DATA_DIR / "q1_fields.npz")
1756     t_s = d["times"]
1757     Cs, Ts = d["Cs"], d["Ts"]
1758     env_t, env_C = d["env_t"], d["env_C"]
1759     C_env = np.interp(t_s, env_t, env_C)
1760     rho_ds = (650.0 + 128.0 * Cs) / (1.0 + Cs)
1761     j_w = rho_ds * HM * (Cs - C_env)
1762     D_s = D2(Cs)
1763     out = np.column_stack([t_s, Cs, Ts, C_env, rho_ds, j_w, D_s])
1764     hdr = ["# surface_flux.csv —— 问题 1 表面量分项 (fig14 的唯一数据源)",
        "# 口径:  $j_w = \rho_{d,s} \cdot h_m \cdot (U_s - C_{env})$  [kg 水/(m2 · s)], 正=脱湿;  $h_m = 8$ 
        e-7 m/s (附录2 延用)",
        "#  $\rho_{d,s} = (650 + 128 \cdot U_s) / (1 + U_s)$  [kg/m3] (附录2 湿密度  $\rho = 650 + 128C$  除
        以  $1 + C$ , 同 latent_scenario.py 口径)",
        "#  $D_s = D_{附2}(C_s) = 7e-9 \cdot \exp(-0.89/C_s)$  [m2/s];  $C_{env}$  为附件1 环境水分
    ]

```



```

        (线性插值到 1 s 网格)",
1767     "# 生成: python plot_all.py fig14",
1768     "t_s,Cs_kg_kg,Ts_K,rho_d_s_kg_m3,j_w_kg_m2_s,D_s_m2_s"]
1769 with open(DATA_DIR / "surface_flux.csv", "w", encoding="utf-8", newline=
    "") as f:
1770     f.write("\n".join(hdr[:5]) + "\n" + hdr[5] + ",C_env_kg_kg\n")
1771     for r in out:
1772         f.write(f"{r[0]:.0f},{r[1]:.6f},{r[2]:.4f},{r[4]:.4f},{r[5]:.6e},{
            r[6]:.6e},{r[3]:.6f}\n")
1773
1774 fig = plt.figure(figsize=(W_DOUBLE, 3.5))
1775 gs = fig.add_gridspec(2, 2, width_ratios=[1.18, 1.0], wspace=0.34,
    hspace=0.16,
1776                     left=0.135, right=0.985, top=0.865, bottom=0.175)
1777 axJ = fig.add_subplot(gs[0, 0])
1778 axD = fig.add_subplot(gs[1, 0], sharex=axJ)
1779 axC = fig.add_subplot(gs[:, 1])
1780 axJ.plot(t_s / 60.0, j_w, color=ROLE["model"], lw=1.2)
1781 axJ.set_ylabel("$j_w$ / (kg/(m$^2$·s))")
1782 axJ.tick_params(labelbottom=False)
1783 style_axis(axJ, grid="y")
1784 tile(axJ, "表面水通量: 0.5 h 内峰值")
1785 panel(axJ, "(a)", dx=-0.145)
1786 axD.plot(t_s / 60.0, D_s / 1e-9, color=ROLE["ref"], lw=1.2) # 以 1e-9 为
    单位显示
1787 axD.set_ylabel("$D_s$ / ($10^{-9}$ m$^2$/s)")
1788 axD.set_xlabel("时间 $t$ / min")
1789 style_axis(axD, grid="y")
1790 axC.plot(Cs, D_s / 1e-9, color=ROLE["ref"], lw=1.2)
1791 axC.plot(Cs[::120], D_s[::120] / 1e-9, ls="none", marker="o", ms=2.6,
    mfc="white",
1792         mec=ROLE["ref"], mew=0.8)
1793 axC.set_xlabel("表面含水率 $C_s$ / (kg/kg)")
1794 axC.set_ylabel("$D_s$ / ($10^{-9}$ m$^2$/s)")
1795 style_axis(axC, grid="y")
1796 panel(axC, "(b)", dx=-0.135)
1797 tile(axC, "表面扩散系数随含水率")
1798 axC.annotate(f"末值 $D_s$={D_s[-1]/1e-9:.2f}x$10^{{-9}}$", xy=(Cs[-1],
    D_s[-1] / 1e-9),
1799             xytext=(0.96, 0.06), textcoords="axes fraction", ha="right", va
    ="bottom",
1800             fontsize=7.5, color="0.25",
1801             arrowprops=dict(arrowstyle="->", lw=0.7, color="0.45"))
1802
1803 print(f" [F14] 问题1 表面量: $j_w$ {float(j_w.min()):.3e} - {float(j_w.max
    ()):~.3e} kg/(m$^2$·s)"
1804       f" (峰值 {t_s[int(np.argmax(j_w))]/60:.2f} min) ; $D_s$ {float(D_s.
    min()):.3e} - "
1805       f"{float(D_s.max()):.3e} m$^2$/s ($C_s$ {float(Cs.min()):.4f}→{float(Cs
    .max()):.4f}) ; "
1806       f"$\\rho_{{d,s}}$ {float(rho_ds.min()):.1f} - {float(rho_ds.max()):.1

```

```

1807         f} kg/m3; "
1808         f"$C_{{env}}$ {float(C_env.min()):.4f} - {float(C_env.max()):.4f} kg/
            kg; "
1809         f"落盘 数据/surface_flux.csv ({len(t_s)} 行)", flush=True)
1810 save(fig, "fig14_表面通量与表面扩散系数")
1811
1812 # ===== F15 收敛与守恒检验 =====
1813 def fig15():
1814     """(a) 问题 1 温度场误差收敛 (双对数 + 二阶参考斜率); (b) t_f 网格收敛 (双
1815         对数);
1816         (c) 守恒相对残差随网格加密 (三问题: 水量 / 能量)。
1817
1818         数据源: v1_convergence.csv、tf_convergence.csv、q4_endo_convergence.csv、
1819         conservation.csv (问题1)、conservation23.csv (问题2•3)、q4_conservation
1820         .csv (问题4)。
1821
1822         head, conv = read_csv_rows("v1_convergence.csv")
1823         rows = [c for c in conv if c[head[0]] != "order(fit)"]
1824         Ns = np.array([float(c[head[0]]) for c in rows])
1825         errs = np.array([[float(c[h]) for h in head[1:]] for c in rows])
1826         order_row = [c for c in conv if c[head[0]] == "order(fit)"][0]
1827         orders = [float(order_row[h]) for h in head[1:]]
1828         _, tf3 = read_csv_rows("tf_convergence.csv")
1829         _, tf4 = read_csv_rows("q4_endo_convergence.csv")
1830         d3 = {r["case"]: float(r["t_f_h"]) for r in tf3}
1831         d4 = {r["case"]: float(r["t_f_h"]) for r in tf4}
1832         N3 = [40, 80, 160]
1833         e3 = [abs(d3[f"N={n}"] - d3["richardson_extrap"]) for n in N3]
1834         e4 = [abs(d4[f"N={n}"] - d4["richardson_extrap"]) for n in N3]
1835         _, cons1 = read_csv_rows("conservation.csv")
1836         _, cons23 = read_csv_rows("conservation23.csv")
1837         _, cons4 = read_csv_rows("q4_conservation.csv")
1838
1839         fig, (axA, axB, axC) = plt.subplots(1, 3, figsize=(W_DOUBLE, 3.7),
1840             gridspec_kw=dict(width_ratios=[1.0, 1.0,
1841                 1.06]))
1842         # 图高 3.4→3.7 in、top 0.865→0.930、bottom 0.185→0.350: (c) 的图例移到面板
1843         外、x 轴标签
1844         # 下方, 需要纵向让位; top 同时抬高以补回面板高度, 三个面板不被压扁 (轴内高
1845         2.15 in vs 原 2.31 in)。
1846         fig.subplots_adjust(left=0.083, right=0.985, top=0.930, bottom=0.350,
1847             wspace=0.34)
1848         mk = ["o", "s", "^"]
1849         for k in range(errs.shape[1]):
1850             col = TIME_CMAP(0.12 + 0.72 * k / max(errs.shape[1] - 1, 1))
1851             axA.loglog(Ns, errs[:, k], color=col, marker=mk[k], ms=3.0, lw=1.1,
1852                 label=f"$t$={ [100, 600, 1800][k] } s (阶 {orders[k]:.2f}) ")
1853             axA.loglog(Ns, errs[0, 0] * (Ns / Ns[0]) ** -2.0, color=ROLE["guide"],
1854                 ls="--", lw=1.0,
1855                 label="斜率 -2 (二阶参考)")

```

```

1849 axA.set_xlabel("网格数 $N$")
1850 axA.set_ylabel("最大偏差 / °")
1851 axA.set_xticks([20, 40, 80, 160, 320])
1852 axA.set_xticklabels(["20", "40", "80", "160", "320"])
1853 axA.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
1854 style_axis(axA, grid="both")
1855 axA.legend(loc="lower left", handlelength=1.6, labelspace=0.24)
1856 panel(axA, "(a)", dx=-0.185)
1857 tile(axA, "收敛: 温度场误差")
1858
1859 for ns, es, col, lab, m in ((N3, e3, ROLE["model"], "问题 3 (固定 $R$)",
1860                             "o"),
1861                             (N3, e4, ROLE["ref"], "问题 4 (内生 $R(t)$)", "s"))
1862                             :
1863                             (N3, e3[-1] * (np.array(N3) / 160.0) ** -1.0, color=ROLE["
1864                             guide"], ls="--",
1865                             lw=1.0, label="斜率 -1 (一阶参考)"))
1866 axB.set_xlabel("网格数 $N$")
1867 axB.set_ylabel("$|t_f(N)-t_f^{\mathrm{Rich}}| / h$")
1868 axB.set_xticks(N3)
1869 axB.set_xticklabels([str(n) for n in N3])
1870 axB.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
1871 style_axis(axB, grid="both")
1872 axB.legend(loc="lower left", handlelength=1.6, labelspace=0.24)
1873 panel(axB, "(b)", dx=-0.185)
1874 tile(axB, "收敛: $t_f$ 随网格")
1875
1876 series = [(float(r["N"]) for r in cons1), [float(r["max_rel_residual"])
1877 for r in cons1],
1878           "水量/能量 (问题 1)", ROLE["data"], "o"),
1879           ([float(r["N"]) for r in cons23],
1880            [float(r["max_rel_residual_water"]) for r in cons23], "水量 (问题
1881            2·3)",
1882            ROLE["model"], "s"),
1883           ([float(r["N"]) for r in cons23],
1884            [float(r["glob_rel_residual_enthalpy"]) for r in cons23], "能量
1885            (问题 2·3)",
1886            ROLE["aux1"], "s"),
1887           ([float(r["N"]) for r in cons4],
1888            [float(r["max_rel_residual_water"]) for r in cons4], "水量 (问题
1889            4)",
1890            ROLE["ref"], "^"),
1891           ([float(r["N"]) for r in cons4],
1892            [float(r["glob_rel_residual_enthalpy"]) for r in cons4], "能量
1893            (问题 4)",
1894            ROLE["aux2"], "^")]
1895 for xs, ys_, lab, col, m in series:
1896     axC.loglog(xs, ys_, color=col, marker=m, ms=3.2, lw=1.1, label=lab)
1897 axC.axhline(1e-10, color=ROLE["guide"], ls=":", lw=0.9)
1898 # “机器精度量级  $10^{-10}$ ”原在右上角 (0.975, 0.86), 被水量 (问题2·3) 蓝线

```

```

穿过：
1892 # 曲线与  $1e-10$  点线之间只剩约 1.5 个字号的高度，故移到左上角该空带内，并加
      白底。
1893 axC.text(0.015, 0.975, "机器精度量级  $10^{-10}$ ", transform=axC.transAxes
      , ha="left",
1894       va="top", fontsize=7.5, color="0.35", zorder=6,
1895       bbox=dict(boxstyle="round,pad=0.18", fc="white", ec="none", alpha
      =0.88))
1896 axC.set_xticks([40, 80, 160])
1897 axC.set_xticklabels(["40", "80", "160"])
1898 axC.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
1899 axC.set_xlabel("网格数  $N$ ")
1900 axC.set_ylabel("相对残差 (各网格最大值)")
1901 # 下界由  $1.8e-14$  放宽到  $1.5e-15$  (仅多一个空 decade, 不改任何数据/数字): 给
      下面那条
1902 # 能量 (问题4) 粉线留出完整的 3 个网格点与  $1e-10$  参考线之间的空带。
1903 axC.set_ylim(1.5e-15, 1.5e-10)
1904 style_axis(axC, grid="both")
1905 # 图例移到 (c) 面板**外**、x 轴标签下方 (2026-09-12 改): 原 5 行单列无框图
      例宽约面板的
1906 # 73%, 放轴内左/右下都会盖住能量 (问题4) 粉线的第 1 个点 ( $N=40, 2.68e-14$ 
      ), 缩字号又低于
1907 # 7.5 pt 的审计下限; 移到轴外后三个点 ( $N=40/80/160$ ) 全部可见。轴外不再需要
      白色衬底,
1908 # 改回全局默认 frameon=False。列数只能取 1: 这 5 个中文标签排 2 列时图例宽
      约 2.4 in,
1909 # 越过图右边界会被裁掉 (ncol=2 实测右端超出画布约 0.32 in)。
1910 axC.legend(loc="upper center", bbox_to_anchor=(0.5, -0.20), ncol=1,
      handlelength=1.4, labelspacing=0.18, borderpad=0.3, fontsize
      =7.5)
1911
1912 panel(axC, "(c)", dx=-0.165)
1913 tile(axC, "守恒: 残差随网格加密")
1914 for _ax in (axA, axB, axC):
1915     log_minor_off(_ax)
1916
1917 print(f" [F15] 收敛: 问题1 阶 {orders} ( $N=20 \rightarrow 320$ , 5 点); 问题3  $|\Delta t_f|$  "
1918       f"{e3[0]:.4f}/{e3[1]:.4f}/{e3[2]:.4f} h、问题4 {e4[0]:.5f}/{e4[1]:.5f}/{e4[2]:.6f} h"
1919       f" (对 Richardson 外推); 守恒: 问题1 max {max(float(r['max_rel_residual']) for r in cons1):.2e}、"
1920       f"问题2·3 水量 max {max(float(r['max_rel_residual_water']) for r in cons23):.2e} / 能量 "
1921       f"{max(float(r['glob_rel_residual_enthalpy']) for r in cons23):.2e} 、问题4 水量 max "
1922       f"{max(float(r['max_rel_residual_water']) for r in cons4):.2e} / 能
      量 "
1923       f"{max(float(r['glob_rel_residual_enthalpy']) for r in cons4):.2e}"
1924       f" (conservation23.csv 的 enthalpy_window 列为逐时窗口口径, 非残差, 不
      入图)", flush=True)
1925 save(fig, "fig15_收敛与守恒检验")
1926

```

```

1927
1928 # ===== F16 中心表面演化与半径收缩 =====
1929 def fig16():
1930     """(a) 中心/表面含水率演化 + 0.15 阈值 + t_f 标注; (b) 半径收缩 + 附件 2
        实测对照。
1931
1932     数据源 数据/q4_endo_fields.npz (t_s/Cc/Cs/R_t)、q4_endo_eval.csv (t_f)、
1933     附件 2 (01-题目/原始文件/附件2.xlsx, 实测 R)。
1934     """
1935     d = np.load(DATA_DIR / "q4_endo_fields.npz")
1936     h = d["t_s"] / 3600.0
1937     Cc, Cs, R_t = d["Cc"], d["Cs"], d["R_t"]
1938     _, ev = read_csv_rows("q4_endo_eval.csv")
1939     kv = {r["item"]: float(r["value"]) for r in ev}
1940     tf_h = kv["tf_first_crossing_h"]
1941     t2, R2 = read_att2()
1942     h2 = t2 / 3600.0
1943
1944     fig, (axA, axB) = plt.subplots(2, 1, figsize=(W_SINGLE, 3.30), sharex=
        True,
1945                                     gridspec_kw=dict(height_ratios=[1.28, 1.0]))
1946     fig.subplots_adjust(left=0.135, right=0.975, top=0.885, bottom=0.155,
        hspace=0.22)
1947     axA.plot(h, Cc, color=ROLE["model"], lw=1.2, label="中心  $\theta=0$ ")
1948     axA.plot(h, Cs, color=ROLE["data"], lw=1.2, ls="--", label="表面  $\theta=1$ ")
1949     axA.axhline(TH, color=ROLE["guide"], ls=":", lw=1.1)
1950     axA.axvline(tf_h, color=ROLE["ref"], ls="--", lw=1.0)
1951     # t_f 标注改到竖线左侧 (右侧是图例样例所在区, 原位置字尾被图例的线样压住)
1952     axA.text(tf_h - 0.8, 2.42, f"$t_f$={tf_h:.4f} h", fontsize=7.5, color=
        ROLE["ref"],
1953             va="top", ha="right")
1954     # 阈值标注移到曲线之外的右侧空段 (中心曲线止于 t_f=50.54 h): 原位置被表面
        曲线穿过
1955     axA.text(72.0, TH + 0.075, f"阈值 {TH} kg/kg", fontsize=7.5, color="0.30",
        ha="right",
1956             va="bottom")
1957     axA.set_ylabel("干基含水率  $\theta$  / (kg/kg)")
1958     axA.set_ylim(0.0, 2.75)
1959     axA.set_xlim(0, 72)
1960     style_axis(axA, grid="y")
1961     axA.legend(loc="upper right", handlelength=1.8, labelspace=0.26)
1962     panel(axA, "(a)", dx=-0.115)
1963     tile(axA, "中心/表面含水率演化 (内生主解)")
1964
1965     axB.plot(h2, R2, ls="none", marker="o", ms=2.6, markevery=3, mfc="white",
        ,
1966             mec=ROLE["data"], mew=0.8, label="附件 2 实测 (每 3 点标一次)")
1967     axB.plot(h, R_t, color=ROLE["model2"], lw=1.3, label="内生闭合预测  $R(t)$ ")
1968     axB.axvline(tf_h, color=ROLE["ref"], ls="--", lw=1.0)
1969     axB.set_xlabel("时间  $t$  / h")

```

```

1970 axB.set_ylabel("药材半径 $R$ / cm")
1971 axB.set_ylim(1.14, 2.06)
1972 style_axis(axB, grid="y")
1973 axB.legend(loc="upper right", handlelength=1.8, labelspace=0.26)
1974 panel(axB, "(b)", dx=-0.115)
1975 tile(axB, "半径收缩: 预测 vs 实测")
1976
1977 i_tf = int(np.argmin(np.abs(h - tf_h)))
1978 ov = (h2 >= h[0]) & (h2 <= h[-1]) # 只在预测覆盖的 0 - 50.54 h 窗口内比
1979 rmse = float(np.sqrt(np.mean((np.interp(h2[ov], h, R_t) - R2[ov]) ** 2))
1980 )
1981 print(f" [F16] $t_f$={tf_h:.6f} h (q4_endo_eval.csv); $t_f$ 时 中心 {Cc[
1982 i_tf]:.4f} / 表面 "
1983 f"{Cs[i_tf]:.4f} kg/kg; $R$ {R_t[0]:.4f}→$t_f$ {R_t[i_tf]:.4f}→末 {
1984 R_t[-1]:.4f} cm"
1985 f" ({h[-1]:.2f} h); 与附件 2 实测散点 ({int(ov.sum())} 点重叠段) RMSE
1986 = {rmse:.4f} cm; "
1987 f"表面在 {float(h[np.argmax(Cs < TH)):.2f} h 首次低于阈值", flush=
1988 True)
1989 save(fig, "fig16_中心表面演化与半径收缩")
1990
1991 # ===== F17 几何模型与边界条件示意图 =====
1992 def fig17():
1993     """几何模型与边界条件示意图（无数据曲线，纯 matplotlib patches 矢量绘制）。
1994
1995     口径常数取自 代码/solver_q1.py: $R_0$=0.02 m、$L$=0.25 m、$h$=25 W/(m2·K)
1996     、
1997     $h_m$=8e-7 m/s、$T_0$=301.15 K (28 °C)、$U_0$=2.55 kg/kg（题目给定与声明假
1998     设）。
1999
2000     """
2001     from solver_q1 import R0 as R0_M, H, HM, T0_K, C0
2002     L_CM = 25.0 # 题目给定: 药材长 25 cm（一维径向模型只用 L/R
2003     比, 见论文表 2）
2004
2005     fig, (axA, axB) = plt.subplots(1, 2, figsize=(W_DOUBLE, 3.3),
2006                                     gridspec_kw=dict(width_ratios=[1.0, 2.15]))
2007     fig.subplots_adjust(left=0.035, right=0.975, top=0.905, bottom=0.075,
2008                         wspace=0.10)
2009
2010     # ---- (a) 横截面 ----
2011     axA.set_aspect("equal")
2012     axA.add_patch(plt.Circle((0, 0), 2.0, fc="#eef3f8", ec=ROLE["model"], lw
2013                             =1.3))
2014     axA.plot([0], [0], marker="+", ms=7, mew=1.2, color=ROLE["data"])
2015     axA.annotate("", xy=(2.0, 0), xytext=(0, 0),
2016                 arrowprops=dict(arrowstyle="-|>", lw=1.0, color=ROLE["data"]))
2017     # $R_0$ 尺寸标注: 原位置 (1.05, 0.12) 左对齐、右端伸到  $x \approx 2.3$ , 被圆周轮廓穿
2018     过;
2019     # 改为在直径箭头正上方居中（居中的文字两端离圆周都留有余量）。
2020     axA.text(1.00, 0.28, "$R_0$=2 cm", fontsize=7.5, ha="center", va="bottom

```

```

    ",
    color=ROLE["data"])
2010
2011 for ang, lab in ((90, "$h$, $h_m$"), (0, "$h$, $h_m$"), (270, "$h$,
    $h_m$"), (180, "$h$, $h_m$")):
2012     a = np.deg2rad(ang)
2013     axA.annotate("", xy=(3.05 * np.cos(a), 3.05 * np.sin(a)),
2014                   xytext=(2.05 * np.cos(a), 2.05 * np.sin(a)),
2015                   arrowprops=dict(arrowstyle="->", lw=0.9, color=ROLE["ref"]))
2016
2017 axA.text(0, 3.62, "环境 $T_a(t)$、$C_{\mathrm{env}}(t)$", fontsize=8, ha=
    "center",
2018         va="center", color=ROLE["ref"])
2019 axA.text(0, -3.72, "对称轴 $r=0$ ($\partial_r=0$)", fontsize=7.5, ha="
    center",
2020         va="center", color="0.30")
2021 axA.set_xlim(-3.6, 3.6)
2022 axA.set_ylim(-4.3, 4.3)
2023 axA.axis("off")
2024 panel(axA, "(a)", dx=0.0)
2025 tile(axA, "横截面：一维径向")
2026 axA.text(-3.55, 2.75, f"$h=\{H:g\} \text{ W}/(\text{m}^2 \cdot \text{K}) \backslash \text{n} h_m=\{HM:g\} \text{ m/s}$",
    fontsize=7.5,
2027         va="center", ha="left", color="0.20")
2028
2029 # ---- (b) 轴向纵剖面（柱体横放：水平向=柱长 L，竖向=直径 2R0） ----
2030 axB.set_aspect("equal")
2031 axB.add_patch(Rectangle((0, -2), L_CM, 4, fc="#eef3f8", ec=ROLE["model"
    ], lw=1.3))
2032 # 竖向尺寸线（柱体右侧）：直径 2R0
2033 axB.annotate("", xy=(L_CM + 0.9, -2), xytext=(L_CM + 0.9, 2),
2034               arrowprops=dict(arrowstyle="<-|>", lw=0.8, color="0.35"))
2035 axB.text(L_CM + 0.9, 2.45, "$2R_0=4 \text{ cm}$", fontsize=8, ha="center", va="
    bottom")
2036 # 水平尺寸线（柱体下方）：柱长 L，两端画延伸线
2037 for xe in (0.0, L_CM):
2038     axB.plot([xe, xe], [-2.0, -3.15], color="0.62", lw=0.6, ls=":",
2039               zorder=1)
2040 axB.annotate("", xy=(0, -3.0), xytext=(L_CM, -3.0),
2041               arrowprops=dict(arrowstyle="<-|>", lw=0.8, color="0.35"))
2042 axB.text(0.5 * L_CM, -3.35, f"$L=\{L\_CM:.0f\} \text{ cm}$", fontsize=8, ha="center
    ", va="top")
2043 for x in (3.0, 8.0, 13.0, 18.0, 22.0): # 沿轴向的代表性对流箭头
2044     for sgn in (1,):
2045         axB.annotate("", xy=(x, sgn * 3.0), xytext=(x, sgn * 2.05),
2046                       arrowprops=dict(arrowstyle="->", lw=0.8, color=ROLE["ref"
    ])))
2047 axB.text(12.5, 3.75, "环境 $T_a(t)$、$C_{\mathrm{env}}(t)$（附件 1）",
    fontsize=8,
2048         ha="center", color=ROLE["ref"])
2049 axB.text(12.5, 0.78, "$\text{u}_{c1}(C)$、$c_p(C)$、$k(C)$、$D(C,T)$（附录 2/3/4
    分问选用）",

```

```

2048         fontsize=7.5, ha="center", color=ROLE["model"])
2049     axB.text(12.5, -0.32, "干基含水率 $U$, 温度 $T$; $r=R$ 处 Robin 边界",
2050             fontsize=7.5, ha="center", color="0.25")
2051     axB.text(12.5, -1.45, "初始: $T_0=28$ °C、$U_0=2.55$ kg/kg (均匀) ",
2052             fontsize=7.5,
2053             ha="center", color="0.25")
2054     axB.set_xlim(-4.2, 31.5)
2055     axB.set_ylim(-4.6, 4.6)
2056     axB.axis("off")
2057     panel(axB, "(b)", dx=0.0)
2058     tile(axB, "轴向: 长柱、表面对流")
2059
2060     print(f" [F17] 示意图常数: $R_0=\{R0\_M*100:.1f\}$ cm、$L=\{L\_CM:.0f\}$ cm (题
2061           目给定)、"
2062           f"$h=\{H:g\}$ W/(m2·K)、$h_m=\{HM:g\}$ m/s、$T_0=\{T0\_K-273.15:.1f\}$ °C、"
2063           f"$U_0=\{C0\}$ kg/kg (solver_q1.py; 无数据曲线) ", flush=True)
2064     save(fig, "fig17_几何模型与边界条件示意图")
2065
2066     FIGS = {"fig1": fig1, "fig2": fig2, "fig3": fig3, "fig5": fig5, "fig6":
2067            fig6,
2068            "fig7": fig7, "fig9": fig9, "fig10": fig10,
2069            "fig11": fig11, "fig12": fig12, "fig13": fig13, "fig14": fig14,
2070            "fig15": fig15, "fig16": fig16, "fig17": fig17}
2071
2072     def main():
2073         ap = argparse.ArgumentParser()
2074         ap.add_argument("only", nargs="*", default=[], help="只出指定图, 如 fig4"
2075                        )
2076         ap.add_argument("--no-cache", action="store_true", help="fig4 对照曲线强
2077                        制重算")
2078         a = ap.parse_args()
2079         plt.rcParams.update(RC)
2080         want = a.only or ["fig1", "fig2", "fig3", "fig4", "fig5", "fig6", "fig7"
2081                           ,
2082                           "fig9", "fig10", "fig11", "fig12", "fig13", "fig14", "fig15"
2083                           ,
2084                           "fig16", "fig17"]
2085         for k in want:
2086             print(f"[{k}] 作图 ...", flush=True)
2087             if k == "fig4":
2088                 fig4(no_cache=a.no_cache)
2089             else:
2090                 FIGS[k]()
2091
2092     print("\n[审计汇总] 图 | 设计尺寸(in) | 纵横比 | 最小字号(pt) | 标题≤28 |
2093           图例项 | 刻度重叠 | "
2094           "柱基线 | 裁切文字 | 图内文字块 | colorbar(须带label) | 3D面板 | 嵌入
2095           位图 | 密集标记 | 结论")

```



```

2089     for r in AUDIT:
2090         print(f" {r['stem']:<22} {r['w']:.1f}x{r['h']:.1f} | {r['aspect']:.2f}
          | "
2091         f"{r['min_font']:.1f} | {max((len(t) for t in r['titles']),
          default=0)} | "
2092         f"{r['legends']} | {len(r['tick_bad'])} | {len(r['bar_base'])} | "
          "
2093         f"{len(r['clipped'])} | {len(r['figtext'])} | {r['cb']} | "
2094         f"{r['ax3d']} | {r['embed_img']} | {r['dense_marker']} | "
2095         f"'PASS' if r['ok'] else 'FAIL'", flush=True)
2096     bad = [r for r in AUDIT if not r["ok"]]
2097     if GLYPH.msgs:
2098         print("[字体] 缺字告警如下（应视为失败）：", flush=True)
2099         for m in GLYPH.msgs:
2100             print(" ", m, flush=True)
2101     else:
2102         print("[字体] 无缺字告警（logging + warnings 双通道）；PDF 嵌入
          TrueType 子集"
2103         "（pdf.fonttype=42）", flush=True)
2104     print(f"[灰度] 灰度预览见 图表/_qa/*_gray.png（逐张目视核对“不靠颜色可区
          分”）", flush=True)
2105     if GLYPH.msgs or bad:
2106         print(f"[门禁] FAIL: 缺字 {len(GLYPH.msgs)} 条，审计不合格 {len(bad)}
          张 "
2107         f"({'', '.join(r['stem'] for r in bad)}))", flush=True)
2108         sys.exit(1)
2109     print(f"[门禁] PASS: {len(AUDIT)} 张图审计全部通过", flush=True)
2110
2111
2112 if __name__ == "__main__":
2113     main()

```

E.19 一键复现

单命令驱动：按既定顺序调用上述全部脚本并汇总校验结果。

代码 20 一键复现（run_all.py, 73 行）

```

1  """run_all.py: 单命令复现全项目（M1 - M4 + 问题 4 内生几何主解）。
2
3  用法:  python run_all.py
4  依次执行 run_q1 → run_q23 → run_q4 → **run_q4_endo**（数据: 问题 4 内生几何主
          解,
5  产出内生版 result4.xlsx）→ **run_q4_endo_fields**（数据: 内生时空场
          q4_endo_fields.npz）
6  → run_m4（判据/敏感性/误差/总账），各自的一条命令产物与门禁全部包含，
7  日志落 ../数据/run_all.log; 结束打印 result1 - 4 的 md5 供两次运行比对。
8
9  口径说明: 问题 4 的**权威主解是内生几何版**（t_f=50.536869 h, 收缩由耦合内生闭
          合 C 预测）;
10 run_q4 产出的附件 2 外生几何版仅作交叉验证。run_m4 最后运行并重建结果总账,
11 其问题 4 行与论文表 6 均取内生来源（见 run_m4.py D5 注释）。

```

```

12 """
13 from __future__ import annotations
14
15 import hashlib
16 import sys
17 import time
18 from pathlib import Path
19
20 CODE_DIR = Path(__file__).resolve().parent
21 sys.path.insert(0, str(CODE_DIR))
22 A_DIR = CODE_DIR.parent
23 DATA_DIR = A_DIR / "数据"
24 sys.path.insert(0, str(DATA_DIR)) # run_q4_endo*.py 在 数据（活依赖链）
25 LOG = DATA_DIR / "run_all.log"
26
27
28 class Tee:
29     def __init__(self, *streams):
30         self.streams = streams
31
32     def write(self, s):
33         for st in self.streams:
34             st.write(s)
35             st.flush()
36
37     def flush(self):
38         for st in self.streams:
39             st.flush()
40
41
42 def main():
43     t0 = time.perf_counter()
44     logf = open(LOG, "w", encoding="utf-8")
45     sys.stdout = sys.stderr = Tee(sys.__stdout__, logf)
46     print(f"==== run_all 开始 {time.strftime('%Y-%m-%d %H:%M:%S')} ====")
47     import run_q1
48     import run_q23
49     import run_q4
50     import run_q4_endo # 数据: 问题 4 内生几何主解 (→ 内生版 result4.xlsx)
51     import run_q4_endo_fields # 数据: 内生时空场 (→ q4_endo_fields.npz)
52     import run_m4
53     for name, mod in (("run_q1(问题1+V1)", run_q1),
54                       ("run_q23(问题2/3+门禁)", run_q23),
55                       ("run_q4(问题4 外生交叉验证+T6/V5/T7)", run_q4),
56                       ("run_q4_endo(问题4 内生几何主解→result4.xlsx)", run_q4_endo),
57                       ("run_q4_endo_fields(内生时空场)", run_q4_endo_fields),
58                       ("run_m4(判据/敏感性/误差/总账)", run_m4)):
59         print(f"\n##### {name} #####", flush=True)
60         t1 = time.perf_counter()
61         mod.main()

```

```

62     print(f"##### {name} 完成, 耗时 {time.perf_counter() - t1:.1f} s
        #####", flush=True)
63     print("\n=== result 工作簿 md5 (两次运行应一致) ===")
64     for fn in ("result1.xlsx", "result2.xlsx", "result3.xlsx", "result4.xlsx
        "):
65         p = A_DIR / "结果" / fn
66         print(f"{fn}: {hashlib.md5(p.read_bytes()).hexdigest()} ({p.stat().
            st_size} B)")
67     print(f"=== run_all 全部完成, 总耗时 {time.perf_counter() - t0:.1f} s
        ===")
68     sys.stdout = sys.stderr = sys.__stdout__
69     logf.close()
70
71
72 if __name__ == "__main__":
73     main()

```

AI 工具使用声明

本参赛队就本次竞赛中 AI 工具的使用情况声明如下：

序号	AI 工具名称	使用场景/用途	涉及论文章节	使用程度（轻度/中度）
1	KimiCode、DeepSeek	程序测试：辅助编写与调试数值求解代码，以及守恒恒等式、网格与时间收敛、退化测试等检验脚本	第五章（模型建立与求解）、第六章（模型检验）、附录 D	中度
2	KimiCode、DeepSeek	绘图：辅助整理数据并生成全文插图	第一章至第七章的插图	中度
3	KimiCode、DeepSeek	数学公式推理验证：辅助核对控制方程与移动域推导、解析谱解、量纲与极限退化	第 5.1 节、附录 A、第 6.1–6.2 节	轻度

使用程度说明：

轻度：仅用于语法检查、文字润色、代码片段解释、公式验证等辅助性工作，不影响论文核心内容与结论。

中度：用于辅助代码编写、文献整理、思路拓展等，核心建模与分析仍由队员独立完成。

本队承诺以上声明内容全部属实，论文核心建模方法、算法设计、结果分析与结论均为队伍独立完成，不存在 AI 工具直接生成核心内容或代写代做的情形。如有不实，愿承担全部责任并接受相应处理。

全体队员签字：_____

日期：_____